INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

Trabajo Terminal

“Prototipo de sistema móvil de detección de insectos descortezadores en árboles mediante visión por computadora. Caso de estudio: Bosque de Chapultepec”

2026-B161

Presentan

Karime Calero Alvarez

Ángel Damián Franco Rayas

Miguel Ángel Gómez López

Directores

Ing. Ariel López Rojas

Dra. Vanessa Alejandra Camacho Vázquez

Ciudad de México, mayo de 2026

INSTITUTO POLITÉCNICO NACIONAL

ESCOM

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA

No de TT: 2026-B161 Ciudad de México, mayo de 2026

Documento Técnico

"Prototipo de sistema móvil de detección de insectos descortezadores en árboles mediante visión por computadora"

Presentan

Karime Calero Alvarez[1]

Ángel Damián Franco Rayas[2]

Miguel Ángel Gómez López[3]

Directores

Ing. Ariel López Rojas

Dra. Vanessa Alejandra Camacho Vázquez

RESUMEN

El presente reporte describe la propuesta de un sistema orientado a la identificación de indicios de infestación por insectos descortezadores en árboles, integrado por una aplicación móvil para la captura y análisis en campo, y una plataforma web complementaria para la consulta histórica, la gestión de registros y el seguimiento de árboles analizados. A lo largo del documento se aborda el contexto del problema, su delimitación computacional, la fundamentación del enfoque de aprendizaje profundo y el análisis y diseño general de la solución propuesta.

Palabras clave: plagas descortezadoras de árboles, visión por computadora, áreas verdes, aprendizaje profundo, aplicación móvil.

[1] kcaleroa1900@alumno.ipn.mx [2] afrancor1900@alumno.ipn.mx [3] mgomezl1801@alumno.ipn.mx

Carta Responsiva

Advertencia

"Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos con derecho de propiedad y por lo tanto, su uso quedará restringido a las aplicaciones que explícitamente se convengan."

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Av. Juan de Dios Bátiz s/n Teléfono: 57296000, extensión 52000.

Índice general

Capítulo 1. Planteamiento del proyecto **10**
1.1 Problemática . . . 10
1.2 Objetivo . . . 11
1.2.1 General . . . 11
1.2.2 Específicos . . . 11
1.3 Justificación . . . 11
1.4 Estado del arte . . . 13
1.5 Metodología . . . 14

Capítulo 2. Contexto y caracterización del caso de estudio **16**
2.1 Justificación de la selección del área de interés . . . 16
2.2 Descripción General del Área de Estudio . . . 16
2.3 Caracterización de los Huespedes . . . 17
2.3.1 Mecanismo de colonización y daño . . . 17
2.3.2 El papel de la resina como barrera de defensa . . . 18
2.4 Caracterización de los Hospederos . . . 18
2.4.1 Condiciones de vulnerabilidad y contexto urbano . . . 19
2.5 Progresión visual de la infestación por insectos descortezadores . . . 20
2.5.1 Señales visuales en el follaje . . . 20
2.5.2 Señales visuales en la corteza . . . 21
2.5.3 Implicaciones para el análisis de color del *dataset* . . . 22

Capítulo 3. Delimitación computacional del problema **23**
3.1 Naturaleza de la información de entrada . . . 23
3.2 La visión por computadora como área de estudio . . . 23
3.2.1 Definición de visión por computadora . . . 23
3.2.2 Tareas de visión por computadora . . . 23
3.2.3 Herramientas y marcos de trabajo utilizadas en visión por computadora 24
3.2.4 Flujo de trabajo común en el desarrollo de sistemas mediante visión por computadora . . . 25
3.3 Justificación del uso de visión por computadora en el proyecto . . . 25

Capítulo 4. Fundamentación del enfoque de aprendizaje profundo **27**
4.1 Modelos básicos de aprendizaje automático en clasificación . . . 27
4.1.1 Aprendizaje automático supervisado y clasificación . . . 27
4.1.2 Principales algoritmos de clasificación . . . 27
4.2 Limitaciones de los enfoques tradicionales en escenarios reales . . . 28
4.3 Aprendizaje profundo en tareas de visión por computadora . . . 29
4.3.1 Qué es el aprendizaje profundo . . . 29
4.3.2 Ventajas del aprendizaje profundo en tareas visuales . . . 30
4.3.3 Aplicación del aprendizaje profundo en visión por computadora . . . 30
4.3.4 Importancia de los datos anotados . . . 31
4.4 Comparación entre aprendizaje automático tradicional y aprendizaje profundo . 31
4.4.1 Similitudes generales entre ambos enfoques . . . 31
4.4.2 Diferencias en el enfoque de resolución de problemas . . . 32
4.4.3 Diferencias en los datos, entrenamiento e infraestructura . . . 32
4.5 Justificación del uso de aprendizaje profundo en el proyecto . . . 33

Capítulo 5. Análisis comparativo de arquitecturas y selección del modelo **34**
5.1 Arquitecturas relevantes para clasificación de imágenes . . . 34
5.2 Relevancia de las arquitecturas para dispositivos móviles . . . 34
5.3 Modelos de referencia para clasificación de imágenes . . . 35
5.3.1 ResNet-50 . . . 35
5.3.2 MobileNetV3 . . . 35
5.3.3 EfficientNet-B0/B1/B3 . . . 35
5.3.4 MobileViT . . . 35
5.4 Transfer learning y fine-tuning . . . 35
5.5 Comparación de modelos candidatos . . . 36
5.6 Selección del modelo definitivo para el proyecto . . . 36
5.7 Arquitectura de EfficientNet-B3 . . . 37
5.8 Arquitectura y funcionamiento de las Redes Neuronales Convolucionales . . . 37
5.8.1 Convolución . . . 38
5.8.2 Normalización por lotes (Batch Normalization) . . . 38
5.8.3 Función de activación ReLU . . . 38
5.8.4 Agrupamiento (Pooling) . . . 39
5.8.5 Aplanado (Flatten) . . . 39
5.8.6 Capas Densas (Fully Connected) . . . 39
5.8.7 Clasificación mediante Softmax . . . 39

Capítulo 6. Conjunto de datos y preprocesamiento **40**
6.1 Recopilación de datos . . . 40
6.2 Características del conjunto de datos . . . 40
6.3 Etiquetado de imágenes . . . 40
6.4 Preprocesamiento de imágenes . . . 41
6.5 Análisis complementario de color . . . 42
6.6 Organización de los datos para entrenamiento, validación y prueba . . . 42

Capítulo 7. Análisis del sistema **43**
7.1 Requerimientos del sistema . . . 43
7.1.1 Requerimientos funcionales . . . 43
7.1.2 Requerimientos no funcionales . . . 46
7.2 Reglas de negocio del sistema . . . 48
7.3 Catálogo de mensajes del sistema . . . 52
7.4 Actores del sistema . . . 53
7.5 Métricas de evaluación del proyecto . . . 54
7.5.1 Métricas de desempeño del modelo . . . 54
7.5.2 Métricas de desempeño del sistema . . . 55
7.5.3 Criterios de aceptación del prototipo . . . 55
7.6 Análisis de riesgos y restricciones . . . 56
7.6.1 Riesgos asociados al conjunto de datos . . . 56
7.6.2 Riesgos técnicos de implementación . . . 56
7.6.3 Restricciones del prototipo . . . 57
7.6.4 Medidas de mitigación . . . 57
7.7 Plan general de pruebas . . . 58
7.7.1 Pruebas del modelo de clasificación . . . 58
7.7.2 Pruebas del sistema . . . 60
7.7.3 Pruebas de integración . . . 62
7.7.4 Recursos disponibles . . . 65

7.7.5 Herramientas para el desarrollo del modelo . . . 65
7.7.6 Herramientas para el procesamiento de imágenes . . . 66
7.7.7 Herramientas para la aplicación móvil . . . 67
7.7.8 Herramientas para la aplicación web . . . 68
7.7.9 Infraestructura en la nube . . . 69
7.8 Análisis financiero . . . 72
7.8.1 Recursos disponibles . . . 72
7.8.2 Costos estimados de infraestructura . . . 72
7.8.3 Costos asociados a servicios en la nube . . . 74
7.8.4 Viabilidad económica del prototipo . . . 76
7.8.5 Justificación del proveedor de nube: AWS sobre Azure y GCP . . . 77
7.9 Proyección a ambiente productivo . . . 80

Capítulo 8. Diseño del sistema **84**
8.1 Arquitectura general del sistema . . . 84
8.2 Estrategia de seguridad y control de acceso . . . 85
8.2.1 Almacenamiento y persistencia políglota . . . 85
8.2.2 Integración con Amazon SageMaker . . . 85
8.2.3 Observabilidad y monitoreo centralizado . . . 86
8.2.4 Entorno de desarrollo y automatización (CI/CD) . . . 86
8.2.5 Flujos principales de comunicación . . . 87
8.2.6 Estructura interna del backend (Clean Architecture) . . . 88
8.3 Arquitectura de la aplicación móvil . . . 90
8.3.1 Componentes de la aplicación . . . 90
8.3.2 Inferencia local y procesamiento de imágenes . . . 90
8.3.3 Estrategia *Offline-First* y sincronización . . . 91
8.4 Arquitectura de la aplicación web . . . 92
8.4.1 Arquitectura del Frontend (SPA) . . . 92
8.4.2 Distribución estática y alojamiento seguro . . . 92
8.4.3 Consumo de API y visualización de datos . . . 92
8.4.4 Autenticación y seguridad en el cliente . . . 93
8.5 Arquitecura del modelo de clasificación y análisis cromático . . . 94
8.6 Entorno de desarrollo y pruebas locales . . . 95
8.6.1 Entorno de desarrollo local . . . 95
8.6.2 Servidor de pruebas *on-premise* (Raspberry Pi) . . . 95
8.6.3 Orquestación y barrera de despliegue . . . 95
8.7 Diagramas de casos de uso . . . 97
8.8 Especificación de casos de uso . . . 106
8.8.1 Casos de uso generales . . . 106
8.8.2 CU-M00: Operación general de la aplicación móvil . . . 114
8.8.3 CU-W00: Consulta general de la plataforma web . . . 117
8.8.4 CU-A00: Gestionar cuentas y dispositivos desde la plataforma web . . . 120
8.9 Diseño de interfaces de la aplicación móvil (UI/UX) . . . 123
8.9.1 Diseño visual y prototipos de alta fidelidad . . . 123
8.9.2 Flujo de navegación y experiencia de usuario . . . 127
8.9.3 Presentación de resultados . . . 127
8.10 Diseño de interfaces de la plataforma web (UI/UX) . . . 128
8.10.1 Módulo de Autenticación . . . 128
8.10.2 Panel administrativo y gestión de usuarios . . . 129
8.10.3 Dashboard y flujo de navegación (Experto de Campo) . . . 133

8.10.4 Análisis espacial e identificación de focos de infestación 134
8.10.5 Presentación de resultados analíticos (D3.js) 134
8.11 Diseño de base de datos 136
8.11.1 Modelo entidad-relación 136
8.11.2 Esquema relacional 137
8.11.3 Diccionario de datos 138

Capítulo 9. Pruebas y validación del sistema **142**
9.1 Pruebas de viabilidad de requisitos funcionales en dispositivos móviles 142
9.1.1 Requisitos de hardware mínimos y recomendados 142
9.1.2 Clasificación de dispositivos por gama de rendimiento 144
9.1.3 Metodología de validación y benchmark con dispositivo de prueba . . . 145
9.1.4 Resultados del benchmark y criterios de aceptación 146
9.1.5 Casos de incompatibilidad y limitaciones técnicas 147
9.2 Pruebas y evaluación del modelo de aprendizaje profundo 150
9.2.1 Configuración experimental y estrategia de entrenamiento 150
9.2.2 Distribución del conjunto de datos 151
9.2.3 Curvas de entrenamiento y convergencia 151
9.2.4 Optimización del umbral de decisión 152
9.2.5 Evaluación final en el conjunto de prueba independiente 152
9.2.6 Exportación a TensorFlow Lite y verificación de paridad numérica . . . 154

10 Conclusiones **155**

11 Trabajo a futuro **156**

12 Referencias **157**

13 Glosario **161**

Índice de figuras

1 Metodología SCRUM. Tomado de [8] 15
2 Escarabajo descortezador del pino *Dendroctonus frontalis*. Tomado de [9] . . . 17
3 a) Tronco de árbol con grumos de resina producidos como respuesta defensiva ante la perforación por descortezadores; b) Red de galerías formadas por *D. frontalis* en el tejido subcortical. Tomado de [9] 18
4 Ciclo de retroalimentación negativa entre el cambio climático, el debilitamiento del arbolado y el incremento en la actividad de insectos descortezadores. Tomado de [9] 19
5 Etapas de coloración del follaje que indican la fase de infestación del arbolado. Se observa la transición de verde (Fase 1) a verde-amarillento (Fase 2) y café-rojizo (Fase 3), con la dirección de expansión del brote indicada. Tomado de [10] 20
6 Señales visuales en la superficie exterior e interior de la corteza. A) hembras alimentándose en el tejido subcortical; B) macho entrando a través del orificio hecho por la hembra; C) grumos de resina acumulados a lo largo del tronco; D) galerías individuales generadas por las larvas durante la alimentación y oviposición. Tomado de [10] 21
7 Algunas tareas de visión por computadora 24
8 Principales algoritmos de clasificación. Tomado de [18] 28

9 Subconjuntos de la Inteligencia artificial. Tomado de [23] 32
10 Diagrama comparativo con diferentes arquitecturas de CNNs. Tomado de [27] . 36
11 Arquitectura CNN de EfficientNet. 37
12 Capas de una CNN 39
13 Muestra de configuración de ajustes de una misma imagen. 41
14 Métricas de evaluación del modelo de clasificación 54
15 Curvas de entrenamiento con dataset de prueba de 1,500 imágenes por clase. . . 58
16 Evaluación del modelo sobre el conjunto de validación. 58
17 Predicciones sobre imágenes no vistas durante el entrenamiento. 59
18 Arquitectura general del sistema. 84
19 Flujo de Integración y Despliegue Continuo (CI/CD). 86
20 Diagrama de secuencia del proceso de sincronización de datos. 87
21 Estructura en capas del *backend* basada en *Clean Architecture*. 89
22 Flujo de decisión y sincronización de datos de la estrategia *Offline-First*. 91
23 Flujo de autenticación y consumo seguro de API en la aplicación web. 93
24 Arquitectura general del modelo 94
25 Arquitectura de entornos: desarrollo local, pruebas *on-premise* y orquestación hacia producción. 96
26 CU-G01–CU-G04: Autenticación móvil / web 97
27 CU-M05–CU-M12: Dashboard móvil 98
28 CU-M06 y CU-M13–CU-M21: Captura y análisis móvil 99
29 CU-M08 y CU-M22–CU-M23: Historial de registros móvil 100
30 CU-M09 y CU-M24–CU-M25: Mapa local de árboles móvil 100
31 CU-M11–CU-M12 y CU-M26–CU-M27, CU-G04: Perfil y configuración móvil 101
32 CU-M10 y CU-M28–CU-M30: Notificaciones y alertas móvil 101
33 CU-W05–CU-W09: Dashboard web 102
34 CU-W06 y CU-W10–CU-W15: Historial general de registros web 103
35 CU-W07 y CU-W16–CU-W18: Mapa general de árboles web 103
36 CU-W08 y CU-W19–CU-W23: Estadísticas de registros web 104
37 CU-W09 y CU-G04: Perfil de usuario web 104
38 CU-A01–CU-A09: Administración web 105
39 Paleta de colores utilizada en el diseño. 123
40 Pantalla de inicio de sesión. 124
41 Pantalla de creación de cuenta. 124
42 Menú principal con resumen de jornada y lista de árboles registrados. 124
43 Detalle de árbol: estado sanitario e historial de capturas. 124
44 Mapa de árboles registrados con indicadores de estado sanitario. 125
45 Selección del tipo de captura: corteza o follaje. 125
46 Selección del árbol al que se asociará la captura. 125
47 Resultado del análisis de corteza: clasificación e índice de confianza. 125
48 Resultado del análisis de corteza: análisis de color. 126
49 Resultado del análisis de follaje: distribución cromática de la zona capturada. . 126
50 Pantalla de alertas y notificaciones sanitarias. 126
51 Pantalla de perfil del técnico. 126
52 Maqueta de inicio de sesión. 128
53 Maqueta de recuperacion de contraseña. 128
54 Maqueta de nueva contraseña. 129
55 CU-A01: Gestión general de cuentas de usuario. 129
56 CU-A02: Creación de nueva cuenta de usuario. 130
57 CU-A03: Edición de datos de cuenta de usuario. 130

58 CU-A04: Desactivación de cuenta de usuario. 131
59 CU-A05: Asignación de rol a cuenta de usuario. 131
60 CU-A06: Gestión de dispositivos asociados a una cuenta. 132
61 CU-A07: Asociación de dispositivo escáner a una cuenta de usuario. 132
62 CU-A08: Desasociación de dispositivo escáner de una cuenta de usuario. . . . 133
63 Interfaz del panel de resumen (Dashboard) 133
64 Maqueta del mapa general de árboles y focos de infestación. 134
65 Interfaces de análisis detallado y estadísticas. 135
66 MER: Sistema móvil y web 137
67 ER: Sistema móvil y web 138
68 Curvas de entrenamiento y validación para el clasificador de corteza (fases 1 y 2). 151
69 Evolución de la exactitud balanceada y la puntuación F_β en validación. . . . 152
70 Evaluación diagnóstica en el conjunto de prueba independiente. 153
71 Distribución de probabilidades predichas en el conjunto de prueba. 154

Índice de cuadros

1 Softwares y trabajos similares a nuestra propuesta 13
2 Comparación de modelos candidatos 36
3 Requerimientos funcionales de la aplicación móvil y la plataforma web 43
4 Requerimientos no funcionales de la aplicación móvil y la plataforma web . . . 46
5 Reglas de negocio de la aplicación móvil y la plataforma web 48
6 Catálogo de mensajes del sistema. 52
7 Plan de pruebas funcionales del sistema. 60
8 Plan de pruebas de integración del sistema. 62
9 Costos estimados de infraestructura física del proyecto. 72
10 Límites de la capa gratuita de AWS aplicables al prototipo. 74
11 Estimación de costos reales del prototipo sin capa gratuita (AWS Pricing Calculator, us-east-1, 2026). 75
12 Resumen de costos totales del prototipo. 76
13 Comparativa de servicios equivalentes entre AWS, Azure y GCP (2025). 77
14 Comparativa de capa gratuita entre AWS, Azure y GCP (2025). 77
15 Comparativa entre arquitectura serverless y contenedores para ambiente productivo. 80
16 Comparativa de instancias Amazon RDS PostgreSQL para ambiente productivo (us-east-1, 2026). 81
17 Estimación de costos en ambiente productivo generada con AWS Pricing Calculator (us-east-1, 2026). 82
18 Resumen de atributos del caso de uso: Iniciar sesión. 106
19 Resumen de atributos del caso de uso: Recuperar contraseña. 108
20 Resumen de atributos del caso de uso: Restablecer contraseña. 110
21 Resumen de atributos del caso de uso: Cerrar sesión. 112
22 Resumen de atributos del caso de uso: Operar aplicación móvil desde el dashboard. 114
23 Resumen de atributos del caso de uso: Consultar información consolidada desde el dashboard web. 117
24 Resumen de atributos del caso de uso: Gestionar cuentas y dispositivos desde la plataforma web. 120
25 Diccionario de datos de la tabla CUENTA 138
26 Diccionario de datos de la tabla DISPOSITIVO 139
27 Diccionario de datos de la tabla CUENTA_DISPOSITIVO 139

28 Diccionario de datos de la tabla ARBOL 139
29 Diccionario de datos de la tabla ZONA_FOLLAJE 140
30 Diccionario de datos de la tabla ANALISIS 140
31 Diccionario de datos de la tabla ANALISIS_CROMATICO 141
32 Requisitos de hardware mínimos y recomendados para la aplicación móvil. . . 143
33 Resultados del benchmark y evaluación frente a umbrales de aceptación. 146
34 Resumen de restricciones técnicas e incompatibilidades de hardware. 149
35 Parámetros de configuración experimental del modelo. 150
36 Métricas de evaluación final en el conjunto de prueba con umbral $\tau = 0.5791$. . 152
37 Prueba de paridad diagnóstica entre el modelo Keras y el modelo TensorFlow Lite. 154

Capítulo 1. Planteamiento del proyecto

1.1 Problemática

El mantenimiento de los bosques de la Ciudad de México en los últimos años ha presentado deficiencias notables debido a la escasez de especialistas y a la falta de tecnologías para su monitoreo. Estos ecosistemas han enfrentado amenazas constantes por la presencia de plagas, entre ellas **los insectos descortezadores que en 2022 en promedio afectaron al 30% de los árboles de varios bosques de la Ciudad**, que afectan la salud de los árboles y pueden provocar pérdidas significativas en el entorno natural. Esto representa un problema, ya que, de acuerdo con la Organización de las Naciones Unidas para la Alimentación y la Agricultura (FAO), los bosques urbanos no solo contribuyen al equilibrio ecológico, sino que también impactan directamente en la calidad de vida de la población al mejorar la calidad del aire, reducir la temperatura y ofrecer espacios de recreación [1].

En este contexto, los bosques constituyen un elemento fundamental de la biodiversidad en la Ciudad de México. La presencia de estos insectos representa un riesgo grave, ya que, al alimentarse de la corteza, interrumpen el flujo de nutrientes, debilitando a los árboles y, en muchos casos, provocando su muerte. Las consecuencias de estas afectaciones trascienden lo ambiental, alcanzando también dimensiones culturales y recreativas, dado que los bosques reciben a miles de visitantes diariamente. Además, cada uno de ellos es un espacio emblemático con un profundo valor histórico y social, ya que albergan museos, áreas deportivas, monumentos y sitios de gran relevancia para la identidad del país. Su deterioro, por tanto, no solo afecta al equilibrio ecológico, sino también al patrimonio cultural y a la calidad de los servicios ambientales que proveen.

La detección temprana de este tipo de plagas es esencial para prevenir daños irreversibles y reducir los costos asociados a la recuperación forestal. Sin embargo, los métodos tradicionales dependen de la inspección visual realizada por especialistas, lo cual requiere tiempo, experiencia y recursos humanos que no siempre están disponibles. Este enfoque limita la capacidad de respuesta ante brotes, favoreciendo la propagación de las plagas. A ello se suma que la extensión de algunos bosques dificulta la supervisión constante de todas sus áreas, lo que aumenta la probabilidad de que las plagas se desarrollen sin ser identificadas a tiempo.

Ante este panorama, el desarrollo de un sistema móvil que administre y monitoree la plaga en arboles afectados. Esta herramienta permitirá no solo administrar la información de los árboles afectados, sino también dar un seguimiento preciso a la evolución de la plaga, todo esto haciendo uso de visión por computadora para que permita identificar automáticamente plagas de insectos descortezadores siendo una solución más adecuada para apoyar a los especialistas en el área. Se trata de una oportunidad relevante para innovar y reforzar las estrategias de conservación, buscando el mantenimiento y recuperación de espacios naturales esenciales en la Ciudad de México. La aplicación de tecnologías basadas en inteligencia artificial y procesamiento de imágenes podría apoyar el monitoreo continuo, optimizar la toma de decisiones y generar bases de datos que apoyen a comprender mejor la dinámica de las plagas, posibilitando estrategias de control más efectivas y sostenibles.

Además, la incorporación de este sistema podría apoyar como modelo para otros espacios verdes del país, fortaleciendo la gestión ambiental urbana y la preservación de áreas naturales que funcionan como pulmones urbanos frente a los desafíos del cambio climático y la urbanización acelerada.

1.2 Objetivo

1.2.1 General

Desarrollar un sistema móvil de visión por computadora, basado en aprendizaje automático, para identificar automáticamente la presencia de plagas de insectos descortezadores y clasificar el estado sanitario del arbolado en un bosque en específico de la Ciudad de México. El sistema utilizará imágenes recolectadas en campo y permitirá la detección temprana de focos de infestación para fomentar su atención.

1.2.2 Específicos

1. Identificar el bosque en la Ciudad de México más apto para la construcción del *dataset*.
2. Construir y etiquetar datos tomados directamente en campo de imágenes del arbolado, identificando casos sanos e infectados.
3. Diseñar una arquitectura en la nube que permita almacenar, procesar y entrenar al modelo de manera eficiente.
4. Entrenar un modelo de visión por computadora capaz de reconocer síntomas de la infestación por descortezadores en los árboles de un bosque en específico de la Ciudad de México.
5. Desarrollar una aplicación móvil para la captura de imágenes y seguimiento de los resultados.

1.3 Justificación

Los bosques de la Ciudad de México abarcan miles de hectáreas y reciben durante cada año a millones de visitantes, quienes caminan día con día recorriendo el arbolado, mientras que, detrás de ese paisaje, plagas aparentemente silenciosas han estado provocando la destrucción masiva de un ecosistema forestal que cumple funciones ecológicas, económicas y sociales fundamentales.

Como se menciona [2], existen plagas que afectan negativamente los bosques de México en mayor o menor medida. Entre estas plagas, el objeto de estudio del proyecto presentado es el escarabajo descortezador. Estos pequeños insectos en un ambiente controlado actúan como reguladores del ecosistema, sin embargo, cuando en los bosques templados se presentan actividades como incendios forestales, tala desmedida o ciertos efectos del cambio climático, se incrementa la proliferación de estos escarabajos, produciendo efectos adversos hasta volverse una plaga. [3]

Conforme a la CONAFOR [3]: “Los descortezadores son pequeños escarabajos que viven debajo de la corteza del árbol y se alimentan del tejido que conduce los nutrientes de este, provocando frecuentemente la muerte del arbolado.”

Debido a lo anterior, la plaga del insecto descortezador es vista como un problema forestal a nivel nacional que afecta una gran cantidad de arbolado, situándose en segundo lugar después de los incendios forestales. Incluso, la CONAFOR considera a 5 estados en *muy alto riesgo*, los cuales son Ciudad de México, Chihuahua, Durango, Jalisco, Estado de México y Michoacán. [4]

Las afectaciones que han tenido los bosques urbanos de la Ciudad de México varían respecto a cada zona, sin embargo, todas presentan problemas relacionados con la salud del arbolado. Un ejemplo representativo es el Bosque de Chapultepec, donde según indica Karla Mora, la SEDEMA estimó que aproximadamente 200 árboles padecían de afecciones, principalmente los fresnos [5].

Incluso, Reséndiz *et al.* [6] reporta lo siguiente:

> En los fresnos también se detectó la presencia del descortezador Hylesinus sp. afectando directamente ramas y tallos. La magnitud del daño causado por el coleóptero en el arbolado provocó incidencias de 2.4%; de acuerdo a Martínez-Trejo et al. (2016) este tipo de escolítidos con hábitos de barrenar y descortezar ramas y tallos, y con fines de alimentación, construyen galerías en ramas jóvenes que son destinadas a la producción, lo mismo que generan galerías de reproducción en troncos y ramas gruesas de árboles debilitados.
>
> Además, en el arbolado de C. lusitanica se presentó el descortezador (Phloeosinus sp.) con afectaciones de 4.08% en la población forestal de esta especie de cedro. Este coleóptero es importante en áreas urbanas donde hay Cupressaceae, la principal característica de este curculiónido radica en que las infestaciones las realiza en árboles que se encuentran debilitados por estrés hídrico o que se encuentran en decadencia, además Tórres y Sánchez (2006) confirman que Phloeosinus sp. afecta a Cupressus arizonica (pp.152-153).

Otro ejemplo de la presencia de esta plaga se observa en el Parque República de Líbano, en donde se tiene registrado que entre 2017 y 2020 se perdieron de 500 a 600 árboles y de 80 a 90 árboles tan solo de marzo a agosto de 2024. [4]

Por otra parte, como se señala en [4], el problema para crear una solución eficiente es que no hay una estrategia colectiva para darle atención a la plaga, ya que el monitoreo de esta se hace por secciones y con un enfoque que no es preventivo, sino más bien reactivo.

Considerando el panorama expuesto, se hace evidente la necesidad de desarrollar un sistema que identifique automáticamente la presencia de plagas de insectos descortezadores en los árboles, con el propósito de identificarlas tempranamente para poder frenar en gran medida su proliferación y dar atención focalizada en áreas ya infestadas.

La propuesta que se presenta es un sistema móvil que, a partir de fotografías tomadas del arbolado de uno de los bosques de la Ciudad de México y analizadas en la nube mediante modelos de visión por computadora, determine si un árbol presenta o no indicios de infestación de insectos descortezadores y mantenga un historial del estado de salud de cada ejemplar.

El sistema *agrega un valor* significativo porque es *replicable*, ya que, a futuro, podría implementarse en otros bosques alrededor de la ciudad o el país, adaptándose a las especies de árboles del lugar y sus plagas más recurrentes. Por otra parte, también es escalable, ya que, debido a que utilizaremos servicios en la nube, el sistema nos permitirá comenzar en un sitio específico y, de manera progresiva, extenderse a otros sitios forestales.

Cabe mencionar que, también se considera como un sistema *adaptativo*, debido a que los modelos que serán utilizados para su elaboración pueden entrenarse y actualizarse con un data-set de nuevas imágenes, manteniendo tanto la precisión de la predicción, como la migración del enfoque del sistema hacia especies de árboles de otros sitios.

Finalmente, dado que el sistema se implementará a través de una aplicación móvil, este será *accesible* tanto para especialistas dedicados al área competente, como para el personal del bosque

encargado de la protección del área natural.

1.4 Estado del arte

A continuación, se presenta la Tabla 1, en la que se expone la clasificación de algunos softwares afines al sistema propuesto en el presente documento, señalando sus principales características, así como las tecnologías empleadas en su desarrollo.

Tabla 1: *Softwares y trabajos similares a nuestra propuesta*

Software / Trabajo	**Características**	**Tecnologías**
Progress in developing a bark beetle identification tool (Marais et al., 2025)	Clasificación de géneros de escarabajos descortezadores a partir de imágenes con múltiples individuos. Usa fotografía macro para captura reproducible. F1-score de 0.99–1.0 en datos de prueba.	Arquitectura MaxViT (Vision Transformer con atención local y global). Dataset disponible en HuggingFace. Transfer learning sobre backbone preentrenado.
Classification of Bark Beetle-Induced Forest Tree Mortality Using Deep Learning (Huo et al., 2022)	Clasifica cuatro etapas de ataque de descortezadores a nivel de árbol individual usando imágenes RGB de UAV. Aborda desbalance de clases con aumentación afín. Accuracy de 98.95%.	Arquitectura RetinaNet con backbone preentrenado para detección de copas. Imágenes RGB capturadas con hexacóptero en bosques del norte de México.
Lightweight YOLOv8 for Apple Leaf Disease Detection (Zhang et al., 2024)	Detecta enfermedades en hojas de manzano en entornos naturales. Optimizado para dispositivos móviles/embebidos. Modifica YOLOv8n reduciendo parámetros sin sacrificar precisión.	YOLOv8n + GhostConv + mecanismo de atención global GAM. MDPI Applied Sciences — DOI: 10.3390/app14156710.
Tree Disease Detection Using Image Classification (IEEE ICCCNT, 2023)	Compara tres arquitecturas CNN para detección de enfermedades en hojas de árbol. Accuracies entre 80–83%. Contexto de bosques reales.	InceptionV3, ResNet y MobileNetV2. DOI: 10.1109/ICC-CNT56998.2023.10307097.
Apple Leaf Disease Diagnosis & Mobile App with DeepLabV3+ (2023)	Segmentación semántica de lesiones en hojas de manzano para medir severidad. Desplegado en smartphone con 9s por imagen. MIoU de 83.85%.	DeepLabV3+ para segmentación. App móvil. MDPI Plants — DOI: 10.3390/plants12040786.

Para este proyecto se presenta el desarrollo de un sistema móvil de visión por computadora basado en un modelo de aprendizaje profundo, entrenado tanto en máquinas locales como en la nube, para la detección automática de plagas descortezadoras en los árboles de uno de los

bosques de la Ciudad de México. A diferencia de los métodos tradicionales de inspección visual, el sistema permitirá un monitoreo constante y digitalizado, generando un historial de árboles afectados, enfocado en la toma de decisiones en la gestión forestal. El modelo se va a entrenar a partir de imágenes recolectadas en campo y buscará optimizar la detección temprana de plagas, como apoyo a los especialistas en las primeras etapas del análisis del lugar y apoyando la toma de decisiones en el cuidado de los bosques urbanos de la Ciudad de México.

1.5 Metodología

Acorde con nuestro proyecto de trabajo terminal el cual implica el uso de diversas tecnologías, servicios en la nube y el desarrollo de módulos interdependientes, se ha decidido adoptar la metodología ágil Scrum. La naturaleza del proyecto, demanda un marco de trabajo flexible que permita adaptarse a las necesidades emergentes. Scrum, conocido por su ligereza y adaptabilidad, es ideal para equipos pequeños y proyectos que buscan entregar valor de forma incremental. Permite gestionar la complejidad y fomentar la colaboración, asegurando que las soluciones evolucionen con el proyecto [7].

El desarrollo de la metodología Scrum se descompone en los siguientes pasos:

1. Reunión inicial de Stakeholders para definir la visión del proyecto.
2. El Product Owner define el *Product Backlog* con los requerimientos en forma de casos de uso.
3. Cada sprint inicia con una reunión de planificación para seleccionar los casos de uso prioritarios.
4. Se crea el Sprint Backlog con las tareas a desarrollar durante el sprint.
5. Durante la ejecución se realizan Daily Standups para revisar avances y obstáculos.
6. Al finalizar cada sprint, se efectúa una Sprint Review para presentar avances al Product Owner.
7. El Product Owner valida y acepta los entregables según los criterios definidos.
8. Se realiza una Sprint Retrospective para identificar oportunidades de mejora.
9. El ciclo se repite desde el paso 3 hasta concluir el proyecto.

Para nuestro trabajo terminal, se plantean 5 sprints a lo largo de las dos fases del trabajo terminal:

1. Trabajo Terminal I
   (a) Primer sprint. Se asignarán las tareas iniciales para definir y ajustar el alcance del proyecto. El primer paso será un análisis comparativo de los bosques de la Ciudad de México con el objetivo de seleccionar el más adecuado para el estudio. A continuación, se realizará la recolección manual de fotografías y una clasificación preliminar para crear el dataset de entrenamiento. Simultáneamente, se establecerá el entorno de trabajo y se avanzará en el diseño de la arquitectura híbrida. El resultado de este sprint será un dataset de imágenes en bruto, el diseño preliminar de la arquitectura híbrida y la documentación básica del proyecto.

(b) Segundo sprint. Se llevará a cabo el análisis de los modelos de aprendizaje automático y la preparación de la arquitectura híbrida. Se organizará el dataset recolectado con una estructura más clara, dejándolo listo para su uso. También se considerarán aspectos clave como la portabilidad y el despliegue, y se planificarán las interfaces de usuario para la aplicación móvil.

2. Trabajo Terminal II

   (a) Tercer sprint. El enfoque principal será la limpieza y preparación final del dataset. Se procesarán las imágenes para normalizarlas y dividirlas en los subconjuntos necesarios. Con los datos listos, se iniciará el entrenamiento del modelo, se harán pruebas iniciales y se comenzará la optimización para mejorar la precisión y el rendimiento. De forma simultánea, se iniciará el desarrollo del backend del sistema.

   (b) Cuarto sprint. Este sprint se centrará en la integración y consumo de la aplicación móvil. Se ajustarán los módulos restantes del sistema para asegurar una comunicación fluida y un funcionamiento coherente entre todas las partes de la solución. Se realizará el despliegue completo y la validación final del sistema con nuevas imágenes de prueba. Finalmente, se consolidará y completará la documentación del proyecto, resumiendo la metodología, la arquitectura, los resultados y las conclusiones.

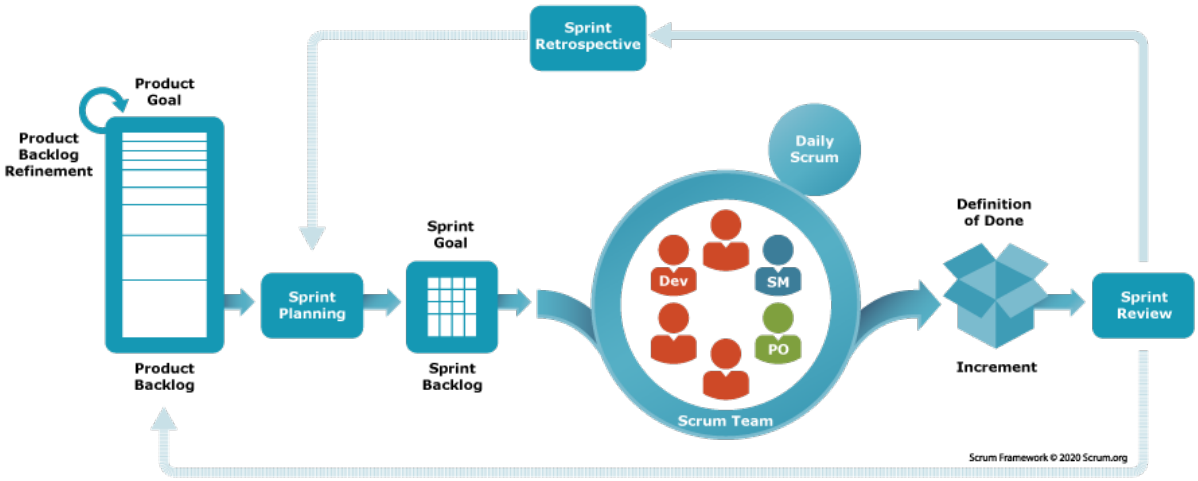


Figura 1: *Metodología SCRUM. Tomado de [8]*

Capítulo 2. Contexto y caracterización del caso de estudio

En este capítulo se presenta el contexto y la caracterización del caso de estudio, describiendo las condiciones del bosque seleccionado, las especies hospederas relevantes y su relación con la plaga de interés. Del mismo modo, se exponen los criterios técnicos que justifican su elección como escenario para la recolección de datos del proyecto.

2.1 Justificación de la selección del área de interés

La determinación del Bosque de Chapultepec como caso de estudio responde a la necesidad de encontrar un equilibrio entre la viabilidad logística, la relevancia biológica del problema y la pertinencia para la detección temprana. A diferencia de otras zonas forestales urbanas de la Ciudad de México —como el Bosque de San Juan de Aragón, con aproximadamente 2,500 árboles infestados y el 60% en tratamiento activo, o Milpa Alta, con más de 2,100 árboles muertos documentados—, Chapultepec representa un escenario donde la plaga está presente pero aún no ha alcanzado niveles críticos de colapso [5]. Esto lo convierte en el entorno idóneo para el desarrollo y validación de un sistema orientado a la **detección temprana**, que es precisamente donde la intervención tecnológica tiene mayor impacto. Los criterios considerados son los siguientes:

- **Incidencia de la plaga:** El sitio presenta niveles de infestación documentados y focalizados por especie. Se reportan incidencias del 2.4% en la población de fresnos (*Hylesinus sp.*) y del 4.08% en cedros (*Phloeosinus sp.*) [6]. Estos porcentajes no reflejan la proporción del arbolado total del bosque, sino la fracción afectada dentro de cada especie en particular. Para dimensionar su relevancia, cabe señalar que el promedio de afectación por descortezadores en el conjunto del arbolado urbano de la Ciudad de México alcanza el 30% [2]; en este contexto, Chapultepec representa un foco activo pero con margen real de intervención preventiva.
- **Diversidad de muestras para el dataset:** La densidad de hospederos en el área, combinada con la variabilidad en el grado de avance de la infestación —desde individuos sanos hasta árboles con síntomas visibles en distintas etapas—, permite capturar imágenes representativas de múltiples estadios del daño. Esta diversidad es fundamental para construir un conjunto de datos con capacidad de generalización durante el entrenamiento del modelo de clasificación [6].
- **Accesibilidad y logística:** Las secciones II y III del bosque permanecen abiertas las 24 horas, mientras que la primera sección opera de martes a domingo en horario extendido. Esta flexibilidad facilita visitas recurrentes para la recolección de datos bajo distintas condiciones de iluminación natural, aspecto crítico para la robustez del sistema de visión por computadora.
- **Impacto ecológico y social:** Los bosques urbanos de esta escala contribuyen directamente a la calidad de vida de la población al mejorar la calidad del aire, regular la temperatura local y ofrecer servicios ecosistémicos esenciales [1]. La pérdida acelerada del arbolado en este sitio tendría consecuencias directas sobre millones de habitantes y visitantes, lo que refuerza la pertinencia de desarrollar herramientas tecnológicas de apoyo a su gestión sanitaria.

2.2 Descripción General del Área de Estudio

El Bosque de Chapultepec es el área verde urbana de mayor extensión y relevancia en la Ciudad de México. Su importancia trasciende lo ambiental, constituyendo un referente del patrimonio histórico, cultural y recreativo del país [5].

- **Localización y extensión:** Se sitúa en las alcaldías Miguel Hidalgo (Secciones I, II y III) y

Álvaro Obregón (Sección IV), abarcando una superficie total aproximada de 866.37 hectáreas, lo que lo posiciona como uno de los pulmones verdes más grandes de América Latina.
- **Valor social:** El área alberga museos, centros deportivos, lagos y monumentos de relevancia nacional, recibiendo a miles de visitantes diariamente [5]. Este contexto subraya la urgencia de mantener la salud del arbolado, cuyo deterioro no solo implica pérdida ecológica sino también afectación directa al uso público del espacio.

2.3 Caracterización de los Huespedes

Dentro de los principales agentes bióticos que amenazan la salud de los bosques templados se encuentran los insectos descortezadores, un grupo de coleópteros de la familia *Curculionidae* cuya característica distintiva es su vida subcortical: todo su ciclo reproductivo y alimenticio transcurre por debajo de la corteza de los árboles que colonizan [9]. A pesar de su reducido tamaño (entre 0.1 y 0.6 cm) su capacidad de daño es considerable, ya que al establecerse en el interior del árbol comprometen directamente los tejidos responsables del transporte de nutrientes.

Aunque en el mundo se han descrito cerca de 3,000 especies de este grupo y en México están presentes 870 de ellas distribuidas en 87 géneros [9], no todas representan una amenaza equivalente para el arbolado. La gran mayoría mantiene poblaciones reducidas y cumple un papel regulador dentro del ecosistema forestal, eliminando individuos enfermos o debilitados y favoreciendo la renovación del dosel. Las especies con capacidad real de provocar la muerte del árbol por sí mismas son menos de 20 en el país, y pertenecen principalmente a los géneros *Dendroctonus* e *Ips* [9].

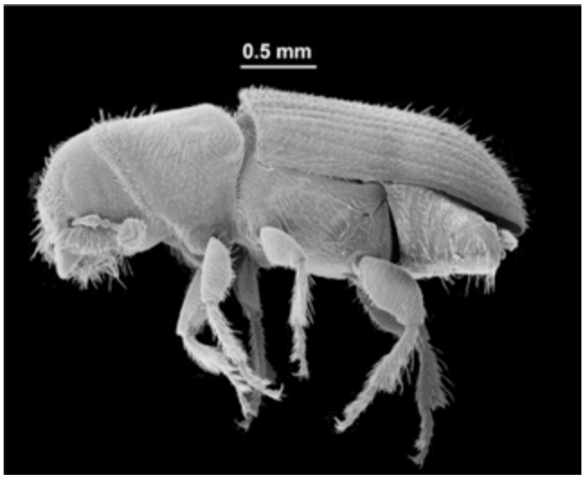


Figura 2: *Escarabajo descortezador del pino Dendroctonus frontalis. Tomado de [9]*

2.3.1 Mecanismo de colonización y daño

El proceso de colonización comienza con la perforación de la corteza por parte de la hembra adulta, que busca establecerse en el floema para alimentarse. La presencia de la hembra atrae al macho mediante señales químicas, y ambos inician la reproducción dentro del árbol. La hembra deposita sus huevos a lo largo de una galería central, cuidando que queden suficientemente separados para que las larvas no interfieran entre sí al alimentarse. Conforme cada larva se desarrolla, excava su propio recorrido subcortical, generando una red ramificada de galerías que se extiende desde el punto de entrada original. Este proceso de excavación y alimentación

progresiva interrumpe los tejidos conductores de nutrientes del árbol de forma creciente hasta comprometer su viabilidad [9].

Un aspecto relevante para el presente proyecto es que cada especie de descortezador genera un patrón de galerías propio en la madera del hospedero [9]. Si bien el sistema desarrollado no busca identificar la especie causante ni el árbol afectado de forma individual, la presencia de estos patrones subcorticales constituye en términos generales una evidencia visual directa de infestación activa. Es precisamente esta asociación entre la existencia de galerías y la clase ”con infestación” la que el modelo de clasificación busca aprender a reconocer a partir de las imágenes del dataset.

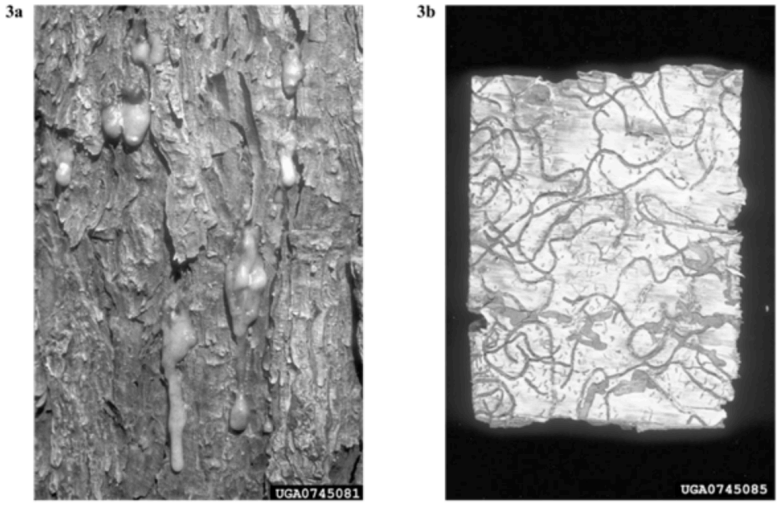


Figura 3: *a) Tronco de árbol con grumos de resina producidos como respuesta defensiva ante la perforación por descortezadores; b) Red de galerías formadas por D. frontalis en el tejido subcortical. Tomado de [9]*

2.3.2 El papel de la resina como barrera de defensa

Ante el ataque de los descortezadores, el árbol activa su principal mecanismo de defensa: la producción de resina, una sustancia viscosa almacenada bajo presión en conductos especializados del xilema. Cuando un insecto perfora la corteza, el árbol libera un flujo de resina que puede expulsar físicamente al atacante y sellar el orificio de entrada al endurecerse en contacto con el aire. Un árbol en condiciones fisiológicas óptimas puede sostener esta respuesta de forma efectiva. Sin embargo, cuando el árbol se encuentra debilitado por factores como la sequía, la edad o el estrés ambiental, la producción de resina disminuye o cesa, dejando al árbol sin su principal línea de defensa y permitiendo que los insectos accedan al floema sin resistencia, lo que a menudo conduce a su muerte [9].

2.4 Caracterización de los Hospederos

Para el desarrollo del sistema, es relevante identificar las especies que presentan mayor susceptibilidad ante los insectos descortezadores en el área de estudio, ya que estas constituyen el objetivo primario de detección y conforman una muestra significativa del dataset de entrenamiento. Si bien el sistema está diseñado para generalizar a otras especies arbóreas, los hospederos con mayor incidencia documentada representan los casos de referencia sobre los que se construye y valida el modelo.

- **Fresnos (*Fraxinus uhdei*):** Es la especie con mayor incidencia documentada de afecciones en el bosque. Su infestación por *Hylesinus sp.* genera patrones de daño visibles en la corteza que constituyen el principal objetivo de detección mediante visión por computadora [6].
- **Cedros Blancos (*Cupressus lusitanica*):** Presentan infestaciones vinculadas al estrés hídrico, que favorece la colonización por *Phloeosinus sp.* Los individuos afectados manifiestan alteraciones visuales características en la textura y coloración de la corteza, señales que el sistema busca aprender a identificar de forma automatizada [6].

2.4.1 Condiciones de vulnerabilidad y contexto urbano

La relación entre el debilitamiento fisiológico del arbolado y la susceptibilidad al ataque por descortezadores adquiere especial relevancia en entornos urbanos como el Bosque de Chapultepec. El estrés hídrico al que están sometidos los árboles en ecosistemas urbanos, agravado por las condiciones de impermeabilización del suelo, la contaminación y los efectos del cambio climático, reproduce el mismo patrón de vulnerabilidad documentado en bosques templados bajo condiciones de sequía [9].

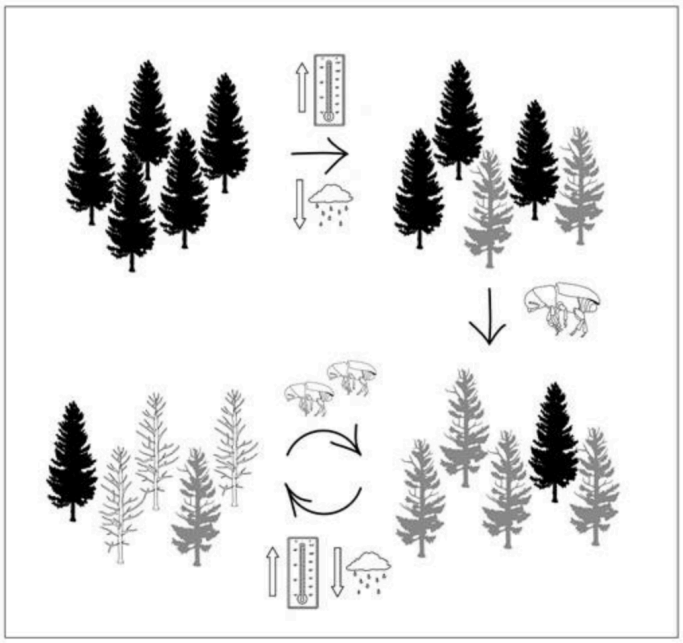


Figura 4: *Ciclo de retroalimentación negativa entre el cambio climático, el debilitamiento del arbolado y el incremento en la actividad de insectos descortezadores. Tomado de [9]*

Este ciclo explica por qué las infestaciones por *Hylesinus* sp. en *Fraxinus uhdei* y por *Phloeosinus* sp. en *Cupressus lusitanica* documentadas en el área de estudio no son fenómenos aislados, sino manifestaciones locales de una dinámica forestal más amplia [9]. Su detección oportuna tiene además una dimensión preventiva, ya que al identificar tempranamente la presencia de estos **descortezadores secundarios**, se posibilita la intervención antes de que el deterioro del arbolado genere condiciones propicias para la colonización por géneros de alto impacto como *Dendroctonus e Ips*, cuya presencia implicaría un escenario de difícil o imposible reversión. En este sentido, el sistema no se limita a detectar infestaciones activas, sino que actúa como barrera preventiva frente a plagas de mayor severidad.

Comprender la biología de estos insectos y el proceso mediante el cual dañan al árbol es, por tanto, el punto de partida necesario para definir las señales visuales de interés que el sistema de

clasificación busca aprender a identificar, descritas en detalle en las secciones siguientes.

2.5 Progresión visual de la infestación por insectos descortezadores

La detección visual de infestaciones por descortezadores puede abordarse desde dos niveles de observación complementarios, cada uno con señales cromáticas y morfológicas propias: el follaje, observable desde captura aérea, y la corteza, analizable a escala de árbol individual. Esta distinción es fundamental para el presente proyecto, ya que ambos niveles corresponden directamente a las imágenes que conforman el *dataset* de entrenamiento descritos en la Sección 2.4.

2.5.1 Señales visuales en el follaje

Las alteraciones en el follaje constituyen la señal observable a escala macro, y son las que se capturan mediante dron en el primer nivel de análisis del sistema. De acuerdo con Gómez-Pineda y Ramírez [10], la progresión documentada en campo sigue la siguiente secuencia cromática.

- **Fase 1 — Colonización reciente.** La coloración del follaje aún es **verde**, sin alteraciones cromáticas visibles desde el dosel. Sin embargo, a nivel de tronco ya se observan grumos frescos de resina blanca o rojiza sobre la corteza o ramas, así como desechos tipo aserrín en la base del árbol, señales de que la colonización subcortical ya ha comenzado.
- **Fase 2 — Deterioro activo.** Conforme los descortezadores se reproducen y alimentan dentro del árbol, el follaje transita de verde a **verde-amarillento o verde limón**. En esta fase la infestación ya está activa y consolidada.
- **Fase 3 — Muerte del árbol.** El follaje adquiere una coloración **roja o café-rojiza** generalizada. A lo largo del tronco se hace visible una gran cantidad de orificios de salida por donde emergieron las nuevas generaciones de insectos. El árbol ya está muerto y la corteza comienza a desprenderse con facilidad [10].

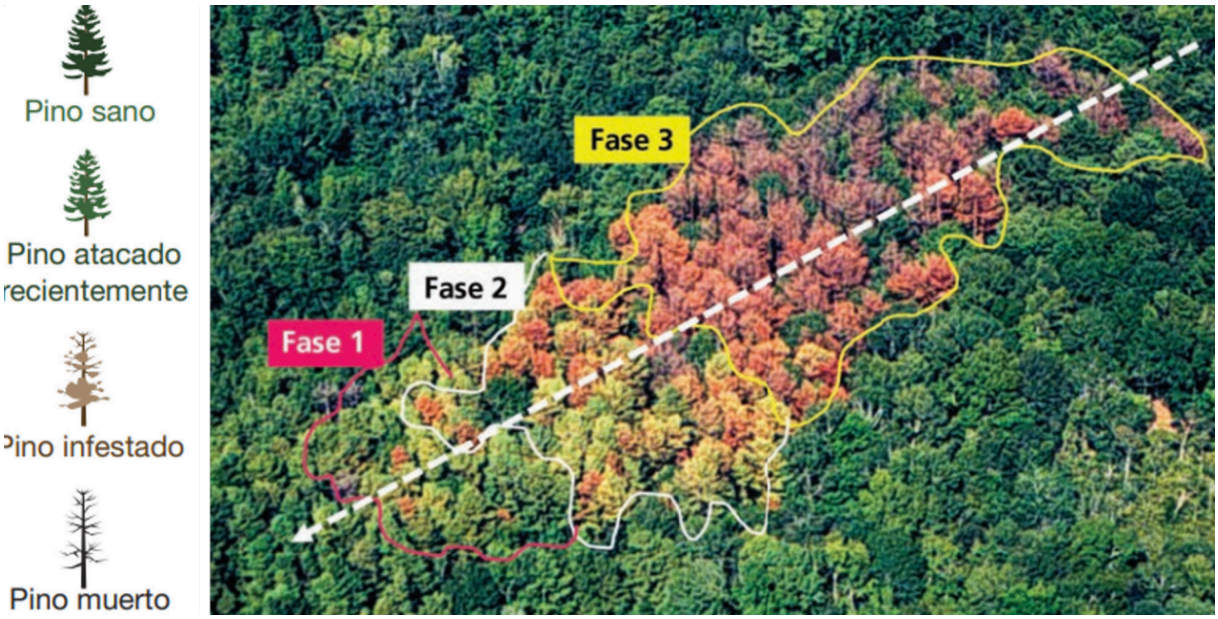


Figura 5: *Etapas de coloración del follaje que indican la fase de infestación del arbolado. Se observa la transición de verde (Fase 1) a verde-amarillento (Fase 2) y café-rojizo (Fase 3), con la dirección de expansión del brote indicada. Tomado de [10]*

Un aspecto metodológicamente relevante de esta progresión es que durante la Fase 1 el follaje aún no presenta alteraciones cromáticas detectables desde el dosel, lo que limita la detección aérea temprana. Esto refuerza la necesidad del segundo nivel de análisis como mecanismo complementario de detección en estadios iniciales de infestación.

2.5.2 Señales visuales en la corteza

A escala de árbol individual, la infestación produce señales visibles tanto en la superficie exterior de la corteza como, de forma más directa, en el tejido subcortical expuesto mediante remoción controlada [10].

En la superficie exterior de la corteza

El proceso de colonización inicia cuando las hembras adultas perforan la corteza y se establecen en el floema. Como respuesta defensiva, el árbol produce resina que expulsa a los atacantes; sin embargo, cuando la infestación supera la capacidad de defensa del árbol, esta respuesta resulta insuficiente [10]. Las señales visibles en la superficie exterior siguen la siguiente progresión:

- **Fase 1 — Respuesta defensiva activa.** Aparecen grumos frescos de resina en tonos **blancos, crema o rojizos** sobre el tronco y las ramas, junto con desechos tipo aserrín en la corteza y en la base del árbol. Estos son los primeros indicadores visibles de colonización activa, observables incluso cuando el follaje aún se mantiene verde.
- **Fase 2 — Colonización avanzada.** Los exudados resinosos aumentan en volumen y comienzan a acumularse en tonos **café-anaranjados**. Simultáneamente aparecen pequeños orificios de entrada y salida de los insectos a lo largo del tronco, señal de que la población de descortezadores ya es numerosa.
- **Fase 3 — Árbol muerto y abandonado.** La corteza presenta una coloración **café-grisácea uniforme**, sin exudados resinosos activos. Los orificios de emergencia son abundantes y evidentes. El árbol ha sido abandonado por los descortezadores primarios y comienza a ser colonizado por especies secundarias.

En el tejido subcortical expuesto

Mediante remoción controlada de la corteza externa, se exponen directamente las galerías excavadas por los insectos para alimentarse y ovipositar.

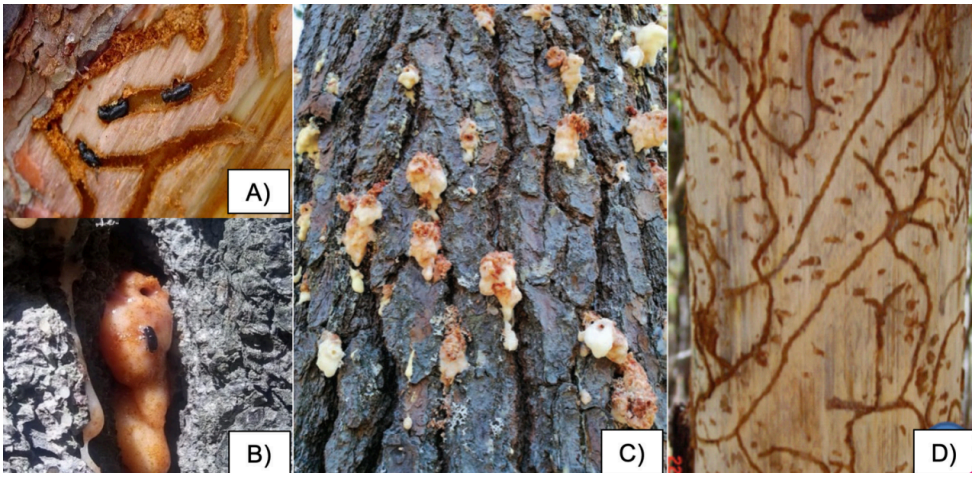


Figura 6: *Señales visuales en la superficie exterior e interior de la corteza. A) hembras alimentándose en el tejido subcortical; B) macho entrando a través del orificio hecho por la hembra; C) grumos de resina acumulados a lo largo del tronco; D) galerías individuales generadas por las larvas durante la alimentación y oviposición. Tomado de [10]*

La hembra excava una galería central mientras deposita huevos a cada lado; al eclosionar, las larvas generan galerías individuales ramificadas que se extienden interrumpiendo los tejidos conductores de nutrientes [10].

Estas estructuras presentan patrones cromáticos característicos: tonos **café-rojizos y anaranjados** en el tejido dañado, con trayectorias de excavación sobre un fondo de madera en tonos **beige y crema**. La presencia de estos patrones constituye la señal más directa y confiable de infestación activa, y representa la clase de mayor interés para el entrenamiento del modelo de clasificación.

2.5.3 Implicaciones para el análisis de color del *dataset*

La diferenciación entre señales de follaje y señales de corteza tiene consecuencias metodológicas directas para el análisis cromático complementario propuesto en el Capítulo 6. Dado que cada tipo de imagen opera en un espacio visual distinto, el análisis de distribución cromática se plantea de forma **separada para cada nivel**, evitando generalizaciones que mezclen señales de naturaleza diferente.

Además, esta progresión muestra que las señales visibles en la corteza aparecen antes que los cambios en el follaje. Es decir, en una etapa temprana, la infestación ya puede observarse en la corteza mientras el árbol todavía conserva un aspecto verde en su copa.Esto es importante porque sugiere que las imágenes de corteza pueden ser más útiles para la detección temprana; sin embargo, también conviene considerar las imágenes del follaje, ya que constituyen un factor importante para complementar el análisis. Por tanto, esto representa un argumento adicional para incorporar ambos niveles de forma complementaria dentro del sistema.

Finalmente, aunque las referencias bibliográficas de base corresponden principalmente a coníferas del centro de México, las señales descritas son respuestas fisiológicas generales al daño en el floema, documentadas en múltiples géneros de árboles, por lo tanto, el presente proyecto las utiliza como **marco conceptual de referencia**.

Capítulo 3. Delimitación computacional del problema

En este capítulo se delimita el problema desde una perspectiva computacional, considerando la naturaleza visual de los datos de entrada, la visión por computadora como área de estudio y la justificación de su aplicación en el proyecto.

3.1 Naturaleza de la información de entrada

La información de entrada del sistema está conformada por **imágenes de árboles recolectadas directamente en campo** en *el bosque de Chapultepec en la Ciudad de México*, seleccionado como caso de estudio.

Estas imágenes constituyen el contenido del dataset, el cual representa la principal fuente de datos para **analizar visualmente** aspectos relacionados con la presencia de afecciones en el estado de salud de los árboles, provocadas por insectos descortezadores.

En este sentido, el sistema debe ser capaz de **identificar ciertos patrones** presentes en la apariencia externa de los árboles a partir de las imágenes, extrayendo características relevantes como variaciones en la textura de la corteza, alteraciones en la coloración, irregularidades superficiales o signos visibles de daño que puedan asociarse con la presencia de la plaga.

Cabe destacar que la elección de imágenes como recurso principal responde a la naturaleza misma del problema, ya que los indicios de afectación suelen manifestarse visualmente en el árbol antes de poder cuantificarse mediante otro tipo de mediciones. Por ello, el análisis computacional debe centrarse en interpretar **información visual compleja**, no estructurada y susceptible a variaciones propias del entorno de captura, tales como la iluminación, la distancia, el ángulo, el fondo y la calidad de la imagen.

3.2 La visión por computadora como área de estudio

3.2.1 Definición de visión por computadora

La visión por computadora es un campo de la inteligencia artificial que hace posible que las máquinas tengan la capacidad de replicar tareas cognitivas humanas fundamentales tales como procesar, describir, analizar e interpretar con el fin de extraer información desde entradas visuales, es decir, imágenes y videos. Para llevar a cabo estas tareas, hace uso del **aprendizaje automático o del aprendizaje profundo**, lo que a su vez permite realizar tareas más complejas y específicas propias del campo, basadas en las fundamentales [11].

3.2.2 Tareas de visión por computadora

Según IBM [12], la visión por computadora incluye las siguientes tareas:

- Reconocimiento de imágenes
- Clasificación de imágenes
- Detección de objetos
- Segmentación de imágenes
- Seguimiento de objetos

- Comprensión de la escena
- Reconocimiento facial
- Estimación de pose
- Reconocimiento óptico de caracteres
- Generación de imágenes
- Inspección visual

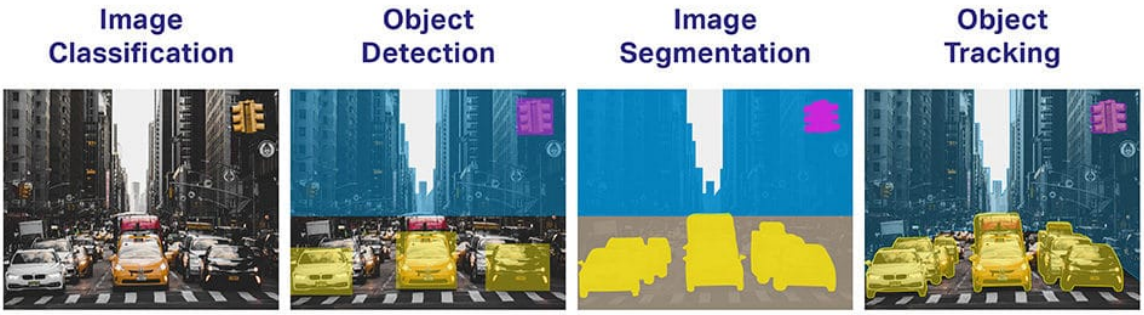


Figura 7: *Algunas tareas de visión por computadora*

Estas tareas representan, sin duda, la gran capacidad de la visión por computadora para poder resolver un amplio catálogo de problemas a partir del análisis de entradas visuales. No obstante, es importante tomar en cuenta el objetivo principal del problema de interés para delimitar el tipo o los tipos de tarea que se requieren abordar y, de esta manera, conseguir el tipo de información que se desea obtener.

3.2.3 Herramientas y marcos de trabajo utilizadas en visión por computadora

IBM [12] señala que, para desarrollar ágilmente un sistema utilizando visión por computadora, es necesario el uso de diferentes herramientas y marcos de trabajo, entre los cuales se encuentran:

- Keras
- OpenCV
- Scikit-image
- TensorFlow
- Torchvision

En cuanto al tratamiento de información, destacan las *bibliotecas* **OpenCv y Scikit-image**, herramientas especializadas en el procesamiento digital de imágenes, detección de objetos, segmentación de imágenes y extracción de características, tareas que son realizadas mediante una gran colección de algoritmos de visión artificial [12].

En cambio, en lo que refiere al diseño, entrenamiento, despliegue y gestión de modelos de visión por computadora, **Keras** destaca siendo una *interfaz de programación* orientada al aprendizaje profundo, la cual puede ejecutarse sobre los marcos de trabajo **TensorFlow** (que también constituye una de las plataformas de aprendizaje automático más extendidas) y **Pytorch**. Dentro de éste último se encuentran herramientas como **Torchvision**, que ofrece transformaciones de imágenes, conjuntos de datos y modelos preentrenados para clasificación, detección y segmentación [12].

3.2.4 Flujo de trabajo común en el desarrollo de sistemas mediante visión por computadora

Una vez definido y delimitado el problema a resolver, se requiere transformar la información visual en resultados útiles para la tarea o tareas específicas en que se plantea dicho problema, esto es, seguir un flujo de trabajo desde la adquisición de datos hasta la generación de una salida interpretable.

De acuerdo con IBM [12], es posible identificar una serie de etapas comunes que conforman el flujo de trabajo de un sistema de visión por computadora:

1. Recopilación de datos
2. Preprocesamiento
3. Selección de modelos
4. Entrenamiento del modelo

Como se puede observar, este flujo suele comenzar con la **recopilación de imágenes** mediante cámaras o sensores, etapa en la cual se obtienen todos los datos visuales que servirán como entrada del sistema [12].

Después, en la etapa del **preprocesamiento**, estos datos visuales se someten a un tratamiento de limpieza en donde se mejora su calidad, por ejemplo, mediante el ajuste del brillo, el contraste, el tamaño, entre otras propiedades que ayuden a resaltar características relevantes para el análisis de los datos [12].

Una vez preparadas las imágenes, se realiza una comparativa entre los modelos candidatos a ser implementados en el sistema, tomando en cuenta que el **modelo que va a ser seleccionado** deberá ser el más conveniente con base en los objetivos del problema a resolver e ídoneo para cumplir con los objetivos de calidad. IBM señala que "seleccionar el modelo de machine learning adecuado es crucial para optimizar la eficiencia y el rendimiento" [12].

Por consiguiente, el **modelo seleccionado es entrenado** con los datos recopilados al inicio. En esta última etapa, se evalúan los objetivos de calidad (como la eficiencia y el rendimiento) de forma iterativa con el fin de encontrar los parámetros óptimos para satisfacer la tarea de visión artificial a la que está sujeto [12].

Finalmente, el sistema genera la salida deseada conforme a la tarea para la cual el modelo fue entrenado.

3.3 Justificación del uso de visión por computadora en el proyecto

A partir del análisis de la naturaleza de los datos de entrada, así como de la información general relacionada con la visión por computadora, es necesario que el desarrollo del presente proyecto se lleve a cabo dentro de este campo de la inteligencia artificial, ya que será de utilidad para poder analizar el contenido visual (imágenes) y extraer características importantes mediante un modelo de aprendizaje automático o profundo.

En secciones anteriores se mencionó que, antes de llevar a cabo el flujo de trabajo común para el desarrollo de un sistema mediante visión por computadora, es necesario definir y delimitar el

problema. Dentro de este proceso, se encuentra principalmente la *selección del tipo de tarea al que estará sujeto el proyecto*. Bajo esta consideración, el problema abordado puede relacionarse principalmente con tres tareas, las cuales son: **clasificación de imágenes, detección de objetos y segmentación de imágenes**.

En primer lugar, la **clasificación de imágenes** consiste en asignar una categoría predefinida a una imagen u objetos dentro de la misma [12]. En este sentido, esta tarea permitiría determinar si un árbol presenta o no indicios de infestación por insectos descortezadores.

En segundo lugar, la **detección de objetos** busca localizar y delimitar la ubicación de objetos dentro de una imagen, así como identificar la categoría a la que pertenecen, haciendo uso de dos técnicas: *la localización de objetos y la clasificación de imágenes* [12]. A través de esta tarea, se podrían identificar y localizar zonas específicas del tronco o de la corteza de los árboles en las que se visualicen indicios de infestación.

Por último, la **segmentación de imágenes** puede definirse como una versión más detallada de la detección de objetos. Si bien también se centra en localizar y delimitar la ubicación de los objetos, así como en identificar la categoría a la que pertenecen, su principal diferencia radica en que esta tarea delimita a nivel de pixel, lo que implica una localización puntual de un objeto, llegando incluso a aproximar su ancho y alto [12]. Desde esta perspectiva, se podría identificar detalladamente la superficie infestada, así como aproximar su extensión.

Conforme al panorama expuesto, las tres tareas tienen relación con el problema general; pese a ello, no todas resultan ser factibles ni viables dentro del alcance del proyecto.

En el caso de la detección de objetos, las imágenes deben estar etiquetadas, proceso que se realiza por humanos a través de una interfaz de anotación, en donde se definen cajas delimitadoras para los objetos con el fin de entrenar a un modelo capaz de reconocer y clasificar zonas de acuerdo con patrones visuales [13].

Por otra parte, en el caso de la segmentación de imágenes, Bergmann afirma que “los conjuntos de datos de entrenamiento deben ser etiquetados previamente por humanos expertos, lo cual es una tarea extremadamente laboriosa” [14]. Incluso, si consideramos que la segmentación es una tarea más compleja que la detección, entonces se puede concluir que el proceso de etiquetado es mucho más preciso que en la detección de objetos, puesto que ya no solo se trabaja con objetos en particular, sino a nivel de pixeles.

Desde luego, lo mencionado anteriormente se convierte en una limitación; tanto la detección de objetos como la segmentación requieren conjuntos de datos con anotaciones más detalladas, así como mayores requerimientos computacionales para entrenamiento. Debido a ello, se tomó la decisión de *plantear el sistema* principalmente como una **tarea de clasificación de imágenes**, de tal suerte que el modelo que se elija sea capaz de determinar si un árbol presenta o no indicios de infestación por plaga de insectos descortezadores.

Capítulo 4. Fundamentación del enfoque de aprendizaje profundo

En este capítulo se analiza la transición del aprendizaje automático tradicional hacia el aprendizaje profundo, con el propósito de fundamentar la elección del enfoque de modelado más adecuado para el problema planteado. Para ello, se revisan los modelos básicos de clasificación, la necesidad de extracción manual de características, las limitaciones de los enfoques tradicionales y las ventajas del aprendizaje profundo en tareas visuales.

4.1 Modelos básicos de aprendizaje automático en clasificación

4.1.1 Aprendizaje automático supervisado y clasificación

El aprendizaje automático constituye una rama de la inteligencia artificial orientada al desarrollo de modelos capaces de identificar patrones en los datos y realizar predicciones o decisiones sin haber sido programados de manera explícita para cada caso [15].

Dentro de este campo existen diversas técnicas que permiten abordar problemas de distinta naturaleza. En términos generales, los **métodos de machine learning** pueden clasificarse en tres paradigmas principales: *aprendizaje supervisado, aprendizaje no supervisado y aprendizaje por refuerzo*. Esta división depende del objetivo con el que se entrena el modelo y, en buena medida, del tipo de datos que intervienen durante dicho proceso [15].

Dentro de estos tres paradigmas mencionados, el que resulta de mayor interés para el presente proyecto es el **aprendizaje supervisado**, ya que el problema planteado parte de imágenes previamente etiquetadas y busca asignarles una categoría específica. Al respecto, Belcic y Stryker señalan que “el aprendizaje supervisado es una técnica de machine learning que utiliza conjuntos de datos etiquetados para entrenar modelos de inteligencia artificial (IA) para identificar los patrones y relaciones subyacentes” [16], de modo que el modelo aprende una relación entre un conjunto de entradas y sus salidas correspondientes, con el propósito de generalizar dicho conocimiento a nuevas muestras no vistas durante el entrenamiento.

Dentro del aprendizaje supervisado las tareas se pueden dividir en **dos tipos principales de problemas**: *regresión y clasificación*. La regresión se emplea cuando la salida esperada corresponde a la predicción de un valor continuo, con base en el análisis de las relación entre variables dependientes e independientes, mientras que la clasificación se utiliza cuando el objetivo consiste en asignar una nueva muestra a una categoría previamente definida o también llamada clase [16]. En este sentido, el problema abordado en el presente trabajo corresponde a una tarea de clasificación (tal como se definió en el capítulo anterior), con enfoque supervisado, ya que se busca determinar si una imagen de un árbol presenta o no indicios visuales de infestación por plaga.

4.1.2 Principales algoritmos de clasificación

Una vez establecido que el problema abordado en este trabajo corresponde a una tarea de clasificación, resulta pertinente revisar los tipos de clasificación y algunos de los algoritmos que tradicionalmente se han utilizado para resolver este tipo de problemas dentro del aprendizaje automático supervisado.

Este tipo de tarea puede presentarse de distintas maneras. Por un lado, se encuentra la **clasificación binaria**, en la que el modelo debe decidir entre dos clases posibles; por otro, la **clasificación**

multiclase, donde la tarea consiste en elegir una categoría entre varias opciones. En problemas más específicos también puede hablarse de clasificación multietiqueta, aunque esta variante no corresponde al alcance del presente trabajo [17].

Dentro de los algoritmos de clasificación más conocidos se encuentra la **regresión logística**, la cual se emplea con frecuencia en problemas de clasificación binaria debido a su capacidad para estimar la probabilidad de pertenencia a una clase. También destacan los **árboles de decisión**, que estructuran la toma de decisiones mediante reglas jerárquicas construidas a partir de las características de entrada. A partir de estos, surgen métodos como el **bosque aleatorio**, que combina múltiples árboles para mejorar la capacidad de generalización del modelo y reducir la dependencia de un solo esquema de decisión [17].

Otro algoritmo ampliamente utilizado es la **máquina de vectores de soporte** (Support Vector Machine, SVM), cuyo propósito es encontrar una frontera de separación adecuada entre clases dentro del espacio de características. De manera similar, el método de **k-vecinos más cercanos**(k-Nearest Neighbors, k-NN) clasifica una observación de acuerdo con la cercanía que guarda respecto a muestras previamente etiquetadas. Asimismo, el clasificador de **Bayes ingenuo**(Naive Bayes) constituye un enfoque probabilístico que asigna clases con base en la probabilidad de ocurrencia de ciertas características dentro de cada categoría [17].

Figura 8: *Principales algoritmos de clasificación. Tomado de [18]*

4.2 Limitaciones de los enfoques tradicionales en escenarios reales

Una vez revisados los principales algoritmos de clasificación tradicionales, es importante analizar las limitaciones que presentan cuando se aplican a problemas de visión por computadora en condiciones reales.

Aunque los algoritmos tradicionales de clasificación han mostrado utilidad en distintos contextos, su aplicación en tareas de visión por computadora presenta limitaciones importantes. Una de las principales radica en que estos métodos suelen requerir una etapa previa de preprocesamiento de la imagen, la cual puede incluir operaciones como segmentación, mejora de imagen y refinamiento, para después realizar una extracción manual de características con la que se construye el modelo de reconocimiento [19].

Bajo esta lógica, el desempeño del sistema no depende únicamente del clasificador empleado, sino también de la calidad de la representación visual que se construye antes de la etapa de clasificación. En otras palabras, si las características extraídas no logran describir de manera adecuada la información contenida en la imagen, la capacidad del modelo para distinguir entre clases se ve limitada [19].

Además, este tipo de extracción manual presenta restricciones para representar estructuras de nivel medio y alto dentro de la imagen, así como información semántica más compleja. A ello se suma que los algoritmos tradicionales pueden resultar costosos en tiempo de procesamiento y difíciles de aplicar en contextos reales o de producción, particularmente cuando las condiciones de trabajo no son controladas [19].

En el caso del presente proyecto, estas limitaciones adquieren mayor relevancia debido a que las imágenes son recolectadas directamente en campo, lo que implica variaciones naturales en iluminación, ángulo de captura, fondo y apariencia de los árboles. En este tipo de escenarios, un enfoque que dependa de manera estricta del preprocesamiento y de la extracción manual de características puede resultar insuficiente para representar la complejidad visual del problema.

A partir de ello, se vuelve necesario considerar métodos con mayor capacidad para aprender representaciones directamente a partir de los datos.

4.3 Aprendizaje profundo en tareas de visión por computadora

Las limitaciones que presentan los enfoques tradicionales en problemas de clasificación de imágenes hacen evidente la necesidad de recurrir a métodos con mayor capacidad de representación y aprendizaje. En este sentido, el aprendizaje profundo ha adquirido un papel central en la visión por computadora, al ofrecer modelos capaces de extraer y jerarquizar características directamente a partir de los datos visuales. Por ello, en el presente apartado se introduce el aprendizaje profundo como paradigma de especial relevancia para el análisis de imágenes y para la solución del problema planteado en este trabajo.

4.3.1 Qué es el aprendizaje profundo

El aprendizaje profundo es una rama del aprendizaje automático basada en redes neuronales artificiales con múltiples capas, diseñadas para aprender patrones complejos a partir de grandes volúmenes de datos. A diferencia de los enfoques tradicionales, en los que un especialista debía definir manualmente las características visuales relevantes, el aprendizaje profundo permite que el propio modelo descubra automáticamente los rasgos más útiles para resolver una tarea. En visión por computadora, este enfoque ha resultado especialmente valioso porque facilita el aprendizaje de representaciones jerárquicas desde la imagen de entrada hasta la decisión final del sistema [20].

Además, el aprendizaje profundo se apoya en arquitecturas como las redes neuronales convolucionales, que procesan la información visual mediante filtros capaces de detectar bordes, texturas, formas y patrones cada vez más complejos. Este esquema de aprendizaje de extremo a extremo ha modificado la forma en que se diseñan los sistemas de visión por computadora, ya que el énfasis ya no está en programar reglas explícitas, sino en entrenar modelos con datos anotados para que aprendan directamente de ellos [20].

Desde otra perspectiva, el avance del aprendizaje profundo en visión por computadora también se explica por su integración con estrategias que buscan aprovechar mejor los datos disponi-

bles. En contextos donde etiquetar imágenes es costoso o lento, se han desarrollado variantes como el aprendizaje activo profundo, cuyo propósito es seleccionar las muestras más valiosas para anotarlas y así mejorar el entrenamiento con menos esfuerzo humano. Esto muestra que el aprendizaje profundo no solo depende de arquitecturas potentes, sino también de métodos eficientes para gestionar los datos de entrenamiento [21].

4.3.2 Ventajas del aprendizaje profundo en tareas visuales

Una de las principales ventajas del aprendizaje profundo en tareas visuales es su **alta capacidad para alcanzar niveles superiores de precisión** en comparación con los métodos tradicionales. Según la literatura revisada, este enfoque ha impulsado mejoras importantes en tareas como clasificación de imágenes, segmentación semántica, detección de objetos e incluso localización y mapeo simultáneo. Esto se debe a que las redes profundas aprenden automáticamente las características más discriminativas, reduciendo la dependencia de la ingeniería manual de atributos [20].

Otra ventaja relevante es su **flexibilidad**. Los modelos basados en aprendizaje profundo pueden volver a entrenarse con conjuntos de datos específicos para distintos problemas, lo que permite adaptarlos a nuevos contextos con relativa facilidad. En contraste, muchos métodos clásicos de visión por computadora tienden a ser más rígidos y dependientes del dominio de aplicación. De esta manera, el aprendizaje profundo ofrece una mayor capacidad de generalización y adaptación cuando se dispone de datos adecuados para el entrenamiento [20].

También destaca su **utilidad en escenarios complejos** donde existen *grandes volúmenes de información visual*. En la revisión sobre aprendizaje activo profundo se señala que el aprendizaje profundo ha permitido avances notables en visión por computadora gracias al uso de conjuntos de datos anotados a gran escala, que proporcionan supervisión directa durante el entrenamiento. Esto ha favorecido el desarrollo de sistemas más robustos para problemas difíciles, aunque también ha puesto en evidencia la necesidad de optimizar el proceso de etiquetado debido a su alto costo en muchas aplicaciones [21].

4.3.3 Aplicación del aprendizaje profundo en visión por computadora

El aprendizaje profundo se aplica ampliamente en la visión por computadora para resolver tareas de reconocimiento, análisis e interpretación de imágenes. Entre sus aplicaciones más importantes se encuentran *la clasificación de imágenes, la detección de objetos, y la segmentación semántica*. Estas tareas se han beneficiado del uso de redes convolucionales, las cuales han demostrado un rendimiento sobresaliente frente a enfoques tradicionales [20].

A nivel más amplio, la revisión sobre aprendizaje activo profundo destaca que la visión por computadora ha experimentado un avance significativo con la incorporación del aprendizaje profundo en aplicaciones que requieren grandes cantidades de datos etiquetados. Entre los campos mencionados se encuentran la conducción autónoma, el análisis de imágenes médicas, la detección de anomalías y otras tareas donde la precisión del modelo resulta crítica. En estos escenarios, el aprendizaje profundo se ha convertido en una herramienta central para extraer información útil a partir de imágenes, videos, nubes de puntos y otros datos visuales [21].

Asimismo, su aplicación no se limita a tareas bidimensionales convencionales. También se ha extendido a contextos emergentes como la *visión 3D, la percepción robótica y los sistemas híbridos* que combinan técnicas tradicionales con redes profundas. Esto confirma que el aprendizaje profundo no solo ha transformado la visión por computadora actual, sino que también sigue

ampliando su alcance hacia nuevos problemas y modalidades de datos [20].

4.3.4 Importancia de los datos anotados

Los datos anotados son fundamentales en el aprendizaje profundo porque proporcionan la supervisión necesaria para que el modelo aprenda a relacionar una entrada visual con una salida correcta. En visión por computadora, las imágenes etiquetadas permiten entrenar sistemas capaces de clasificar, detectar o segmentar con mayor precisión. De acuerdo con las fuentes revisadas, el crecimiento del aprendizaje profundo ha estado fuertemente vinculado a la disponibilidad de grandes conjuntos de datos anotados, ya que estos hacen posible entrenar modelos complejos con un mejor desempeño [20], [21].

Sin embargo, aunque los datos anotados son indispensables, su obtención representa uno de los principales desafíos del área. En muchas aplicaciones, el etiquetado manual requiere tiempo, conocimiento especializado y altos costos. La revisión sobre aprendizaje activo profundo subraya que en tareas como la segmentación urbana o el análisis de imágenes médicas, la anotación puede ser extremadamente demandante, incluso hasta superar los 90 minutos por imagen en ciertos casos. Esto convierte a los datos anotados en un recurso valioso y limitado, cuya calidad influye directamente en el rendimiento del modelo [21].

Por esa razón, no solo importa disponer de datos anotados, sino contar con datos bien seleccionados y de alta calidad. En este sentido, enfoques como el aprendizaje activo profundo buscan reducir la carga de anotación al priorizar las muestras más informativas para el entrenamiento. Así, la importancia de los datos anotados no radica únicamente en su cantidad, sino en su relevancia, precisión y utilidad para mejorar la capacidad de generalización de los modelos de visión por computadora [21].

4.4 Comparación entre aprendizaje automático tradicional y aprendizaje profundo

Una vez expuestos los modelos tradicionales de clasificación, sus limitaciones en escenarios reales y el panorama general del aprendizaje profundo en tareas de visión por computadora, resulta adecuado establecer una comparación directa entre ambos enfoques. Esta comparación tiene como propósito precisar, de forma puntual, las diferencias en la manera en que el aprendizaje automático tradicional y el aprendizaje profundo representan la información, aprenden características y responden ante problemas visuales complejos. De esta manera, se busca reforzar con mayor claridad la justificación del aprendizaje profundo como el enfoque más adecuado para el problema planteado en el presente proyecto.

4.4.1 Similitudes generales entre ambos enfoques

Tanto el aprendizaje automático tradicional como el aprendizaje profundo se apoyan en conjuntos de datos para entrenar algoritmos capaces de encontrar relaciones entre entradas y salidas. En ambos casos, el modelo aprende a partir de ejemplos de entrenamiento con el fin de generalizar ese conocimiento hacia nuevos datos no vistos. Asimismo, ambos enfoques forman parte de la inteligencia artificial y la ciencia de datos, y pueden emplearse para resolver tareas que resultarían costosas o poco prácticas mediante métodos tradicionales de programación [22].

Figura 9: *Subconjuntos de la Inteligencia artificial. Tomado de [23]*

De igual manera, tanto el **machine learning** como el **aprendizaje profundo** se apoyan en fundamentos matemáticos y estadísticos para construir sus modelos. Esto incluye el uso de técnicas relacionadas con el análisis de regresión, el álgebra lineal, el cálculo y otros métodos numéricos que permiten optimizar el proceso de aprendizaje. En ambos casos, la calidad y cantidad de los datos disponibles influye directamente en el desempeño final del sistema [22].

4.4.2 Diferencias en el enfoque de resolución de problemas

A pesar de compartir una base común, la principal diferencia entre ambos enfoques radica en la forma en que abordan la representación de la información. En el aprendizaje automático tradicional, es habitual que exista una etapa de ingeniería de características, en la cual una persona selecciona, diseña o extrae manualmente los atributos que serán utilizados por el modelo. Esto implica que parte del conocimiento necesario para resolver el problema debe ser definido explícitamente antes del entrenamiento [22].

En cambio, el aprendizaje profundo reduce esta dependencia al permitir que el propio modelo aprenda automáticamente las representaciones más relevantes a partir de los datos. Esto se logra mediante arquitecturas basadas en redes neuronales profundas, las cuales están compuestas por múltiples capas capaces de transformar progresivamente la información de entrada en representaciones de mayor nivel de abstracción. De esta manera, la extracción de características y la clasificación dejan de ser procesos separados y pasan a integrarse dentro de un mismo esquema de aprendizaje [22].

Bajo esta premisa, puede decirse que el aprendizaje profundo representa una evolución del aprendizaje automático hacia modelos con mayor capacidad de representación. Mientras que los algoritmos tradicionales suelen depender de variables explícitas o de características previamente construidas, los modelos profundos pueden aprender por sí mismos patrones más complejos y menos evidentes dentro de los datos [22].

4.4.3 Diferencias en los datos, entrenamiento e infraestructura

Otra diferencia importante entre ambos enfoques tiene que ver con el tipo de datos para el cual suelen resultar más adecuados. El aprendizaje automático tradicional suele ofrecer buenos resultados en problemas bien definidos que trabajan con **datos estructurados**, como tablas o regis-

tros con variables numéricas y categóricas claramente establecidas. Por otro lado, el aprendizaje profundo resulta más apropiado cuando se trabaja con **datos no estructurados**, tales como imágenes, audio o texto, donde se requiere un nivel mayor de abstracción para identificar patrones relevantes [22].

En lo que respecta al entrenamiento, los modelos de aprendizaje profundo suelen demandar más datos, más tiempo de cómputo y una infraestructura más compleja que los modelos tradicionales. Esto se debe a que las redes neuronales profundas involucran una mayor cantidad de operaciones matemáticas y parámetros ajustables, lo que incrementa el costo computacional del entrenamiento. Por el contrario, muchos **algoritmos clásicos de machine learning** pueden entrenarse con menor volumen de datos y con menores requerimientos computacionales [22].

Sin embargo, esta mayor complejidad también se traduce en una mayor capacidad para modelar relaciones no lineales y patrones de alta complejidad. Por ello, aunque el aprendizaje profundo exige más recursos, su uso se justifica cuando la naturaleza del problema supera la capacidad representacional de los enfoques tradicionales [22].

4.5 Justificación del uso de aprendizaje profundo en el proyecto

A partir de la comparación entre el aprendizaje automático tradicional y el aprendizaje profundo, resulta posible identificar con mayor claridad por qué este último constituye el enfoque más adecuado para el problema planteado en el presente trabajo.

En tareas de clasificación de imágenes, la información de entrada no se encuentra organizada como un conjunto reducido de variables explícitas, sino distribuida en una estructura visual compleja que incluye color, textura, bordes, formas y relaciones espaciales. En este tipo de escenarios, el aprendizaje automático tradicional depende de la calidad de la representación construida manualmente, mientras que el aprendizaje profundo ofrece la posibilidad de aprender dichas representaciones directamente desde la imagen [22].

Esta capacidad resulta ser valiosa cuando se trabaja con imágenes recolectadas en condiciones reales, donde existe variabilidad en iluminación, ángulo, fondo y calidad de captura. En dichas condiciones, la posibilidad de que el modelo aprenda representaciones más complejas y adaptativas constituye una ventaja importante frente a enfoques basados exclusivamente en características diseñadas manualmente [22].

En consecuencia, aunque el aprendizaje automático tradicional y el aprendizaje profundo comparten fundamentos relacionados con el aprendizaje a partir de datos, sus diferencias en la forma de representar la información, en el proceso de entrenamiento y en los requerimientos de infraestructura hacen que el aprendizaje profundo resulte más adecuado para problemas visuales complejos. Esta distinción es particularmente relevante para el presente proyecto, donde la entrada del sistema corresponde a imágenes capturadas en campo y el objetivo consiste en identificar patrones visuales asociados a la posible presencia de plaga.

Además, el objetivo del proyecto no se reduce a realizar una clasificación en condiciones ideales, sino a construir una solución que pueda funcionar en un contexto real de uso, orientado a la detección temprana de plagas en árboles. Esto exige un enfoque que sea capaz de responder de forma más consistente ante la diversidad visual presente en las imágenes del caso de estudio. En este sentido, el aprendizaje profundo no solo representa una alternativa técnicamente viable, sino también una elección metodológica coherente con la naturaleza del problema y con las condiciones reales bajo las cuales se espera que opere el sistema.

Capítulo 5. Análisis comparativo de arquitecturas y selección del modelo

La selección de una arquitectura adecuada es una decisión central en cualquier sistema de clasificación de imágenes basado en Deep Learning. Este capítulo presenta un análisis de las principales familias de arquitecturas relevantes para el problema, con el objetivo de justificar la elección del modelo empleado en el proyecto.

5.1 Arquitecturas relevantes para clasificación de imágenes

Desde hace ya varios años, las Redes Neuronales Convolucionales (CNNs) han sido el estándar para clasificación de imágenes con el uso de modelos de Deep Learning. Estas se componen de capas de nodos que contienen una capa de entrada, una o más capas ocultas y una capa de salida. Cada nodo se conecta a otro y tiene un peso y umbral asociados. Si la salida de cualquier nodo individual está por encima del valor del umbral especificado, ese nodo se activa y envía datos a la siguiente capa de la red; de lo contrario, no se pasa ningún dato a la siguiente capa. Las CNN proporcionan un enfoque escalable para las tareas de clasificación de imágenes y reconocimiento de objetos, aprovechando principios del álgebra lineal, específicamente la multiplicación de matrices, para identificar patrones dentro de una imagen. Sin embargo, pueden ser muy exigentes desde el punto de vista computacional, ya que requieren unidades de procesamiento gráfico (GPU) para entrenar los modelos[24].

A partir de esta base, las CNN pueden combinarse de formas específicas para construir arquitecturas más complejas. Un ejemplo es YOLO, que secciona la imagen en regiones y aplica múltiples CNNs para detectar objetos de forma individual, lo cual resulta útil cuando la imagen contiene múltiples elementos de interés simultáneos.

Más recientemente, han surgido los Transformers de Visión (ViT), que aplican el principio de atención propio de los Transformers al análisis de imágenes. La principal diferencia respecto a las CNN es que la entrada al modelo es una proyección lineal de parches de la imagen, en lugar de aplicar convoluciones sobre píxeles adyacentes[25].

Estas tres familias de arquitecturas, CNNs clásicas, arquitecturas basadas en detección de regiones y Transformers de visión, representan el panorama actual de modelos para el análisis de imágenes, y serán el punto de referencia para la selección del modelo en este proyecto.

5.2 Relevancia de las arquitecturas para dispositivos móviles

El despliegue de modelos de aprendizaje profundo en dispositivos móviles introduce restricciones que no existen en entornos de cómputo en la nube. Un dispositivo móvil cuenta con recursos limitados de memoria, procesamiento y batería, por lo que no cualquier arquitectura es viable para ejecutarse de forma local.

Los factores más relevantes a considerar son el tamaño del modelo, medido en megabytes y número de parámetros, y la latencia de inferencia, es decir, el tiempo que tarda el modelo en procesar una imagen y producir un resultado. Modelos como ResNet-50 superan los 90 MB, mientras que arquitecturas diseñadas para entornos restringidos como MobileNet o EfficientNet-B0 reducen este tamaño considerablemente manteniendo una precisión competitiva.

Estas arquitecturas ligeras logran su eficiencia principalmente mediante convoluciones separables en profundidad (depthwise separable convolutions) [26], que reducen el número de opera-

ciones necesarias sin perder capacidad de extracción de características. Adicionalmente, TensorFlow Lite permite convertir modelos entrenados en Keras a un formato optimizado (.tflite) compatible con Android de forma directa, lo que facilita considerablemente su integración en la aplicación móvil.

5.3 Modelos de referencia para clasificación de imágenes

En esta sección se describen los modelos candidatos para el prototipo, considerando su arquitectura, eficiencia y viabilidad en dispositivos móviles.

5.3.1 ResNet-50

Es una arquitectura CNN profunda de 50 capas ampliamente utilizada como baseline en tareas de clasificación de imágenes. Cuenta con aproximadamente 25 millones de parámetros y un tamaño cercano a los 98 MB, lo que la hace poco viable para despliegue móvil directo, pero útil como referencia de comparación durante el entrenamiento.

5.3.2 MobileNetV3

Es una evolución de la familia MobileNet optimizada mediante búsqueda de arquitectura neuronal (NAS). Incorpora capas de atención ligeras y activaciones h-swish que mejoran su eficiencia. Con menos de 5 MB, es una opción sólida para inferencia en Android.

5.3.3 EfficientNet-B0/B1/B3

Esta aplica un escalado compuesto que ajusta simultáneamente la profundidad, el ancho y la resolución del modelo en proporción balanceada. EfficientNet-B0 parte de apenas 5.3 MB y ofrece una precisión superior a MobileNetV3 en ImageNet con un tamaño comparable, posicionándose como una de las mejores opciones para esta aplicación [27].

5.3.4 MobileViT

Es una arquitectura híbrida que combina convoluciones locales con bloques de atención tipo Transformer. Aunque más ligera que un ViT estándar, su costo computacional es mayor que el de MobileNetV3 o EfficientNet, y su soporte en TFLite es menos maduro, lo que limita su viabilidad práctica para este proyecto.

5.4 Transfer learning y fine-tuning

El transfer learning es una técnica que consiste en reutilizar un modelo previamente entrenado en un conjunto de datos de gran escala como punto de partida para una tarea distinta. **En lugar de inicializar los pesos de forma aleatoria, el modelo hereda representaciones ya aprendidas**, como bordes, texturas y formas, lo que reduce significativamente el tiempo de entrenamiento y la cantidad de datos necesarios.

Para este proyecto, el uso de transfer learning es especialmente relevante dado que el dataset se construye a partir de imágenes recolectadas directamente en campo, lo que limita su tamaño. Entrenar un modelo desde cero en estas condiciones resultaría en sobreajuste y un rendimiento deficiente.

La estrategia adoptada combina dos fases: en la primera se aplica feature extraction, manteniendo los pesos del modelo base congelados y entrenando únicamente la cabeza de clasificación. En la segunda fase se realiza fine-tuning, descongelando las últimas capas del modelo base y continuando el entrenamiento con una tasa de aprendizaje reducida, permitiendo que el modelo se adapte a las características visuales específicas.

5.5 Comparación de modelos candidatos

A continuación se presenta una comparación de los modelos candidatos evaluados, considerando los criterios discutidos a lo largo del capítulo. La precisión reportada corresponde al benchmark ImageNet y sirve como referencia del potencial de cada arquitectura antes del fine-tuning.

Tabla 2: *Comparación de modelos candidatos*

Modelo	Parámetros	Tamaño	Top-1 ImageNet	TFLite
ResNet-50	25.6 M	98 MB	76.0%	Parcial
MobileNetV3	5.4 M	4.9 MB	75.2%	Nativo
EfficientNet-B0	5.3 M	5.3 MB	77.1%	Nativo
EfficientNet-B1	7.8 M	7.8 MB	79.1%	Nativo
EfficientNet-B3	12.3 M	48 MB	81.6%	Nativo
MobileViT-S	5.6 M	22 MB	78.4%	Limitado

5.6 Selección del modelo definitivo para el proyecto

Con base en la comparación, se selecciona EfficientNet-B3 como el modelo definitivo para este proyecto. Esta arquitectura ofrece la mayor precisión en ImageNet entre los candidatos viables para móvil, con un tamaño de 48 MB que resulta perfectamente manejable para su despliegue mediante TFLite en Android.

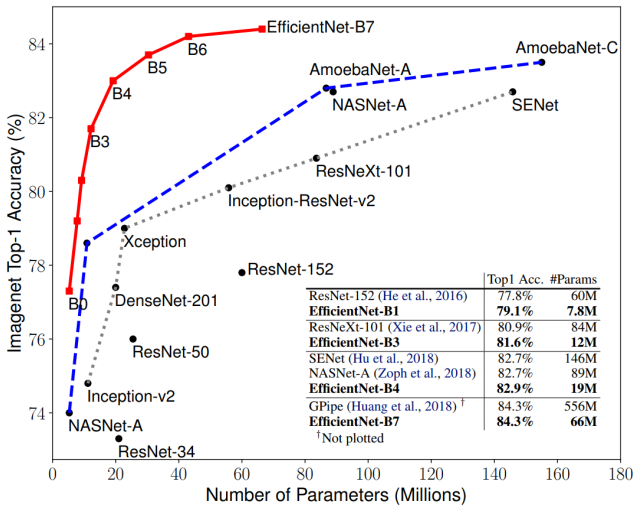


Figura 10: *Diagrama comparativo con diferentes arquitecturas de CNNs. Tomado de [27]*

Adicionalmente, EfficientNet-B3 está disponible como modelo preentrenado en `tf.keras.apps`, lo que facilita su uso con la estrategia de transfer learning y fine-tuning descrita en la sección anterior.

5.7 Arquitectura de EfficientNet-B3

EfficientNet-B3 es una red neuronal convolucional cuyo funcionamiento sigue el flujo clásico de una CNN: las imágenes de entrada pasan por sucesivas capas que extraen patrones visuales de forma progresiva, desde características simples como bordes y texturas hasta representaciones más complejas, para finalmente producir una clasificación.

Lo que distingue a EfficientNet-B3 de una CNN convencional son tres elementos clave en su diseño. Primero, su bloque fundamental es el **MBConv** (*Mobile Inverted Bottleneck Convolution*), que reemplaza la convolución clásica separando la detección de patrones espaciales de la combinación de canales, logrando la misma capacidad representativa con un menor costo computacional. Segundo, cada bloque MBConv incorpora un mecanismo de **Squeeze-and-Excitation**, que pondera la importancia de cada canal de forma adaptativa según el contenido de la imagen, amplificando las características más relevantes y suprimiendo las irrelevantes. Tercero, la arquitectura base fue obtenida mediante **búsqueda de arquitectura neuronal** (NAS), lo que garantiza que el punto de partida ya sea óptimo antes de aplicar cualquier escalado.

A partir de esta base (EfficientNet-B0), el modelo B3 se obtiene aplicando **Compound Scaling**: un método que escala simultáneamente la profundidad (número de capas), el ancho (número de filtros por capa) y la resolución de entrada, en proporción balanceada. Esto contrasta con enfoques tradicionales que escalan solo una dimensión, y explica por qué EfficientNet-B3 alcanza un 81.6% de precisión en ImageNet con apenas 12.3 M de parámetros.

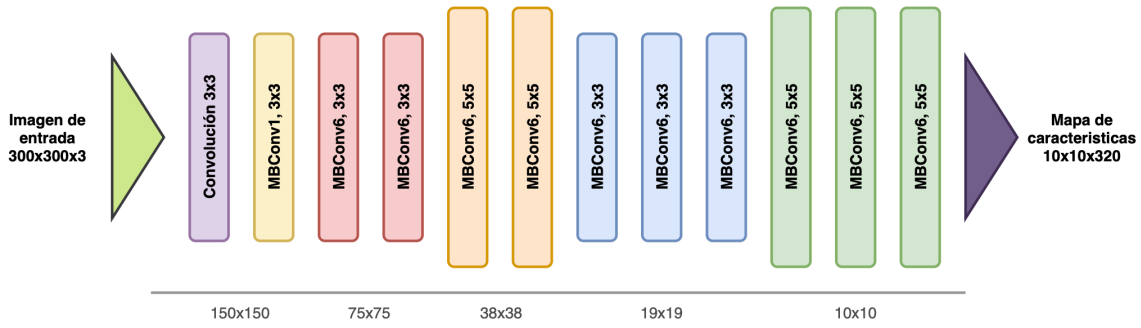


Figura 11: *Arquitectura CNN de EfficientNet.*

5.8 Arquitectura y funcionamiento de las Redes Neuronales Convolucionales

Las Redes Neuronales Convolucionales (CNN) son arquitecturas de aprendizaje profundo especializadas en el procesamiento de datos enfocados a imágenes y videos. Su funcionamiento se basa en la extracción de representaciones visuales de complejidad creciente; las primeras capas identifican características básicas como bordes, mientras que las capas subsecuentes reconocen texturas, partes y, finalmente, objetos completos [28].

Sus aplicaciones principales abarcan la clasificación de imágenes, la detección de objetos, la

segmentación semántica y el reconocimiento facial, consolidándose como el estándar en tareas de visión por computadora. Desde una perspectiva histórica, la arquitectura LeNet-5 (LeCun, 1989) sentó las bases para el reconocimiento de dígitos escritos a mano. Posteriormente, la arquitectura AlexNet detonó el auge del aprendizaje profundo moderno al ganar la competencia ImageNet, reduciendo el error top-5 de aproximadamente 26% a 15% [29].

Para procesar la información visual, el flujo completo de una CNN típica avanza desde la imagen de entrada hasta la obtención de probabilidades mediante la siguiente secuencia:

$$\text{Imagen} \xrightarrow{\text{Conv+BN+ReLU+Pool}\times n} \text{Features} \xrightarrow{\text{Flatten}} \text{Vector} \xrightarrow{\text{FC}} \text{Logits} \xrightarrow{\text{Softmax}} \text{Probabilidades} \quad (1)$$

Este flujo se compone de las siguientes operaciones y capas fundamentales:

5.8.1 Convolución

Es la operación central de la red, consistente en un filtro (o kernel) que recorre la imagen multiplicando sus valores por los píxeles adyacentes mediante un producto punto. Esta operación indica qué tan presente está un patrón específico en esa región de la imagen. El resultado es un mapa de características (*feature map*), el cual conforma una nueva matriz que señala la intensidad del patrón en cada zona:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n) \quad (2)$$

5.8.2 Normalización por lotes (Batch Normalization)

Consiste en la normalización de los valores de salida de cada capa convolucional para evitar que las activaciones sean numéricamente muy grandes o muy pequeñas. Esta técnica resuelve el desplazamiento interno de covariables (*internal covariate shift*) [30] al calcular la media μ y la varianza σ^2 del lote (*batch*) actual:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (3)$$

5.8.3 Función de activación ReLU

La Unidad Lineal Rectificada (ReLU) introduce no linealidad en el modelo, lo que resulta esencial para aprender patrones y relaciones complejas. Su regla de operación convierte cualquier valor negativo a cero y mantiene los valores positivos sin cambios:

$$f(x) = \max(0, x) \quad (4)$$

Sin el uso de ReLU u otra función no lineal, el apilamiento de múltiples capas convolucionales sería equivalente a tener una sola capa, ya que la composición de funciones lineales sigue siendo lineal.

5.8.4 Agrupamiento (Pooling)

Esta capa reduce las dimensiones espaciales de la imagen tomando los valores más representativos de cada región. La variante más común, *Max Pooling*, extrae el valor máximo. Esta reducción disminuye el costo computacional y aporta invarianza traslacional a la red:

$$P(i,j) = \max_{(m,n)\in R_{ij}} f(m,n) \tag{5}$$

5.8.5 Aplanado (Flatten)

Convierte la matriz bidimensional resultante de las operaciones anteriores en un vector unidimensional. Funciona como un puente estructural entre la etapa de extracción de características convolucionales y la etapa de clasificación. Por ejemplo, un tensor con dimensiones $7 \times 7 \times 512$ se transforma en un vector de 25,088 valores:

$$\text{Tensor}(H \times W \times C) \rightarrow \text{Vector}(H \cdot W \cdot C) \tag{6}$$

5.8.6 Capas Densas (Fully Connected)

Actúan como una red neuronal artificial tradicional donde cada neurona se conecta con todas las activaciones de la capa anterior. Su propósito es combinar la información extraída a lo largo de la red para procesar la clasificación final:

$$y = W \cdot x + b \tag{7}$$

5.8.7 Clasificación mediante Softmax

Aplicada en la capa de salida para problemas multiclase, la función Softmax convierte los valores de salida continuos (*logits*) en probabilidades que suman 1.

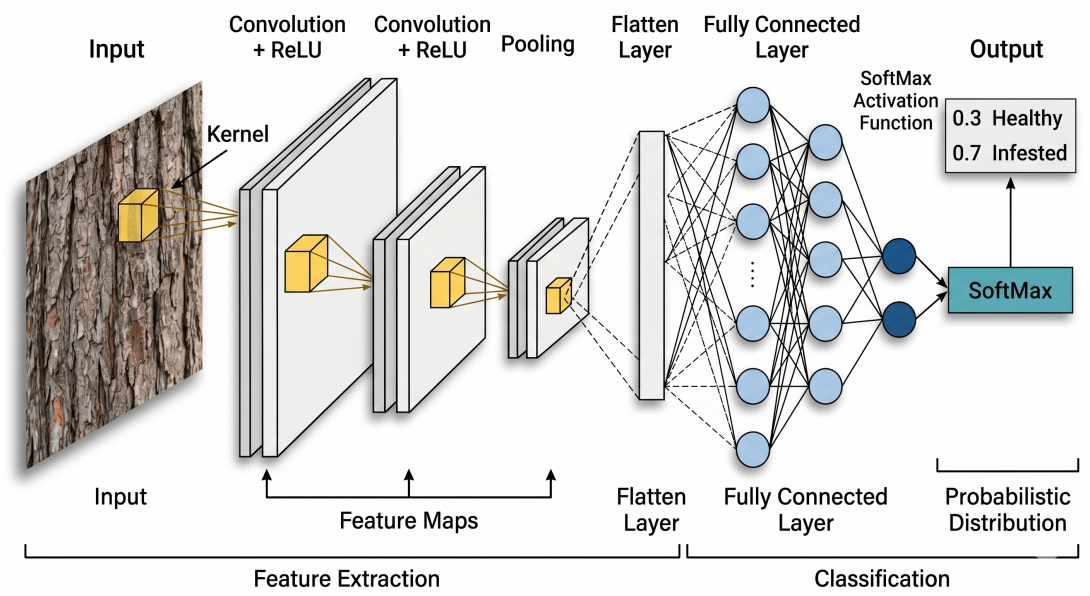


Figura 12: *Capas de una CNN*

Capítulo 6. Conjunto de datos y preprocesamiento

Los conjuntos de datos en el contexto de Machine Learning requieren una organización clara de clases y un formato de imagen bien definido, especialmente cuando se trabaja con un modelo preentrenado cuyos pesos han sido congelados tras haber sido entrenado con un formato específico.

6.1 Recopilación de datos

La recopilación de imágenes para el dataset se compone de dos fuentes principales. La primera corresponde a capturas propias realizadas en campo en distintos bosques de la Ciudad de México, con énfasis en el Bosque de Chapultepec como caso de estudio. La segunda proviene de iNaturalist, plataforma comunitaria de ciencia ciudadana donde usuarios de todo el mundo documentan observaciones de flora y fauna. Dado que la plataforma permite filtrar por taxonomía, fue posible identificar registros asociados a la familia de insectos descortezadores y recuperar las imágenes que resultaron de mayor utilidad para los objetivos del proyecto.

El conjunto de datos se organiza en dos tipos de imágenes según el nivel de observación al que corresponden. El primero agrupa imágenes de follaje capturadas desde una perspectiva aérea, utilizadas para identificar zonas del arbolado con posibles afecciones a partir del análisis cromático descrito en la Sección 2.5.1. El segundo agrupa imágenes de corteza tomadas directamente sobre el tronco de los árboles, cuyo propósito es alimentar el modelo de clasificación para determinar si un árbol presenta o no indicios de infestación.

Las imágenes de follaje se mantienen en una sola clase, ya que su uso no requiere una separación por etapa de infestación sino que sirven como referencia para definir los rangos cromáticos del análisis de color. Las imágenes de corteza, en cambio, se organizan de forma binaria en dos clases: *sano* e *infestado*. Todas las imágenes han sido estandarizadas a una resolución de 300×300 píxeles en formato JPG, correspondiente al tamaño de entrada requerido por EfficientNet-B3.

6.2 Características del conjunto de datos

6.3 Etiquetado de imágenes

El etiquetado del conjunto de datos se define a partir de la estructura de carpetas en la que se organizan las imágenes, separadas en dos clases: *sano* e *infestado*. Este esquema es compatible con el método de carga `image_dataset_from_directory` de TensorFlow/Keras, que infiere automáticamente la etiqueta de cada imagen a partir del nombre de la carpeta que la contiene. Dado que el problema se plantea como una tarea de clasificación de imagen completa, no fue necesario el uso de herramientas de anotación/recorte externa como LabelImg o Roboflow, ya que no se requiere delimitar regiones específicas dentro de la imagen.

El criterio de asignación de clase cambia según la funete de la imagen. Las imágenes provenientes de iNaturalist ya contaban con una identificación taxonómica asociada a la presencia del insecto descortezador en el árbol, por lo que su clase fue validada durante el proceso de limpieza del dataset descrito en la Sección 6.2. Las imágenes recolectadas directamente en campo fueron confirmadas con el apoyo de un experto, quien verificó la presencia de las señales visuales características de infestación descritas en la Sección 2.5, entre ellas grumos de resina, galerías en

la corteza y alteraciones en la coloración superficial del tronco.

6.4 Preprocesamiento de imágenes

El preprocesamiento de las imágenes del conjunto de datos se divide en dos etapas: la estandarización de las imágenes y el aumento de datos.

La estandarización consiste en garantizar que todas las imágenes del dataset cumplan con el formato de entrada requerido por EfficientNet-B3. Como se establece en la RN-09, la resolución estándar es de 300×300 píxeles, por lo que todas las imágenes fueron redimensionadas a esta resolución y convertidas al formato JPG. Adicionalmente, los valores de píxel se normalizan en el rango $[0, 1]$ dividiendo entre 255, lo cual es una práctica estándar para estabilizar el entrenamiento de redes neuronales profundas.

La segunda etapa corresponde al aumento de datos o *data augmentation*, que consiste en aplicar transformaciones aleatorias sobre las imágenes durante el entrenamiento con el objetivo de que el modelo aprenda a generalizar ante la variabilidad visual presente en las capturas de campo. A diferencia de la estandarización, estas transformaciones no se aplican sobre el dataset almacenado en disco sino en tiempo real durante el entrenamiento, mediante capas de aumento integradas directamente en el modelo con `tf.keras.layers`.

Las transformaciones seleccionadas para las imágenes de corteza son las siguientes:

- **Flip horizontal y vertical**: dado que las fotografías de corteza pueden tomarse desde cualquier ángulo en campo, los patrones de galerías e indicios de infestación son igualmente reconocibles en cualquier orientación.
- **Rotaciones**: por la misma razón anterior, se aplican rotaciones de hasta 90° para simular diferentes ángulos de captura.
- **Variaciones de brillo y contraste**: simulan las distintas condiciones de iluminación natural presentes durante las jornadas de captura en el bosque.
- **Zoom leve**: simula variaciones en la distancia de captura respecto al tronco del árbol.

Figura 13: *Muestra de configuración de ajustes de una misma imagen.*

No se aplican deformaciones fuertes ya que los patrones generados por las larvas en la corteza podrian perderse en el proceso, afectando de manera negativa el aprendizaje del modelo.

6.5 Análisis complementario de color

El análisis complementario de color tiene como objetivo identificar en la imagen la presencia de tonalidades asociadas a las fases de infestación descritas en la Sección 2.5, sirviendo como información adicional al resultado del modelo de clasificación. Este análisis se ejecuta de forma local en el dispositivo de manera concurrente al modelo TensorFlow Lite, sin requerir conexión a internet, tal como se establece en la RN-49.

El análisis se realiza sobre el espacio de color HSV (*Hue, Saturation, Value*), que a diferencia del espacio RGB permite aislar el tono cromático en un solo canal, facilitando la definición de rangos de color más precisos en variaciones de brillo e iluminación propias de las capturas en campo.

Para las imágenes de corteza, el análisis clasifica los píxeles en rangos cromáticos correspondientes a las fases de colonización documentadas: tonos blancos y crema asociados a la respuesta defensiva activa del árbol, tonos café-anaranjados relacionados con una colonización avanzada, y tonos café-rojizos y grisáceos presentes en etapas de daño severo o muerte del árbol. El resultado se expresa como el porcentaje de píxeles pertenecientes a cada rango sobre el total de píxeles analizados en la imagen, conforme a lo establecido en la RN-51.

6.6 Organización de los datos para entrenamiento, validación y prueba

El conjunto de datos de corteza se divide en tres subconjuntos: entrenamiento, validación y prueba, siguiendo una proporción de 80/10/10 respectivamente. Esta distribución maximiza la cantidad de imágenes disponibles para el entrenamiento, lo cual es relevante dado el tamaño del dataset, mientras que los subconjuntos de validación y prueba mantienen una muestra suficiente para evaluar el desempeño del modelo de forma confiable.

El subconjunto de entrenamiento es el que el modelo utiliza directamente para ajustar sus parámetros durante el proceso de aprendizaje. El subconjunto de validación se emplea para monitorear el desempeño del modelo al final de cada época y detectar posibles problemas de sobreajuste durante el entrenamiento. Finalmente, el subconjunto de prueba se reserva completamente hasta la evaluación final del modelo, garantizando que los resultados reportados reflejen el desempeño real ante datos no vistos en ninguna etapa previa.

La división se realiza de forma aleatoria pero estratificada, asegurando que la proporción entre la clase *sano* e *infestado* se mantenga equivalente en los tres subconjuntos, lo cual es especialmente importante para mantener un balance entre ambas clases en el dataset.

Capítulo 7. Análisis del sistema

En esta sección se hace el analisis de los requerimientos, componenetes del sistema y pruebas del mismo con razón de identificar los riesgos, restricciones y buscar las medidas de mitigación necesarias.

7.1 Requerimientos del sistema

El presente proyecto tiene como producto principal el desarrollo de un sistema móvil para la detección de insectos descortezadores mediante visión por computadora. Sin embargo, durante el análisis del problema se identificó que una aplicación móvil por sí sola no es suficiente para cubrir todas las necesidades del proceso de monitoreo forestal. El Experto de campo requiere una herramienta ágil, funcional sin conexión a internet y orientada a la captura y análisis inmediato en sitio; posteriormente, puede utilizar la plataforma web para consultar la información consolidada, revisar historiales, visualizar mapas generales y analizar estadísticas derivadas de los registros sincronizados.

Por esta razón, el sistema se compone de dos subsistemas complementarios: una aplicación móvil Android diseñada bajo un enfoque offline-first, que constituye el núcleo del prototipo para el trabajo en campo, y una plataforma web orientada a la consulta y supervisión de la información recolectada. Ambos subsistemas se relacionan mediante una base de datos central en la nube. La aplicación móvil almacena los registros en una base local SQLite y posteriormente los sincroniza de forma ascendente hacia la nube cuando existe conexión a internet. La plataforma web consulta la información consolidada en dicha base central, pero no modifica los registros operativos generados desde los dispositivos móviles.

7.1.1 Requerimientos funcionales

Los requisitos funcionales definen las capacidades y comportamientos específicos que el sistema debe satisfacer, diferenciados por componente: la aplicación móvil, orientada al monitoreo en campo, y el panel web, destinado a la gestión y consulta de resultados.

Con el propósito de reflejar claramente a qué subsistema aplica cada requerimiento, se ha adoptado un esquema de etiquetado de requisitos funcionales con el ámbito de aplicación correspondiente: aplicación móvil (**MOV**), aplicación web (**WEB**) o ambos entornos (**MOV/WEB**).

La Tabla 3 resume estos requisitos.

Tabla 3: *Requerimientos funcionales de la aplicación móvil y la plataforma web*

ID	Ámbito	Descripción
RF-01	MOV/WEB	El sistema debe permitir al usuario registrarse e iniciar sesión utilizando un correo electrónico y una contraseña.
RF-02	MOV/WEB	El sistema debe permitir al usuario solicitar la recuperación de su contraseña a través de un enlace enviado a su correo electrónico.
RF-03	MOV	El sistema debe permitir la entrada de imágenes de dos formas: capturándolas directamente desde la aplicación o subiéndolas desde la galería del dispositivo.

Tabla 3 – continuación

ID	Ámbito	Descripción
RF-04	MOV	El sistema debe contar con herramientas de edición básicas antes de enviar la imagen a procesar.
RF-05	MOV	El sistema debe registrar automáticamente la ubicación geográfica del árbol fotografiado, obtenida a partir de los metadatos de la fotografía o del GPS del dispositivo en el momento de la captura.
RF-06	MOV	El sistema debe almacenar localmente un registro de todas las imágenes ingresadas por el usuario, incluyendo su resultado de clasificación, *timestamp* y ubicación, para su posterior sincronización con la nube.
RF-07	MOV	El sistema debe sincronizar los registros almacenados localmente hacia la nube cuando el usuario cuente con conexión a internet, conservando la fecha y hora original de captura como referencia del registro.
RF-08	MOV/WEB	La aplicación móvil deberá incorporar un modelo en formato TensorFlow Lite embebido en el dispositivo para realizar una clasificación local e inmediata de la imagen capturada cuando no se tenga acceso a internet. La aplicación web deberá incorporar un modelo en formato Keras3 para realizar una clasificación en la nube de la imagen capturada en la aplicación movil.
RF-09	MOV	El sistema debe emitir y mostrar en pantalla una clasificación inmediata del estado sanitario del árbol, indicando si está sano o infectado, generada por el modelo local embebido en el dispositivo, sin requerir conexión a internet.
RF-10	WEB	El sistema debe permitir agrupar y dar seguimiento a un conjunto de muestras específicas a lo largo del tiempo, generando un historial del estado de salud de cada ejemplar.
RF-11	MOV/WEB	El sistema debe mostrar alertas o indicadores visuales con el analisis complementario de color que faciliten la detección de focos de infestación.
RF-12	MOV	El sistema debe mostrar un resumen estadístico de la jornada actual, incluyendo el número total de fotografías tomadas en el día, la cantidad de árboles clasificados como sanos y la cantidad de posibles infecciones detectadas.
RF-13	MOV	El sistema debe proveer acceso directo y visible a la función principal: “Nueva Captura”.
RF-14	MOV	El sistema debe mostrar notificaciones o alertas cuando el usuario se encuentre en las proximidades de una zona previamente catalogada como foco de infestación.
RF-15	MOV	La sincronización manual estará disponible únicamente cuando existan errores de sincronización y el dispositivo cuente con conexión activa. En caso contrario, el sistema deberá deshabilitar esta opción e informar al usuario la causa.
RF-16	WEB	El sistema debe permitir generar un reporte básico en formato PDF de las capturas realizadas.
RF-17	WEB	El sistema debe permitir descargar el reporte generado.

Tabla 3 – continuación

ID	Ámbito	Descripción
RF-18	MOV	El sistema debe conservar en el almacenamiento local del dispositivo los registros asociados a las imágenes capturadas, excepto estas últimas.
RF-19	MOV	El sistema debe permitir al usuario asociar una nueva captura a un árbol ya existente en el registro local, o bien crear un nuevo registro de árbol al momento de realizar una captura.
RF-20	WEB	El sistema debe mostrar el historial completo y permanente de todos los árboles registrados, incluyendo sus capturas, resultados de clasificación, observaciones y fechas de subida y sincronización, sin restricción de periodo de tiempo.
RF-21	WEB	El sistema debe mostrar estadísticas generales del estado del arbolado registrado, incluyendo totales por estado sanitario, distribución por especie y evolución temporal de los focos de infestación detectados.
RF-22	MOV/WEB	El sistema movil debe ejecutar un análisis complementario de color sobre la imagen capturada y mostrar su resultado como información complementaria al resultado de clasificación, tanto en la app móvil como en la app web.
RF-23	MOV	El sistema debe permitir al usuario operativo editar la información de un registro de árbol, incluyendo la actualización de su estado sanitario y la adición de observaciones o notas de campo.
RF-24	WEB	El sistema debe mostrar un mapa general con la ubicación de todos los árboles registrados, permitiendo filtrarlos por estado sanitario, especie, fecha de captura y dispositivo.
RF-25	MOV	El sistema debe mostrar un mapa general con la ubicación de todos los árboles registrados del dispositivo, permitiendo filtrarlos por estado sanitario, especie y fecha de captura.
RF-26	WEB	El sistema debe permitir al administrador gestionar las cuentas de usuario del sistema, incluyendo la creación, edición, desactivación y asignación de roles.
RF-27	MOV/WEB	El sistema deberá permitir que una cuenta de experto de campo pueda estar asociada a uno o varios dispositivos móviles. Cada dispositivo deberá mantener una base de datos local independiente en SQLite para almacenar árboles, capturas, análisis y ubicaciones registrados offline. Al recuperar conexión, la aplicación móvil deberá sincronizar los registros locales con la base de datos central en la nube, conservando la identificación del dispositivo de origen.
RF-28	MOV	Cuando dos dispositivos asociados a la misma cuenta registren árboles con el mismo nombre, el sistema no deberá asumir que se trata del mismo árbol físico.

7.1.2 Requerimientos no funcionales

Los requisitos no funcionales establecen las restricciones de calidad, rendimiento y operación bajo las cuales el sistema debe funcionar, independientemente de las funcionalidades que provee. La Tabla 4 los detalla.

Tabla 4: *Requerimientos no funcionales de la aplicación móvil y la plataforma web*

ID	Categoría	Descripción
RNF-01	Rendimiento	El sistema debe devolver el resultado del análisis (sano/infectado) en un tiempo máximo de X segundos una vez que la imagen ha sido cargada exitosamente en la nube.
RNF-02	Portabilidad	El tamaño de la aplicación móvil instalada no debe exceder los 50 MB, asegurando que pueda ser instalada en dispositivos de gama media sin saturar el almacenamiento.
RNF-03	Usabilidad	Las transiciones entre las pantallas de la aplicación, como pasar del mapa al historial del árbol, no deben demorar más de X segundos.
RNF-04	Eficiencia	La aplicación debe estar optimizada para no realizar procesos en segundo plano innecesarios, minimizando el consumo de batería del dispositivo durante las jornadas de inspección en el bosque.
RNF-05	Seguridad	Toda comunicación entre la aplicación móvil y los servicios en la nube, incluyendo el envío de credenciales, imágenes y coordenadas, debe realizarse a través de protocolos seguros como HTTPS.
RNF-06	Seguridad	Las contraseñas de los usuarios deben almacenarse en la base de datos utilizando algoritmos de hashing para evitar su exposición.
RNF-07	Seguridad	El sistema debe cerrar automáticamente la sesión del usuario tras un periodo prolongado de inactividad, por ejemplo 24 horas, para proteger los datos en caso de extravío del dispositivo.
RNF-08	Precisión	El modelo de visión por computadora debe alcanzar y mantener un porcentaje mínimo de precisión de X% en la correcta clasificación de los árboles infestados por descortezadores.
RNF-09	Arquitectura	El sistema debe respetar una arquitectura cliente-servidor, delegando el procesamiento pesado del modelo de inteligencia artificial a la nube.
RNF-10	Persistencia	El sistema debe contar con una base de datos relacional o no relacional optimizada para manejar consultas rápidas del historial forestal y referenciar los archivos de imágenes.
RNF-11	Implementación	La aplicación cliente debe ser desarrollada de forma nativa para el sistema operativo Android utilizando el lenguaje de programación **Kotlin**, garantizando un acceso óptimo al hardware del dispositivo, como cámara, GPS y almacenamiento local.

Tabla 4 – continuación

ID	Categoría	Descripción
RNF-12	Implementación	El modelo de visión por computadora para la detección de insectos descortezadores debe ser programado en Python, utilizando *frameworks* especializados en aprendizaje profundo, como TensorFlow o PyTorch.
RNF-13	Rendimiento	Debido a que el análisis de imágenes puede tomar algunos segundos, el backend en Python debe manejar las peticiones de forma asíncrona para no bloquear la interfaz de usuario en la aplicación Kotlin.
RNF-14	Mantenibilidad	El proyecto móvil debe utilizar Gradle para la gestión de librerías, mientras que el entorno de Python debe usar gestores como *pip* o *conda* para asegurar la replicabilidad del entorno de desarrollo.
RNF-15	Persistencia	La aplicación cliente en Android debe implementar **SQLite**, preferentemente mediante la librería *Room*, para el almacenamiento estructurado y local de la información. Esto permitirá guardar temporalmente los datos de las capturas cuando el usuario se encuentre en zonas sin conexión a internet, garantizando el uso básico *offline*.
RNF-16	Persistencia	El backend del sistema debe utilizar **PostgreSQL** como motor de base de datos relacional principal. Esta base de datos será la encargada de almacenar de forma centralizada, segura y escalable las credenciales de acceso de los usuarios, así como el historial completo y digitalizado del estado sanitario de cada árbol evaluado.
RNF-17	Integridad	El sistema debe cambiar el estado local de los registros en SQLite a “Sincronizado” o eliminarlos localmente una vez que el servidor backend confirme que la información se guardó exitosamente en PostgreSQL, evitando así la duplicidad de datos.

7.2 Reglas de negocio del sistema

Las reglas de negocio definen los criterios que rigen el comportamiento del sistema en relación con la detección de plagas y el procesamiento de imágenes. Reflejan decisiones metodológicas independientes de la tecnología utilizada y se presentan en la Tabla 5.

Tabla 5: *Reglas de negocio de la aplicación móvil y la plataforma web*

ID	Tipo	Descripción
RN-01	Acceso	Solo los usuarios con rol **Administrador** podrán crear cuentas nuevas. El registro no será abierto al público.
RN-02	Acceso	El rol **Experto de campo** podrá iniciar sesión en la app móvil y web. El rol **Administrador** solo podrá iniciar sesión en la app web.
RN-03	Sesión	La sesión en la app móvil persistirá activa hasta que el usuario cierre sesión manualmente. No expirará por inactividad.
RN-04	Seguridad	El enlace de recuperación de contraseña tendrá una vigencia máxima de 24 horas a partir de su generación.
RN-05	Seguridad	El enlace de recuperación será de un solo uso; una vez utilizado quedará invalidado automáticamente.
RN-06	Seguridad	El sistema permitirá un máximo de 3 solicitudes de recuperación de contraseña por usuario en un periodo de 30 días naturales.
RN-07	Imagen	Solo se aceptarán imágenes en formato JPG, JPEG, PNG, WEBP, BMP y HEIC.
RN-08	Imagen	El tamaño máximo de imagen aceptado será de 10 MB.
RN-09	Imagen	La resolución mínima aceptada será de 300×300 píxeles, correspondiente al tamaño de entrada requerido por EfficientNet-B3. Imágenes con resolución menor serán rechazadas por el sistema con un mensaje informativo al usuario.
RN-10	Edición	La herramienta de edición disponible antes del análisis será recorte de la imagen.
RN-11	Edición	La herramienta de edición se aplicará únicamente antes del análisis. Una vez procesada la imagen, no podrá modificarse.
RN-12	Edición	La edición de la imagen es opcional; el usuario podrá omitirla y proceder directamente al análisis.
RN-13	Ubicación	Si la imagen proviene de la galería y contiene metadatos de ubicación, el sistema utilizará esas coordenadas como ubicación del árbol.
RN-14	Ubicación	Si la imagen no contiene metadatos de ubicación, el sistema utilizará las coordenadas del GPS del dispositivo en el momento del registro.
RN-15	Ubicación	Si no es posible obtener la ubicación por ninguna de las dos vías, el registro se guardará sin coordenadas y se marcará como **Ubicación no disponible**, sin bloquear el flujo de captura.
RN-16	Registros	El usuario podrá eliminar manualmente un registro local antes de sincronizarlo. Esta acción será irreversible y requerirá confirmación explícita.

Tabla 5 – continuación

ID	Tipo	Descripción
RN-17	Registros	Cada registro local tendrá uno de los siguientes estados: **Pendiente de sincronización**, **Sincronizado** o **Error de sincronización**.
RN-18	Registros	Cada registro local deberá contener como mínimo: imagen, resultado del modelo, timestamp de captura, coordenadas GPS y árbol asociado.
RN-19	Sincronización	La sincronización automática se activará cuando el dispositivo detecte conexión a internet, ya sea por datos móviles o Wi-Fi.
RN-20	Sincronización	En caso de fallo durante la sincronización, el sistema reintentará automáticamente hasta 3 veces antes de marcar el registro como **Error de sincronización**.
RN-21	Sincronización	La fecha y hora de sincronización se almacenará como un campo adicional separado del timestamp original de captura. Ambos valores serán visibles en la app web.
RN-22	Modelo	El resultado generado por el modelo se mostrará al experto inmediatamente después de la captura, etiquetado explícitamente como **Offline**.
RN-23	Modelo	El sistema clasificará el árbol en dos categorías únicamente: **Sano** o **Infestado**.
RN-24	Modelo	El modelo TFLite solo emitirá una clasificación si el nivel de confianza supera el 75%. Si no supera ese umbral, el resultado se mostrará como **No determinado** y se recomendará al experto retomar la captura.
RN-25	Alertas	De acuerdo con la NOM-019-SEMARNAT-2017 [31], se considerará **foco de infestación activo** cuando se registren 3 o más árboles clasificados como infectados dentro de una superficie de 1,000 m^2.
RN-26	Interfaz	Los indicadores visuales de estado sanitario utilizarán el siguiente código de color: **verde** para sano, **rojo** para infestado y **amarillo** para no determinado.
RN-27	Alertas	De acuerdo con la NOM-019-SEMARNAT-2017 [31], el sistema considerará que el experto está en proximidad de un foco de infestación cuando se encuentre a 100 metros o menos de las coordenadas de un árbol catalogado como foco.
RN-28	Alertas	La alerta de proximidad se generará usando únicamente el GPS del dispositivo y los registros locales almacenados en SQLite, funcionando sin conexión a internet.
RN-29	Alertas	No se emitirá más de una alerta por foco en un periodo de 60 minutos para evitar notificaciones repetitivas mientras el experto permanezca en la zona.
RN-30	Jornada	El resumen de jornada considerará únicamente los registros cuyo timestamp de captura corresponda al día en curso, desde las 00:00 hasta las 23:59.
RN-31	Jornada	El resumen de jornada se actualizará en tiempo real conforme el experto realice nuevas capturas durante la jornada.

Tabla 5 – continuación

ID	Tipo	Descripción
RN-32	Sincronización	El sistema debe permitir al usuario reintentar manualmente la sincronización de los registros marcados con estado **Error de sincronización**, siempre que el dispositivo cuente con conexión a internet activa.
RN-33	Reportes	Los reportes generados desde la app web incluirán el historial sincronizado en la nube de un periodo de tiempo o de una zona específica.
RN-34	Reportes	Todo reporte deberá incluir como mínimo: identificador del árbol, nombre del árbol, fecha y hora de captura, resultado de clasificación, coordenadas GPS y nombre del experto que realizó la captura.
RN-35	Almacenamiento	La eliminación automática de registros del almacenamiento local solo podrá aplicarse a registros con estado **Sincronizado**. Los registros en estado **Pendiente** o **Error** no deberán eliminarse automáticamente hasta que sean sincronizados o eliminados manualmente por el usuario.
RN-36	Historial	El historial visible en la app móvil mostrará únicamente los registros del dia actual.
RN-37	Árboles	No puede existir una captura sin un árbol asociado. La selección o creación del árbol es un paso obligatorio antes de proceder al análisis.
RN-38	Árboles	Al crear un nuevo árbol, el sistema generará automáticamente un identificador único. El usuario deberá asignar manualmente un nombre al árbol y seleccionar su especie de una lista predefinida basada en las especies hospederas documentadas para el Bosque de Chapultepec, entre ellas **Fresno** (*Fraxinus uhdei*) y **Cedro blanco** (*Cupressus lusitanica*), además de otras que el Administrador podrá gestionar desde la app web.
RN-39	Árboles	El nombre asignado al árbol podrá complementarse con una descripción libre opcional, por ejemplo: *Pino 1 con bordes irregulares en corteza*, para facilitar su identificación en campo.
RN-40	Usuarios	Solo el rol **Administrador** podrá crear, editar, desactivar y asignar roles a usuarios.
RN-41	Usuarios	El **Experto de campo** podrá editar registros de árboles y agregar observaciones en la app movil.
RN-42	Historial	El historial en la app web no tendrá restricción de tiempo; mostrará todos los registros sincronizados desde el inicio del sistema.
RN-43	Alertas	El radio de proximidad para alertas definido en RN-29 será configurable únicamente desde la app web por el **Administrador**.
RN-44	Alertas	De acuerdo con la NOM-019-SEMARNAT-2017 [31], el sistema deberá considerar como periodos de mayor riesgo los meses de enero a mayo y de septiembre a noviembre, pudiendo emitir alertas estacionales en la app web durante estos periodos.

Tabla 5 – continuación

ID	Tipo	Descripción
RN-45	Modelo	Las evidencias visuales de infestación que el sistema buscará identificar mediante el modelo de clasificación incluyen: presencia de grumos de resina rojiza en el fuste, cambios de coloración del follaje de verde a verde amarillento o rojizo, y presencia visible de galerías o daños en la corteza.
RN-46	Análisis cromático	El análisis complementario de color se ejecutará de forma local en el dispositivo de forma concurrente al modelo TensorFlow Lite, utilizando OpenCV, sin requerir conexión a internet.
RN-47	Análisis cromático	El análisis de color del follaje clasificará los píxeles en tres rangos cromáticos correspondientes a las fases de infestación documentadas: verde, verde-amarillento o verde limón, y café-rojizo. El resultado se expresará como porcentaje de cada rango sobre el total de píxeles analizados.
RN-48	Análisis cromático	El análisis de color de la corteza clasificará los píxeles en los rangos cromáticos asociados a las fases de colonización subcortical: blanco/crema/rojizo, café-anaranjado y café-grisáceo. El resultado se expresará como porcentaje de cada rango sobre el total de píxeles analizados.
RN-49	Análisis cromático	El análisis cromático se ejecutará de forma independiente según el tipo de imagen procesada, ya sea follaje o corteza, dado que cada nivel opera en un espacio visual distinto. No se realizarán comparaciones ni agregaciones cromáticas entre imágenes de diferente naturaleza dentro de un mismo análisis.
RN-50	Almacenamiento	La base de datos local SQLite de la aplicación móvil no deberá almacenar imágenes como datos binarios de forma permanente. Únicamente deberá guardar metadatos del análisis y una referencia, ruta temporal o URL de la evidencia fotográfica.
RN-51	Historial	La evidencia fotográfica completa solo estará disponible para consulta desde la plataforma web después de que el registro haya sido sincronizado correctamente con la nube.
RN-52	Sincronización	La sincronización de datos se realizará únicamente desde la base local del dispositivo móvil hacia la base central en la nube. El sistema no contemplará la descarga automática de registros desde la nube hacia otros dispositivos móviles.
RN-53	Historial	La aplicación web permitirá consultar y analizar los registros sincronizados desde los dispositivos móviles, pero no modificará los registros operativos generados en la app móvil.
RN-54	Usuarios	Las funciones CRUD del Administrador aplicarán únicamente sobre datos administrativos o de configuración del sistema almacenados en la nube.

7.3 Catálogo de mensajes del sistema

La Tabla 6 presenta el catálogo de mensajes utilizados por el sistema para informar al usuario sobre errores, validaciones o estados relevantes durante la ejecución de los casos de uso.

Tabla 6: *Catálogo de mensajes del sistema.*

Código	Mensaje
MSG1	Dato requerido faltante.
MSG2	El correo electrónico no está registrado.
MSG3	Contraseña incorrecta.
MSG4	La cuenta se encuentra desactivada.
MSG5	El dispositivo no está autorizado para esta cuenta.
MSG6	No fue posible enviar el correo de recuperación.
MSG7	El enlace o código de recuperación no es válido.
MSG8	El enlace o código de recuperación ha expirado.
MSG9	Las contraseñas no coinciden.
MSG10	La contraseña no cumple con los requisitos de seguridad.
MSG11	No existe una sesión activa.
MSG12	No fue posible cerrar la sesión correctamente.
MSG13	No existen registros para la jornada actual.
MSG14	No fue posible cargar la información local.
MSG15	No existen registros con el estado sanitario seleccionado.
MSG16	No fue posible cargar los registros recientes.
MSG17	No fue posible acceder a la cámara.
MSG18	No fue posible importar la imagen seleccionada.
MSG19	Imagen no válida para el análisis.
MSG20	No fue posible registrar la ubicación GPS.
MSG21	No fue posible guardar el análisis en la base local.
MSG22	No fue posible obtener la clasificación del estado sanitario.
MSG23	No fue posible recortar la imagen.
MSG24	Área de recorte no válida.
MSG25	No fue posible sincronizar el registro.
MSG26	No existen registros sincronizados disponibles.
MSG27	No fue posible cargar la información consolidada.
MSG28	No fue posible aplicar los filtros seleccionados.
MSG29	No se cuenta con permisos de administración.
MSG30	No fue posible cargar la información de usuarios.
MSG31	El correo electrónico ya se encuentra registrado.
MSG32	No fue posible guardar los cambios de la cuenta.
MSG33	El dispositivo ya se encuentra asociado a una cuenta.
MSG34	No fue posible asociar el dispositivo.
MSG35	No fue posible desasociar el dispositivo.

7.4 Actores del sistema

Un actor representa cualquier entidad externa que interactúa con el sistema, ya sea una persona, un rol organizacional o un sistema externo. En el contexto de UML, los actores no forman parte del sistema en sí, sino que se relacionan con él a través de los casos de uso, iniciando acciones o recibiendo resultados de las mismas.

Para el presente sistema, los actores fueron definidos considerando los roles operativos involucrados en el proceso de monitoreo forestal. Por consiguiente, se identificaron los siguientes actores:

1. **Experto de campo**
   Persona encargada de utilizar la aplicación móvil en campo y la plataforma web para consultar la información sincronizada. Este actor registra árboles, realiza análisis del estado sanitario y consulta los datos generados durante la jornada.
   - Iniciar sesión en la aplicación móvil o web.
   - Recuperar y restablecer contraseña.
   - Capturar o importar imágenes de árboles.
   - Recortar o repetir una imagen antes del análisis.
   - Asociar una imagen a un árbol nuevo o existente.
   - Registrar datos de árboles nuevos.
   - Modificar datos de árboles existentes desde la aplicación móvil.
   - Obtener la clasificación del estado sanitario del árbol.
   - Consultar el análisis complementario de color.
   - Consultar registros recientes de la jornada.
   - Consultar el historial local de análisis de la jornada.
   - Consultar el mapa local de árboles.
   - Filtrar registros, árboles o mapas según el estado sanitario u otros criterios disponibles.
   - Consultar notificaciones y alertas.
   - Reintentar manualmente la sincronización de registros con error.
   - Consultar información consolidada desde la plataforma web.
   - Consultar historial general, mapa general, estadísticas y perfil de usuario en la web.
   - Cerrar sesión.

2. **Administrador**
   Persona encargada de gestionar las cuentas de usuario y los dispositivos asociados desde la plataforma web. Este actor no modifica los registros operativos capturados desde la aplicación móvil, sino que administra la información de acceso y configuración relacionada con las cuentas.
   - Iniciar sesión en la plataforma web.
   - Recuperar y restablecer contraseña.
   - Crear cuentas de usuario.
   - Editar cuentas de usuario.
   - Desactivar cuentas de usuario.
   - Asignar roles de usuario.
   - Gestionar dispositivos asociados a una cuenta.
   - Asociar dispositivos móviles a una cuenta.
   - Desasociar dispositivos móviles de una cuenta.
   - Consultar dispositivos asociados a una cuenta.
   - Cerrar sesión.

7.5 Métricas de evaluación del proyecto

La evaluación del proyecto se plantea considerando tres niveles: el desempeño del modelo de clasificación, el desempeño técnico del sistema y el cumplimiento de los criterios de aceptación del prototipo. Esta división permite valorar tanto la calidad del análisis automático como la funcionalidad general de la aplicación móvil y la plataforma web.

7.5.1 Métricas de desempeño del modelo

Para evaluar el modelo de clasificación se consideran métricas que permitan medir su capacidad para identificar correctamente el estado sanitario a partir de las imágenes capturadas. Las principales métricas son:

- **Exactitud:** mide el porcentaje de predicciones correctas respecto al total de imágenes evaluadas.
- **Precisión:** indica qué proporción de las imágenes clasificadas como positivas realmente corresponden a dicha clase.
- **Sensibilidad:** mide la capacidad del modelo para detectar correctamente casos con presencia de daño o riesgo.
- **F1-score:** combina precisión y sensibilidad en una sola métrica, útil cuando existen diferencias entre clases.
- **Matriz de confusión:** permite observar los aciertos y errores del modelo por cada clase evaluada.

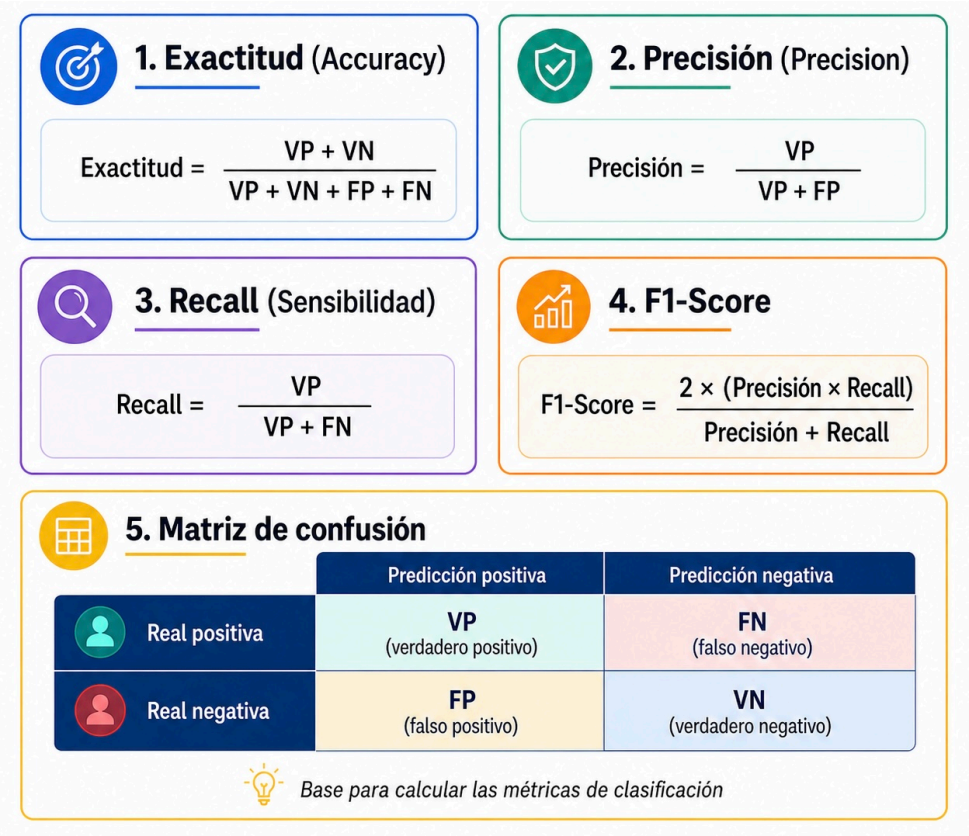


Figura 14: *Métricas de evaluación del modelo de clasificación*

Estas métricas permiten determinar si el modelo es adecuado para apoyar la detección de posibles afectaciones por insectos descortezadores, considerando imágenes de corteza de arboles.

7.5.2 Métricas de desempeño del sistema

El desempeño del sistema se evalúa considerando la respuesta de la aplicación móvil, la operación offline y el proceso de sincronización con la nube. Las métricas propuestas son:

- **Tiempo de respuesta del análisis:** tiempo transcurrido desde que se captura o selecciona una imagen hasta que se muestra el resultado del análisis.
- **Tiempo de guardado local:** tiempo requerido para almacenar un análisis en la base local SQLite.
- **Tiempo de sincronización:** tiempo necesario para enviar registros pendientes desde la aplicación móvil hacia la nube.
- **Tasa de sincronización exitosa:** porcentaje de registros locales que se sincronizan correctamente.
- **Disponibilidad offline:** capacidad de la aplicación móvil para capturar, analizar y guardar registros sin conexión a internet.
- **Consistencia de datos:** verificación de que los registros sincronizados en la nube coincidan con los registros generados localmente.

Estas métricas permiten comprobar que el sistema cumple con el enfoque *offline-first* y que la información generada en campo puede conservarse y sincronizarse correctamente.

7.5.3 Criterios de aceptación del prototipo

Los criterios de aceptación definen las condiciones mínimas que debe cumplir el prototipo para considerarse funcional. Para este proyecto se establecen los siguientes criterios:

- El usuario puede iniciar sesión y acceder a las funciones correspondientes a su rol.
- La aplicación móvil permite capturar o importar imágenes para realizar análisis.
- El sistema permite seleccionar el tipo de análisis: follaje o corteza.
- Los análisis de corteza pueden asociarse a un árbol nuevo o existente.
- Los análisis de follaje pueden asociarse a una zona de monitoreo.
- El sistema genera un resultado de clasificación y un nivel de confianza.
- La aplicación almacena localmente los análisis realizados sin conexión a internet.
- Los registros pendientes se sincronizan con la nube cuando existe conexión disponible.
- La plataforma web permite consultar registros, mapas, historial y estadísticas consolidadas.
- La evidencia fotográfica puede consultarse desde la plataforma web después de la sincronización.

El cumplimiento de estos criterios permitirá validar que el prototipo cubre las funcionalidades esenciales definidas para el monitoreo forestal mediante aplicación móvil y plataforma web.

7.6 Análisis de riesgos y restricciones

El desarrollo de un prototipo de detección de insectos descortezadores mediante visión por computadora implica riesgos en distintos niveles: desde la calidad y representatividad de los datos de entrenamiento, hasta las condiciones reales de operación en campo dentro del Bosque de Chapultepec. A continuación se identifican los principales riesgos técnicos y operativos, así como las restricciones inherentes a la naturaleza del prototipo.

7.6.1 Riesgos asociados al conjunto de datos

La calidad del modelo de clasificación depende directamente de la calidad y diversidad del dataset utilizado. Se identifican los siguientes riesgos:

- **Sesgo de dominio.** Los datasets utilizados en la prueba preliminar (*bark-complete* y *bark_infected*) provienen de condiciones de captura distintas a las del Bosque de Chapultepec: diferentes especies, ángulos de captura, condiciones de iluminación y etapas de infestación. Esto genera un sesgo de dominio que limita la capacidad del modelo para generalizar a imágenes capturadas en campo.
- **Desbalance entre etapas de infestación.** Las imágenes de corteza infestada pueden corresponder a distintas fases del proceso de colonización (galería inicial, salida de adultos, daño avanzado). Si el dataset no cubre todas las etapas de forma balanceada, el modelo tenderá a detectar únicamente los casos más evidentes, fallando en infestaciones tempranas que son las más relevantes para la intervención oportuna.
- **Variabilidad estacional y ambiental.** Las condiciones visuales de la corteza varían según la época del año, la humedad y la exposición solar. Un dataset capturado en una sola temporada puede no representar la apariencia de la corteza en otras condiciones, reduciendo la robustez del modelo en uso continuo.

7.6.2 Riesgos técnicos de implementación

Más allá del modelo, la integración del sistema en un entorno móvil y su operación en campo introducen riesgos técnicos adicionales:

- **Variabilidad en la captura de imágenes.** A diferencia de un dataset controlado, las imágenes capturadas por usuarios en campo presentan variaciones de distancia, ángulo, enfoque y exposición. El modelo puede degradar su rendimiento ante imágenes que no cumplan las condiciones mínimas de calidad, por lo que se requiere un módulo de validación de imagen previo a la inferencia.
- **Rendimiento del modelo en dispositivos de gama baja.** El archivo `bark_classifier.tflite` fue optimizado mediante cuantización float16 para reducir su tamaño y acelerar la inferencia en móvil. Sin embargo, en dispositivos con procesadores limitados o sin soporte de aceleración neuronal (NPU), los tiempos de inferencia pueden ser inaceptables para una experiencia de usuario fluida.
- **Deriva del modelo en el tiempo.** Las poblaciones de insectos descortezadores evolucionan y los patrones de daño pueden cambiar con el tiempo o ante nuevas especies invasoras. Un modelo entrenado estáticamente puede perder precisión progresivamente si no se implementa un mecanismo de reentrenamiento periódico con datos actualizados.
- **Integración con el backend y la plataforma web.** A nivel del sistema completo, la comunicación entre la aplicación móvil, el servidor de procesamiento y la plataforma web representa un punto de fallo potencial. Latencias elevadas o errores en la sincronización de registros

podrían afectar la trazabilidad histórica de los análisis realizados en campo.

7.6.3 Restricciones del prototipo

El presente desarrollo constituye un prototipo funcional con alcance delimitado, por lo que existen restricciones inherentes que deben considerarse al interpretar sus resultados y proyectar su evolución:

- **Dependencia de condiciones de captura.** El prototipo asume que el usuario captura la imagen en condiciones mínimas de calidad: distancia adecuada, iluminación suficiente y encuadre centrado en la corteza. No se implementa aún un mecanismo robusto de validación de imagen en tiempo real que guíe al usuario durante la captura.
- **Ausencia de uso sostenido por usuarios reales.** Al tratarse de un prototipo en etapa inicial, no ha sido sometido a uso continuo por parte de los técnicos o guardabosques que operarían el sistema en campo. Esta falta de uso sostenido dificulta la detección de errores de usabilidad, casos no contemplados durante el desarrollo, y mejoras necesarias que solo emergen con el uso real. En consecuencia, tanto el modelo como la interfaz podrían presentar deficiencias que no son visibles en condiciones de prueba controlada.
- **Cobertura geográfica restringida.** El prototipo está diseñado y validado para el Bosque de Chapultepec como caso de estudio. Su aplicación directa en otras áreas verdes de la Ciudad de México o del país requeriría validación adicional, dado que las especies arbóreas, condiciones ambientales y tipos de plaga pueden diferir significativamente.

7.6.4 Medidas de mitigación

Frente a los riesgos y restricciones identificados, se proponen las siguientes medidas para reducir su impacto durante el desarrollo del prototipo y orientar su evolución hacia un sistema más robusto:

- **Campañas de captura en campo.** Para mitigar el sesgo de dominio y el volumen insuficiente de datos, se recolectan imágenes directamente en el Bosque de Chapultepec, cubriendo las especies arbóreas presentes, distintas etapas de infestación y variadas condiciones de iluminación y temporada. Esto permitirá construir un dataset representativo del dominio real de operación.
- **Aumento de datos orientado al dominio.** Complementariamente a la captura en campo, se aplicarán técnicas de aumento de datos que simulen condiciones reales: variaciones de brillo, rotaciones, recortes aleatorios y ajustes de color, con el objetivo de incrementar artificialmente la diversidad del conjunto de entrenamiento sin requerir captura adicional para cada variante.
- **Validación de imagen previa a la inferencia.** Se implementará un módulo de preprocesamiento que ajuste la imagen capturada (nitidez, exposición y dimensiones) antes de ejecutar la inferencia, reduciendo predicciones erróneas por imágenes de baja calidad.
- **Modo offline en la aplicación móvil.** Para garantizar la operación en zonas del bosque con conectividad limitada, la aplicación almacenará localmente los registros generados y los sincronizará con la plataforma web una vez restablecida la conexión, asegurando la trazabilidad de los análisis sin depender de red en tiempo real.
- **Pruebas con usuarios reales en campo.** Se planea involucrar a técnicos forestales o personal del bosque en sesiones de prueba, con el objetivo de identificar errores de usabilidad y casos no contemplados que solo emergen con el uso real. La información recopilada retroalimentará tanto el diseño de la interfaz como los criterios de reentrenamiento del modelo.

7.7 Plan general de pruebas

El presente plan general de pruebas tiene como finalidad validar el comportamiento del prototipo en sus principales niveles de funcionamiento: el modelo de clasificación, las funcionalidades del sistema y la integración entre los componentes de la arquitectura. Debido a que la solución combina una aplicación móvil, inferencia con TensorFlow Lite, procesamiento local, servicios en la nube y una plataforma web, resulta necesario definir pruebas que permitan comprobar tanto la precisión del modelo como la correcta operación del sistema completo.

7.7.1 Pruebas del modelo de clasificación

Como parte del proceso de desarrollo, se realizó una prueba preliminar del modelo de clasificación binaria utilizando EfficientNet-B3 con un subconjunto reducido de datos: 1,500 imágenes por clase (corteza sana y corteza infestada), balanceadas manualmente a partir de los conjuntos disponibles. El objetivo de esta prueba no fue obtener un modelo definitivo, sino validar la viabilidad del enfoque y establecer una línea base para el entrenamiento final.

Figura 15: *Curvas de entrenamiento con dataset de prueba de 1,500 imágenes por clase.*

Las curvas de entrenamiento (Figura 15) muestran un comportamiento prometedor: la exactitud y el AUC del conjunto de entrenamiento mejoran de forma sostenida a lo largo de las dos fases, alcanzando valores superiores al 90% en entrenamiento. Sin embargo, las métricas de validación siguen una tendencia positiva pero con una brecha visible respecto al conjunto de entrenamiento, especialmente durante la fase de fine-tuning. Esta separación entre curvas de entrenamiento y validación indica que el modelo aprende representaciones útiles, pero la variabilidad limitada del dataset restringe su capacidad de generalización. El comportamiento no refleja una limitación de la arquitectura, sino una restricción directa en el volumen y diversidad de muestras disponibles.

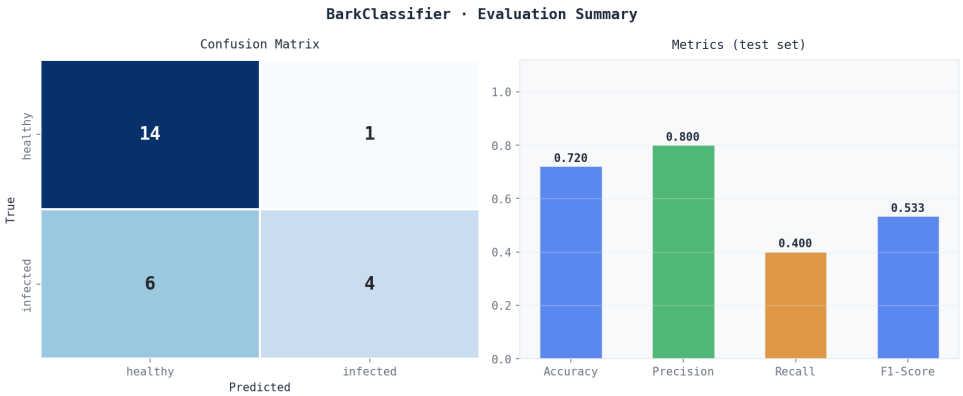


Figura 16: *Evaluación del modelo sobre el conjunto de validación.*

Los resultados de evaluación (Figura 16) reportan una exactitud del 72%, con una precisión del 80% y un recall del 40%. La precisión indica que cuando el modelo predice infestación, acierta en 4 de cada 5 casos; sin embargo, el recall bajo revela que detecta únicamente la mitad de los árboles realmente infestados. Este patrón es característico de modelos entrenados con datos insuficientes para la clase positiva: el modelo es conservador al emitir predicciones de infestación, lo que reduce los falsos positivos a costa de aumentar los falsos negativos. En el contexto de detección de plagas, mejorar el recall es prioritario, ya que un árbol infestado no detectado representa un riesgo de propagación.

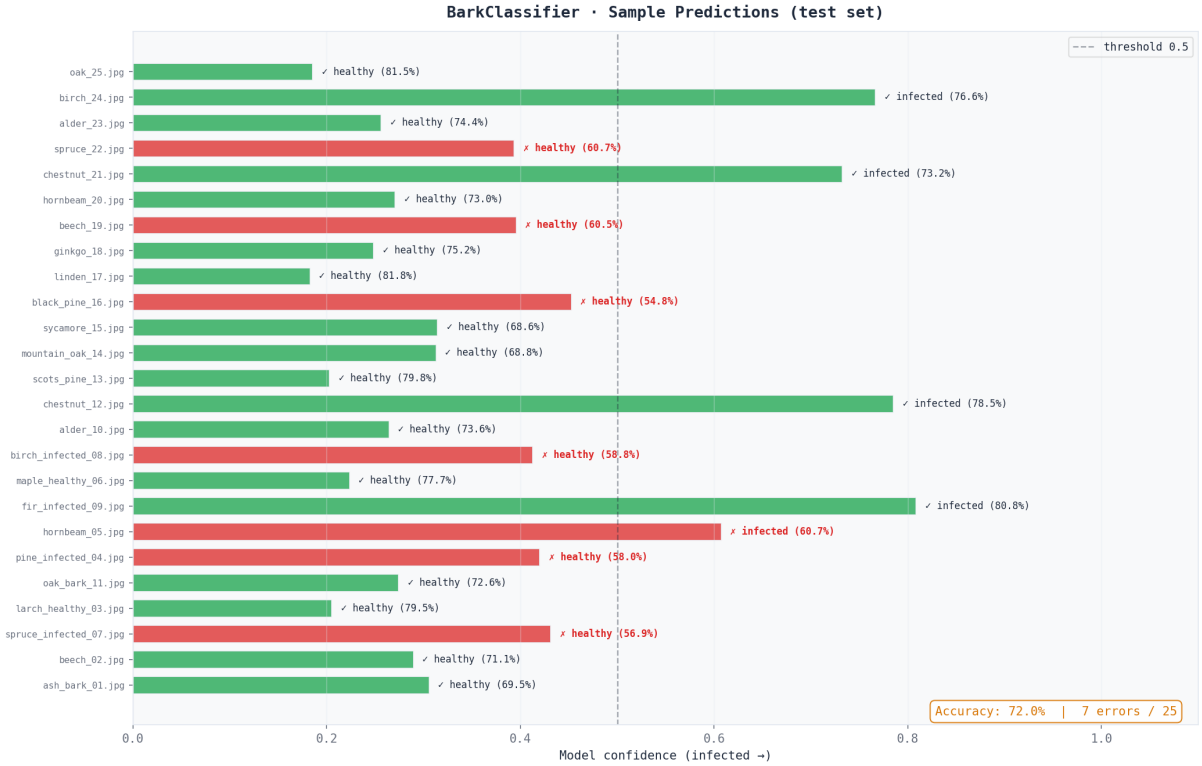


Figura 17: *Predicciones sobre imágenes no vistas durante el entrenamiento.*

Las predicciones sobre imágenes nuevas (Figura 17) confirman lo observado: el modelo opera con niveles de confianza moderados, generalmente entre el 68% y el 85% en los aciertos, mientras que los errores se concentran cerca del umbral de decisión (51%–62%), lo que indica que el modelo duda en casos ambiguos en lugar de cometer errores categóricos. De 25 muestras, 7 fueron clasificadas incorrectamente, todas con confianza baja. Este comportamiento es una señal positiva: sugiere que el modelo ha aprendido representaciones parcialmente discriminativas y que, con mayor volumen de datos, la frontera de decisión se definiría con mayor claridad.

Se estima que incrementar el conjunto de entrenamiento a entre 10,000 y 20,000 imágenes por clase permitiría al modelo mejorar su generalización y alcanzar niveles de exactitud y recall mejores.

7.7.2 Pruebas del sistema

Las pruebas del sistema verificarán que cada funcionalidad del sistema cumpla con los requerimientos funcionales definidos en la sección 7.1.1, evaluando el comportamiento del sistema desde la perspectiva del usuario final. Dado que el sistema se encuentra en fase de diseño, esta sección presenta el plan de pruebas funcionales que se ejecutará durante la fase de implementación del prototipo.

Herramientas utilizadas:

- **Espresso:** Framework oficial de Google para pruebas de interfaz de usuario en Android. Se utilizará para verificar los flujos de navegación, la captura de imágenes, la visualización de resultados y el comportamiento offline de la aplicación móvil desarrollada con Kotlin y Jetpack Compose.
- **Postman + Newman:** Se utilizará para diseñar y ejecutar colecciones de pruebas sobre los endpoints REST expuestos por Amazon API Gateway. Newman permitirá automatizar la ejecución de estas pruebas desde el pipeline de GitHub Actions.

La Tabla 7 presenta los casos de prueba funcionales planificados para el sistema.

Tabla 7: *Plan de pruebas funcionales del sistema.*

ID	Caso de prueba	Pasos	Resultado esperado
PT-01	Login exitoso (app móvil)	Ingresar email y contraseña válidos en la pantalla de login.	El sistema autentica al usuario y muestra la pantalla principal.
PT-02	Login fallido (credenciales incorrectas)	Ingresar credenciales incorrectas.	El sistema muestra el mensaje MSG03 y no permite el acceso.
PT-03	Captura de imagen desde cámara	Presionar ”Nueva Captura” y tomar fotografía.	La imagen se captura, se procesa con TFLite y se muestra el resultado de clasificación.
PT-04	Clasificación offline	Con conexión desactivada, capturar imagen de árbol.	El modelo TFLite genera resultado etiquetado como **Offline** sin requerir internet.
PT-05	Guardado local en SQLite	Realizar captura sin conexión a internet.	El registro se guarda en SQLite con estado **Pendiente de sincronización**.
PT-06	Sincronización automática	Activar conexión a internet con registros pendientes.	El sistema sincroniza automáticamente y actualiza el estado a **Sincronizado**.
PT-07	Sincronización manual	Con registros en estado **Error de sincronización**, presionar botón de reintento.	El sistema reintenta la sincronización y actualiza el estado.

ID	Caso de prueba	Pasos	Resultado esperado
PT-08	Alerta de proximidad a foco	Acercarse a coordenadas de un foco de infestación registrado.	El sistema emite notificación de proximidad sin requerir internet.
PT-09	Resumen de jornada	Consultar el resumen de capturas del día.	Se muestran correctamente el total de capturas, árboles sanos e infestados del día en curso.
PT-10	Login exitoso (plataforma web)	Ingresar credenciales válidas en la plataforma web.	Amazon Cognito emite token JWT y Angular redirige al dashboard.
PT-11	Consulta de historial web	Acceder al historial completo de detecciones desde la plataforma web.	Se muestran todos los registros sincronizados sin restricción de tiempo.
PT-12	Generación de reporte PDF	Seleccionar rango de fechas y generar reporte desde la plataforma web.	El sistema genera y permite descargar el reporte en formato PDF.
PT-13	Visualización de mapa	Consultar el mapa general de árboles registrados.	Se muestran los marcadores en las coordenadas correctas con su estado sanitario.
PT-14	Gestión de usuarios (administrador)	Crear cuenta de experto de campo desde la plataforma web.	La cuenta se crea correctamente y el usuario puede iniciar sesión.
PT-15	Descarga de modelo TFLite	Conectar dispositivo a internet con versión de modelo desactualizada.	El sistema descarga la nueva versión del modelo desde Amazon S3.

7.7.3 Pruebas de integración

Las pruebas de integración verificarán la correcta comunicación entre los componentes del sistema, asegurando que los datos fluyan de forma íntegra y consistente a través de las distintas capas de la arquitectura. Esta sección presenta el plan de pruebas de integración que se ejecutará durante la fase de implementación.

Herramientas utilizadas:

- **Micronaut Test:** Framework oficial de Micronaut para pruebas de integración del backend. Permite levantar el contexto completo de la aplicación en un entorno de pruebas controlado, verificando el comportamiento real de los controladores REST, casos de uso y repositorios.
- **Testcontainers:** Biblioteca que levanta instancias reales de PostgreSQL en contenedores Docker durante la ejecución de las pruebas, eliminando la necesidad de mocks de base de datos y garantizando que las pruebas reflejen el comportamiento real del sistema en producción.
- **JUnit 5:** Framework base para la definición y ejecución de los casos de prueba de integración del backend.
- **Newman:** Ejecutor de colecciones Postman desde línea de comandos, integrado en el pipeline de GitHub Actions para automatizar las pruebas de los endpoints REST en cada despliegue.

La Tabla 8 presenta los casos de prueba de integración planificados.

Tabla 8: *Plan de pruebas de integración del sistema.*

ID	Caso de prueba	Componentes involucrados	Resultado esperado
PI-01	Sincronización móvil → API Gateway	App móvil (Retrofit) → API Gateway	La petición POST /sync llega correctamente a API Gateway con el token JWT en el header.
PI-02	Validación JWT en API Gateway	API Gateway → Amazon Cognito	API Gateway valida el token JWT con Cognito y rechaza peticiones sin token con HTTP 401.
PI-03	API Gateway → Lambda	API Gateway → AWS Lambda (Micronaut)	La función Lambda se invoca correctamente y procesa la petición en menos de 5 segundos incluyendo cold start.
PI-04	Lambda → RDS (escritura)	AWS Lambda → Amazon RDS (PostgreSQL)	Los datos de detección se persisten correctamente en PostgreSQL mediante Micronaut Data JPA y Testcontainers.
PI-05	Lambda → S3 (carga de imagen)	AWS Lambda → Amazon S3	La imagen se carga correctamente en S3 y se retorna la URL pública para su almacenamiento en RDS.

Tabla 8 – continuación

ID	Caso de prueba	Componentes involucrados	Resultado esperado
PI-06	Lambda → RDS (lectura)	AWS Lambda → Amazon RDS	La consulta del historial de detecciones retorna los registros correctos desde PostgreSQL.
PI-07	SageMaker → S3 (publicación modelo)	Amazon SageMaker → Amazon S3	El modelo `.tflite` se publica correctamente en S3 y es accesible para descarga desde el dispositivo móvil.
PI-08	App móvil → S3 (descarga modelo)	App móvil → Amazon S3	El dispositivo descarga correctamente la nueva versión del modelo `.tflite` y lo reemplaza localmente.
PI-09	Angular → API Gateway (petición protegida)	Angular (HttpInterceptor) → API Gateway	El interceptor adjunta el JWT correctamente y la petición recibe respuesta HTTP 200 con datos.
PI-10	CloudFront → S3 (frontend)	Amazon CloudFront → Amazon S3	CloudFront sirve los artefactos estáticos de Angular con HTTPS y caché correctamente configurado.
PI-11	Flujo completo de sincronización	App móvil → API Gateway → Lambda → RDS → S3	El ciclo completo de sincronización se ejecuta sin errores y el estado en SQLite cambia a **Sincronizado**.
PI-12	Flujo completo de consulta web	Angular → API Gateway → Lambda → RDS	La plataforma web consulta y muestra correctamente el historial de detecciones sincronizadas.
PI-13	TFLite + OpenCV en dispositivo	TFLite (EfficientNet-B3) → OpenCV (análisis cromático)	El modelo TFLite y el análisis cromático de OpenCV se ejecutan de forma concurrente y retornan resultados en menos de 3 segundos sin conexión a internet.

Tabla 8 – continuación

ID	Caso de prueba	Componentes involucrados	Resultado esperado
PI-14	SQLite → Room (escritura local)	Room → SQLite	Los registros de detección se persisten correctamente en SQLite mediante Room, incluyendo estado, coordenadas, timestamp y referencia de imagen.
PI-15	SQLite → Room (lectura y filtrado)	Room → SQLite	Las consultas de registros pendientes de sincronización retornan únicamente los registros con estado **Pendiente** o **Error de sincronización**.
PI-16	Token refresh en Angular	Angular (HttpInterceptor) → Amazon Cognito	Cuando el Access Token expira, el HttpInterceptor solicita automáticamente un nuevo token a Cognito usando el Refresh Token sin interrumpir la sesión del usuario.
PI-17	D3.js con datos reales de la API	Angular (D3.js) → API Gateway → Lambda → RDS	Las visualizaciones de D3.js renderizan correctamente los datos de distribución de focos de infestación y evolución temporal obtenidos desde la API.

7.7.4 Recursos disponibles

El desarrollo del sistema de detección de insectos descortezadores requiere la integración de un conjunto heterogéneo de tecnologías que abarcan desde el entrenamiento de modelos de aprendizaje profundo hasta el despliegue de infraestructura en la nube y el desarrollo de aplicaciones móviles nativas. Las herramientas seleccionadas responden a criterios de madurez tecnológica, compatibilidad entre componentes, disponibilidad de licencias gratuitas para uso académico y adecuación al enfoque offline-first del sistema.

En las siguientes subsecciones se describen las tecnologías empleadas organizadas por capa del sistema: herramientas para el desarrollo del modelo de clasificación, procesamiento de imágenes, desarrollo de la aplicación móvil, desarrollo de la aplicación web e infraestructura en la nube.

7.7.5 Herramientas para el desarrollo del modelo

El modelo de clasificación del sistema se desarrollará utilizando Python como lenguaje principal, apoyándose en un conjunto de herramientas especializadas en aprendizaje profundo y visión por computadora.

TensorFlow y Keras constituirán el núcleo del proceso de entrenamiento. TensorFlow es un framework de código abierto desarrollado por Google, ampliamente utilizado en la industria y la academia para la construcción y entrenamiento de modelos de aprendizaje profundo. Keras actúa como interfaz de alto nivel sobre TensorFlow, simplificando la definición de arquitecturas de redes neuronales convolucionales como EfficientNet-B3, seleccionada para la clasificación de imágenes de árboles en este proyecto.

TensorFlow Lite es la versión optimizada de TensorFlow orientada a dispositivos móviles y embebidos. Una vez entrenado el modelo con Keras, este se convertirá al formato `.tflite`, reduciendo su tamaño y optimizando su rendimiento para ejecutarse directamente en el dispositivo Android sin requerir conexión a internet. Esta conversión es fundamental para el enfoque offline-first del sistema.

Scikit-learn se empleará como herramienta complementaria durante el proceso de entrenamiento y evaluación del modelo. Aportará métricas de evaluación como precisión, recall, F1-score y matriz de confusión, permitiendo analizar el desempeño del modelo de clasificación de forma rigurosa antes de su conversión a formato TFLite.

Amazon SageMaker se empleará como plataforma de registro y versionado del modelo. Una vez que el modelo sea entrenado en los equipos de cómputo disponibles, se registrará en SageMaker Model Registry para llevar control de versiones y garantizar la trazabilidad de cada iteración antes de publicarlo en Amazon S3 para su distribución al dispositivo móvil.

El entrenamiento se realizará sobre los equipos de cómputo del equipo de desarrollo, aprovechando las capacidades de procesamiento local disponibles para la experimentación y ajuste de hiperparámetros del modelo.

7.7.6 Herramientas para el procesamiento de imágenes

El procesamiento de imágenes del sistema se apoyará en herramientas especializadas que permitirán preparar, transformar y analizar las imágenes tanto durante el entrenamiento del modelo como en la ejecución en el dispositivo móvil.

OpenCV será la biblioteca principal para el procesamiento de imágenes en el sistema. Durante el entrenamiento, permitirá realizar operaciones de redimensionamiento, conversión de espacios de color, normalización de píxeles y transformaciones geométricas sobre las imágenes del conjunto de datos, facilitando la construcción de pipelines de preprocesamiento compatibles con EfficientNet-B3.

En la aplicación móvil, OpenCV se utilizará para ejecutar el análisis complementario de color de forma local en el dispositivo. Este análisis clasificará los píxeles de la imagen en rangos cromáticos asociados a las distintas fases de infestación documentadas, tanto para imágenes de follaje como de corteza, operando de forma concurrente al modelo TFLite y sin requerir conexión a internet, tal como se establece en la regla de negocio RN-46.

7.7.7 Herramientas para la aplicación móvil

La aplicación móvil se desarrollará de forma nativa para el sistema operativo Android, utilizando un conjunto de herramientas modernas que garantizan rendimiento, mantenibilidad y compatibilidad con el enfoque offline-first del sistema.

Kotlin es el lenguaje de programación oficial para el desarrollo de aplicaciones Android, respaldado por Google desde 2017. Su sintaxis concisa, su interoperabilidad con Java 21 y su soporte nativo para corrutinas lo convierten en la opción más adecuada para el desarrollo de aplicaciones móviles modernas que requieren manejo eficiente de operaciones asíncronas como la sincronización de datos y la inferencia del modelo.

Jetpack Compose es el framework moderno de Google para la construcción de interfaces de usuario en Android. A diferencia del enfoque tradicional basado en XML, Jetpack Compose permite definir la interfaz de forma declarativa en Kotlin, reduciendo la cantidad de código necesario y facilitando la creación de interfaces reactivas que respondan al estado de la aplicación en tiempo real.

TensorFlow Lite se integrará directamente en la aplicación para ejecutar el modelo de clasificación EfficientNet-B3 de forma local en el dispositivo. El modelo en formato `.tflite` se almacenará en el dispositivo y se actualizará ocasionalmente desde Amazon S3 cuando exista una nueva versión disponible.

Room es la biblioteca de persistencia oficial de Android, construida sobre SQLite. Se utilizará para gestionar la base de datos local del dispositivo, almacenando los registros de árboles, capturas, resultados de clasificación y estados de sincronización de forma estructurada y eficiente mientras el dispositivo opera sin conexión a internet.

MapLibre GL Android es la biblioteca de código abierto utilizada para la visualización de mapas vectoriales dentro de la aplicación. Permite renderizar mapas de forma completamente offline a partir de un archivo `.pmtiles` descargado localmente en el dispositivo, el cual contiene los datos cartográficos de la zona de operación. Sobre este mapa se renderizan pines de ubicación generados a partir de las coordenadas GPS extraídas de los metadatos EXIF de cada imagen capturada, sin requerir conexión a internet en ningún momento durante su uso.

Retrofit y OkHttp conforman el cliente HTTP que gestionará la comunicación entre la aplicación móvil y la API REST en la nube durante la sincronización de datos. Retrofit facilita la definición de endpoints REST de forma declarativa, mientras que OkHttp actúa como cliente HTTP subyacente, gestionando las conexiones, los tiempos de espera y los reintentos automáticos.

La autenticación del experto de campo se gestionará mediante **JWT propio**, generado por el backend al momento del inicio de sesión y almacenado localmente en el dispositivo. Este token se incluirá en cada petición de sincronización, permitiendo que la aplicación opere de forma segura sin depender de una conexión permanente a internet.

7.7.8 Herramientas para la aplicación web

La aplicación web se desarrollará como una Single Page Application (SPA) orientada a la visualización y administración de los datos sincronizados desde los dispositivos móviles. El stack seleccionado prioriza la integración nativa entre herramientas, la mantenibilidad del código y la capacidad de representar datos geoespaciales y estadísticos de forma clara.

Angular es el framework principal para el desarrollo del frontend. Desarrollado y mantenido por Google, Angular ofrece una arquitectura basada en componentes, inyección de dependencias nativa y un sistema de módulos que facilita la organización del código en aplicaciones de mediana y gran escala. Su integración con TypeScript garantiza tipado estático y detección temprana de errores durante el desarrollo.

Angular Material es la biblioteca oficial de componentes de interfaz de usuario para Angular, basada en los principios de Material Design. Proveerá los componentes visuales del sistema como tablas, formularios, diálogos, navegación y tarjetas, asegurando una interfaz consistente, accesible y alineada con los estándares modernos de diseño.

D3.js se utilizará para la visualización de datos estadísticos y geoespaciales del sistema. Su capacidad para construir visualizaciones personalizadas basadas en datos permitirá representar la distribución de focos de infestación, la evolución temporal del estado sanitario del arbolado y las estadísticas generales del sistema de forma interactiva y precisa.

MapLibre GL JS es la biblioteca de código abierto utilizada para la visualización de mapas vectoriales en la aplicación web. Permite renderizar mapas interactivos directamente en el navegador y representar mediante pines georreferenciados las ubicaciones de los registros sincronizados desde los dispositivos móviles, facilitando la supervisión espacial del estado sanitario del arbolado desde la plataforma de administración.

HttpClient es el módulo nativo de Angular para la gestión de peticiones HTTP. Se utilizará para consumir los endpoints REST expuestos por Amazon API Gateway, gestionando la comunicación entre el frontend y el backend de forma eficiente mediante observables y soporte para interceptores, los cuales permitirán adjuntar automáticamente el token de autenticación de Amazon Cognito en cada petición.

Amazon Cognito gestionará la autenticación de los usuarios de la plataforma web, incluyendo los roles de Experto de campo y Administrador. Proveerá el flujo de inicio de sesión, recuperación de contraseña y gestión de tokens JWT, eliminando la necesidad de implementar estos mecanismos de forma manual en el backend.

La seguridad en las comunicaciones entre el cliente web y los servicios en la nube se garantizará mediante **AWS Certificate Manager (ACM)**, el servicio de Amazon que provee y renueva automáticamente certificados SSL/TLS sin costo adicional. ACM se integrará con Amazon CloudFront y Amazon API Gateway para asegurar que toda la comunicación se realice a través de HTTPS, cumpliendo con el requerimiento no funcional RNF-05.

7.7.9 Infraestructura en la nube

La infraestructura del sistema se desplegará sobre Amazon Web Services (AWS), adoptando un modelo serverless que elimina la necesidad de gestionar servidores y permite escalar automáticamente según la demanda. La elección de AWS como proveedor de nube se fundamenta en un conjunto de factores técnicos, económicos y estratégicos que lo posicionan como la opción más adecuada para el desarrollo y evolución del prototipo, tal como se justifica en detalle en la sección 7.8.5.

Capa gratuita (AWS Free Tier). AWS ofrece una capa gratuita que cubre los servicios principales utilizados en este proyecto. A diferencia de otros proveedores, AWS distingue entre dos tipos de ofertas gratuitas [32]:

- **Siempre gratuitos (Always Free):** Sin fecha de expiración, disponibles indefinidamente mientras se respeten los límites mensuales establecidos. Los servicios del sistema cubiertos bajo esta categoría son:
  - **AWS Lambda:** 1 millón de invocaciones y 400,000 GB-segundos de cómputo al mes.
  - **Amazon API Gateway:** 1 millón de llamadas REST al mes.
  - **Amazon CloudFront:** 1 TB de transferencia de datos y 10 millones de solicitudes HTTP/S al mes.
  - **Amazon S3:** 5 GB de almacenamiento estándar, 20,000 solicitudes GET y 2,000 solicitudes PUT al mes.
  - **Amazon Cognito:** 50,000 usuarios activos mensuales sin costo.
- **Gratuitos por 12 meses:** Disponibles desde la creación de la cuenta AWS. En esta categoría se encuentra **Amazon RDS**, que ofrece 750 horas mensuales de instancia `db.t2.micro` con PostgreSQL, suficiente para cubrir la fase de desarrollo y pruebas del prototipo.

Esta disponibilidad permite desarrollar, desplegar y validar el prototipo completo sin incurrir en costos de infraestructura durante la fase de desarrollo, lo cual representa una ventaja significativa para un proyecto académico con recursos limitados.

Modelo de pago por uso. Fuera de la capa gratuita, AWS opera bajo un modelo de pago por uso donde únicamente se factura por los recursos efectivamente consumidos, sin compromisos de gasto mínimo ni contratos de largo plazo. Esto permite que el costo del sistema crezca de forma proporcional al uso real. Por ejemplo, AWS Lambda cobra $0.20 USD por cada millón de invocaciones adicionales y $0.0000166667 USD por GB-segundo de cómputo, lo que resulta en costos mínimos para un prototipo con tráfico reducido [33].

Facilidad de uso y ecosistema integrado. AWS provee una consola de administración unificada, documentación extensa y una amplia comunidad de soporte. La integración nativa entre sus servicios reduce considerablemente la complejidad operativa: API Gateway invoca Lambda directamente, Lambda accede a RDS y S3 dentro de la misma región sin costos de transferencia adicionales, y CloudFront sirve el contenido de S3 de forma transparente. Esta cohesión entre servicios acelera los tiempos de desarrollo y despliegue del sistema.

Escalabilidad automática. El modelo serverless adoptado mediante AWS Lambda escala automáticamente en función de la demanda, creando instancias adicionales de forma transparente sin intervención manual. Este comportamiento es especialmente relevante considerando que el sistema podría extenderse a otros bosques o reservas naturales en el futuro, incrementando el número de dispositivos móviles que sincronizan datos simultáneamente.

Disponibilidad global y baja latencia. AWS cuenta con más de 30 regiones geográficas y más de 400 puntos de presencia (PoP) de CloudFront distribuidos globalmente [34]. Para el caso de estudio del Bosque de Chapultepec, la región `us-east-1` (Norte de Virginia) ofrece baja

latencia desde México con alta disponibilidad garantizada mediante zonas de disponibilidad redundantes.

Proyección a un ambiente productivo. El prototipo está diseñado desde su arquitectura para escalar hacia un ambiente productivo sin cambios estructurales. En una fase de producción, el sistema podría incorporar los siguientes servicios adicionales dentro del mismo ecosistema AWS:

- **Amazon RDS Multi-AZ:** Alta disponibilidad de la base de datos mediante replicación automática en múltiples zonas de disponibilidad.
- **AWS WAF:** Firewall de aplicaciones web para protección contra ataques como inyección SQL y cross-site scripting.
- **Amazon CloudWatch:** Monitoreo en tiempo real de métricas, logs y alertas operativas del sistema.
- **Amazon Route 53:** Gestión de dominios y enrutamiento de tráfico con alta disponibilidad.
- **AWS Budgets:** Control y alertas de costos para evitar gastos inesperados al superar los límites de la capa gratuita.

Esta trayectoria de crecimiento es posible gracias a que AWS ofrece todos estos servicios dentro del mismo ecosistema, sin necesidad de migrar a otro proveedor ni rediseñar la arquitectura base del sistema. El análisis comparativo detallado entre AWS, Microsoft Azure y Google Cloud Platform, así como la justificación económica y técnica de esta elección, se presentan en la sección 7.8.5 del análisis financiero.

A continuación se describen los servicios que conforman la arquitectura en la nube del sistema.

Amazon API Gateway actuará como punto de entrada unificado para todas las peticiones REST del sistema, tanto de la aplicación móvil durante la sincronización como de la aplicación web. Se encargará de enrutar cada petición hacia la función Lambda correspondiente, aplicando políticas de control de acceso y limitación de tasa.

AWS Lambda alojará el backend del sistema desarrollado con Micronaut. Al ser serverless, cada invocación es independiente y el escalado es automático, eliminando la necesidad de aprovisionar o gestionar servidores. El código se desplegará automáticamente mediante el pipeline de CI/CD configurado en GitHub Actions.

Amazon RDS (PostgreSQL) será el motor de base de datos relacional central del sistema. Almacenará de forma persistente los datos sincronizados desde los dispositivos móviles, incluyendo registros de árboles, capturas, resultados de clasificación, coordenadas GPS e información de usuarios.

Amazon S3 gestionará el almacenamiento de objetos no estructurados del sistema: las imágenes capturadas por los dispositivos móviles, los artefactos estáticos compilados de la aplicación Angular y el modelo en formato `.tflite` distribuido a los dispositivos.

Amazon CloudFront distribuirá globalmente el frontend Angular mediante una red de distribución de contenido (CDN), sirviendo los artefactos estáticos almacenados en S3 con baja latencia desde ubicaciones geográficamente cercanas al usuario.

Amazon Cognito gestionará la identidad y autenticación de los usuarios de la plataforma web, proveyendo flujos de inicio de sesión, recuperación de contraseña y emisión de tokens JWT para el control de acceso por roles.

Amazon SageMaker se empleará exclusivamente como plataforma de registro y versionado del modelo de clasificación. Permitirá llevar control de las versiones del modelo entrenado antes de

publicarlo en Amazon S3 para su distribución al dispositivo móvil, sin participar en el proceso de inferencia en tiempo real.

AWS Certificate Manager (ACM) proveerá y renovará automáticamente los certificados SSL/TLS asociados a CloudFront y API Gateway, garantizando que todas las comunicaciones entre los clientes y la nube se realicen a través de HTTPS, cumpliendo con el requerimiento no funcional RNF-05.

Amazon CloudWatch se empleará para el monitoreo del sistema durante el desarrollo y las pruebas del prototipo. Permitirá consultar los logs generados por las funciones Lambda, monitorear el uso de Amazon RDS y recibir alertas ante errores o comportamientos inesperados del backend. Dentro de la capa gratuita, CloudWatch ofrece 5 GB de ingesta de logs y 10 métricas personalizadas al mes sin costo adicional [32].

GitHub Actions automatizará el pipeline de integración y entrega continua (CI/CD) del backend. Ante cada cambio en el repositorio, ejecutará las pruebas automatizadas, compilará el proyecto Micronaut y desplegará la nueva versión en AWS Lambda de forma transparente.

Adicionalmente, el sistema contará con un **servidor local Raspberry Pi** utilizado como entorno de pruebas durante el desarrollo, permitiendo validar el comportamiento del backend y la sincronización de datos en un entorno controlado antes del despliegue en la nube.

7.8 Análisis financiero

El análisis financiero del proyecto contempla los recursos disponibles para su desarrollo, los costos estimados de infraestructura física, los costos asociados a los servicios en la nube y una evaluación de la viabilidad económica del prototipo. Adicionalmente, se incluye una justificación de la selección de AWS como proveedor de nube frente a sus principales competidores, así como una proyección de costos para un ambiente productivo.

7.8.1 Recursos disponibles

El desarrollo del prototipo se llevará a cabo con recursos propios del equipo de trabajo, conformado por tres integrantes. A continuación se describen los recursos disponibles para el proyecto.

Recursos humanos. El equipo está integrado por tres desarrolladores estudiantes de Ingeniería en Sistemas Computacionales, quienes aportarán su tiempo y conocimiento en las áreas de desarrollo móvil, backend, infraestructura en la nube y aprendizaje automático. El trabajo se realizará de forma colaborativa utilizando equipos de cómputo personales de cada integrante.

Recursos de hardware. Los recursos físicos disponibles para el proyecto son los siguientes:

- **Equipos de cómputo personales:** Cada integrante cuenta con su propio equipo para el desarrollo, entrenamiento del modelo y pruebas del sistema.
- **Servidor local Raspberry Pi con disco duro externo:** Adquirido con un costo de $5,000 MXN, se utilizará como entorno de pruebas local para validar el comportamiento del backend antes del despliegue en la nube.
- **Dron:** Adquirido con un costo de $3,000 MXN, se utilizará para la captura aérea de imágenes del arbolado del Bosque de Chapultepec durante las jornadas de recolección del conjunto de datos.
- **Dispositivo Android:** Disponible para pruebas de la aplicación móvil en condiciones reales de campo.

Recursos de software. Todas las herramientas de desarrollo seleccionadas son de código abierto o cuentan con licencias gratuitas para uso académico, incluyendo Android Studio, IntelliJ IDEA Community, Python, TensorFlow, OpenCV, Scikit-learn, Angular CLI y GitHub Actions. Esto elimina cualquier costo asociado a licencias de software durante el desarrollo del prototipo.

7.8.2 Costos estimados de infraestructura

La Tabla 9 presenta el desglose de los costos estimados de infraestructura física del proyecto, considerando los recursos adquiridos y los gastos operativos durante el desarrollo del prototipo.

Tabla 9: *Costos estimados de infraestructura física del proyecto.*

Concepto	**Cantidad**	**Costo unitario (MXN)**	**Total (MXN)**
Raspberry Pi + disco duro externo	1	$5,000.00	$5,000.00
Dron para captura de imágenes	1	$3,000.00	$3,000.00

Tabla 9 – continuación

Concepto	Cantidad	Costo unitario (MXN)	Total (MXN)
Transporte al Bosque de Chapultepec (metro, por integrante por visita)	3 integrantes × 8 visitas	$10.00	$240.00
Consumo eléctrico estimado (3 equipos, 8 hrs/día, 8 meses)	—	—	$2,400.00
Consumo eléctrico Raspberry Pi (continuo, 8 meses)	—	—	$320.00
Total			**$10,960.00**

El consumo eléctrico de los equipos de cómputo se estimó considerando un consumo promedio de 150W por equipo, 8 horas diarias de uso durante aproximadamente 200 días de desarrollo, a una tarifa promedio de $1.00 MXN por kWh correspondiente a la tarifa doméstica de la Comisión Federal de Electricidad (CFE). El consumo de la Raspberry Pi se estimó en 5W de consumo continuo durante los 8 meses de desarrollo del prototipo.

El costo de transporte considera el uso de transporte público (metro) con un costo aproximado de $10.00 MXN por viaje por integrante, con más de 8 visitas planificadas al Bosque de Chapultepec para la recolección del conjunto de datos de imágenes.

7.8.3 Costos asociados a servicios en la nube

La justificación detallada de la selección de AWS como proveedor de infraestructura en la nube, incluyendo la comparativa con Microsoft Azure y Google Cloud Platform, se presenta en la sección 7.8.5.

Los servicios en la nube del sistema se desplegarán sobre Amazon Web Services (AWS), aprovechando la capa gratuita disponible durante la fase de prototipo. La Tabla 10 detalla los límites gratuitos de cada servicio utilizado y su aplicabilidad al prototipo.

Tabla 10: *Límites de la capa gratuita de AWS aplicables al prototipo.*

Servicio	**Límite gratuito**	**Tipo**	**Costo estimado**
AWS Lambda	1M invocaciones/mes y 400,000 GB-seg/mes	Siempre gratuito	$0.00 USD
Amazon API Gateway	1M llamadas REST/mes	Siempre gratuito	$0.00 USD
Amazon S3	5 GB almacenamiento, 20,000 GET, 2,000 PUT/mes	Siempre gratuito	$0.00 USD
Amazon CloudFront	1 TB transferencia y 10M solicitudes/mes	Siempre gratuito	$0.00 USD
Amazon Cognito	50,000 usuarios activos/mes	Siempre gratuito	$0.00 USD
Amazon RDS (PostgreSQL)	750 hrs/mes instancia db.t2.micro	12 meses	$0.00 USD
Amazon SageMaker	250 hrs/mes instancia t2.micro	2 meses	$0.00 USD
AWS Certificate Manager	Certificados SSL/TLS ilimitados	Siempre gratuito	$0.00 USD
Amazon CloudWatch	5 GB logs y 10 métricas/mes	Siempre gratuito	$0.00 USD
Total con capa gratuita			**$0.00 USD**

Durante los primeros 12 meses de desarrollo del prototipo, todos los servicios AWS utilizados se mantendrán dentro de los límites de la capa gratuita, resultando en un costo de infraestructura en la nube de **$0.00 USD**. El sistema contará con un máximo de 3 dispositivos móviles activos y un número reducido de usuarios en la plataforma web, lo que garantiza que las invocaciones de Lambda, las solicitudes a API Gateway y el almacenamiento en RDS y S3 no superen los umbrales establecidos [32].

No obstante, la Tabla 11 presenta la estimación del costo real de los servicios sin considerar la capa gratuita, generada mediante AWS Pricing Calculator [35]. Este escenario representa el costo máximo en que incurriría el prototipo si la cuenta AWS excediera los límites gratuitos o si los 12 meses del free tier expiraran durante el desarrollo.

Tabla 11: *Estimación de costos reales del prototipo sin capa gratuita (AWS Pricing Calculator, us-east-1, 2026).*

Servicio	**Configuración**	**Mensual (USD)**	**Mensual (MXN)**
AWS Lambda	ARM64, 100K invocaciones, 128 MB, 300ms	$0.00	$0.00
Amazon API Gateway	100K llamadas REST/mes	$0.10	$1.75
Amazon RDS PostgreSQL	db.t2.micro, Single-AZ, 20 GB gp2	$24.64	$431.20
Amazon S3	5 GB estándar, 20K GET, 2K PUT	$0.13	$2.28
Amazon CloudFront	Free Plan	$0.00	$0.00
Amazon Cognito	10 usuarios activos/mes	$1.00	$17.50
Amazon CloudWatch	1 GB logs, 5 métricas	$2.00	$35.00
Total mensual		**$27.87 USD**	**$487.73 MXN**
Total 12 meses		**$334.44 USD**	**$5,852.70 MXN**

El costo dominante en este escenario corresponde a **Amazon RDS** ($24.64 USD/mes), que representa el 88% del gasto mensual total. Sin embargo, dado que RDS está cubierto por el free tier durante los primeros 12 meses, el costo real esperado del prototipo sin RDS asciende únicamente a **$3.23 USD/mes ($56.53 MXN/mes)**, lo que confirma la viabilidad económica del prototipo incluso fuera de la capa gratuita.

La estimación completa puede consultarse en el siguiente enlace generado por AWS Pricing Calculator:

https://calculator.aws/#/estimate?id=f1906b6e6936514d8325160110c97fc1d367d5de

[35]

7.8.4 Viabilidad económica del prototipo

El análisis financiero del proyecto demuestra que el desarrollo del prototipo es económicamente viable dentro de las condiciones y recursos disponibles del equipo. La Tabla 12 presenta el resumen consolidado de los costos totales estimados.

Tabla 12: *Resumen de costos totales del prototipo.*

Categoría	**Total estimado (MXN)**
Infraestructura física (hardware)	$8,000.00
Transporte y logística	$240.00
Consumo eléctrico	$2,720.00
Licencias de software	$0.00
Servicios en la nube (AWS Free Tier)	$0.00
Total general	**$10,960.00 MXN**

El costo total estimado del prototipo asciende a $10,960.00 MXN, distribuido entre tres integrantes del equipo, lo que representa aproximadamente $3,653.33 MXN por integrante. Este monto es accesible para un proyecto académico de titulación, considerando que los costos más significativos corresponden a adquisiciones de hardware que ya han sido realizadas.

Los factores que contribuyen a la viabilidad económica del proyecto son:

- **Cero costo en servicios en la nube:** La capa gratuita de AWS cubre el 100% de los servicios de infraestructura requeridos durante la fase de prototipo, eliminando uno de los principales rubros de gasto en proyectos de esta naturaleza.
- **Cero costo en licencias de software:** Todas las herramientas de desarrollo seleccionadas son de código abierto o cuentan con versiones gratuitas para uso académico.
- **Dataset propio:** La recolección del conjunto de imágenes se realizará directamente en el Bosque de Chapultepec con el dron y los dispositivos del equipo, evitando la adquisición de datasets de pago.
- **Hardware reutilizable:** Tanto la Raspberry Pi como el dron son recursos que podrán seguir siendo utilizados más allá del prototipo, por lo que su costo representa una inversión con valor residual.

En caso de que el sistema evolucione hacia un ambiente productivo, los costos de infraestructura aumentarían proporcionalmente al volumen de usuarios y datos gestionados. El análisis de costos en ambiente productivo y la proyección financiera a largo plazo se presentan en la sección 7.9.

7.8.5 Justificación del proveedor de nube: AWS sobre Azure y GCP

La selección de Amazon Web Services (AWS) como proveedor de infraestructura en la nube para este sistema no es arbitraria. Se fundamenta en un análisis comparativo de los tres principales proveedores del mercado: AWS, Microsoft Azure y Google Cloud Platform (GCP), evaluando sus servicios equivalentes, modelos de precios, madurez del ecosistema y adecuación al contexto del proyecto.

Posición en el mercado. AWS es el proveedor de nube líder a nivel mundial con una cuota de mercado del 31% en 2025, seguido por Azure con 25% y GCP con 11%, según datos de Synergy Research Group [36]. Esta posición de liderazgo se traduce en un ecosistema más maduro, mayor disponibilidad de documentación, comunidad de soporte más amplia y mayor cantidad de integraciones con herramientas de terceros como Micronaut, Angular y GitHub Actions.

Comparativa de servicios equivalentes. La Tabla 13 presenta los servicios equivalentes de cada proveedor para los componentes utilizados en el sistema.

Tabla 13: *Comparativa de servicios equivalentes entre AWS, Azure y GCP (2025).*

Componente	**AWS**	**Microsoft Azure**	**Google Cloud (GCP)**
Funciones serverless	AWS Lambda	Azure Functions	Cloud Functions
API Gateway	Amazon API Gateway	Azure API Management	Cloud API Gateway / Apigee
Base de datos relacional	Amazon RDS	Azure SQL Database	Cloud SQL
Almacenamiento de objetos	Amazon S3	Azure Blob Storage	Cloud Storage
CDN	Amazon CloudFront	Azure CDN	Cloud CDN
Autenticación	Amazon Cognito	Azure Active Directory B2C	Firebase Authentication
ML / Versionado de modelos	Amazon SageMaker	Azure Machine Learning	Vertex AI
CI/CD	GitHub Actions + AWS CodeDeploy	Azure DevOps	Cloud Build
Certificados SSL/TLS	AWS Certificate Manager	App Service Managed Certificates	Google-managed SSL
Monitoreo	Amazon CloudWatch	Azure Monitor	Cloud Monitoring
WAF	AWS WAF	Azure Web Application Firewall	Cloud Armor

Comparativa de capa gratuita. La Tabla 14 compara los límites gratuitos de los servicios más relevantes para el sistema entre los tres proveedores [36].

Tabla 14: *Comparativa de capa gratuita entre AWS, Azure y GCP (2025).*

Servicio	**AWS**	**Azure**	**GCP**
Funciones serverless	**1M invocaciones/mes siempre gratis**	1M invocaciones/mes (12 meses)	2M invocaciones/mes siempre gratis

Tabla 14 – continuación

Servicio	AWS	Azure	GCP
Almacenamiento objetos	**5 GB siempre gratis**	5 GB (12 meses)	5 GB siempre gratis
Base de datos relacional	750 hrs db.t2.micro (12 meses)	250 GB Azure SQL (12 meses)	1 instancia db-f1-micro siempre gratis
CDN	**1 TB transferencia siempre gratis**	15 GB (12 meses)	10 GB/mes siempre gratis
Autenticación	**50,000 usuarios activos/mes siempre gratis**	50,000 MAU (siempre gratis)	10,000 usuarios/mes (Firebase)
API Gateway	**1M llamadas REST/mes siempre gratis**	No tiene capa gratuita específica	2M llamadas/mes (Cloud Endpoints)
Certificado SSL	**Gratis e ilimitado con servicios AWS**	Gratis solo en App Service	Gratis con servicios GCP

Modelo de precios pay-as-you-go. AWS fue pionero en el modelo de pago por uso en la nube pública, y sigue siendo el más granular de los tres proveedores. Lambda factura por milisegundo de ejecución, S3 por GB almacenado y por número de solicitudes, y CloudFront por GB transferido, lo que garantiza que el sistema pague exactamente por lo que consume sin desperdiciar recursos. Esto contrasta con modelos de otros proveedores donde algunos servicios tienen cargos mínimos independientemente del uso [37].

Razones concretas para elegir AWS sobre Azure y GCP.

- **Madurez del ecosistema serverless:** AWS Lambda fue el primer servicio de funciones serverless del mercado, lanzado en 2014, con dos años de ventaja sobre Azure Functions (2016) y Google Cloud Functions (2016). Esta madurez se traduce en mejor documentación, más ejemplos de integración con Micronaut y mayor estabilidad del servicio [38].
- **Integración nativa con GitHub Actions:** El pipeline CI/CD del proyecto utiliza GitHub Actions, que tiene integración oficial y más desarrollada con AWS que con GCP. Azure DevOps es el competidor directo pero implica adoptar todo el ecosistema Microsoft.
- **Amazon SageMaker vs alternativas:** Para el versionado del modelo de clasificación, SageMaker Model Registry ofrece una integración más directa con S3 y Lambda que Vertex AI (GCP) o Azure Machine Learning, simplificando el flujo de publicación del modelo `.tflite`.
- **CloudFront vs Azure CDN vs Cloud CDN:** Amazon CloudFront ofrece 1 TB de transferencia gratuita de forma permanente, mientras que Azure CDN solo cubre 15 GB durante los primeros 12 meses y Cloud CDN ofrece 10 GB/mes. Para el frontend Angular del sistema, CloudFront representa la opción más económica a largo plazo [36].
- **Amazon Cognito vs alternativas:** Cognito ofrece 50,000 usuarios activos mensuales de forma gratuita e indefinida, comparable a Azure AD B2C pero con una integración más directa con API Gateway y Lambda sin configuración adicional.
- **Reservaciones de RDS:** AWS ofrece Reserved Instances para RDS con hasta un 72% de descuento sobre el precio bajo demanda, permitiendo planificar costos a largo plazo con certeza. GCP ofrece descuentos similares mediante Committed Use Discounts, pero la flexibilidad de instancia de AWS es mayor [37].
- **Comunidad y soporte académico:** AWS cuenta con el programa *AWS Educate* y *AWS Academy*, que proporcionan créditos y recursos educativos para proyectos académicos, alineado con el contexto de titulación de este prototipo.
- **Azure no es la opción óptima para este stack:** Azure es la mejor opción cuando se trabaja

dentro del ecosistema Microsoft (Windows Server, SQL Server, .NET, Azure DevOps). El stack del presente sistema, basado en Java 2 1/Micronaut, Angular, PostgreSQL y GitHub Actions, no se beneficia de las ventajas específicas de Azure, lo que haría su adopción innecesariamente compleja sin ventaja económica o técnica.
- **GCP no es la opción óptima para este stack:** GCP destaca en cargas de trabajo de análisis de datos masivos con BigQuery, orquestación de contenedores con GKE y modelos de ML con Vertex AI. Para un sistema de sincronización móvil con backend REST y base de datos relacional, estas ventajas no son relevantes y su ecosistema serverless es menos maduro que el de AWS para integraciones con Micronaut y PostgreSQL.

En conclusión, AWS representa la opción más adecuada para este proyecto por su madurez en el ecosistema serverless, la amplitud de su capa gratuita, el modelo pay-as-you-go granular, la integración nativa con las herramientas del stack seleccionado y su trayectoria como líder del mercado de nube pública con más de 18 años de operación continua [36].

7.9 Proyección a ambiente productivo

El prototipo desarrollado está diseñado arquitectónicamente para escalar hacia un ambiente productivo sin cambios estructurales. Esta sección presenta un análisis comparativo de las opciones de infraestructura disponibles, una justificación de las instancias de base de datos seleccionadas y una estimación de los costos mensuales de infraestructura en la nube para un escenario productivo con más de 100 dispositivos móviles activos, más de 200 usuarios en la plataforma web y más de 50,000 imágenes sincronizadas mensualmente.

Análisis comparativo: serverless vs contenedores

Una decisión arquitectónica fundamental para el ambiente productivo es la elección entre un modelo serverless basado en AWS Lambda y un modelo basado en contenedores con Docker y Kubernetes (Amazon EKS). La Tabla 15 resume las diferencias principales entre ambos enfoques.

Tabla 15: *Comparativa entre arquitectura serverless y contenedores para ambiente productivo.*

Criterio	**AWS Lambda (serverless)**	**Docker + Kubernetes (EKS)**
Administración de infraestructura	Nula, AWS gestiona todo automáticamente	Alta, requiere configuración y mantenimiento del cluster
Escalado	Automático e instantáneo por invocación	Automático pero requiere configuración de HPA y nodos
Cold starts	2–5 segundos con Micronaut en el primer arranque	Sin cold starts, los contenedores están siempre activos
Costo base mensual	$0.00 si no hay tráfico	$150 USD solo por el cluster EKS, independiente del tráfico
Load balancer	No necesario, API Gateway lo reemplaza	Necesario (ALB), costo adicional de $20 USD/mes
Límite de ejecución	15 minutos máximo por función	Sin límite de tiempo de ejecución
Ideal para	Tráfico intermitente o impredecible	Tráfico constante y predecible de alto volumen
Complejidad operativa	Baja	Alta

Para el escenario productivo del sistema de detección de insectos descortezadores, con 100+ dispositivos sincronizando datos de forma **ocasional** cuando existe conectividad a internet, el modelo serverless con AWS Lambda continúa siendo la opción más adecuada. El tráfico de sincronización es por naturaleza intermitente, que es exactamente el escenario donde Lambda ofrece mayor ventaja económica y operativa sobre Kubernetes.

Sin embargo, si el sistema llegara a escalar a **miles de dispositivos** con sincronizaciones frecuentes y continuas, o si se requiriera integrar procesos de larga duración como el análisis masivo de imágenes en batch, la migración hacia una arquitectura basada en contenedores con Amazon EKS, un Application Load Balancer (ALB) y Auto Scaling Groups sería la evolución natural. Esta transición no requeriría rediseñar la arquitectura base, ya que Micronaut está diseñado pa-

ra ejecutarse tanto en Lambda como en contenedores Docker sin cambios significativos en el código.

Análisis comparativo de instancias de base de datos

La selección de la instancia de Amazon RDS para el ambiente productivo requiere un balance entre rendimiento, disponibilidad y costo. La Tabla 16 presenta un análisis de las principales opciones disponibles para PostgreSQL en la región `us-east-1`.

Tabla 16: *Comparativa de instancias Amazon RDS PostgreSQL para ambiente productivo (us-east-1, 2026).*

Instancia	vCPUs	RAM (GB)	Single-AZ (USD/mes)	Multi-AZ (USD/mes)	Caso de uso
db.t3.micro	2	1	$15.33	$30.66	Desarrollo y pruebas, no recomendado para producción
db.t3.small	2	2	$28.49	$56.98	Producción ligera, hasta 50 usuarios concurrentes
db.t3.medium	**2**	**4**	**$52.56**	**$105.12**	**Producción estándar, 100–200 usuarios, recomendado**
db.t3.large	2	8	$105.12	$210.24	Alta concurrencia, 500+ usuarios
db.m5.large	2	8	$138.00	$276.00	Producción intensiva, cargas sostenidas sin bursting

La instancia **db.t3.medium** fue seleccionada para el ambiente productivo por las siguientes razones:

- **Memoria suficiente:** Con 4 GB de RAM puede mantener en caché las consultas más frecuentes del sistema, como el historial de detecciones y los registros de árboles, reduciendo la latencia de respuesta.
- **CPU con bursting:** La familia T3 opera en modo *unlimited*, permitiendo ráfagas de CPU cuando la carga aumenta durante las sincronizaciones masivas, sin requerir una instancia de mayor tamaño de forma permanente.
- **Costo equilibrado:** Representa el punto óptimo entre rendimiento y costo para el volumen estimado de 100+ dispositivos y 200+ usuarios, con reservación anual para reducir el costo efectivo mensual [39].
- **Escalabilidad hacia Aurora:** Si el sistema escalara a miles de dispositivos, la migración hacia **Amazon Aurora PostgreSQL** no requeriría cambios en el código del backend, ya que Aurora es compatible con el protocolo PostgreSQL.

Estimación de costos mensuales

La siguiente estimación fue generada mediante la herramienta oficial AWS Pricing Calculator [40] para la región `us-east-1` (US East - N. Virginia), vigente a mayo de 2026. El tipo de cambio utilizado para la conversión es de $17.50 MXN por dólar estadounidense (USD).

Supuestos del escenario productivo

- **Dispositivos móviles activos:** 100+ dispositivos sincronizando datos de forma periódica.
- **Usuarios plataforma web:** 200+ usuarios con sesiones activas.
- **Imágenes sincronizadas:** 50,000+ imágenes/mes con un tamaño promedio de 2 MB por imagen, equivalente a 100 GB de almacenamiento mensual nuevo.
- **Invocaciones Lambda:** 5 millones de invocaciones/mes con arquitectura ARM64 (Graviton2), 500ms de duración promedio y 512 MB de memoria asignada.
- **Base de datos:** Instancia `db.t3.medium` PostgreSQL con Reserved Instance de 1 año y pago parcial anticipado, 100 GB de almacenamiento SSD gp3.

Tabla 17: *Estimación de costos en ambiente productivo generada con AWS Pricing Calculator (us-east-1, 2026).*

Servicio	**Configuración**	**Upfront (USD)**	**Mensual (USD)**	**Mensual (MXN)**
AWS Lambda	5M invocaciones, ARM64, 512 MB, 500ms	$0.00	$0.80	$14.00
Amazon API Gateway	5M llamadas REST/mes	$0.00	$17.50	$306.25
Amazon RDS PostgreSQL	db.t3.medium, Reserved 1 año, 100 GB gp3	$845.00	$63.15	$1,105.13
Amazon S3	500 GB almacenamiento estándar	$0.00	$11.83	$206.93
Amazon CloudFront	Pro Plan (1 distribución)	$0.00	$15.00	$262.50
Amazon CloudWatch	Logs y métricas	$0.00	$3.00	$52.50
AWS WAF	1 Web ACL, reglas básicas	$0.00	$13.00	$227.50
Amazon Cognito	200+ usuarios activos/mes	$0.00	$10.00	$175.00
Total		**$845.00**	**$134.28**	**$2,349.90**

El costo mensual recurrente en ambiente productivo asciende a **$134.28 USD ($2,349.90 MXN)**, con un pago único inicial de **$845.00 USD ($14,787.50 MXN)** correspondiente a la reservación anual de la instancia Amazon RDS. El costo total estimado para los primeros 12 meses de operación en producción es de **$2,456.36 USD ($42,986.30 MXN)**, incluyendo el pago anticipado de RDS.

La reservación anual de RDS representa un ahorro significativo respecto al precio bajo demanda: la instancia `db.t3.medium` en configuración on-demand costaría aproximadamente $105 USD/mes ($1,260 USD/año), mientras que con Reserved Instance el costo efectivo se reduce a $845 + ($63.15 $\times$ 12) = $1,602.80 USD/año, un ahorro de alrededor del **27%** respecto al precio on-demand [39].

Estrategias de optimización de costos

Para reducir el costo mensual en producción, se pueden aplicar las siguientes estrategias dentro del mismo ecosistema AWS:

- **Reserved Instances en RDS:** Ya aplicado en esta estimación, con un ahorro del 27% respecto al precio on-demand.
- **Lambda ARM64 (Graviton2):** Las funciones Lambda configuradas con arquitectura ARM64 ofrecen un 20% de reducción en costos de cómputo sin cambios en el código de Micronaut.
- **S3 Intelligent-Tiering:** Para imágenes antiguas con acceso poco frecuente, S3 Intelligent-Tiering reduce automáticamente el costo de almacenamiento hasta un 40%.
- **CloudFront caché:** Configurar políticas de caché agresivas para los artefactos Angular reduce las solicitudes al origen S3, disminuyendo tanto el costo de CloudFront como el de S3.

La estimación completa puede consultarse en el siguiente enlace generado por AWS Pricing Calculator:

https://calculator.aws/#/estimate?id=65c11c7fb1e7ea52aab349e587a999504e736112

[40]

Capítulo 8. Diseño del sistema

El diseño del sistema propone una arquitectura distribuida en tres componentes principales: la aplicación móvil, la plataforma web y la infraestructura en Amazon Web Services (AWS).

8.1 Arquitectura general del sistema

Para garantizar su operatividad en entornos sin conexión, como el Bosque de Chapultepec, el cliente móvil emplea una estrategia *offline-first*. La captura de imágenes y la inferencia para la detección de insectos se ejecutan localmente mediante TensorFlow Lite, sincronizando los datos hacia la nube de forma eventual y segura al recuperar la conectividad.

La arquitectura en la nube se basa en un enfoque *serverless* mediante Amazon API Gateway y AWS Lambda, con autenticación gestionada por Amazon Cognito. El ciclo de vida del modelo de Deep Learning recae en Amazon SageMaker, utilizando Amazon S3 y RDS para la persistencia de datos. El cliente web distribuye su contenido a través de Amazon CloudFront. Finalmente, para asegurar un entorno de grado de producción, la arquitectura integra AWS Certificate Manager (ACM) para el cifrado SSL/TLS, GitHub Actions para despliegues continuos (CI/CD) y una capa transversal de monitoreo centralizado con Amazon CloudWatch.

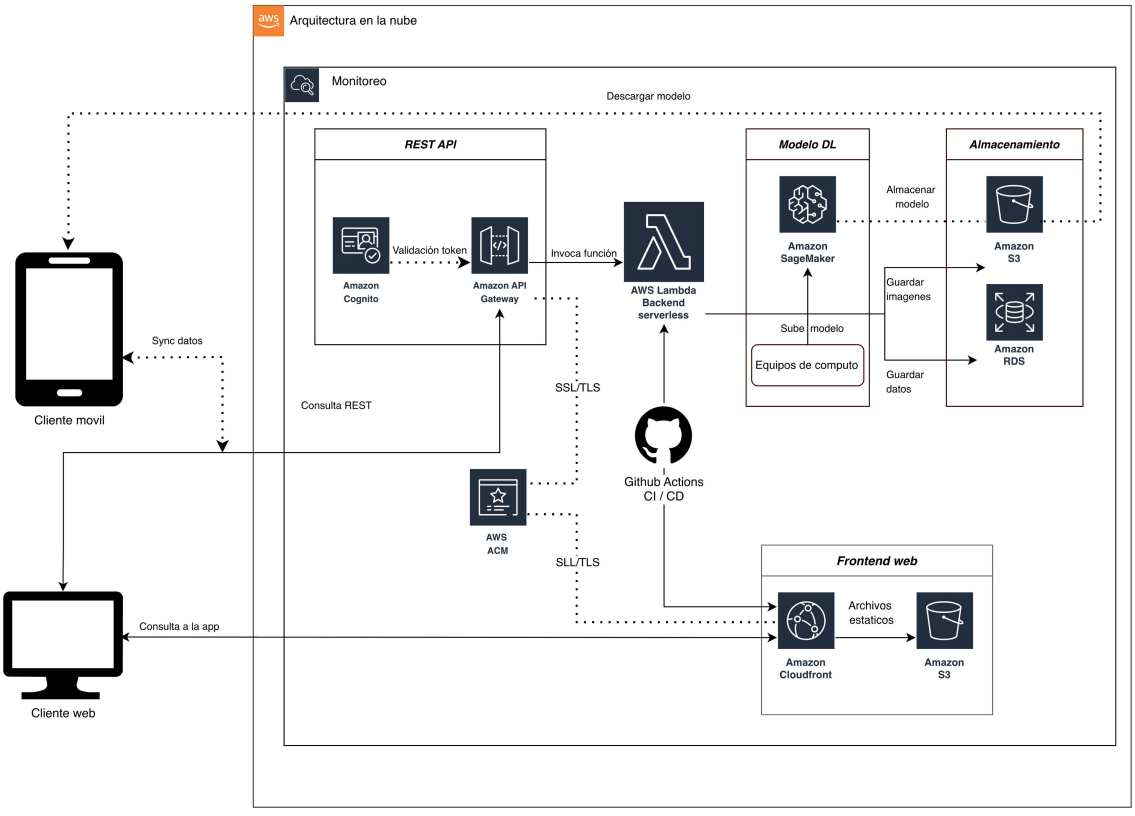


Figura 18: *Arquitectura general del sistema.*

8.2 Estrategia de seguridad y control de acceso

Para proteger la integridad de los datos y restringir el acceso a los recursos de la nube, se implementará una estrategia de seguridad perimetral y una gestión de identidad híbrida.

La protección de los datos en tránsito se garantizará mediante AWS Certificate Manager (ACM), el cual aprovisionará y gestionará certificados SSL/TLS. Estos certificados se adjuntarán a los puntos de entrada públicos (Amazon API Gateway y Amazon CloudFront) para forzar que toda comunicación desde los clientes se realice exclusivamente bajo el protocolo HTTPS.

En cuanto al control de acceso, el sistema adoptará un enfoque híbrido para la autenticación:

- **Plataforma Web:** Se utilizará Amazon Cognito para gestionar el grupo de usuarios administradores. Cognito proporcionará flujos seguros de inicio de sesión y emitirá tokens que la aplicación web enviará en cada petición.
- **Aplicación Móvil:** Dado el enfoque *offline* del cliente móvil, el *backend* (desarrollado en Micronaut) será el encargado de generar y validar firmas de JSON Web Tokens (JWT) propios. Esto aligerará la carga en el dispositivo móvil y permitirá una validación rápida y sin dependencias externas durante los procesos de sincronización.

Ambos tipos de tokens serán interceptados y validados por Amazon API Gateway antes de invocar cualquier función Lambda, protegiendo así la lógica de negocio subyacente.

8.2.1 Almacenamiento y persistencia políglota

El sistema requerirá manejar tanto información relacional estructurada como archivos binarios pesados. Para optimizar el rendimiento y los costos, se adoptará una arquitectura de persistencia políglota dividida en dos servicios:

1. **Amazon RDS (PostgreSQL):** Actuará como la fuente de la verdad para los datos transaccionales. Almacenará los registros de usuarios, el inventario de árboles evaluados, la metadata de los reportes de detección y las configuraciones del sistema. Se eligió PostgreSQL por su robustez, cumplimiento ACID y capacidad para manejar consultas geoespaciales si el proyecto lo requiere en el futuro.

2. **Amazon S3:** Proporcionará almacenamiento de objetos altamente escalable para datos no estructurados. Se configurarán *buckets* con políticas de acceso estricto para almacenar tres tipos de artefactos: las imágenes fotográficas capturadas por el dispositivo móvil, las versiones compiladas de la aplicación web Angular y las versiones del modelo `.tflite` listas para su descarga.

8.2.2 Integración con Amazon SageMaker

A diferencia de las arquitecturas clásicas de *Machine Learning* en la nube, Amazon SageMaker no se utilizará para realizar inferencias en tiempo real, ya que esta tarea recaerá en el dispositivo móvil. En su lugar, SageMaker fungirá exclusivamente como un *Model Registry* (Registro de Modelos).

El entrenamiento intensivo del modelo (basado en EfficientNet-B3 o YOLO) se ejecutará en equipos de cómputo locales. Una vez que el modelo alcance las métricas de precisión deseadas mediante evaluación con *Scikit-learn*, sus pesos y arquitectura se subirán a Amazon SageMaker

para versionarlo de manera formal. Finalmente, SageMaker orquestará la exportación del modelo al formato optimizado TensorFlow Lite (`.tflite`) y lo depositará en Amazon S3, quedando disponible para que los dispositivos móviles descarguen la actualización cuando dispongan de conexión a internet.

8.2.3 Observabilidad y monitoreo centralizado

Para garantizar la fiabilidad del entorno *serverless*, se implementará una capa transversal de observabilidad utilizando Amazon CloudWatch. Este servicio centralizará la telemetría del sistema sin requerir la instalación de agentes adicionales.

CloudWatch recolectará de manera automática los registros de ejecución (*logs*) emitidos por las funciones AWS Lambda en Java/Micronaut, facilitando la depuración de errores. Asimismo, monitoreará las métricas críticas de Amazon API Gateway, tales como la latencia de las peticiones, la tasa de errores HTTP 4xx/5xx y el volumen de tráfico. Se configurarán alarmas automatizadas para notificar a los administradores ante cualquier anomalía operativa, como un pico inusual en los tiempos de arranque (*cold starts*) o intentos masivos de acceso denegado.

8.2.4 Entorno de desarrollo y automatización (CI/CD)

El ciclo de vida del desarrollo de *software* integrará prácticas de DevOps para asegurar transiciones seguras entre el código local y la nube.

A nivel local, los desarrolladores utilizarán contenedores **Docker** para levantar instancias aisladas de PostgreSQL, asegurando la paridad entre el entorno de desarrollo y producción. Como paso intermedio, se configurará una **Raspberry Pi** que actuará como un servidor *on-premise* de pruebas e integración, permitiendo validar el comportamiento de la aplicación Micronaut en un entorno Linux real antes de impactar los servicios de AWS.

El despliegue hacia la nube estará completamente automatizado mediante **GitHub Actions**. Se construirá un *pipeline* de Integración y Despliegue Continuo (CI/CD) que reaccionará a los cambios en la rama principal del repositorio. Las etapas automatizadas incluirán: la compilación del código fuente, la ejecución de pruebas unitarias, el empaquetado del artefacto Java 21 y, finalmente, la actualización automática del código en AWS Lambda mediante la interfaz de línea de comandos (AWS CLI).

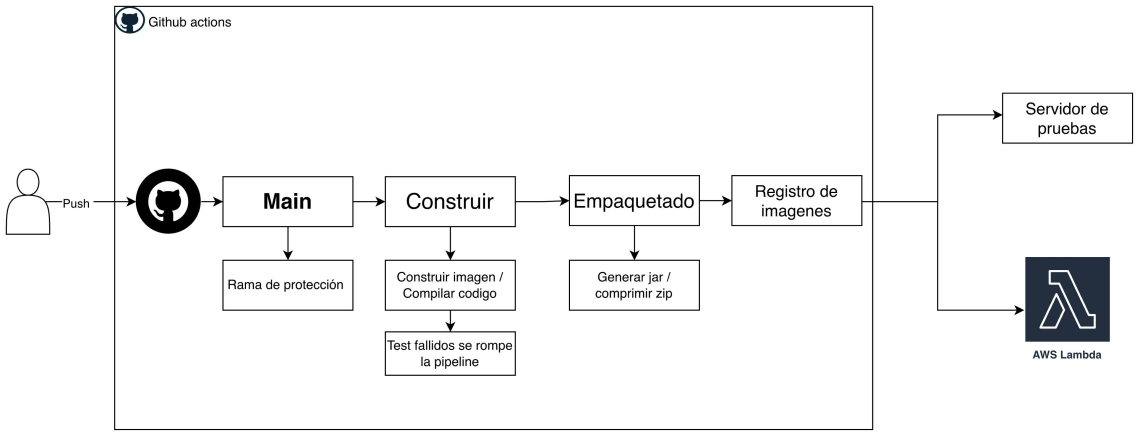


Figura 19: *Flujo de Integración y Despliegue Continuo (CI/CD).*

8.2.5 Flujos principales de comunicación

El flujo de información más crítico de la arquitectura es el proceso asíncrono de sincronización entre el cliente móvil *offline* y la nube. Este proceso se ejecutará de manera secuencial siguiendo los principios de las arquitecturas basadas en eventos:

1. El dispositivo móvil, tras operar en campo sin red, detecta conexión a internet e inicia el proceso de sincronización leyendo los registros locales almacenados en SQLite.
2. El cliente (mediante Retrofit) realiza una petición REST segura al *endpoint* correspondiente en Amazon API Gateway, adjuntando el JWT propio en las cabeceras de autorización.
3. API Gateway intercepta la petición y verifica la firma del token. Si es válido, canaliza el cuerpo de la petición (texto e imágenes codificadas) hacia AWS Lambda.
4. La función Lambda inicializa el contexto de Micronaut. Extrae las imágenes, las decodifica y las transmite hacia Amazon S3, obteniendo como respuesta una URL (URI) permanente.
5. Posteriormente, Lambda construye el objeto de dominio del reporte, asocia la URL de S3 al registro y lo persiste de manera transaccional en Amazon RDS (PostgreSQL).
6. Finalmente, Lambda retorna un código HTTP 200 OK hacia la aplicación móvil, la cual procede a marcar los registros locales como ”sincronizados” para evitar duplicidades futuras.

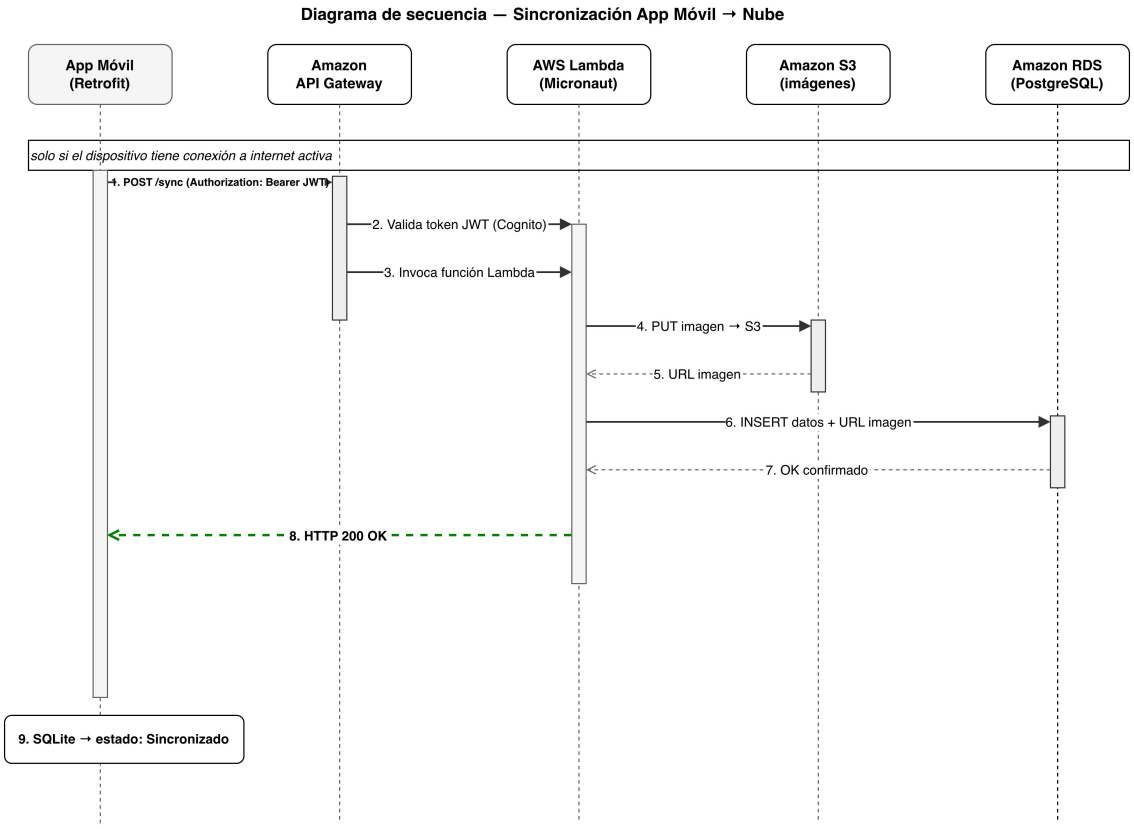


Figura 20: *Diagrama de secuencia del proceso de sincronización de datos.*

8.2.6 Estructura interna del backend (Clean Architecture)

El *backend* alojado en AWS Lambda se implementará utilizando el lenguaje Java 21 acompañado del *framework* Micronaut. La decisión de usar Micronaut responde a su capacidad para resolver la inyección de dependencias en tiempo de compilación (*Ahead of Time - AoT*), eliminando la sobrecarga en tiempo de ejecución característica de *frameworks* tradicionales como Spring, lo cual es vital para mantener la latencia baja en un modelo *serverless*.

Para asegurar la alta cohesión, el bajo acoplamiento y la facilidad de mantenimiento, la base de código se estructurará siguiendo los principios de *Clean Architecture* (Arquitectura Limpia). Las dependencias del código apuntarán exclusivamente hacia adentro, organizándose en las siguientes capas concéntricas:

- **Capa de Dominio (*Domain*):** Constituirá el núcleo del sistema. Contendrá las entidades de negocio puras (`Deteccion`, `Arbol`, `Usuario`, `Captura`) y sus invariantes, sin dependencias hacia ninguna tecnología o librería externa. También define las interfaces abstractas de repositorio (`DeteccionRepository`, `ArbolRepository`) que las capas externas deberán implementar, aplicando el principio de inversión de dependencias.
- **Capa de Casos de Uso (*Use Cases*):** Orquestará el flujo de datos desde y hacia las entidades. Implementará la lógica de la aplicación, como registrar un nuevo hallazgo geolocalizado, procesar la sincronización masiva proveniente del dispositivo móvil o consultar el historial de detecciones de un árbol específico. Cada caso de uso es una clase independiente con una única responsabilidad, lo que facilita las pruebas unitarias aisladas.
- **Capa de Adaptadores (*Interface Adapters*):** Actuará como traductora entre el mundo exterior y los casos de uso. Aquí residirán los controladores REST (`SyncController`, `DetectionController`, `TreeController`, `UserController`) que transforman las peticiones HTTP de Amazon API Gateway en objetos de dominio, así como los *mappers* que convierten entre entidades y DTOs de transferencia.
- **Capa de Infraestructura (*Infrastructure*):** Será la capa más externa, encargada de proveer las implementaciones concretas. Contendrá los módulos de Micronaut Data JPA e Hibernate para la persistencia en PostgreSQL sobre Amazon RDS, la integración del SDK de AWS para la carga de imágenes en Amazon S3, y los componentes de seguridad encargados de la validación de tokens JWT. También incluirá la configuración de Amazon CloudWatch para el monitoreo de logs y métricas en tiempo de ejecución.

La estructura de paquetes del proyecto refleja directamente esta separación de capas:

```
com.descortezadores.backend/
  domain/
    entities/          (Deteccion, Arbol, Usuario, Captura)
    repositories/      (interfaces abstractas)
    rules/             (invariantes y reglas de negocio)
  usecases/
    sync/              (SyncDeteccionUseCase)
    detection/         (RegistrarDeteccionUseCase)
    history/           (ConsultarHistorialUseCase)
  adapters/
    controllers/       (SyncController, DetectionController)
    mappers/           (DeteccionMapper, ArbolMapper)
    repositories/      (DeteccionRepositoryImpl, S3ImageRepository)
  infrastructure/
    persistence/       (Micronaut Data JPA, Hibernate)
```

```
aws/              (S3Client, CognitoValidator)
security/         (JwtValidator)
config/           (MicronautConfig, DataSourceConfig)
```

Esta organización garantiza que cualquier cambio en la infraestructura, como migrar de PostgreSQL a Aurora o cambiar el proveedor de almacenamiento, no afecte la lógica de negocio ni los casos de uso del sistema.

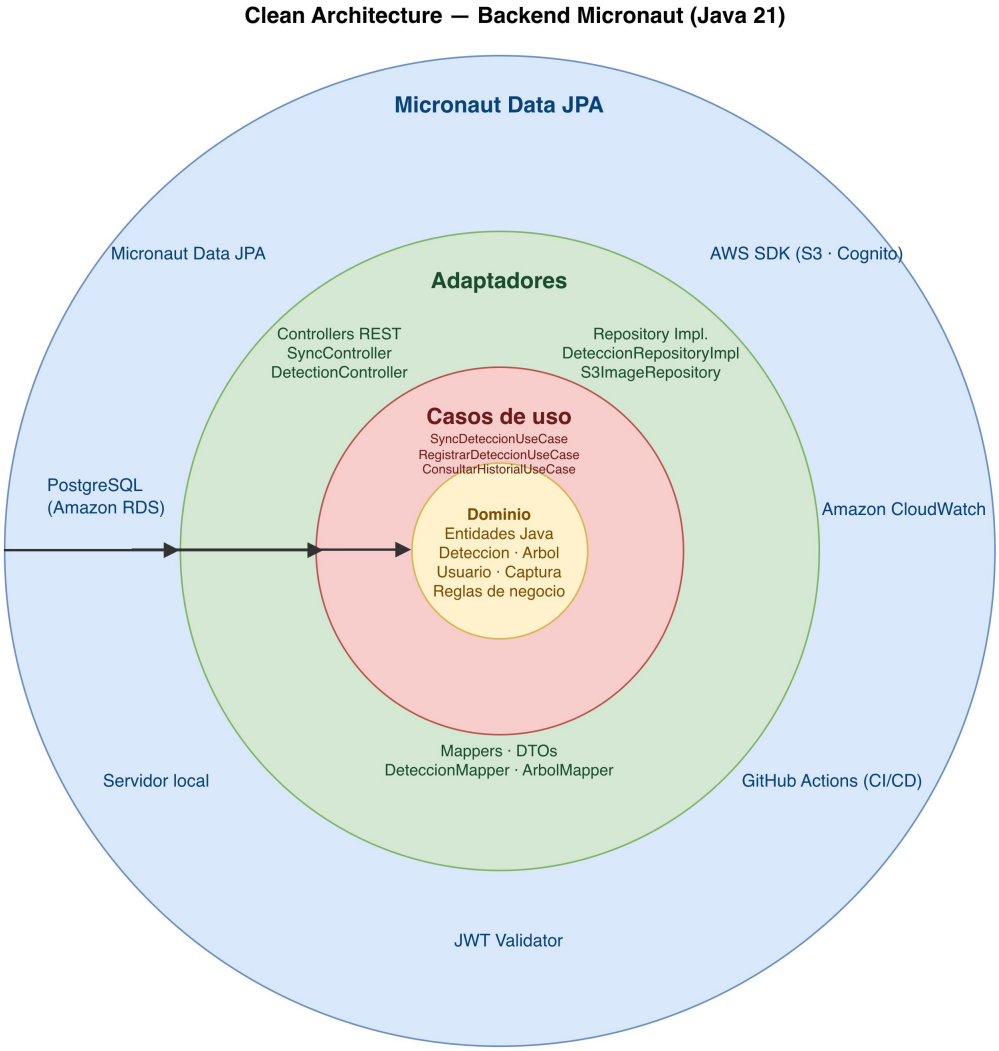


Figura 21: *Estructura en capas del backend basada en Clean Architecture.*

8.3 Arquitectura de la aplicación móvil

La aplicación móvil constituirá la principal herramienta de trabajo en campo para la recolección de datos y el análisis de los árboles. Estará diseñada de forma nativa para el sistema operativo Android, priorizando la eficiencia en el consumo de recursos (batería y memoria) y la capacidad de operar de manera autónoma sin depender de conectividad a la red.

8.3.1 Componentes de la aplicación

El desarrollo de la aplicación se realizará utilizando **Kotlin** como lenguaje de programación principal, alineándose con los estándares modernos y las recomendaciones oficiales de Google para el desarrollo nativo en Android.

Para el diseño y la construcción de la interfaz gráfica, se prescindirá del sistema tradicional de vistas basadas en XML en favor de **Jetpack Compose**. Este *toolkit* de UI declarativa permitirá construir interfaces reactivas, escalables y modulares, reduciendo significativamente la cantidad de código y facilitando la gestión del estado de la aplicación en pantallas complejas, como la vista de captura fotográfica y el despliegue de resultados.

La arquitectura interna de la aplicación seguirá el patrón de diseño *Model-View-ViewModel* (MVVM), recomendado para el ecosistema Android. Esto garantizará una estricta separación de responsabilidades entre la lógica de presentación (gestionada por Compose), la lógica de preparación de datos (*ViewModels*) y la lógica de acceso a datos (*Repositories*).

8.3.2 Inferencia local y procesamiento de imágenes

La característica central del cliente móvil será su capacidad de analizar fotografías para detectar la presencia de insectos descortezadores de manera estrictamente local (*on-device*). Este enfoque eliminará la latencia de red y permitirá que el sistema funcione en zonas boscosas sin cobertura.

Es imperativo señalar que el análisis de cada fotografía se ejecutará **una única vez** en el dispositivo. El sistema no realizará reevaluaciones posteriores sobre la misma imagen una vez que se haya emitido el diagnóstico inicial, garantizando así un uso eficiente de la batería y del poder de cómputo del teléfono. El flujo de este procesamiento único constará de dos etapas fundamentales:

1. **Preprocesamiento con OpenCV:** Al momento de la captura, la imagen pasará por una tubería de preprocesamiento utilizando la librería OpenCV para Android. Esta etapa incluirá técnicas de análisis cromático y filtrado de ruido para resaltar las áreas de interés (e.g., manchas de resina o aserrín en la corteza), además de redimensionar y normalizar los tensores para que coincidan con la entrada de la red neuronal.
2. **Inferencia con TensorFlow Lite:** La imagen procesada será evaluada de inmediato por el modelo de Deep Learning (basado en arquitecturas eficientes como EfficientNet-B3 o YOLO), el cual estará integrado en la aplicación en formato `.tflite`. La API de **TensorFlow Lite** aprovechará la aceleración de *hardware* del dispositivo para emitir el diagnóstico en tiempo real. Este resultado quedará asociado de forma permanente al registro de la captura.

8.3.3 Estrategia *Offline-First* y sincronización

Para garantizar la integridad y la persistencia de los diagnósticos generados en áreas sin internet, la aplicación implementará una estricta estrategia *Offline-First*. Bajo este paradigma, el dispositivo móvil actuará como la fuente primaria de verdad a corto plazo.

La persistencia local de los datos estructurados (información del árbol, coordenadas geográficas, fecha y el diagnóstico único emitido por TFLite) se gestionará utilizando **SQLite** a través de la librería **Room**. Room proporcionará una capa de abstracción robusta, verificando las consultas SQL en tiempo de compilación y exponiendo los datos mediante flujos asíncronos (*Coroutines* y *Flows* de Kotlin). Las imágenes asociadas a cada reporte se almacenarán de forma segura en el almacenamiento interno del dispositivo.

Cuando el sistema operativo notifique a la aplicación sobre la disponibilidad de una conexión a internet estable, se detonará un proceso en segundo plano (*background worker*) para sincronizar los registros locales pendientes con la nube.

Para estas comunicaciones de red, se utilizará la combinación de **Retrofit** y **OkHttp**. Retrofit facilitará la construcción de las peticiones REST hacia Amazon API Gateway. Por su parte, OkHttp se empleará para interceptar estas peticiones e inyectar automáticamente el **token JWT propio** en las cabeceras HTTP de autorización. Una vez que el *backend* en AWS confirme la recepción e inserción exitosax de los datos (HTTP 200), la aplicación actualizará el estado de los registros en SQLite como ”sincronizados”, completando así el ciclo de vida de la información capturada en campo.

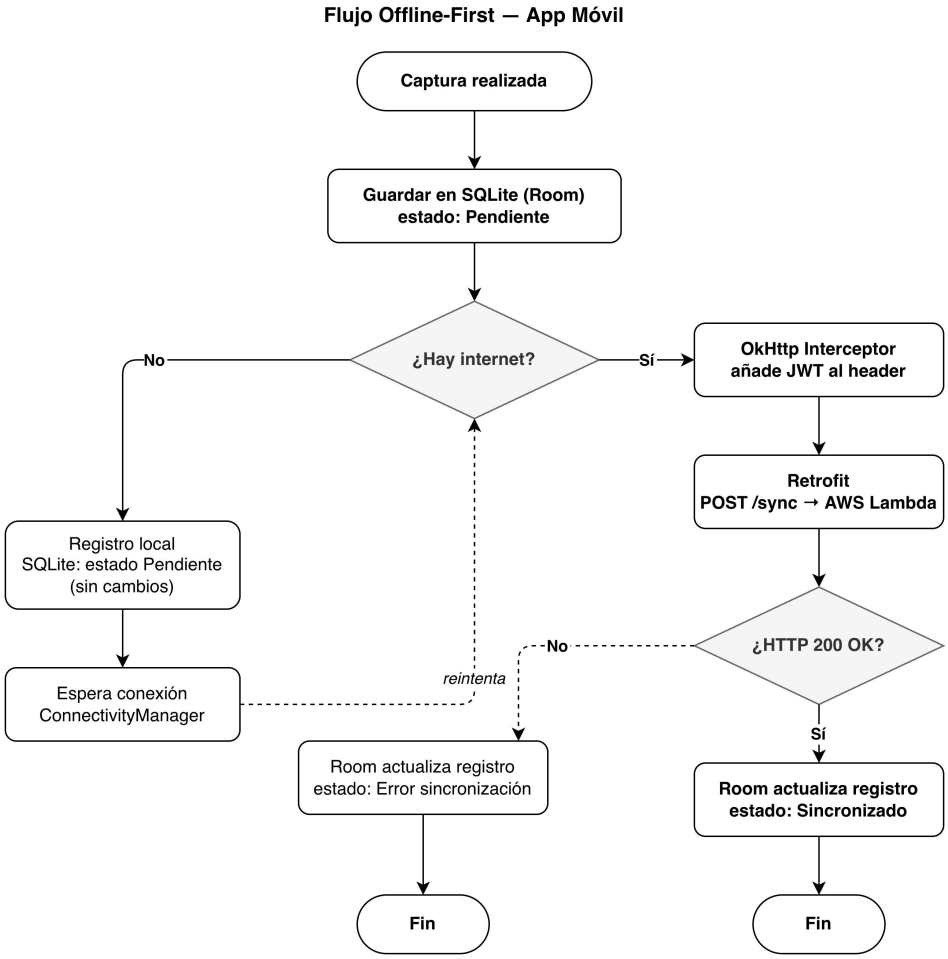


Figura 22: *Flujo de decisión y sincronización de datos de la estrategia Offline-First.*

8.4 Arquitectura de la aplicación web

La aplicación web fungirá como el panel de control central (*dashboard*) del sistema. Estará dirigida a los administradores, investigadores y tomadores de decisiones, permitiéndoles visualizar el historial de detecciones, analizar el estado fitosanitario del bosque y gestionar los registros recolectados por los dispositivos móviles en campo.

8.4.1 Arquitectura del Frontend (SPA)

El desarrollo del *frontend* se llevará a cabo utilizando el *framework* **Angular**, adoptando el paradigma de *Single Page Application* (SPA). Esta arquitectura garantizará una experiencia de usuario fluida, ya que la aplicación cargará una única página HTML y actualizará dinámicamente sus vistas sin necesidad de recargar el navegador en cada interacción.

La interfaz de usuario se construirá de manera modular utilizando **Angular Material**, una biblioteca de componentes de UI que implementa los lineamientos de *Material Design*. Esto asegurará consistencia visual, accesibilidad nativa y un diseño responsivo que se adaptará automáticamente a diferentes resoluciones de pantalla (escritorio y tabletas).

8.4.2 Distribución estática y alojamiento seguro

Aprovechando la naturaleza de la SPA, la aplicación web no requerirá un servidor de aplicaciones tradicional para su alojamiento. En su lugar, el código fuente de Angular se compilará en un conjunto de archivos estáticos (HTML, CSS y JavaScript) que se almacenarán en un *bucket* de **Amazon S3** configurado exclusivamente para alojamiento estático.

Para garantizar una entrega rápida y segura del contenido a los usuarios, se desplegará **Amazon CloudFront** como Red de Entrega de Contenido (CDN). CloudFront almacenará en caché los artefactos estáticos en sus nodos perimetrales globales, reduciendo drásticamente la latencia de carga. Además, CloudFront se integrará con **AWS Certificate Manager (ACM)** para adjuntar un certificado SSL/TLS válido, forzando que todo el tráfico hacia la plataforma web viaje encriptado bajo el protocolo HTTPS.

8.4.3 Consumo de API y visualización de datos

La comunicación entre la aplicación web y el *backend serverless* se gestionará a través del módulo **HttpClient** nativo de Angular. Los servicios del *frontend* realizarán peticiones asíncronas hacia los *endpoints* expuestos en Amazon API Gateway para recuperar la información consolidada en la base de datos PostgreSQL y las imágenes almacenadas en S3.

Dado que el objetivo principal de la plataforma web es el análisis de resultados, se integrará la biblioteca **D3.js** (Data-Driven Documents). D3.js se utilizará para renderizar representaciones gráficas interactivas y complejas basadas en datos, tales como la distribución espacial de los árboles infectados, gráficas de series de tiempo sobre la propagación de insectos descortezadores y diagramas de calor. La manipulación directa del DOM que ofrece D3.js permitirá generar visualizaciones estadísticas altamente personalizadas que no serían posibles con bibliotecas de gráficos genéricas.

8.4.4 Autenticación y seguridad en el cliente

El acceso a la plataforma web estará estrictamente controlado mediante **Amazon Cognito**. El flujo de seguridad se implementará de la siguiente manera:

1. **Inicio de sesión:** Los usuarios administradores se autenticarán ingresando sus credenciales en un formulario conectado al *User Pool* de Cognito.
2. **Gestión de Tokens:** Tras una autenticación exitosa, Cognito devolverá tokens de seguridad (ID Token y Access Token) basados en el estándar JWT. Estos tokens se almacenarán de forma segura en el navegador del cliente.
3. **Interceptores HTTP:** Se configurará un *HttpInterceptor* en Angular que capturará todas las peticiones salientes hacia el API Gateway y adjuntará automáticamente el *Access Token* en la cabecera `Authorization`.
4. **Protección de rutas (*Guards*):** A nivel de navegación, se implementarán *Route Guards* de Angular (`CanActivate`) para verificar la validez del token local antes de permitir el acceso a las vistas administrativas. Si un usuario no está autenticado o su sesión ha expirado, será redirigido inmediatamente a la pantalla de *login*.

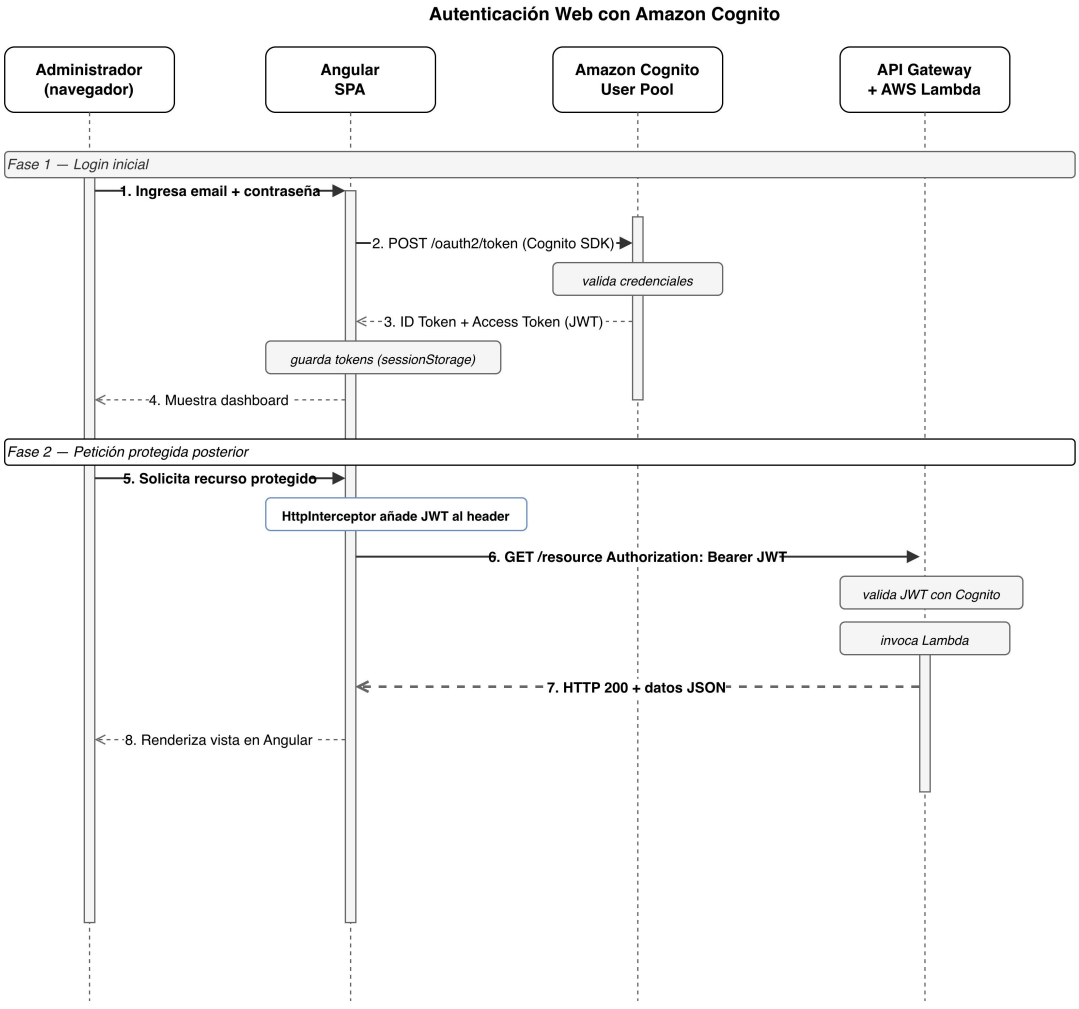


Figura 23: *Flujo de autenticación y consumo seguro de API en la aplicación web.*

8.5 Arquitecura del modelo de clasificación y análisis cromático

Este modelo aborda el proceso de entrada y salida para la identificación de patrones tanto de forma local como apoyado de AWS.

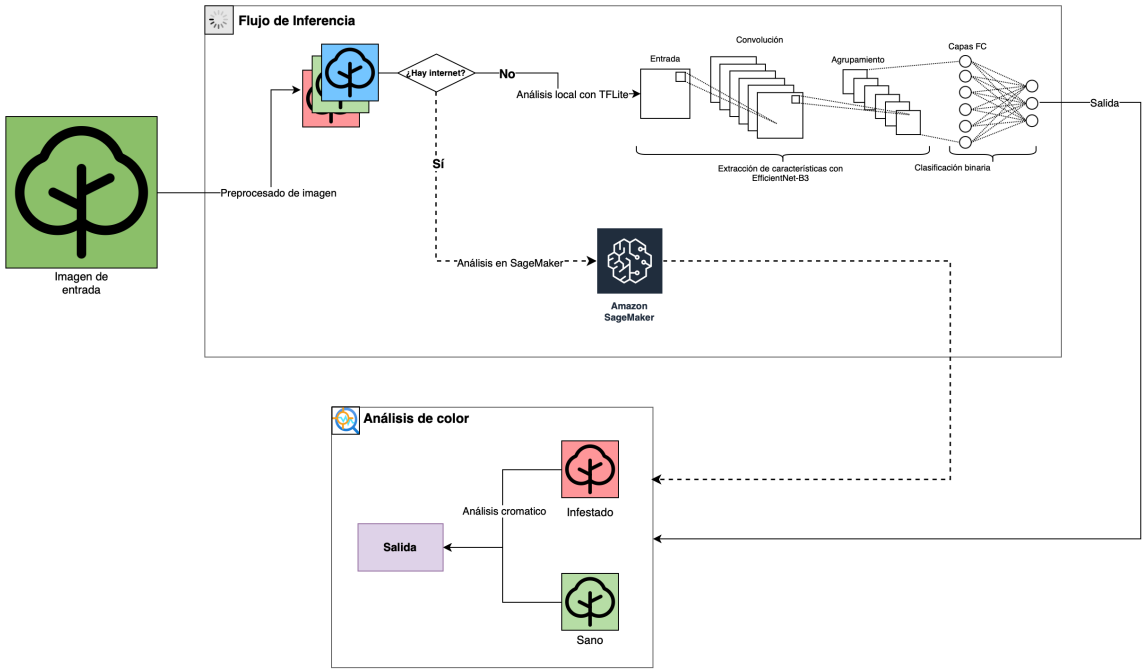


Figura 24: *Arquitectura general del modelo*

La Figura 24 ilustra el flujo completo de inferencia del sistema, desde la captura de la imagen hasta la generación del resultado final. El proceso se divide en dos etapas principales: el flujo de inferencia mediante el modelo de clasificación y el análisis cromático complementario.

En la primera etapa, la imagen de entrada —correspondiente a una fotografía de la corteza de un árbol capturada desde la aplicación móvil— es sometida a un preprocesado que la adecúa al formato requerido por el modelo: redimensionamiento a 300×300 píxeles y normalización mediante `efficientnet.preprocess_input`. Una vez preprocesada, el sistema evalúa el acceso a internet para determinar la ruta de inferencia. Si no hay conexión, el análisis se ejecuta localmente en el dispositivo mediante el modelo local con TFLite embebido en la aplicación móvil, garantizando la operación en zonas del Bosque de Chapultepec con cobertura de red limitada. Si hay conexión disponible, la imagen es enviada a Amazon SageMaker para su procesamiento en la nube, lo que permite aprovechar mayor capacidad de cómputo y facilitar el reentrenamiento del modelo con nuevas muestras capturadas. En ambos casos, la imagen atraviesa la arquitectura EfficientNet-B3, que extrae características visuales mediante capas de convolución y agrupamiento, y las transfiere a las capas densas que ejecutan la clasificación binaria, produciendo una salida con la probabilidad de infestación.

En la segunda etapa, el resultado de la clasificación independientemente de si provino del dispositivo o de SageMaker— es enviado al módulo de análisis de color. Este módulo aplica un análisis cromático sobre la imagen original para complementar la predicción: si el árbol es clasificado como infestado, se analiza la distribución de tonalidades en la corteza en busca de patrones asociados a daño por descortezadores, como cambios en la pigmentación derivados de la pérdida de resina. El resultado combinado de ambos análisis es consolidado en la salida final del sistema, que reporta el estado del árbol (sano o infestado).

8.6 Entorno de desarrollo y pruebas locales

Previo al despliegue definitivo en la infraestructura de Amazon Web Services, el ciclo de vida del *software* contempla una etapa iterativa de desarrollo y pruebas en un entorno local controlado. Esta estrategia permite validar la lógica de negocio, la integración de los componentes y la ejecución de los modelos de visión por computadora sin incurrir en latencia de red ni en costos de facturación en la nube.

8.6.1 Entorno de desarrollo local

El entorno de trabajo en la estación de desarrollo emulará los servicios clave de producción mediante el uso de virtualización ligera y herramientas de línea de comandos:

- **Backend y Base de Datos:** Se utilizará **Docker** para instanciar contenedores del motor **PostgreSQL**, aislando la base de datos del sistema operativo anfitrión. El *backend* desarrollado en **Micronaut** se gestionará y compilará utilizando la herramienta de construcción **Gradle**, ejecutándose localmente para probar rápidamente los *endpoints* REST y las transacciones mediante clientes HTTP.
- **Frontend Web:** La aplicación **Angular** se servirá mediante su servidor de desarrollo local (`Angular CLI`), consumiendo la API de Micronaut expuesta en `localhost`.
- **Aplicación Móvil:** El cliente Android se compilará a través de las herramientas nativas del SDK. Las pruebas iniciales del preprocesamiento con **OpenCV** y la inferencia con **TensorFlow Lite** se validarán inyectando bancos de imágenes de prueba directamente en el entorno local del dispositivo o emulador, antes de proceder con las pruebas en campo.

8.6.2 Servidor de pruebas *on-premise* (Raspberry Pi)

Como puente de transición entre el entorno local y el despliegue final en la nube, el sistema integrará una **Raspberry Pi** configurada como un servidor de pruebas (*staging environment*).

Al basarse también en arquitectura ARM, este dispositivo garantizará una transición fluida y consistente para las imágenes construidas en la estación de trabajo principal. Se desplegarán en la Raspberry Pi los contenedores de la base de datos, las versiones pre-lanzamiento del *backend* (Micronaut/Gradle) y la aplicación *frontend* (Angular).

Este entorno *on-premise* permitirá al equipo realizar pruebas de integración continuas de toda la pila tecnológica en una red de área local real, comprobando que la sincronización asíncrona del dispositivo móvil funcione correctamente al reconectarse a la red.

8.6.3 Orquestación y barrera de despliegue

El puente definitivo entre los entornos locales y la nube será gestionado por **GitHub Actions**. Como se ilustra en la arquitectura de despliegue, el *pipeline* automatizado actuará como un guardián de calidad estricto: si las pruebas unitarias o de integración fallan, el proceso se aborta inmediatamente y el artefacto es descartado (*no deploy*).

Únicamente si el código aprueba los filtros de calidad, GitHub Actions orquestará la distribución hacia los entornos correspondientes, bifurcando los artefactos de producción: el empaquetado del *backend* se enviará hacia AWS Lambda (con acceso a Amazon RDS y S3), mientras que los

estáticos del *frontend* se dirigirán a Amazon S3 para ser distribuidos globalmente a través de Amazon CloudFront y protegidos por AWS Certificate Manager.

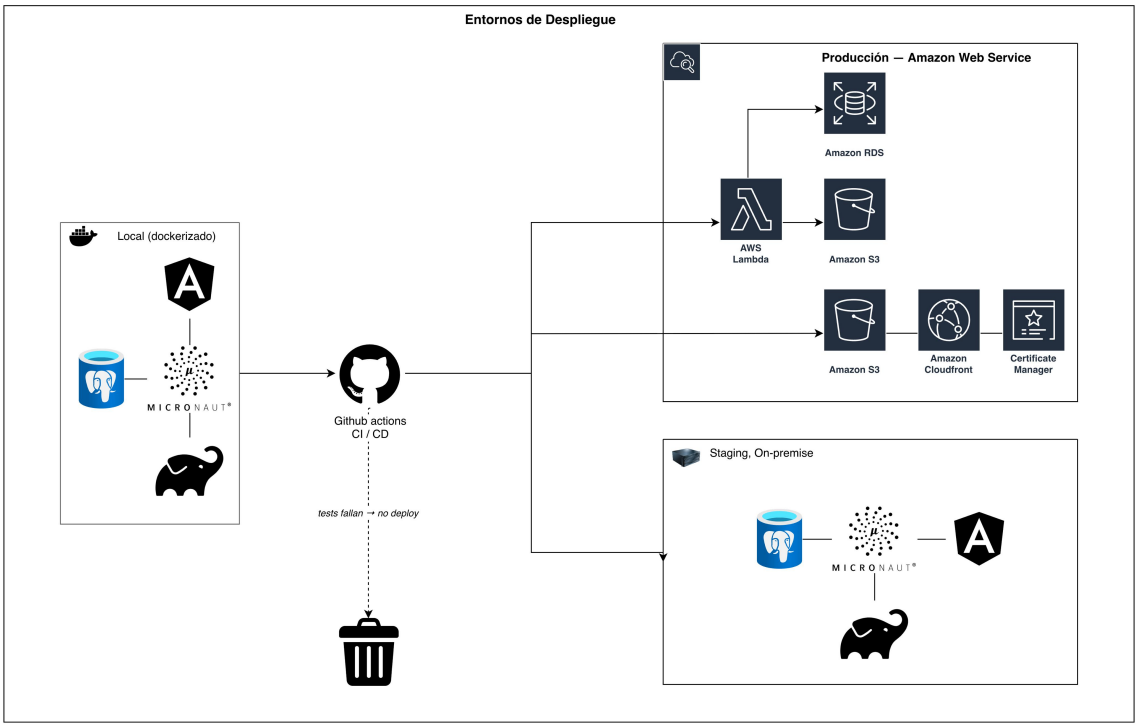


Figura 25: *Arquitectura de entornos: desarrollo local, pruebas on-premise y orquestación hacia producción.*

8.7 Diagramas de casos de uso

Un diagrama de casos de uso es una representación visual perteneciente al lenguaje de modelado UML *(Unified Modeling Language)* que describe las interacciones entre los actores del sistema y las funcionalidades que este ofrece. Su propósito es comunicar, de forma clara y estructurada, qué puede hacer el sistema y quién puede hacerlo, sin entrar en detalles de implementación. Cada caso de uso representa una funcionalidad específica que aporta valor al actor que la ejecuta, y las relaciones entre casos de uso permiten expresar dependencias obligatorias mediante *«include»* o comportamientos opcionales mediante *«extend»*.

Dado que el sistema se compone de dos subsistemas complementarios, se presentan cuatro tipos de nomenclatura de etiquetado de casos de uso:

- **CU-GX — General**: permite identificar los elementos compartidos entre la app móvil y la plataforma web.
- **CU-MX — App móvil**: detalla las funcionalidades disponibles para el actor Experto en la aplicación Android, abarcando desde el inicio de sesión hasta el flujo completo de captura, análisis y sincronización de registros en campo.
- **CU-WX —Plataforma web**: detalla las funcionalidades disponibles para el actor Experto en la plataforma web, orientada al monitoreo histórico del arbolado, la gestión de registros y la generación de reportes.
- **CU-AX — Administración**: detalla las funcionalidades exclusivas del actor Administrador dentro de la plataforma web, incluyendo la gestión de usuarios.

A continuación se presentan los diagramas de casos de uso que conforman el modelo de interacción del sistema.

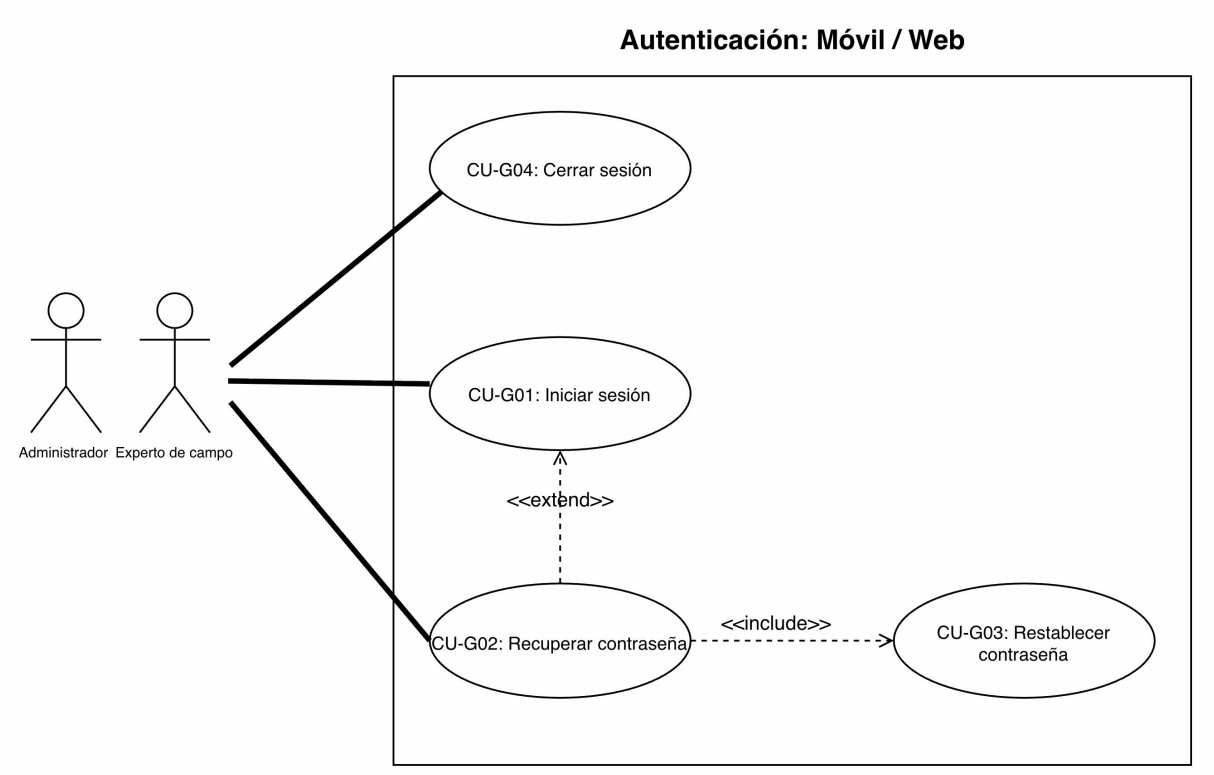


Figura 26: *CU-G01–CU-G04: Autenticación móvil / web*

El diagrama representa el flujo general de autenticación del sistema. El Experto de campo puede acceder desde la aplicación móvil y la plataforma web, mientras que el Administrador accede

únicamente desde la plataforma web. Ambos actores comparten los casos de uso de iniciar sesión, recuperar contraseña, restablecer contraseña y cerrar sesión. La recuperación de contraseña extiende al inicio de sesión cuando el usuario no recuerda sus credenciales, y el restablecimiento forma parte del proceso de recuperación mediante correo electrónico.

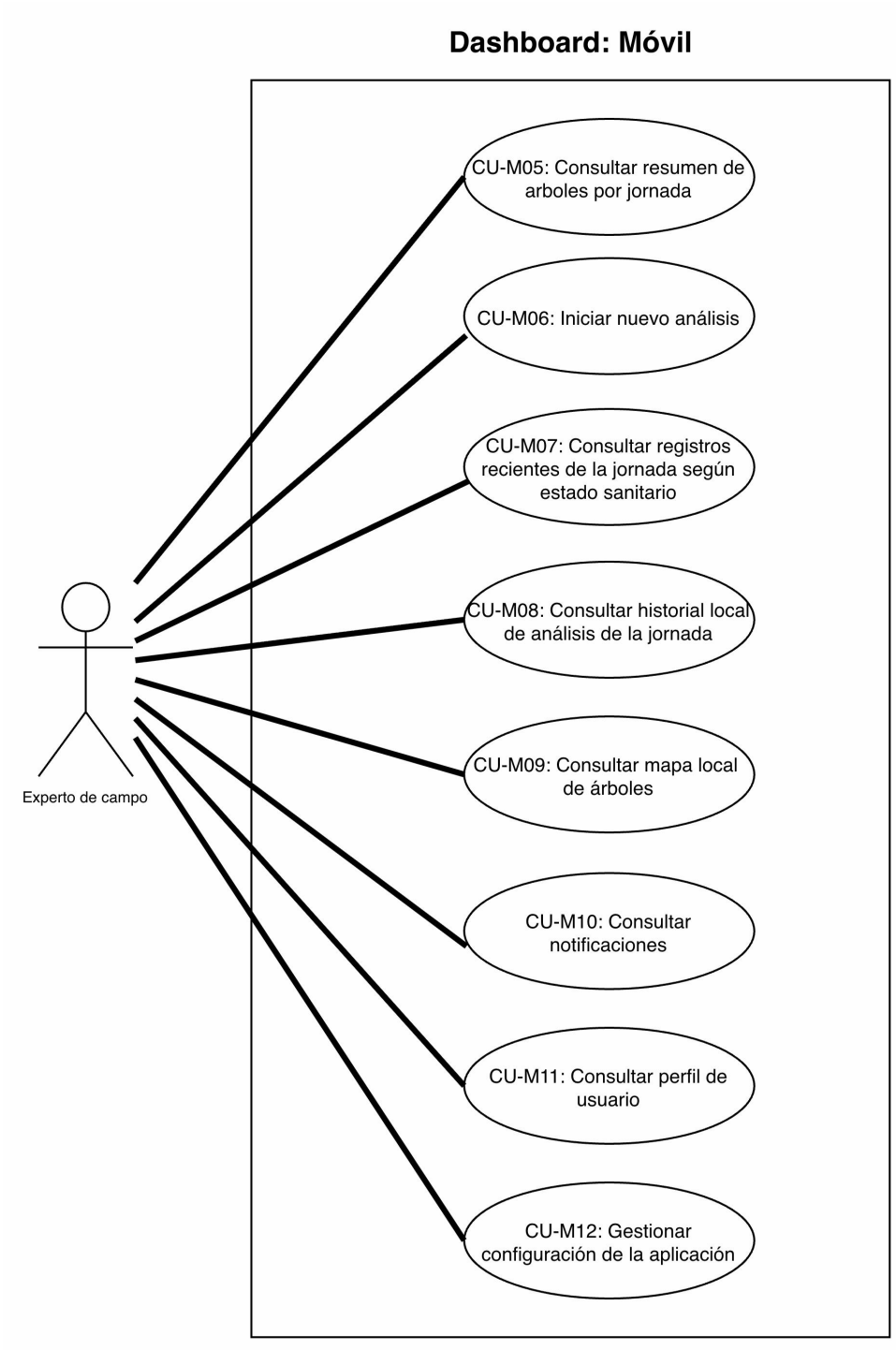


Figura 27: *CU-M05–CU-M12: Dashboard móvil*

El diagrama muestra las acciones principales disponibles para el Experto de campo desde la pantalla inicial de la aplicación móvil. Desde el dashboard puede consultar el resumen de árboles por jornada, iniciar un nuevo análisis, revisar registros recientes según estado sanitario, consultar el historial local, acceder al mapa local de árboles, revisar notificaciones, consultar su perfil y gestionar la configuración de la aplicación.

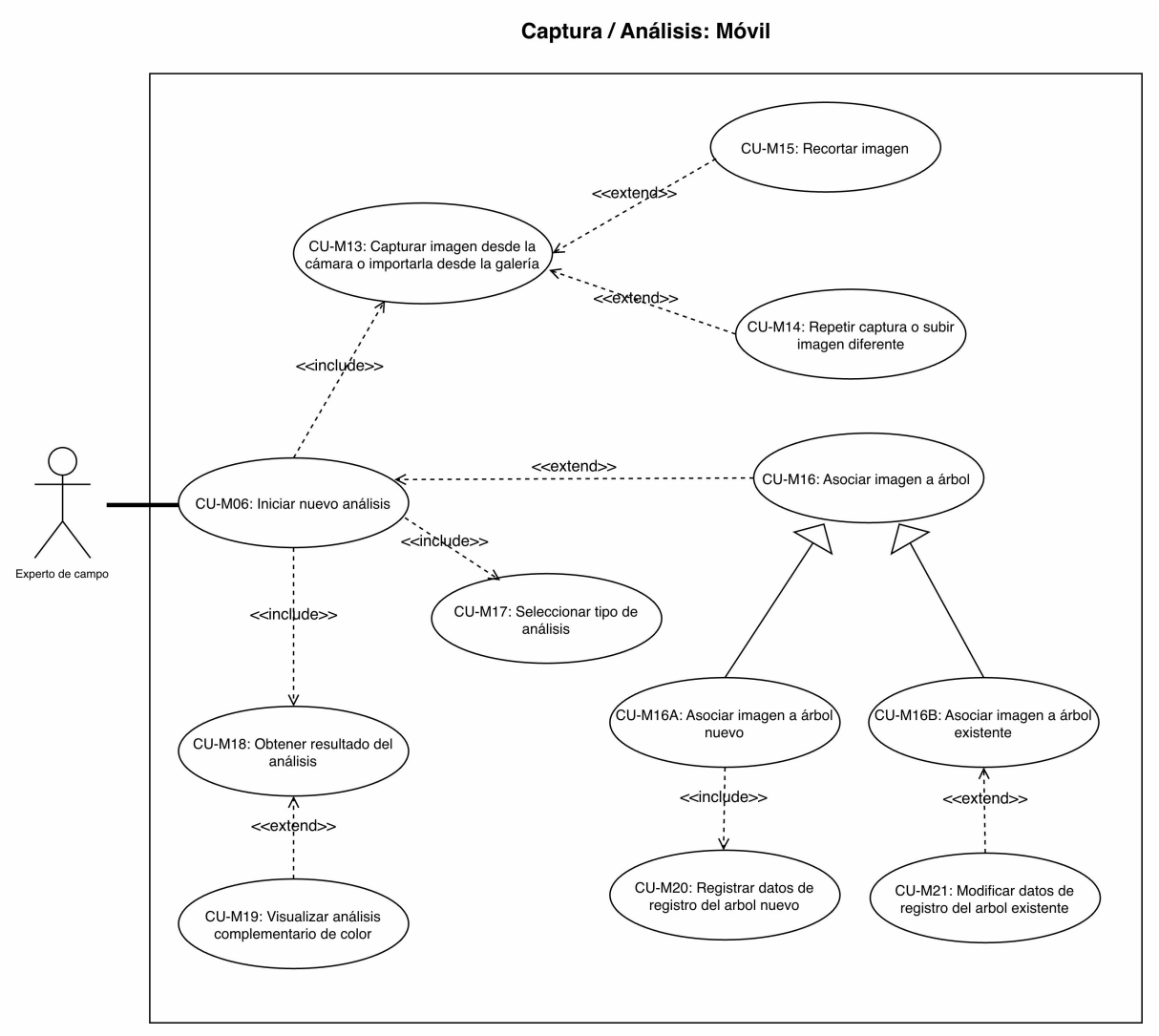


Figura 28: *CU-M06 y CU-M13–CU-M21: Captura y análisis móvil*

El diagrama muestra las acciones disponibles para el Experto de campo al iniciar un nuevo análisis desde la aplicación móvil. El usuario puede capturar una imagen desde la cámara o importarla desde la galería, recortarla o repetir la captura si es necesario. La imagen debe asociarse a un árbol, ya sea nuevo o existente. Si el árbol es nuevo, se registran sus datos; si ya existe, sus datos pueden modificarse. El sistema obtiene la clasificación del estado sanitario y, cuando aplica, permite visualizar el análisis complementario de color.

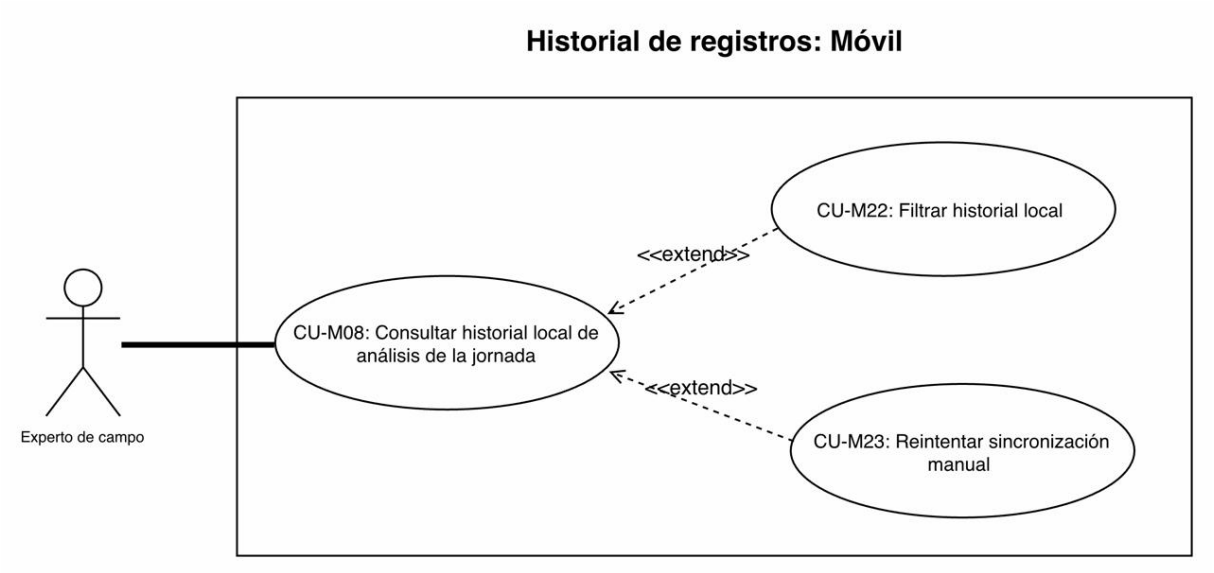


Figura 29: *CU-M08 y CU-M22–CU-M23: Historial de registros móvil*

El diagrama muestra las funciones disponibles en el historial local de la aplicación móvil. El Experto de campo puede consultar los análisis realizados durante la jornada en el dispositivo, filtrar el historial local y reintentar manualmente la sincronización cuando algún registro presente error. El historial móvil utiliza únicamente la base local SQLite del dispositivo en uso.

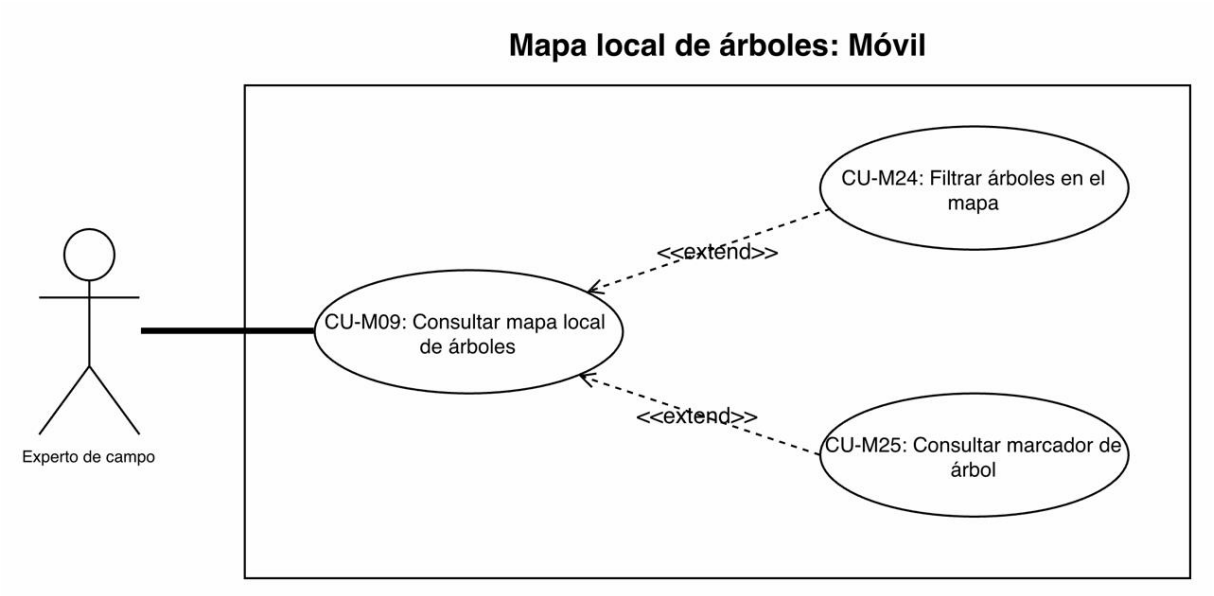


Figura 30: *CU-M09 y CU-M24–CU-M25: Mapa local de árboles móvil*

El diagrama muestra las acciones disponibles en el mapa local de la aplicación móvil. El Experto de campo puede consultar los árboles registrados en el dispositivo, aplicar filtros en el mapa y revisar la información de un marcador seleccionado. La información mostrada corresponde únicamente a los registros almacenados en la base local SQLite del dispositivo móvil en uso.

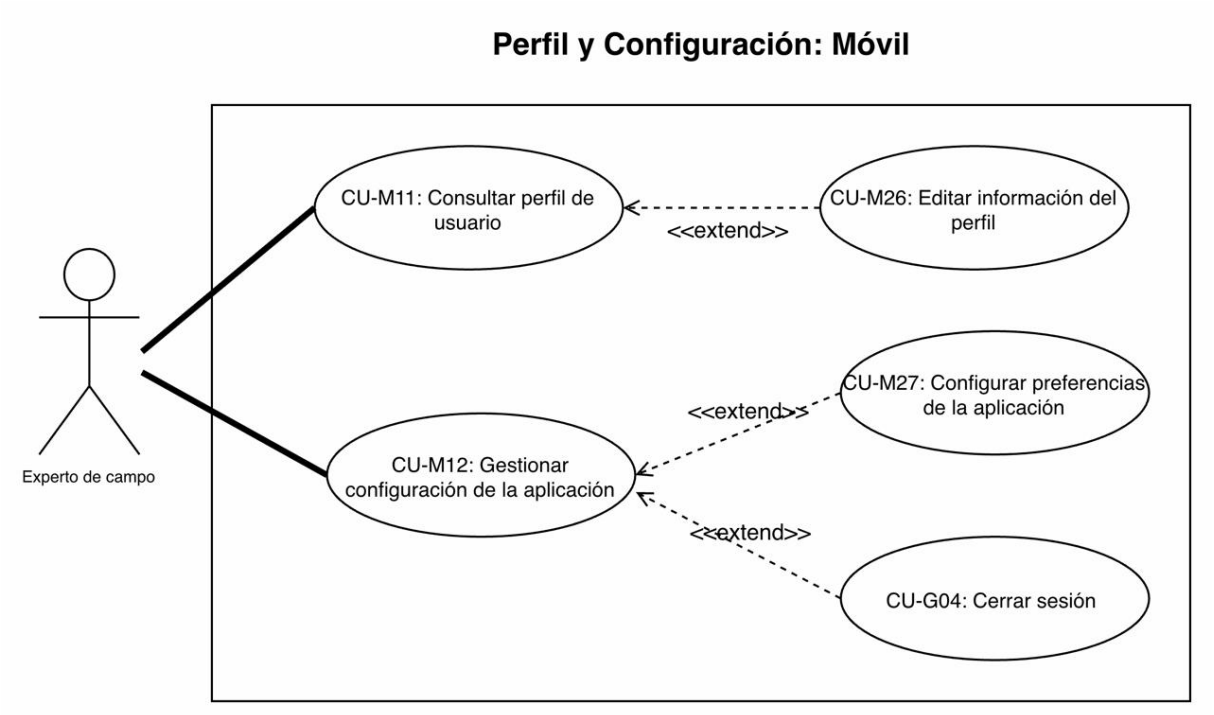


Figura 31: *CU-M11–CU-M12 y CU-M26–CU-M27, CU-G04: Perfil y configuración móvil*

El diagrama muestra las funciones disponibles para el Experto de campo dentro del perfil y configuración de la aplicación móvil. Desde esta sección puede consultar y editar información de su perfil, gestionar la configuración de la aplicación, configurar preferencias y cerrar sesión. El cierre de sesión reutiliza el caso de uso general de autenticación.

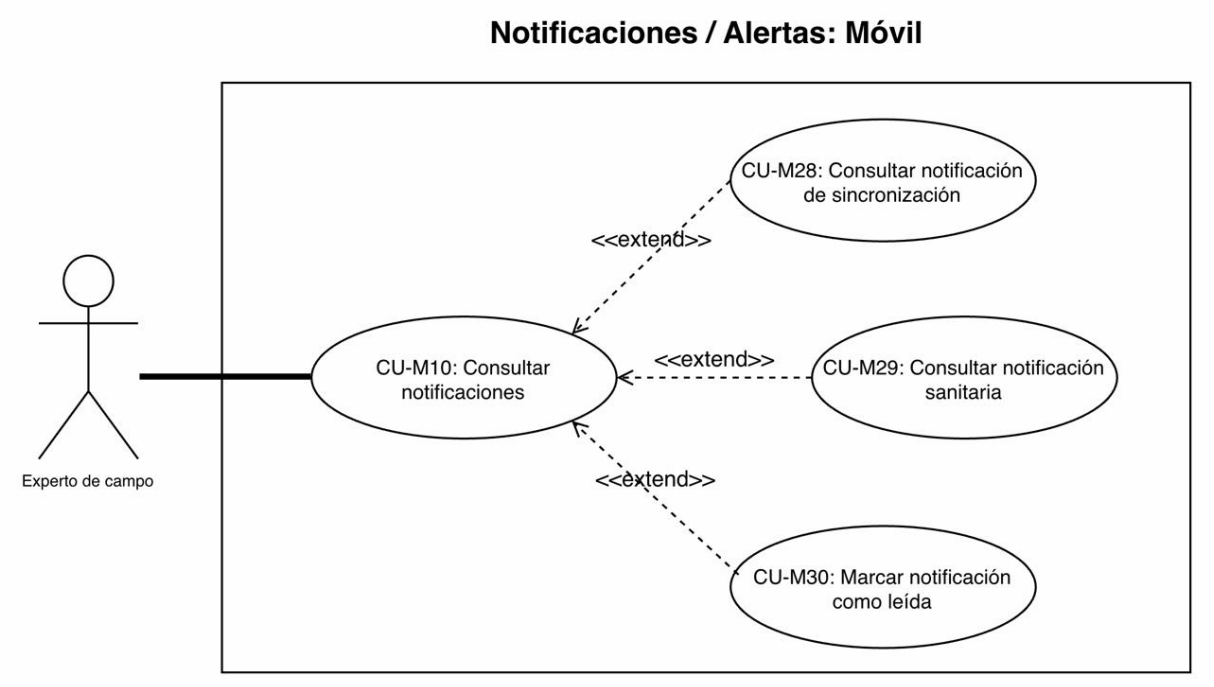


Figura 32: *CU-M10 y CU-M28–CU-M30: Notificaciones y alertas móvil*

El diagrama muestra las acciones disponibles en la sección de notificaciones de la aplicación móvil. El Experto de campo puede consultar notificaciones generales, revisar notificaciones de sincronización, consultar notificaciones sanitarias y marcar notificaciones como leídas. Las

notificaciones de sincronización informan sobre registros pendientes, sincronizados o con error, mientras que las sanitarias alertan sobre resultados relevantes del análisis o posibles riesgos asociados al estado sanitario de los árboles.

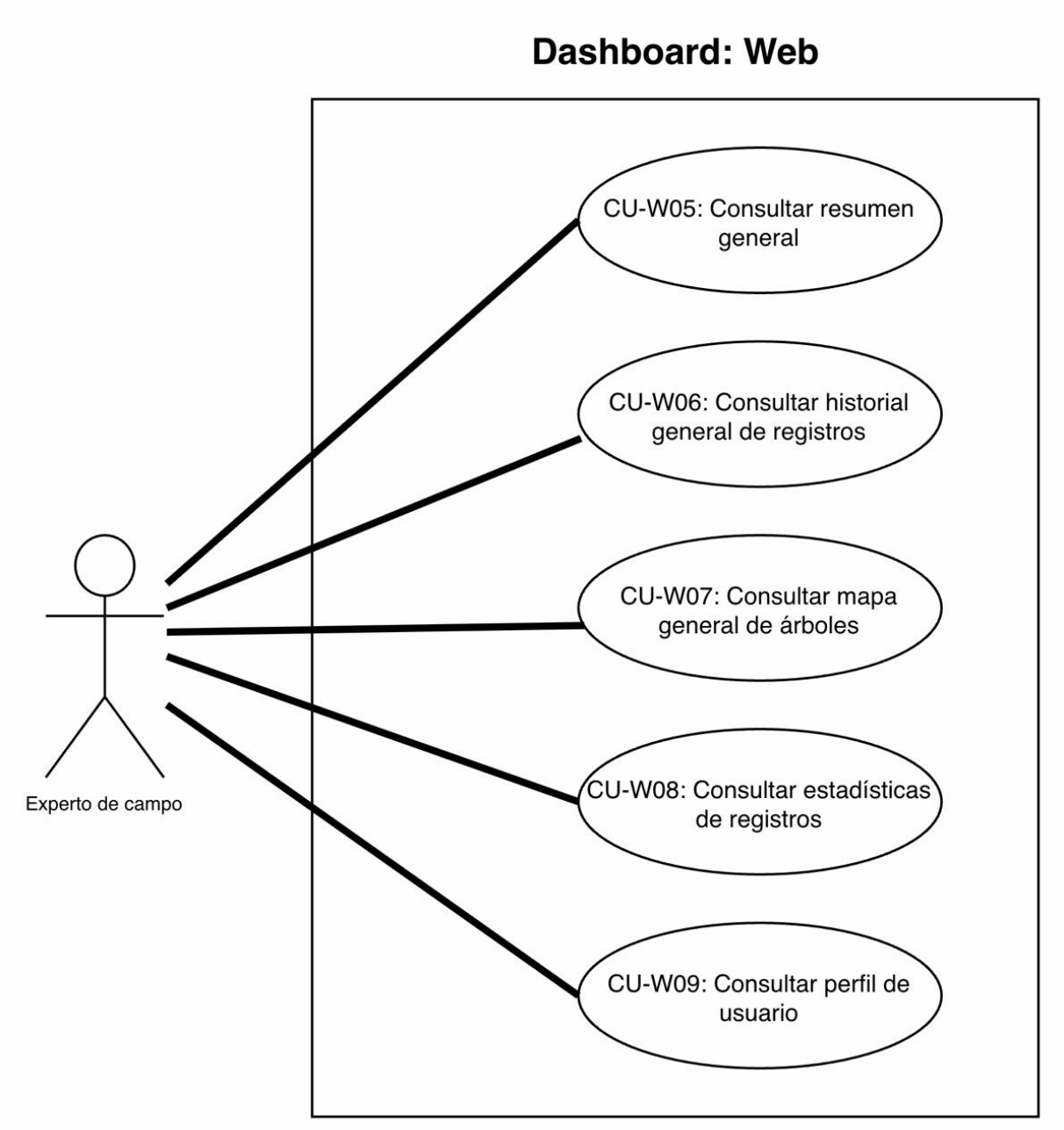


Figura 33: *CU-W05–CU-W09: Dashboard web*

El diagrama representa las acciones principales disponibles para el Experto de campo desde el dashboard de la plataforma web. A diferencia del dashboard móvil, esta vista consulta información consolidada en la nube. Desde ella, el usuario puede consultar el resumen general, acceder al historial general de registros, visualizar el mapa general de árboles, revisar estadísticas y consultar su perfil.

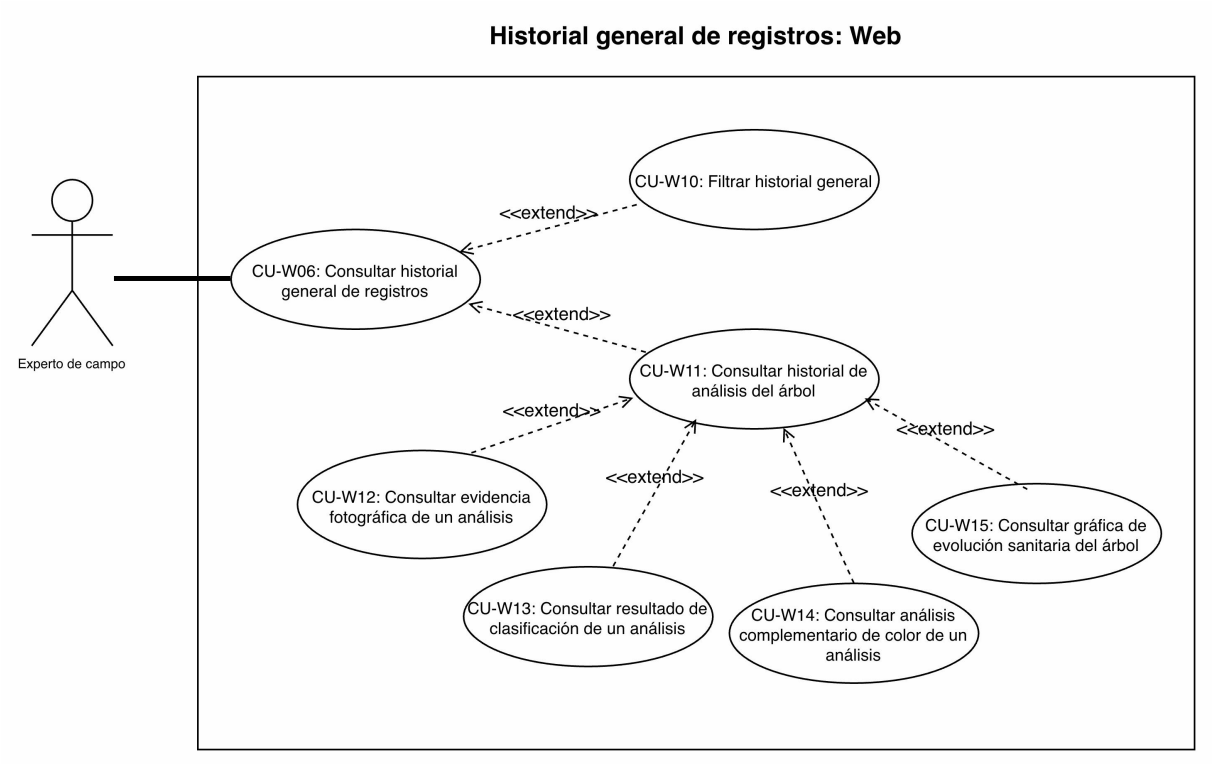


Figura 34: *CU-W06 y CU-W10–CU-W15: Historial general de registros web*

El diagrama representa la consulta del historial general de registros desde la plataforma web. El Experto de campo puede consultar los registros sincronizados en la nube, aplicar filtros generales y acceder al historial de análisis de un árbol. Desde dicho historial puede consultar la evidencia fotográfica, el resultado de clasificación, el análisis complementario de color y la gráfica de evolución sanitaria del árbol. La plataforma web funciona como una vista consolidada de consulta, sin edición de registros operativos.

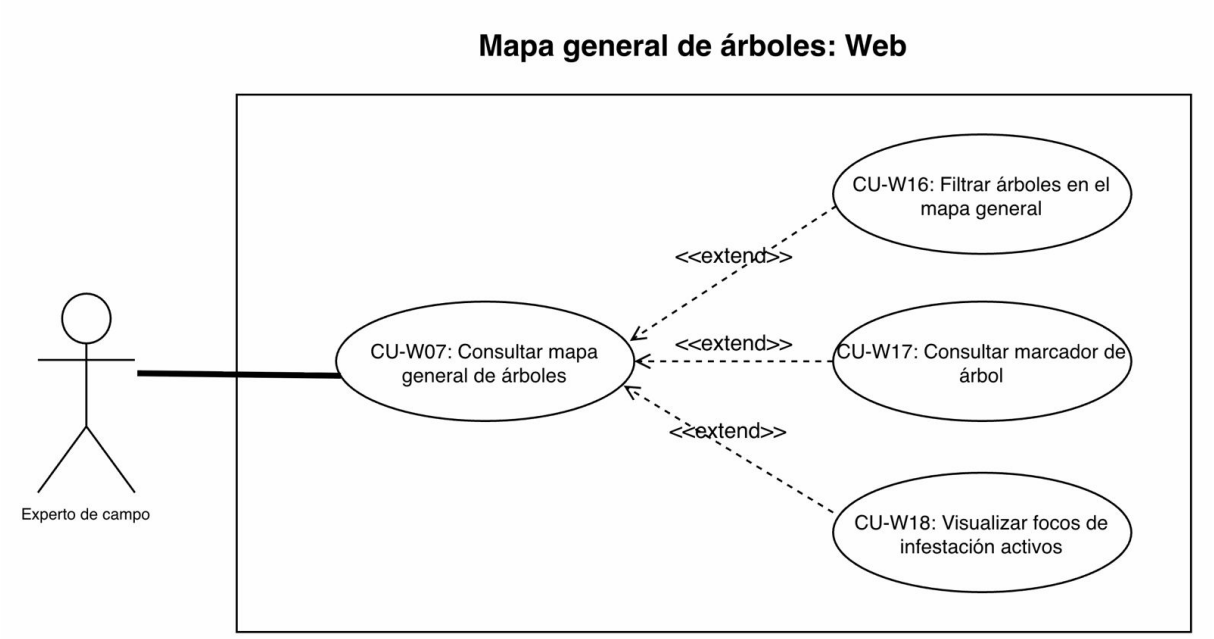


Figura 35: *CU-W07 y CU-W16–CU-W18: Mapa general de árboles web*

El diagrama representa la consulta del mapa general de árboles desde la plataforma web. El

Experto de campo puede visualizar los árboles sincronizados desde todos los dispositivos asociados a su cuenta, aplicar filtros sobre el mapa, consultar marcadores específicos y visualizar focos de infestación activos generados a partir de los registros consolidados en la nube.

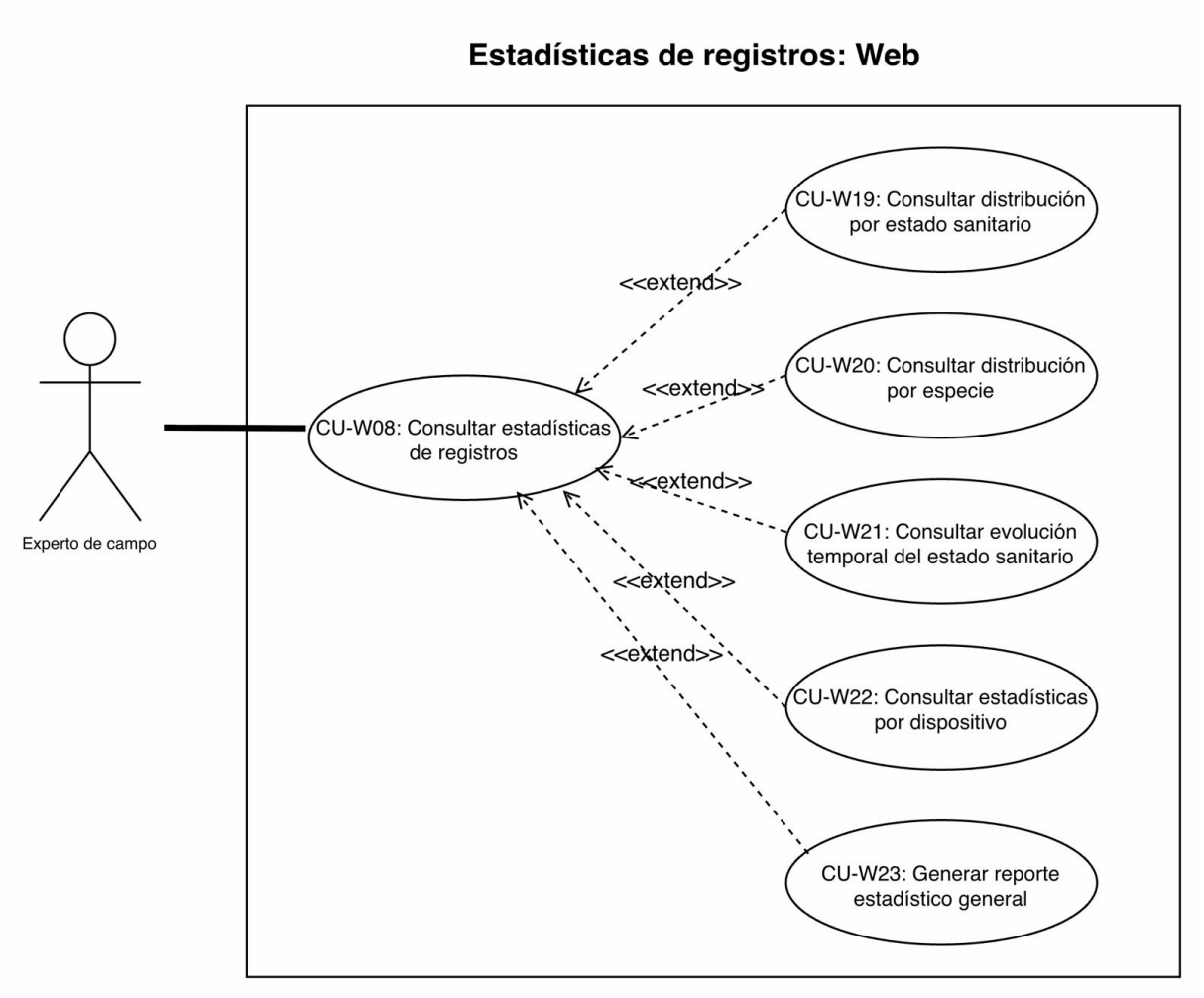


Figura 36: *CU-W08 y CU-W19–CU-W23: Estadísticas de registros web*

El diagrama muestra las funciones estadísticas disponibles para el Experto de campo en la plataforma web. El usuario puede consultar estadísticas generadas a partir de los registros sincronizados en la nube, incluyendo distribución por estado sanitario, distribución por especie, evolución temporal del estado sanitario y estadísticas por dispositivo de origen. Esta sección es únicamente de consulta y no modifica los registros almacenados.

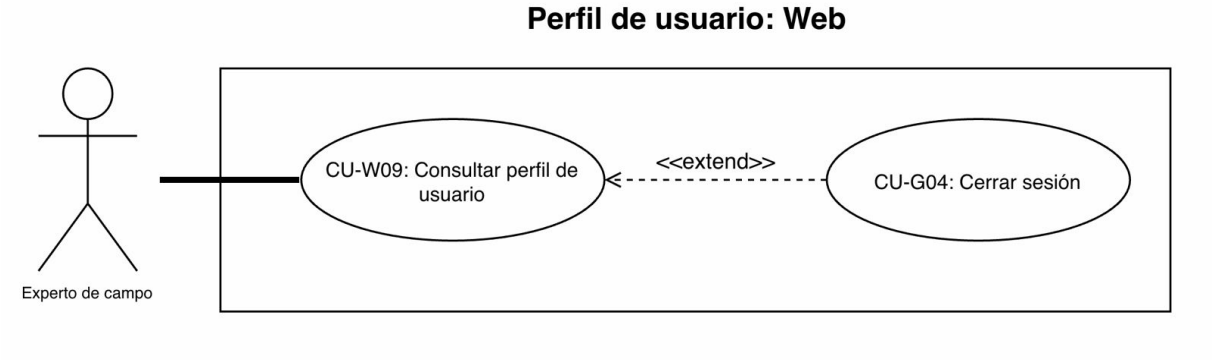


Figura 37: *CU-W09 y CU-G04: Perfil de usuario web*

El diagrama representa la consulta del perfil de usuario desde la plataforma web. El Experto de campo puede consultar la información de su cuenta y cerrar sesión. No se contemplan acciones de edición desde esta sección, ya que la plataforma web se mantiene como una vista de consulta y supervisión.

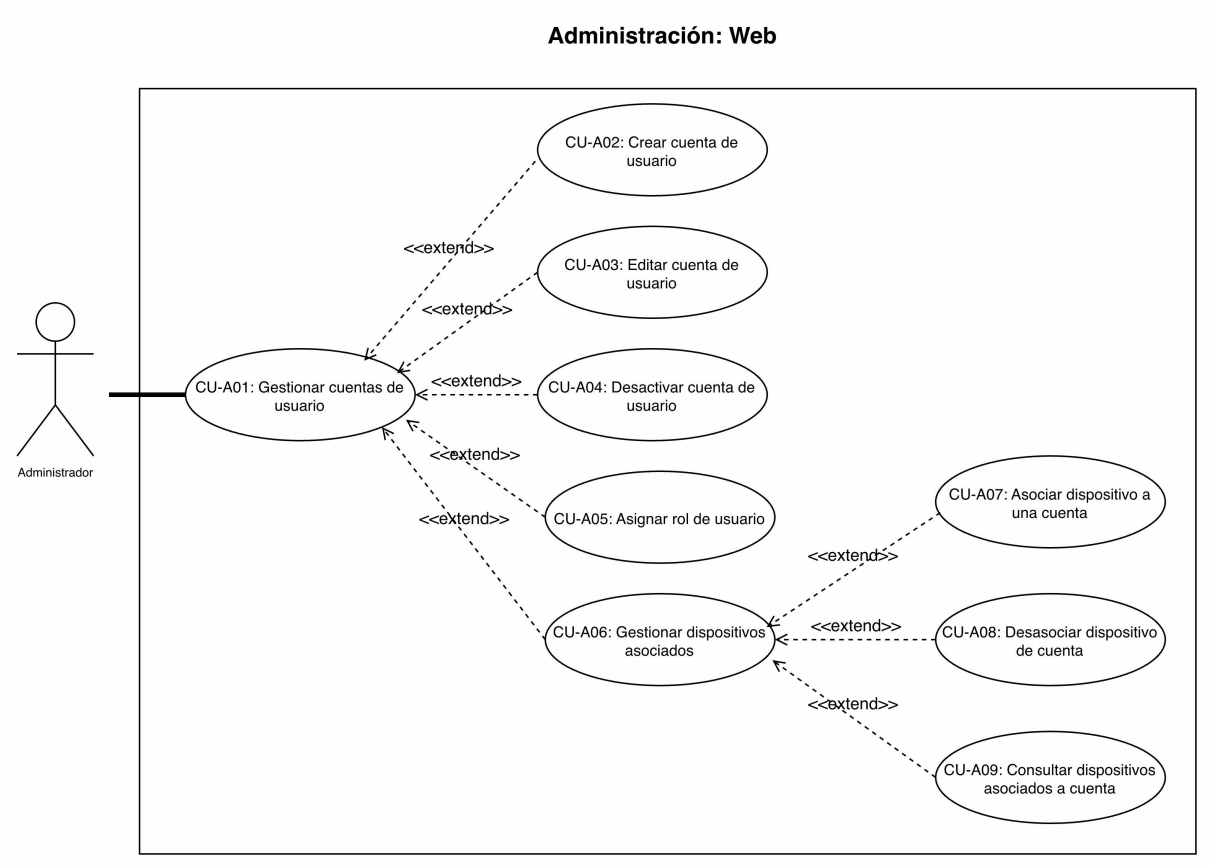


Figura 38: *CU-A01–CU-A09: Administración web*

El diagrama muestra las funciones disponibles para el Administrador dentro de la plataforma web. Desde este módulo puede gestionar cuentas de usuario, crear, editar o desactivar cuentas, asignar roles y administrar los dispositivos asociados a cada cuenta. La gestión de dispositivos permite consultar, asociar o desasociar dispositivos móviles autorizados para operar con una cuenta de experto.

8.8 Especificación de casos de uso

8.8.1 Casos de uso generales

CU-G01 Iniciar sesión

Caso de uso: CU-G01 Iniciar sesión	
Resumen de atributos	
Actor:	*Experto de campo, Administrador*
Propósito:	Permitir el acceso al sistema mediante credenciales válidas, de acuerdo con el rol del usuario y la plataforma utilizada.
Entradas	• Correo electrónico • Contraseña
Salidas	Se muestra la pantalla principal correspondiente al rol y plataforma del usuario.
Precondiciones	• Debe existir una cuenta registrada en el sistema. • La cuenta debe encontrarse activa. • En la aplicación móvil, el dispositivo debe estar asociado a la cuenta del Experto de campo.
Postcondiciones	El usuario accede al sistema con los permisos correspondientes a su rol.
Errores	• MSG1: Dato requerido faltante. • MSG2: Correo electrónico no registrado. • MSG3: Contraseña incorrecta. • MSG4: Cuenta desactivada. • MSG5: Dispositivo no autorizado para esta cuenta.
Tipo	Primario
Fuente	Requerimientos funcionales del sistema
Observaciones	El Experto de campo puede iniciar sesión desde la aplicación móvil o desde la plataforma web. El Administrador accede únicamente desde la plataforma web.

Tabla 18: *Resumen de atributos del caso de uso: Iniciar sesión.*

Trayectorias del caso de uso

Trayectoria Principal

1. El **Usuario** abre la aplicación móvil o la plataforma web.
2. El **Sistema** muestra la pantalla de inicio de sesión.
3. El **Usuario** ingresa su correo electrónico.
4. El **Usuario** ingresa su contraseña.

5. El **Usuario** pulsa el botón **[Iniciar sesión]**.
6. El **Sistema** valida que los campos obligatorios no estén vacíos. Si algún campo obligatorio está vacío, ir a la **Trayectoria Alternativa A**.
7. El **Sistema** verifica que el correo electrónico exista. Si el correo no existe, ir a la **Trayectoria Alternativa B**.
8. El **Sistema** verifica que la contraseña ingresada sea correcta. Si la contraseña es incorrecta, ir a la **Trayectoria Alternativa C**.
9. El **Sistema** verifica que la cuenta se encuentre activa. Si la cuenta está desactivada, ir a la **Trayectoria Alternativa D**.
10. Si el acceso se realiza desde la aplicación móvil, el **Sistema** verifica que el dispositivo esté asociado a la cuenta del Experto de campo. Si el dispositivo no está autorizado, ir a la **Trayectoria Alternativa E**.
11. El **Sistema** identifica el rol del usuario.
12. El **Sistema** muestra la pantalla correspondiente:
    - Dashboard móvil, si el Experto de campo accede desde la aplicación móvil.
    - Dashboard web, si el Experto de campo accede desde la plataforma web.
    - Módulo de administración, si el Administrador accede desde la plataforma web.
13. Fin del caso de uso.

Trayectoria Alternativa A (Campos obligatorios vacíos)

1. El **Sistema** muestra el mensaje **MSG1**: “Dato requerido faltante.”
2. El **Sistema** permite al **Usuario** ingresar los datos faltantes.
3. Fin del caso de uso.

Trayectoria Alternativa B (Correo no registrado)

1. El **Sistema** muestra el mensaje **MSG2**: “El correo electrónico no está registrado.”
2. El **Sistema** permite al **Usuario** ingresar otro correo electrónico.
3. Fin del caso de uso.

Trayectoria Alternativa C (Contraseña incorrecta)

1. El **Sistema** muestra el mensaje **MSG3**: “Contraseña incorrecta.”
2. El **Sistema** permite al **Usuario** ingresar nuevamente la contraseña.
3. Fin del caso de uso.

Trayectoria Alternativa D (Cuenta desactivada)

1. El **Sistema** muestra el mensaje **MSG4**: "La cuenta se encuentra desactivada."
2. El **Sistema** impide el acceso al sistema.
3. Fin del caso de uso.

Trayectoria Alternativa E (Dispositivo no autorizado)

1. El **Sistema** muestra el mensaje **MSG5**: "El dispositivo no está autorizado para esta cuenta."
2. El **Sistema** impide el acceso desde el dispositivo móvil.
3. Fin del caso de uso.

CU-G02 Recuperar contraseña

Caso de uso: CU-G02 Recuperar contraseña	
Resumen de atributos	
Actor:	*Experto de campo, Administrador*
Propósito:	Permitir al usuario solicitar un mecanismo de recuperación de acceso cuando no recuerda su contraseña.
Entradas	• Correo electrónico asociado a la cuenta
Salidas	El sistema envía un enlace o código de recuperación al correo electrónico registrado.
Precondiciones	• Debe existir una cuenta registrada con el correo electrónico ingresado. • La cuenta debe estar activa. • El usuario debe tener acceso al correo electrónico asociado.
Postcondiciones	Se genera una solicitud de recuperación y el usuario puede continuar con el restablecimiento de contraseña.
Errores	• MSG1: Dato requerido faltante. • MSG2: Correo electrónico no registrado. • MSG4: Cuenta desactivada. • MSG6: No fue posible enviar el correo de recuperación.
Tipo	Secundario
Fuente	Requerimientos funcionales del sistema
Observaciones	Este caso de uso extiende al inicio de sesión cuando el usuario no recuerda su contraseña. El restablecimiento de contraseña se realiza mediante el enlace o código enviado al correo electrónico.

Tabla 19: *Resumen de atributos del caso de uso: Recuperar contraseña.*

Trayectorias del caso de uso

Trayectoria Principal

1. El **Usuario** se encuentra en la pantalla de inicio de sesión.
2. El **Usuario** selecciona la opción **[Recuperar contraseña]**.
3. El **Sistema** muestra la pantalla de recuperación de contraseña.
4. El **Usuario** ingresa el correo electrónico asociado a su cuenta.
5. El **Usuario** pulsa el botón **[Enviar]**.
6. El **Sistema** valida que el campo de correo electrónico no esté vacío. Si el campo está vacío, ir a la **Trayectoria Alternativa A**.
7. El **Sistema** verifica que el correo electrónico esté asociado a una cuenta registrada. Si el correo no existe, ir a la **Trayectoria Alternativa B**.
8. El **Sistema** verifica que la cuenta se encuentre activa. Si la cuenta está desactivada, ir a la **Trayectoria Alternativa C**.
9. El **Sistema** genera un enlace o código de recuperación.
10. El **Sistema** envía el enlace o código al correo electrónico registrado. Si el correo no puede enviarse, ir a la **Trayectoria Alternativa D**.
11. El **Sistema** muestra un mensaje indicando que se envió la información de recuperación.
12. Fin del caso de uso.

Trayectoria Alternativa A (Correo no ingresado)

1. El **Sistema** muestra el mensaje **MSG1**: “Dato requerido faltante.”
2. El **Sistema** permite al **Usuario** ingresar el correo electrónico.
3. Fin del caso de uso.

Trayectoria Alternativa B (Correo no registrado)

1. El **Sistema** muestra el mensaje **MSG2**: “El correo electrónico no está registrado.”
2. El **Sistema** permite al **Usuario** ingresar otro correo electrónico.
3. Fin del caso de uso.

Trayectoria Alternativa C (Cuenta desactivada)

1. El **Sistema** muestra el mensaje **MSG4**: “La cuenta se encuentra desactivada.”
2. El **Sistema** impide continuar con el proceso de recuperación.
3. Fin del caso de uso.

Trayectoria Alternativa D (Error al enviar correo)

1. El **Sistema** muestra el mensaje **MSG6**: “No fue posible enviar el correo de recuperación.”
2. El **Sistema** permite al **Usuario** intentar nuevamente el envío.
3. Fin del caso de uso.

CU-G03 Restablecer contraseña

Caso de uso: CU-G03 Restablecer contraseña	
Resumen de atributos	
Actor:	*Experto de campo, Administrador*
Propósito:	Permitir al usuario definir una nueva contraseña después de haber solicitado la recuperación de acceso.
Entradas	• Enlace o código de recuperación • Nueva contraseña • Confirmación de nueva contraseña
Salidas	La contraseña de la cuenta se actualiza correctamente y el usuario puede iniciar sesión con la nueva contraseña.
Precondiciones	• El usuario debe haber solicitado la recuperación de contraseña. • El enlace o código de recuperación debe ser válido. • El enlace o código de recuperación no debe estar expirado.
Postcondiciones	La cuenta queda asociada a la nueva contraseña definida por el usuario.
Errores	• MSG1: Dato requerido faltante. • MSG7: Enlace o código de recuperación inválido. • MSG8: Enlace o código de recuperación expirado. • MSG9: Las contraseñas no coinciden. • MSG10: La contraseña no cumple con los requisitos de seguridad.
Tipo	Secundario
Fuente	Requerimientos funcionales del sistema
Observaciones	Este caso de uso forma parte del proceso de recuperación de contraseña. Requiere que el usuario haya recibido previamente un enlace o código de recuperación en su correo electrónico.

Tabla 20: *Resumen de atributos del caso de uso: Restablecer contraseña.*

Trayectorias del caso de uso

Trayectoria Principal

1. El **Usuario** abre el enlace de recuperación recibido por correo electrónico o ingresa el código de recuperación correspondiente.
2. El **Sistema** valida que el enlace o código de recuperación sea válido. Si el enlace o código es inválido, ir a la **Trayectoria Alternativa A**.
3. El **Sistema** valida que el enlace o código de recuperación no haya expirado. Si el enlace o código expiró, ir a la **Trayectoria Alternativa B**.
4. El **Sistema** muestra la pantalla para restablecer contraseña.
5. El **Usuario** ingresa la nueva contraseña.
6. El **Usuario** confirma la nueva contraseña.
7. El **Usuario** pulsa el botón **[Restablecer contraseña]**.
8. El **Sistema** valida que los campos obligatorios no estén vacíos. Si algún campo obligatorio está vacío, ir a la **Trayectoria Alternativa C**.
9. El **Sistema** verifica que la nueva contraseña y la confirmación coincidan. Si las contraseñas no coinciden, ir a la **Trayectoria Alternativa D**.
10. El **Sistema** valida que la nueva contraseña cumpla con los requisitos de seguridad definidos. Si la contraseña no cumple con los requisitos, ir a la **Trayectoria Alternativa E**.
11. El **Sistema** actualiza la contraseña de la cuenta.
12. El **Sistema** invalida el enlace o código de recuperación utilizado.
13. El **Sistema** muestra un mensaje indicando que la contraseña fue restablecida correctamente.
14. Fin del caso de uso.

Trayectoria Alternativa A (Enlace o código inválido)

1. El **Sistema** muestra el mensaje **MSG7**: “El enlace o código de recuperación no es válido.”
2. El **Sistema** impide continuar con el restablecimiento de contraseña.
3. Fin del caso de uso.

Trayectoria Alternativa B (Enlace o código expirado)

1. El **Sistema** muestra el mensaje **MSG8**: “El enlace o código de recuperación ha expirado.”
2. El **Sistema** solicita al **Usuario** iniciar nuevamente el proceso de recuperación de contraseña.
3. Fin del caso de uso.

Trayectoria Alternativa C (Campos obligatorios vacíos)

1. El **Sistema** muestra el mensaje **MSG1**: "Dato requerido faltante."
2. El **Sistema** permite al **Usuario** ingresar los datos faltantes.
3. Fin del caso de uso.

Trayectoria Alternativa D (Contraseñas no coinciden)

1. El **Sistema** muestra el mensaje **MSG9**: "Las contraseñas no coinciden."
2. El **Sistema** permite al **Usuario** ingresar nuevamente la contraseña y su confirmación.
3. Fin del caso de uso.

Trayectoria Alternativa E (Contraseña insegura)

1. El **Sistema** muestra el mensaje **MSG10**: "La contraseña no cumple con los requisitos de seguridad."
2. El **Sistema** permite al **Usuario** ingresar una nueva contraseña.
3. Fin del caso de uso.

CU-G04 Cerrar sesión

Caso de uso: CU-G04 Cerrar sesión	
Resumen de atributos	
Actor:	*Experto de campo, Administrador*
Propósito:	Permitir al usuario finalizar de forma segura su sesión activa en la aplicación móvil o plataforma web.
Entradas	• Solicitud de cierre de sesión
Salidas	El sistema finaliza la sesión activa y muestra la pantalla de inicio de sesión.
Precondiciones	• El usuario debe haber iniciado sesión previamente. • Debe existir una sesión activa.
Postcondiciones	La sesión del usuario queda finalizada y no se permite el acceso a funciones protegidas sin iniciar sesión nuevamente.
Errores	• MSG11: No existe una sesión activa. • MSG12: No fue posible cerrar la sesión correctamente.
Tipo	Secundario
Fuente	Requerimientos funcionales del sistema
Observaciones	El cierre de sesión puede ejecutarse desde la configuración de la aplicación móvil o desde el perfil de usuario en la plataforma web.

Tabla 21: *Resumen de atributos del caso de uso: Cerrar sesión.*

Trayectorias del caso de uso

Trayectoria Principal

1. El **Usuario** se encuentra dentro de una sesión activa en la aplicación móvil o plataforma web.
2. El **Usuario** accede a la sección donde se encuentra la opción de cierre de sesión.
3. El **Usuario** selecciona la opción **[Cerrar sesión]**.
4. El **Sistema** solicita confirmación para cerrar la sesión.
5. El **Usuario** confirma la acción.
6. El **Sistema** valida que exista una sesión activa. Si no existe una sesión activa, ir a la **Trayectoria Alternativa A**.
7. El **Sistema** finaliza la sesión del usuario. Si no es posible finalizar la sesión, ir a la **Trayectoria Alternativa B**.
8. El **Sistema** redirige al usuario a la pantalla de inicio de sesión.
9. Fin del caso de uso.

Trayectoria Alternativa A (Sesión no activa)

1. El **Sistema** muestra el mensaje **MSG11**: “No existe una sesión activa.”
2. El **Sistema** redirige al usuario a la pantalla de inicio de sesión.
3. Fin del caso de uso.

Trayectoria Alternativa B (Error al cerrar sesión)

1. El **Sistema** muestra el mensaje **MSG12**: “No fue posible cerrar la sesión correctamente.”
2. El **Sistema** permite al usuario intentar nuevamente el cierre de sesión.
3. Fin del caso de uso.

Trayectoria Alternativa C (Cancelar cierre de sesión)

1. El **Usuario** cancela la confirmación de cierre de sesión.
2. El **Sistema** mantiene la sesión activa.
3. El **Sistema** regresa al usuario a la pantalla anterior.
4. Fin del caso de uso.

8.8.2 CU-M00: Operación general de la aplicación móvil

Con el fin de evitar una especificación repetitiva y excesivamente extensa, los casos de uso derivados del dashboard móvil se agrupan en el caso de uso general **CU-M00: Operación general de la aplicación móvil**. Esta decisión se debe a que el dashboard funciona como el punto central de navegación de la app, desde el cual el Experto de campo accede a las funciones principales del sistema: captura y análisis de árboles, consulta del historial local, mapa local, notificaciones, perfil y configuración.

Los casos de uso específicos de cada módulo se representan en los diagramas correspondientes; sin embargo, para la especificación textual se documentan de forma integrada dentro de CU-M00, ya que comparten condiciones generales de operación, como el inicio de sesión, el uso de la base local SQLite, la operación offline-first y la sincronización ascendente hacia la nube.

Tabla 22: *Resumen de atributos del caso de uso: Operar aplicación móvil desde el dashboard.*

Caso de uso: CU-M00 Operar aplicación móvil desde el dashboard	
Resumen de atributos	
Actor:	*Experto de campo*
Propósito:	Permitir al Experto de campo acceder desde el dashboard móvil a las funciones principales de captura, análisis, consulta local, mapa, notificaciones, perfil y configuración de la aplicación.
Entradas	• Solicitud de acceso a una función del dashboard móvil. • Datos capturados o seleccionados por el usuario según la función utilizada.
Salidas	El sistema muestra la información solicitada o ejecuta la función seleccionada, utilizando la base local SQLite del dispositivo cuando corresponda.
Precondiciones	• El Experto de campo debe haber iniciado sesión. • El dispositivo debe estar asociado a la cuenta del Experto de campo. • La base local SQLite debe estar disponible en el dispositivo.
Postcondiciones	• El usuario accede a la función seleccionada desde el dashboard. • Si se genera un análisis, este se almacena localmente y queda pendiente de sincronización. • Si existe conexión a internet, los registros locales pueden sincronizarse de forma ascendente hacia la nube.

Caso de uso: CU-M00 Operar aplicación móvil desde el dashboard (continuación)	
Resumen de atributos	
Funciones incluidas	• CU-M05: Consultar resumen de árboles por jornada. • CU-M06: Iniciar nuevo análisis. • CU-M07: Consultar registros recientes de la jornada según estado sanitario. • CU-M08: Consultar historial local de análisis de la jornada. • CU-M09: Consultar mapa local de árboles. • CU-M10: Consultar notificaciones. • CU-M11: Consultar perfil de usuario. • CU-M12: Gestionar configuración de la aplicación.
Errores	• MSG13: No existen registros para la jornada actual. • MSG14: No fue posible cargar la información local. • MSG17: No fue posible acceder a la cámara. • MSG18: No fue posible importar la imagen seleccionada. • MSG19: Imagen no válida para el análisis. • MSG21: No fue posible guardar el análisis en la base local. • MSG25: No fue posible sincronizar el registro.
Tipo	Primario para el Experto de campo
Fuente	Requerimientos funcionales del sistema
Observaciones	La aplicación móvil opera bajo un enfoque *offline-first*. Cada dispositivo cuenta con una base local SQLite independiente. La sincronización de registros se realiza únicamente de forma ascendente, desde el dispositivo móvil hacia la nube.

Trayectorias del caso de uso

Trayectoria Principal

1. El **Experto de campo** inicia sesión en la aplicación móvil.
2. El **Sistema** muestra el dashboard móvil.
3. El **Sistema** consulta la base local SQLite del dispositivo para mostrar el resumen de la jornada y los registros recientes.
4. El **Experto de campo** selecciona una función disponible en el dashboard.
5. El **Sistema** muestra el módulo correspondiente:
   - Captura y análisis móvil.
   - Historial local de análisis.
   - Mapa local de árboles.
   - Notificaciones y alertas.
   - Perfil de usuario.
   - Configuración de la aplicación.

6. El **Experto de campo** realiza la acción correspondiente dentro del módulo seleccionado.
7. El **Sistema** guarda, consulta o actualiza la información local según la acción realizada.
8. Fin del caso de uso.

Trayectoria Alternativa A (Sin registros locales)

1. El **Sistema** no encuentra registros para la jornada actual.
2. El **Sistema** muestra el resumen con valores en cero y sin tarjetas recientes.
3. El **Sistema** mantiene disponibles las funciones principales del dashboard.
4. Fin del caso de uso.

Trayectoria Alternativa B (Error al consultar información local)

1. El **Sistema** no puede consultar la base local SQLite.
2. El **Sistema** muestra el mensaje correspondiente indicando que no fue posible cargar la información local.
3. Fin del caso de uso.

Trayectoria Alternativa C (Error de sincronización)

1. El **Sistema** intenta sincronizar un registro local hacia la nube.
2. El **Sistema** no logra completar la sincronización.
3. El **Sistema** marca el registro con estado de error de sincronización.
4. El **Experto de campo** puede reintentar manualmente la sincronización desde el historial local.
5. Fin del caso de uso.

8.8.3 CU-W00: Consulta general de la plataforma web

El caso de uso **CU-W00: Consulta general de la plataforma web** agrupa las funcionalidades principales disponibles para el Experto de campo desde el dashboard web. Esta agrupación se realiza porque la plataforma web funciona como un entorno de consulta y supervisión de la información consolidada en la nube, proveniente de los registros sincronizados desde los dispositivos móviles asociados a la cuenta.

Aunque los diagramas muestran módulos específicos como historial general, mapa general, estadísticas y perfil de usuario, su especificación textual se concentra en CU-W00 para evitar duplicidad de información. Estos módulos comparten el mismo contexto de operación: el usuario debe haber iniciado sesión en la plataforma web, la información consultada proviene de la base central en la nube y las acciones realizadas son de consulta, filtrado o visualización.

La plataforma web no contempla la edición de registros operativos capturados desde la aplicación móvil, debido a que el sistema utiliza una sincronización ascendente, es decir, desde la base local del dispositivo móvil hacia la nube. Por ello, CU-W00 describe de manera compacta el comportamiento general de consulta web, mientras que los diagramas detallan visualmente las funciones derivadas disponibles en cada módulo.

Tabla 23: *Resumen de atributos del caso de uso: Consultar información consolidada desde el dashboard web.*

Caso de uso: CU-W00 Consultar información consolidada desde el dashboard web	
Resumen de atributos	
Actor:	*Experto de campo*
Propósito:	Permitir al Experto de campo consultar desde la plataforma web la información consolidada en la nube, proveniente de los dispositivos móviles asociados a su cuenta.
Entradas	• Solicitud de consulta desde el dashboard web. • Criterios de filtro, cuando el usuario los aplique.
Salidas	El sistema muestra información consolidada sobre registros, árboles, mapas, estadísticas e historial de análisis sincronizados en la nube.
Precondiciones	• El Experto de campo debe haber iniciado sesión en la plataforma web. • Deben existir registros sincronizados en la nube para mostrar información consolidada.
Postcondiciones	El usuario visualiza la información consolidada solicitada sin modificar los registros operativos almacenados en la nube.
Funciones incluidas	• CU-W05: Consultar resumen general. • CU-W06: Consultar historial general de registros. • CU-W07: Consultar mapa general de árboles. • CU-W08: Consultar estadísticas de registros. • CU-W09: Consultar perfil de usuario.

Caso de uso: CU-W00 Consultar información consolidada desde el dashboard web (continuación)	
Resumen de atributos	
Errores	• MSG26: No existen registros sincronizados disponibles. • MSG27: No fue posible cargar la información consolidada. • MSG28: No fue posible aplicar los filtros seleccionados.
Tipo	Primario para el Experto de campo
Fuente	Requerimientos funcionales del sistema
Observaciones	La plataforma web funciona como una vista de consulta y supervisión. No se contemplan acciones de edición sobre los registros operativos capturados desde la aplicación móvil, debido a que la sincronización definida es ascendente.

Trayectorias del caso de uso

Trayectoria Principal

1. El **Experto de campo** inicia sesión en la plataforma web.
2. El **Sistema** muestra el dashboard web.
3. El **Sistema** consulta la base de datos en la nube.
4. El **Sistema** muestra el resumen general de registros sincronizados.
5. El **Experto de campo** selecciona una función disponible en el dashboard web.
6. El **Sistema** muestra el módulo correspondiente:
   - Historial general de registros.
   - Mapa general de árboles.
   - Estadísticas de registros.
   - Perfil de usuario.
7. El **Experto de campo** aplica filtros o consulta detalles, si lo requiere.
8. El **Sistema** muestra la información consolidada solicitada.
9. Fin del caso de uso.

Trayectoria Alternativa A (Sin registros sincronizados)

1. El **Sistema** no encuentra registros sincronizados asociados a la cuenta.
2. El **Sistema** muestra el mensaje **MSG26**: “No existen registros sincronizados disponibles.”
3. El **Sistema** muestra el dashboard web sin información consolidada.
4. Fin del caso de uso.

Trayectoria Alternativa B (Error al cargar información consolidada)

1. El **Sistema** no puede consultar la base de datos en la nube.
2. El **Sistema** muestra el mensaje **MSG27**: “No fue posible cargar la información consolidada.”
3. Fin del caso de uso.

Trayectoria Alternativa C (Error al aplicar filtros)

1. El **Experto de campo** aplica filtros sobre la información consultada.
2. El **Sistema** no puede procesar los filtros seleccionados.
3. El **Sistema** muestra el mensaje **MSG28**: “No fue posible aplicar los filtros seleccionados.”
4. El **Sistema** conserva la vista anterior.
5. Fin del caso de uso.

8.8.4 CU-A00: Gestionar cuentas y dispositivos desde la plataforma web

El caso de uso **CU-A00: Gestionar cuentas y dispositivos desde la plataforma web** agrupa las funciones administrativas disponibles para el Administrador. Esta agrupación se realiza porque las acciones de crear, editar, desactivar cuentas, asignar roles y administrar dispositivos asociados forman parte de un mismo proceso general de gestión de usuarios dentro de la plataforma web.

Aunque en el diagrama se muestran casos de uso específicos para cada acción administrativa, su especificación textual se concentra en CU-A00 para evitar repetir condiciones comunes, como el inicio de sesión del Administrador, la validación de permisos y el almacenamiento de cambios en la base central en la nube.

Las acciones incluidas en CU-A00 no modifican los registros operativos capturados desde la aplicación móvil. Es decir, el Administrador gestiona información de acceso, roles y dispositivos autorizados, pero no altera los análisis, evidencias, ubicaciones o registros generados por los expertos de campo. Esta separación permite mantener la sincronización ascendente del sistema, donde los datos operativos se envían desde la aplicación móvil hacia la nube, sin requerir sincronización descendente hacia las bases locales SQLite de los dispositivos.

Tabla 24: *Resumen de atributos del caso de uso: Gestionar cuentas y dispositivos desde la plataforma web.*

Caso de uso: CU-A00 Gestionar cuentas y dispositivos desde la plataforma web	
Resumen de atributos	
Actor:	*Administrador*
Propósito:	Permitir al Administrador gestionar las cuentas de usuario, roles y dispositivos móviles asociados a cada cuenta desde la plataforma web.
Entradas	• Solicitud de gestión administrativa. • Datos de cuenta de usuario. • Rol asignado al usuario. • Identificador o información del dispositivo móvil.
Salidas	El sistema crea, actualiza, desactiva o consulta cuentas de usuario y dispositivos asociados, según la acción realizada por el Administrador.
Precondiciones	• El Administrador debe haber iniciado sesión en la plataforma web. • La cuenta del Administrador debe estar activa. • El Administrador debe contar con permisos de administración.
Postcondiciones	• La cuenta de usuario queda creada, modificada, desactivada o con rol asignado, según corresponda. • Los dispositivos móviles quedan asociados o desasociados de la cuenta seleccionada. • Los cambios administrativos quedan almacenados en la base de datos central en la nube.

Caso de uso: CU-A00 Gestionar cuentas y dispositivos desde la plataforma web (continuación)	
Resumen de atributos	
Funciones incluidas	• CU-A01: Gestionar cuentas de usuario. • CU-A02: Crear cuenta de usuario. • CU-A03: Editar cuenta de usuario. • CU-A04: Desactivar cuenta de usuario. • CU-A05: Asignar rol de usuario. • CU-A06: Gestionar dispositivos asociados. • CU-A07: Asociar dispositivo a cuenta. • CU-A08: Desasociar dispositivo de cuenta. • CU-A09: Consultar dispositivos asociados a una cuenta.
Errores	• MSG29: No se cuenta con permisos de administración. • MSG30: No fue posible cargar la información de usuarios. • MSG31: El correo electrónico ya se encuentra registrado. • MSG32: No fue posible guardar los cambios de la cuenta. • MSG33: El dispositivo ya se encuentra asociado a una cuenta. • MSG34: No fue posible asociar el dispositivo. • MSG35: No fue posible desasociar el dispositivo.
Tipo	Primario para el Administrador
Fuente	Requerimientos funcionales del sistema
Observaciones	Las funciones administrativas se realizan sobre la base central en la nube. Estas acciones no modifican los registros operativos capturados desde la aplicación móvil ni implican sincronización descendente hacia las bases locales SQLite de los dispositivos.

Trayectorias del caso de uso

Trayectoria Principal

1. El **Administrador** inicia sesión en la plataforma web.
2. El **Sistema** valida que la cuenta tenga permisos de administración. Si no cuenta con permisos, ir a la **Trayectoria Alternativa A**.
3. El **Sistema** muestra el módulo de administración.
4. El **Administrador** selecciona la sección de gestión de cuentas de usuario.
5. El **Sistema** muestra la lista de cuentas registradas.
6. El **Administrador** selecciona una acción administrativa:
   - Crear cuenta de usuario.
   - Editar cuenta de usuario.
   - Desactivar cuenta de usuario.
   - Asignar rol de usuario.
   - Gestionar dispositivos asociados.

7. El **Administrador** ingresa o modifica la información requerida según la acción seleccionada.
8. El **Sistema** valida la información ingresada. Si la información no es válida, ir a la **Trayectoria Alternativa B**.
9. El **Sistema** guarda los cambios en la base de datos central en la nube. Si no es posible guardar los cambios, ir a la **Trayectoria Alternativa C**.
10. El **Sistema** muestra una confirmación de la acción realizada.
11. Fin del caso de uso.

Trayectoria Alternativa A (Sin permisos de administración)

1. El **Sistema** muestra el mensaje **MSG29**: "No se cuenta con permisos de administración."
2. El **Sistema** impide el acceso al módulo de administración.
3. Fin del caso de uso.

Trayectoria Alternativa B (Información inválida)

1. El **Sistema** detecta datos incompletos, duplicados o no válidos.
2. El **Sistema** muestra el mensaje correspondiente según el error detectado.
3. El **Sistema** permite al **Administrador** corregir la información.
4. Fin del caso de uso.

Trayectoria Alternativa C (Error al guardar cambios)

1. El **Sistema** no puede guardar los cambios en la base de datos central.
2. El **Sistema** muestra el mensaje **MSG32**: "No fue posible guardar los cambios de la cuenta."
3. El **Sistema** permite al **Administrador** intentar nuevamente.
4. Fin del caso de uso.

Trayectoria Alternativa D (Dispositivo ya asociado)

1. El **Administrador** intenta asociar un dispositivo a una cuenta.
2. El **Sistema** detecta que el dispositivo ya se encuentra asociado a una cuenta.
3. El **Sistema** muestra el mensaje **MSG33**: "El dispositivo ya se encuentra asociado a una cuenta."
4. El **Sistema** cancela la asociación del dispositivo.
5. Fin del caso de uso.

8.9 Diseño de interfaces de la aplicación móvil (UI/UX)

La interfaz de la aplicación móvil EcoVerde fue desarrollada siguiendo los lineamientos de **Material Design 3**, el sistema de diseño de Google para aplicaciones Android.

8.9.1 Diseño visual y prototipos de alta fidelidad

Este sistema proporciona componentes, colores y tipografía coherentes que garantizan una experiencia de usuario consistente y accesible.

Lenguaje de diseño. La paleta de colores se construyó a partir de un verde bosque (`#2d6a3e`) como color principal, del cual se derivan los demás tonos de la interfaz. El color comunica el estado sanitario de forma directa: verde para árboles sanos, rojo para infestados y naranja para capturas sin resultado definitivo. La tipografía utilizada es **Roboto Flex**, tipografía oficial de Material Design 3. La aplicación soporta tema claro y oscuro para adaptarse a distintas condiciones de iluminación en campo.

Los prototipos presentados en las siguientes figuras representan el diseño final de cada pantalla de la aplicación.

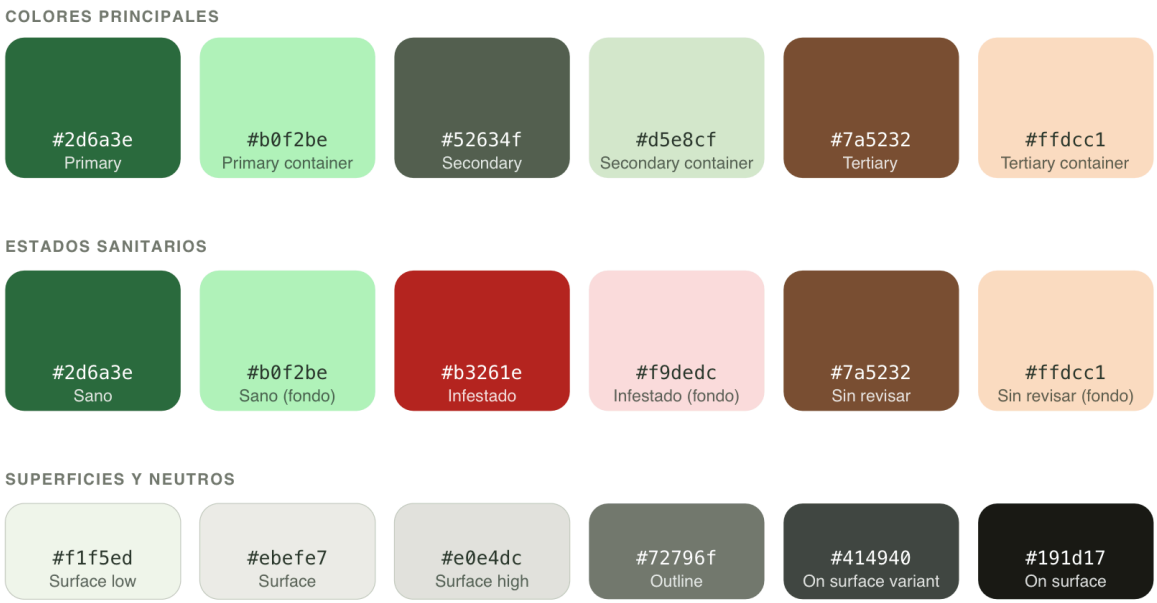


Figura 39: *Paleta de colores utilizada en el diseño.*

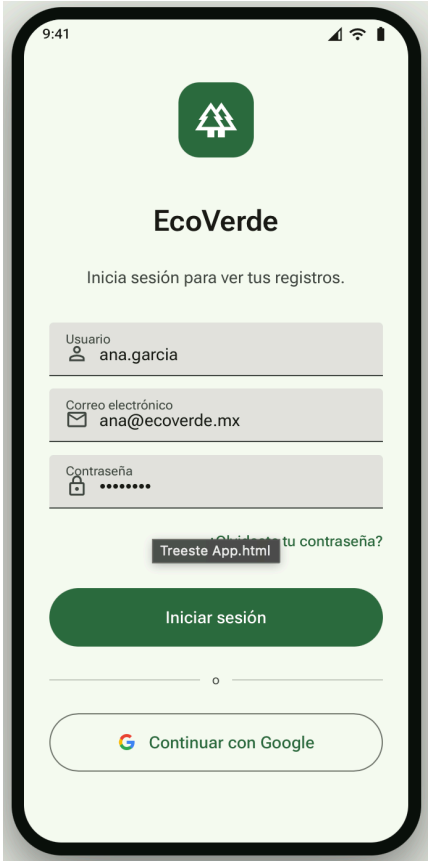


Figura 40: *Pantalla de inicio de sesión.*

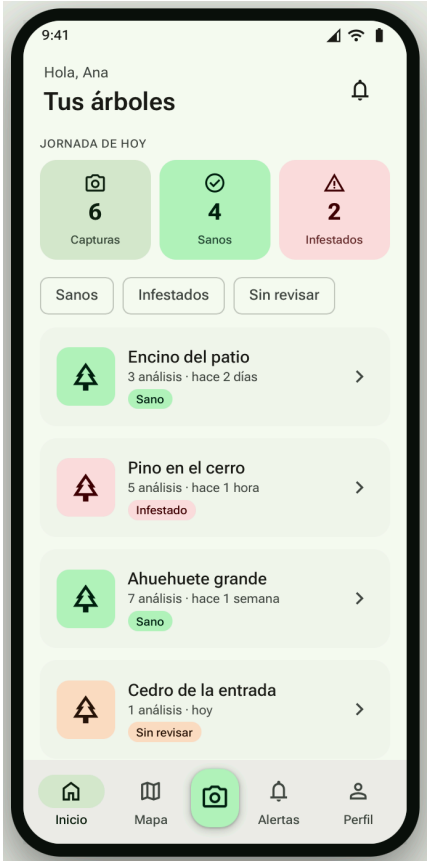


Figura 41: *Pantalla de creación de cuenta.*

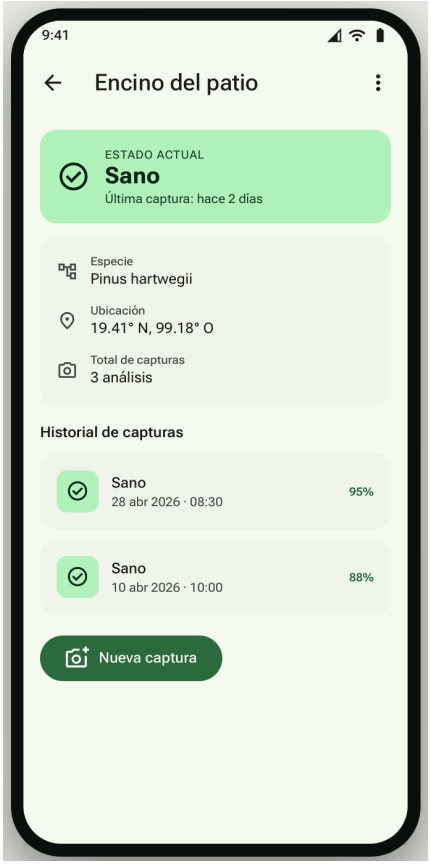


Figura 42: *Menú principal con resumen de jornada y lista de árboles registrados.*

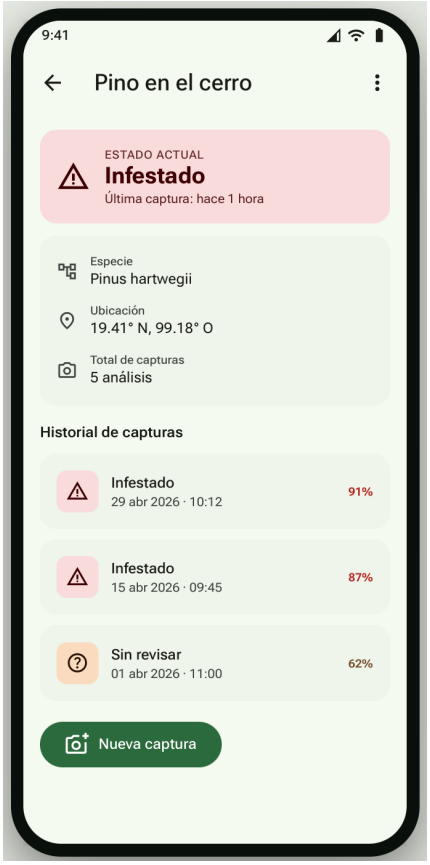


Figura 43: *Detalle de árbol: estado sanitario e historial de capturas.*

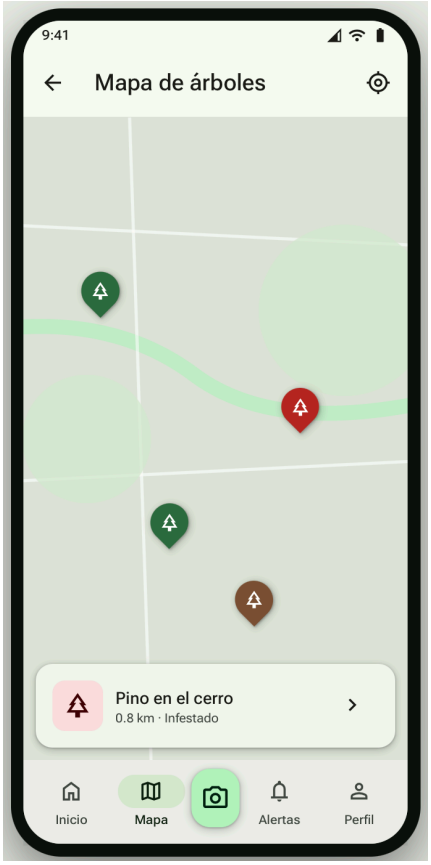


Figura 44: *Mapa de árboles registrados con indicadores de estado sanitario.*

Figura 45: *Selección del tipo de captura: corteza o follaje.*

Figura 46: *Selección del árbol al que se asociará la captura.*

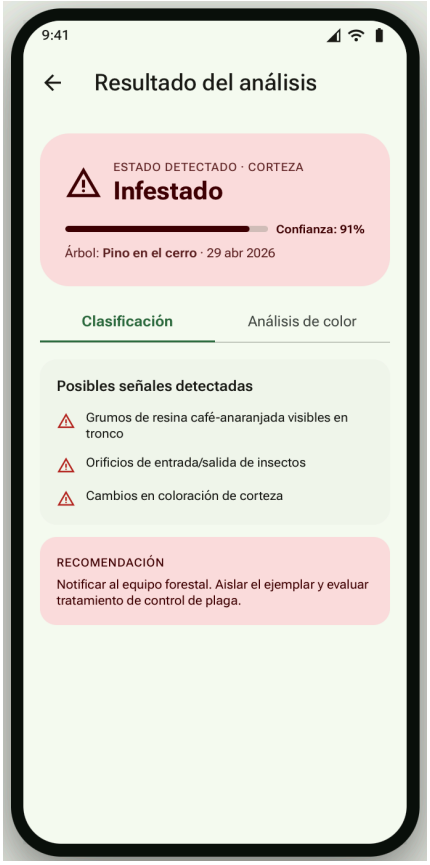


Figura 47: *Resultado del análisis de corteza: clasificación e índice de confianza.*

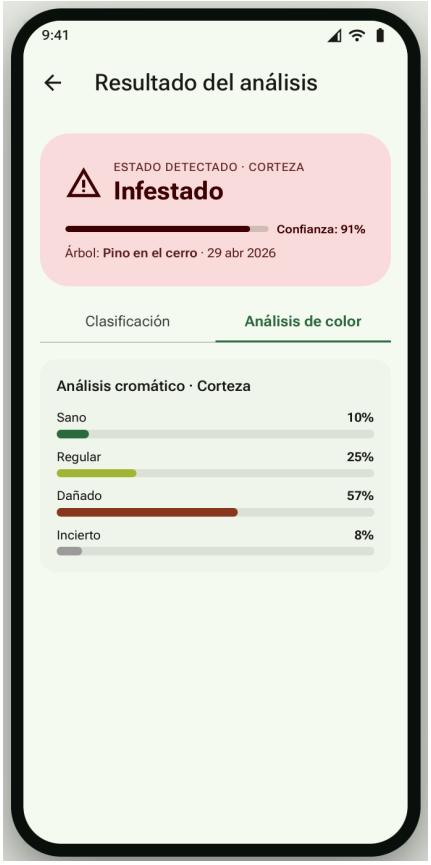


Figura 48: *Resultado del análisis de corteza: análisis de color.*

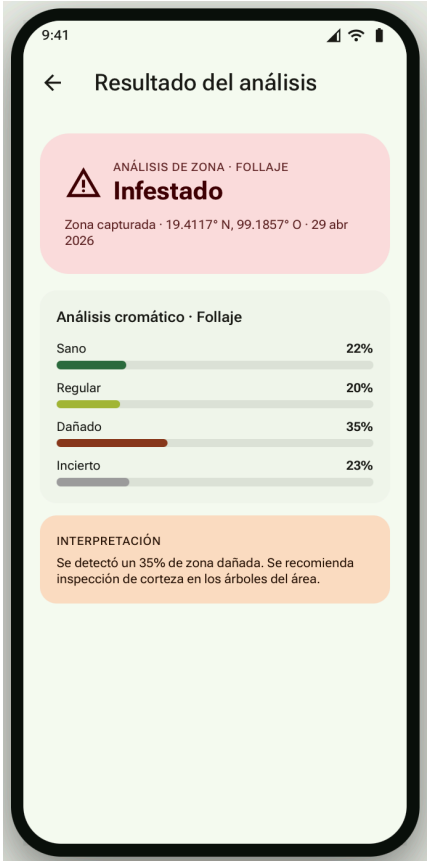


Figura 49: *Resultado del análisis de follaje: distribución cromática de la zona capturada.*

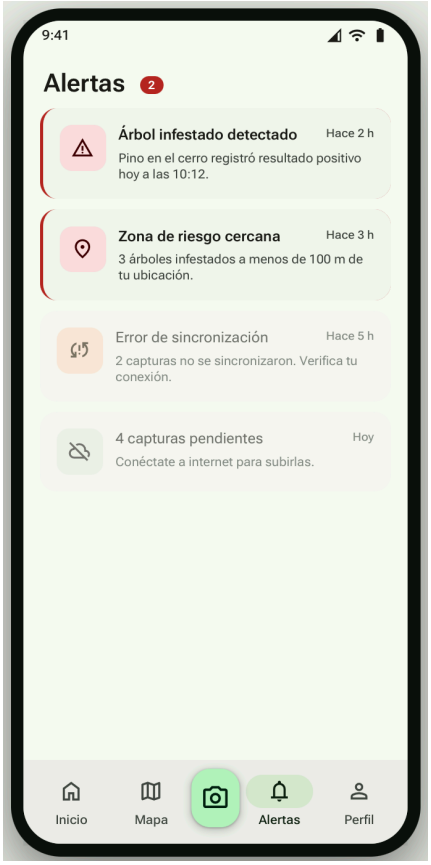


Figura 50: *Pantalla de alertas y notificaciones sanitarias.*

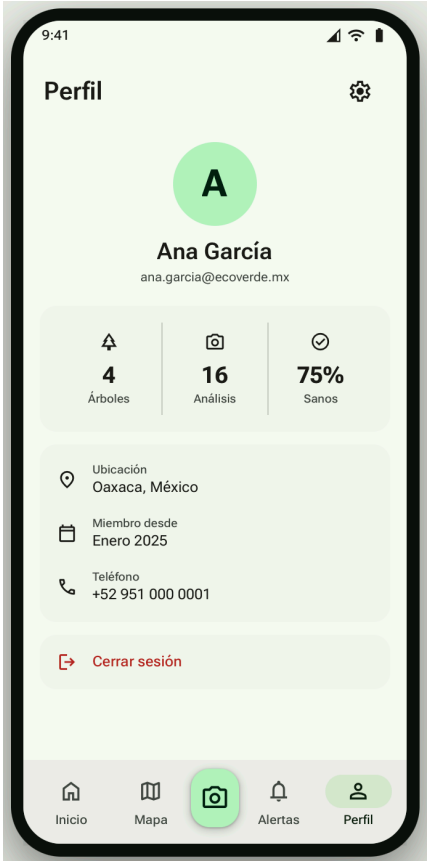


Figura 51: *Pantalla de perfil del técnico.*

8.9.2 Flujo de navegación y experiencia de usuario

La navegación de EcoVerde está organizada en tres niveles:

1. **Autenticación:** El técnico inicia sesión con correo y contraseña o mediante cuenta Google. Un flujo de incorporación de tres pasos presenta las funciones principales de la aplicación antes del primer acceso.
2. **Navegación principal:** Una barra de navegación inferior permite acceder a las secciones de *Inicio*, *Mapa*, *Alertas* y *Perfil*. El botón central activa el flujo de nueva captura, siendo la acción más frecuente durante la jornada de campo.
3. **Flujo de captura:** Consta de cuatro pasos secuenciales:
   (a) Selección del tipo de muestra: corteza o follaje.
   (b) Asociación a un árbol registrado, o registro de uno nuevo (solo para corteza).
   (c) Captura fotográfica con guía de encuadre en pantalla.
   (d) Presentación del resultado del análisis.

8.9.3 Presentación de resultados

La pantalla de resultados traduce la salida del modelo de inferencia en información directamente útil para el técnico. Su contenido varía según el tipo de captura.

Resultado de corteza. Se muestra la clasificación del árbol (*Sano*, *Infestado* o *Sin revisar*), el índice de confianza del modelo, las señales visuales detectadas en la imagen y una recomendación de acción. Una pestaña adicional presenta el análisis cromático de la corteza.

Resultado de follaje. Al tratarse de una captura de zona y no de un árbol individual, se presenta únicamente el análisis cromático de la imagen, con la distribución porcentual de superficie sana, regular, dañada e incierta. La ubicación geográfica de la captura queda registrada en el encabezado para su posterior consulta en el mapa.

8.10 Diseño de interfaces de la plataforma web (UI/UX)

El desarrollo de la plataforma web está orientado a proveer tanto al experto de campo como a los administradores un entorno eficiente para el análisis forestal y la gestión del sistema. El diseño visual y la experiencia de usuario (UX) se estructuraron buscando mantener una alta coherencia e identidad visual con la aplicación móvil. Por este motivo, se implementó la misma paleta cromática (enfocada en tonos de naturaleza, tierra y acentos de alerta), logrando consistencia bajo los lineamientos de *Material Design*.

8.10.1 Módulo de Autenticación

El punto de entrada al sistema web requiere un control de acceso estricto. Se diseñaron interfaces minimalistas para el inicio de sesión y la recuperación de credenciales, asegurando que el usuario pueda autenticarse rápidamente o restablecer su acceso sin fricción.

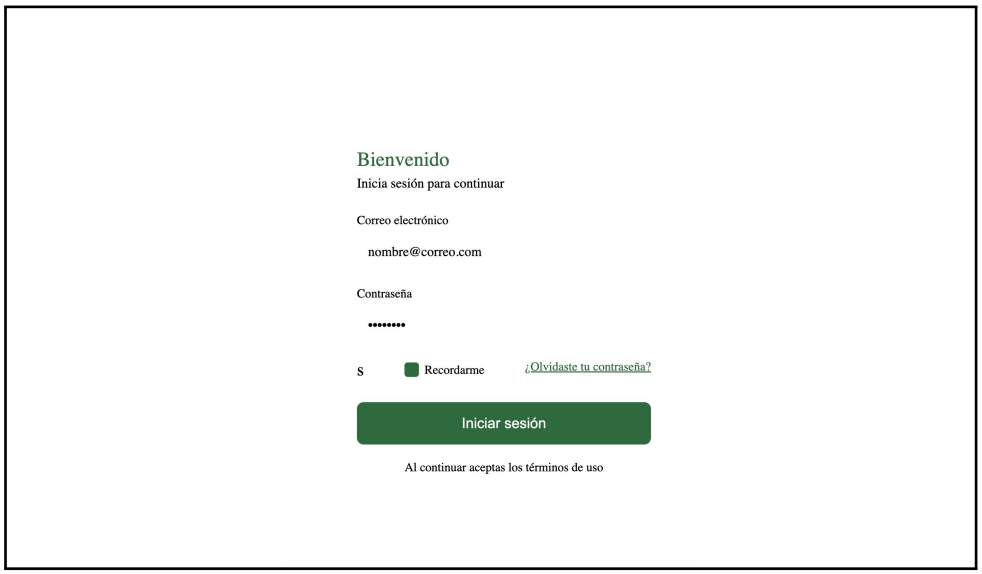


Figura 52: *Maqueta de inicio de sesión.*

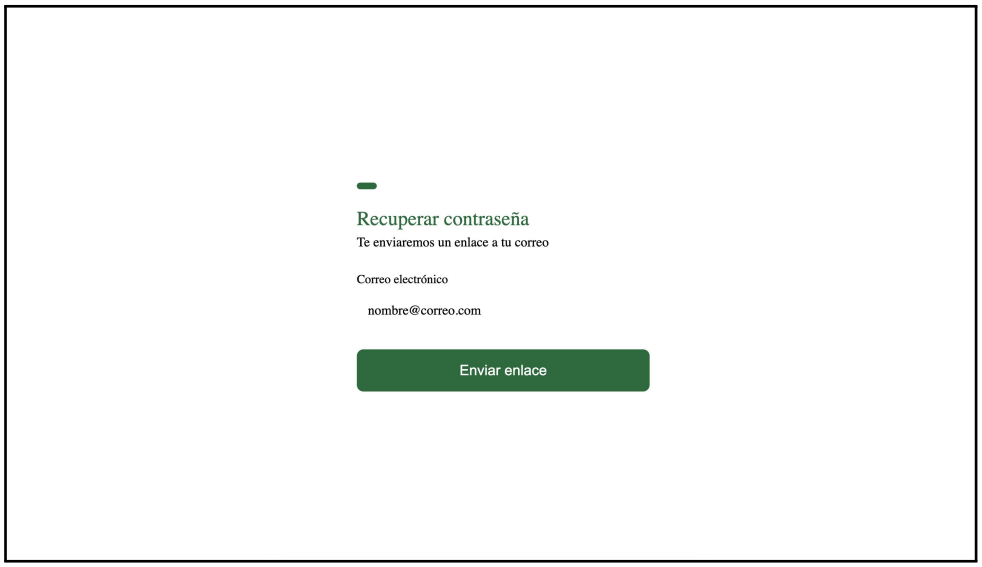


Figura 53: *Maqueta de recuperacion de contraseña.*

Figura 54: *Maqueta de nueva contraseña.*

8.10.2 Panel administrativo y gestión de usuarios

Para el rol de Administrador, se definió un módulo de control interno. Esta interfaz permite la gestión integral del sistema mediante vistas tabulares y formularios modales para la creación, edición y desactivación de cuentas de usuario, así como la asignación de roles y la gestión de los dispositivos de escaneo vinculados a cada cuenta.

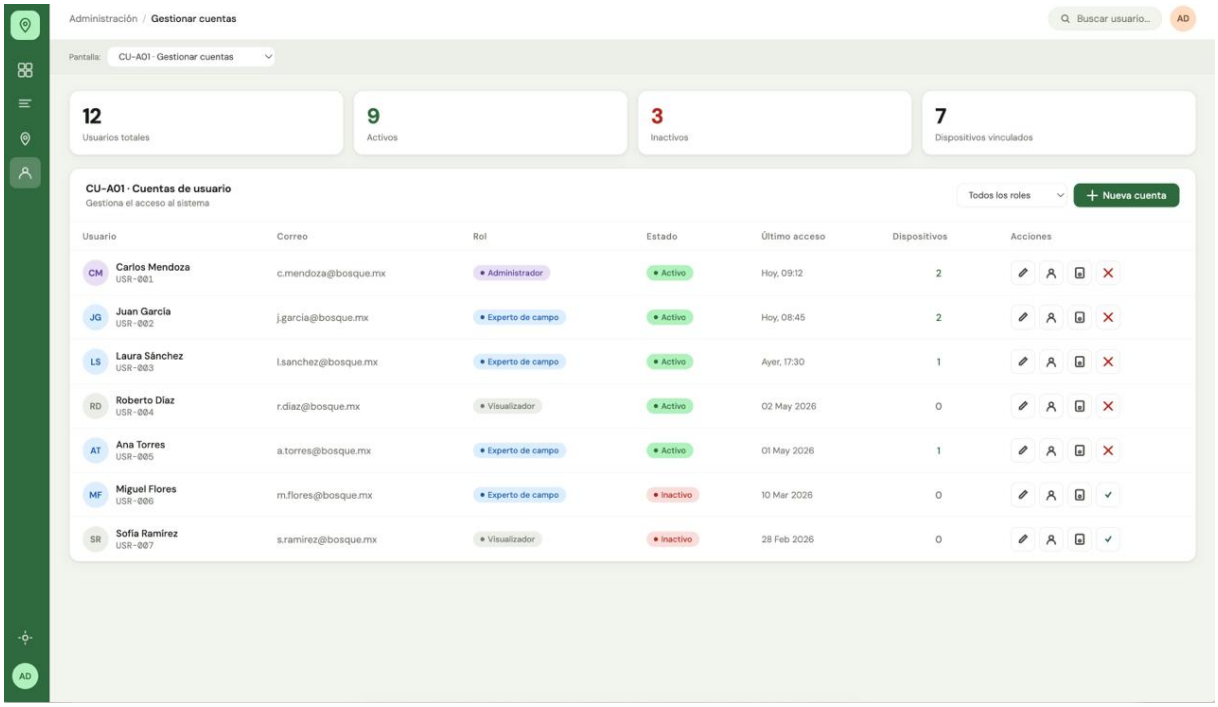


Figura 55: *CU-A01: Gestión general de cuentas de usuario.*

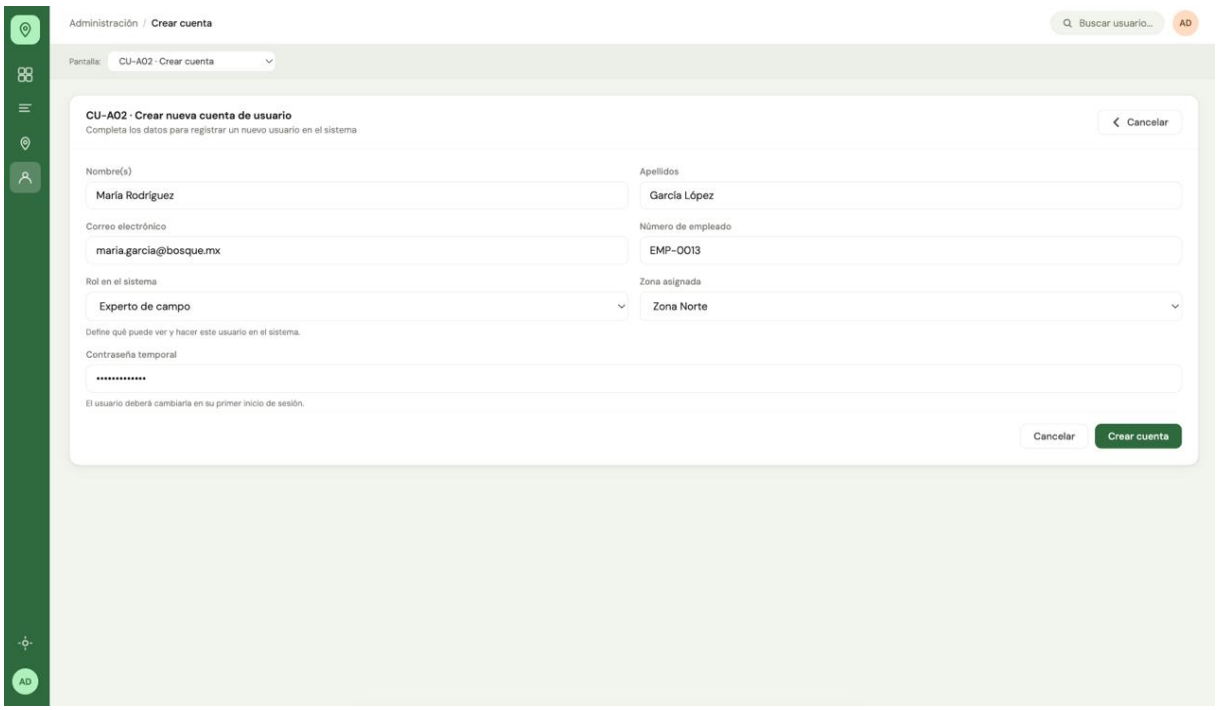


Figura 56: *CU-A02: Creación de nueva cuenta de usuario.*

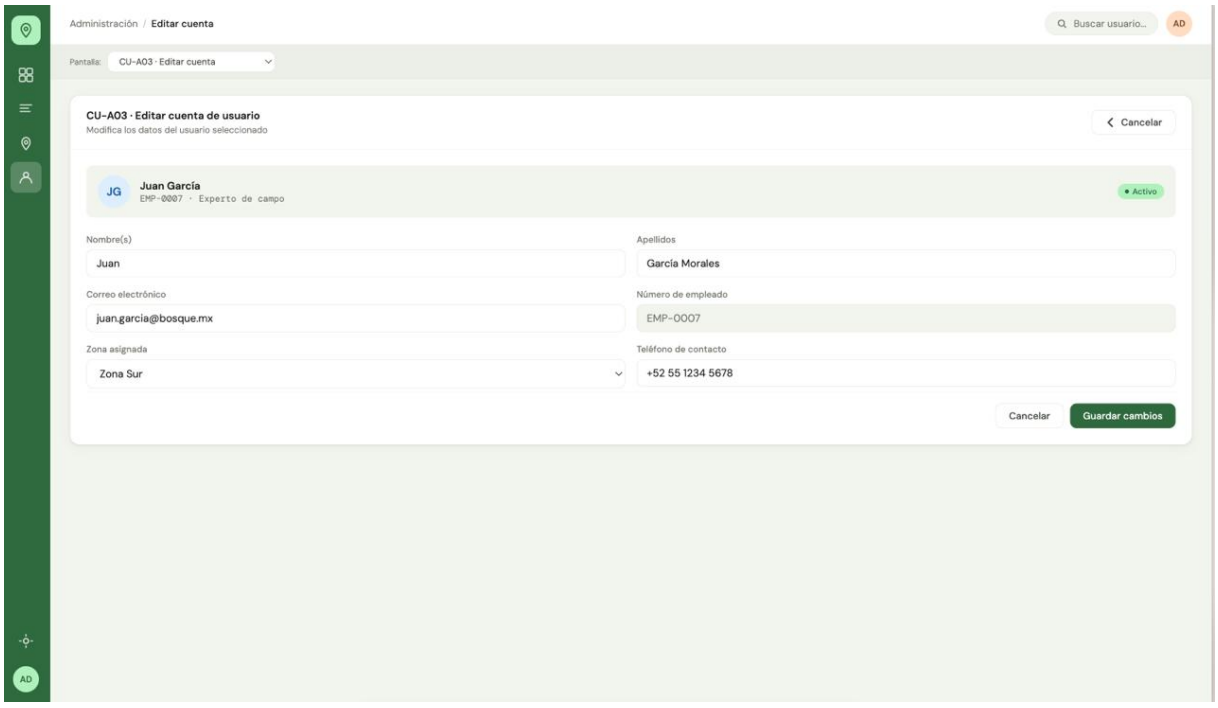


Figura 57: *CU-A03: Edición de datos de cuenta de usuario.*

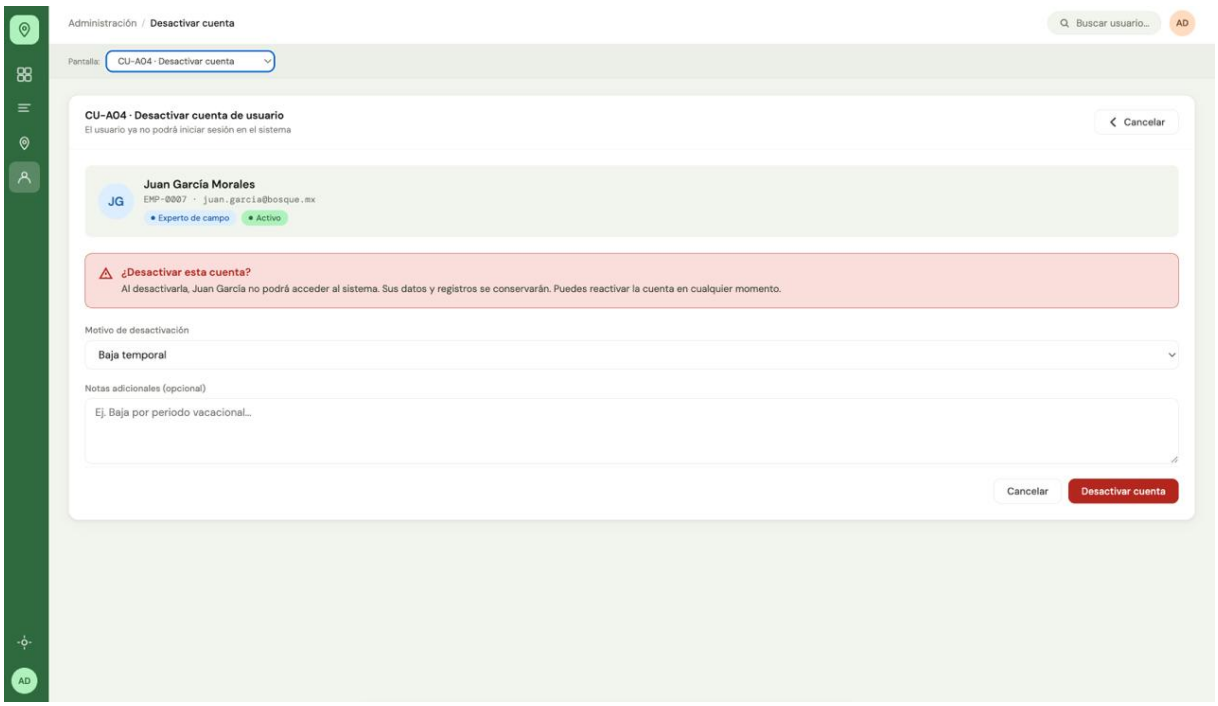


Figura 58: *CU-A04: Desactivación de cuenta de usuario.*

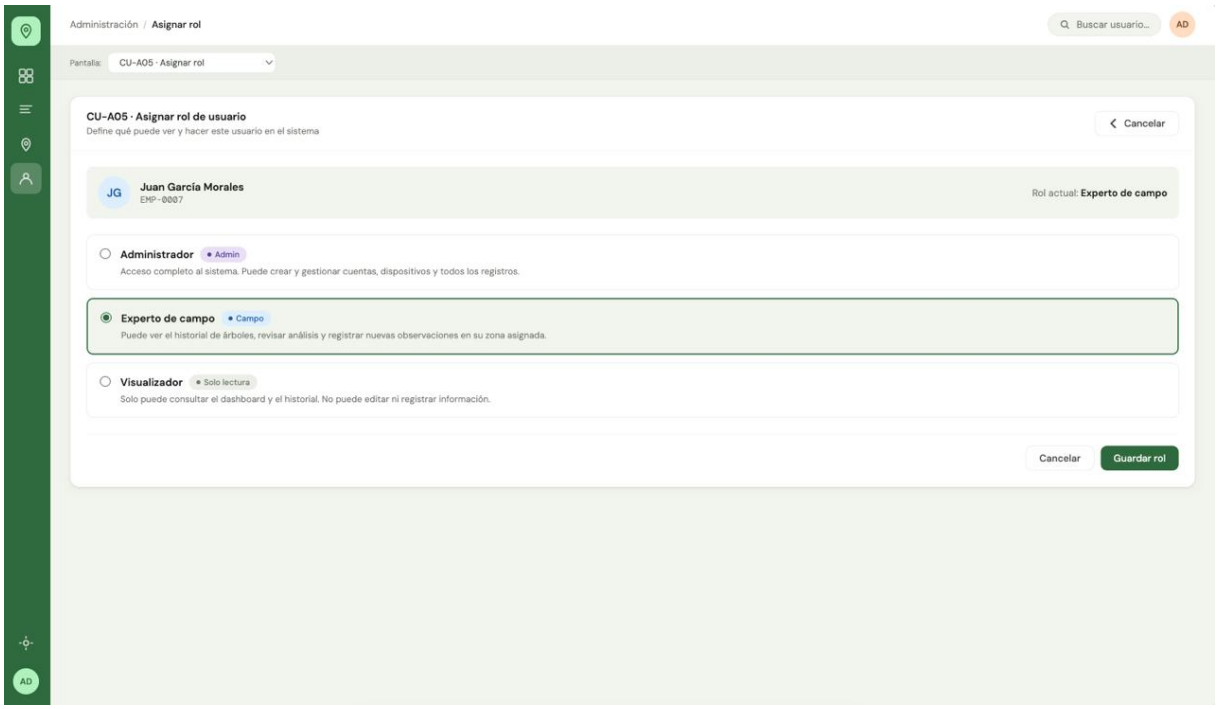


Figura 59: *CU-A05: Asignación de rol a cuenta de usuario.*

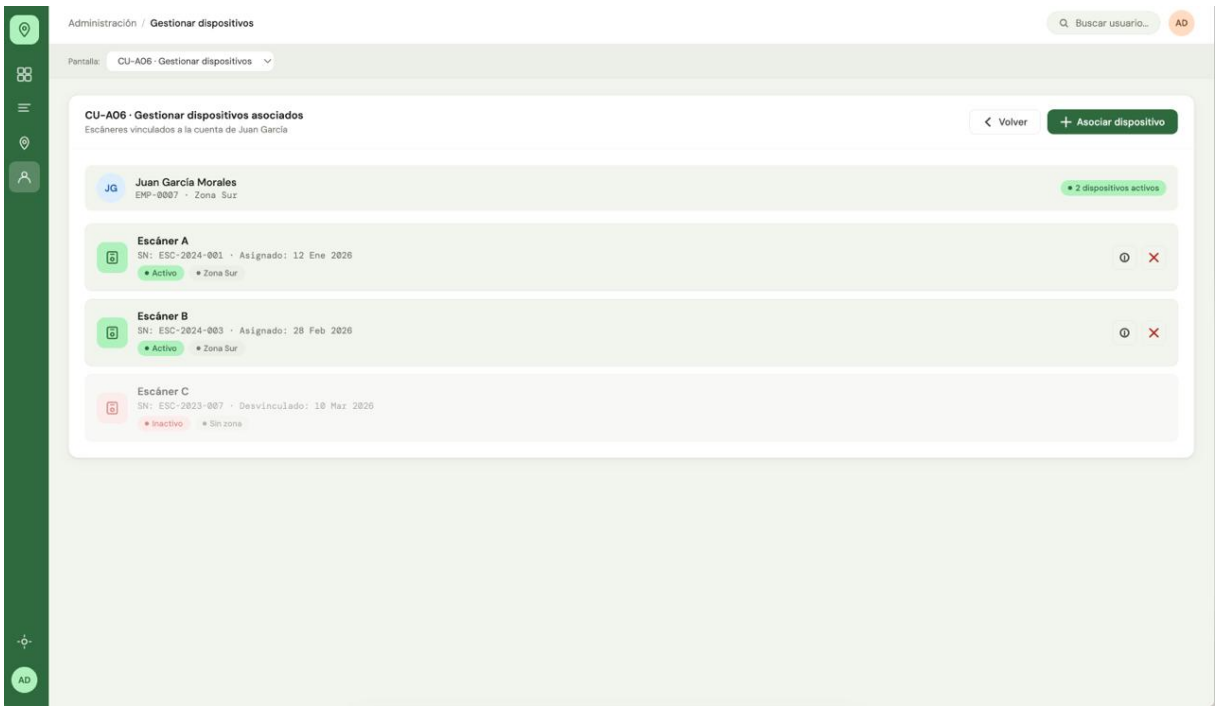


Figura 60: *CU-A06: Gestión de dispositivos asociados a una cuenta.*

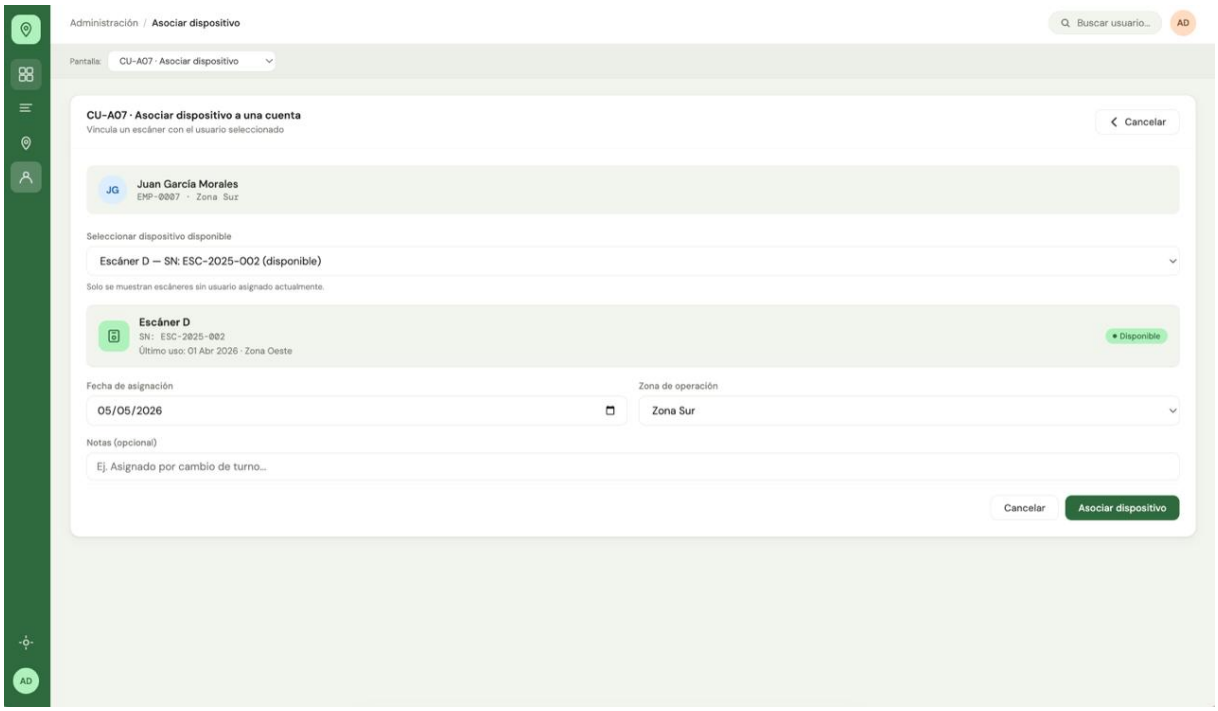


Figura 61: *CU-A07: Asociación de dispositivo escáner a una cuenta de usuario.*

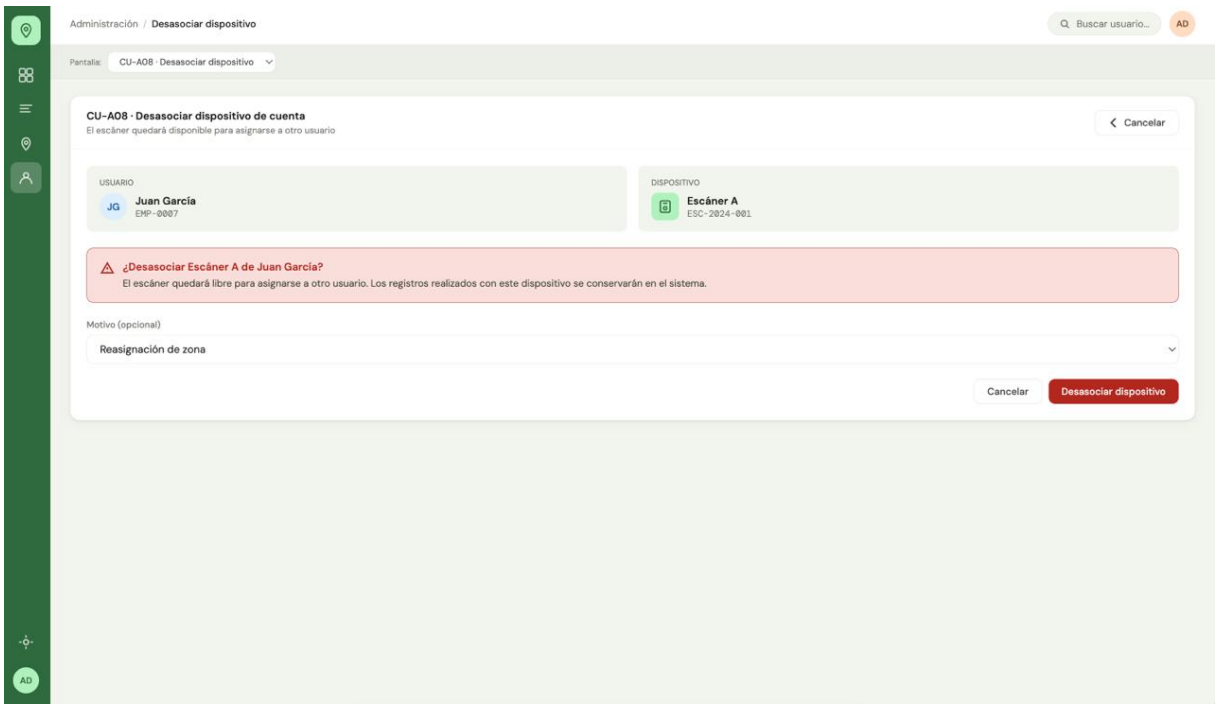


Figura 62: *CU-A08: Desasociación de dispositivo escáner de una cuenta de usuario.*

8.10.3 Dashboard y flujo de navegación (Experto de Campo)

El flujo de navegación para el experto de campo minimiza la cantidad de clics necesarios para acceder a la información crítica. Mediante un menú lateral (*sidebar*) dinámico, el usuario puede transitar entre el resumen general, el historial de registros, el mapa interactivo y las estadísticas.

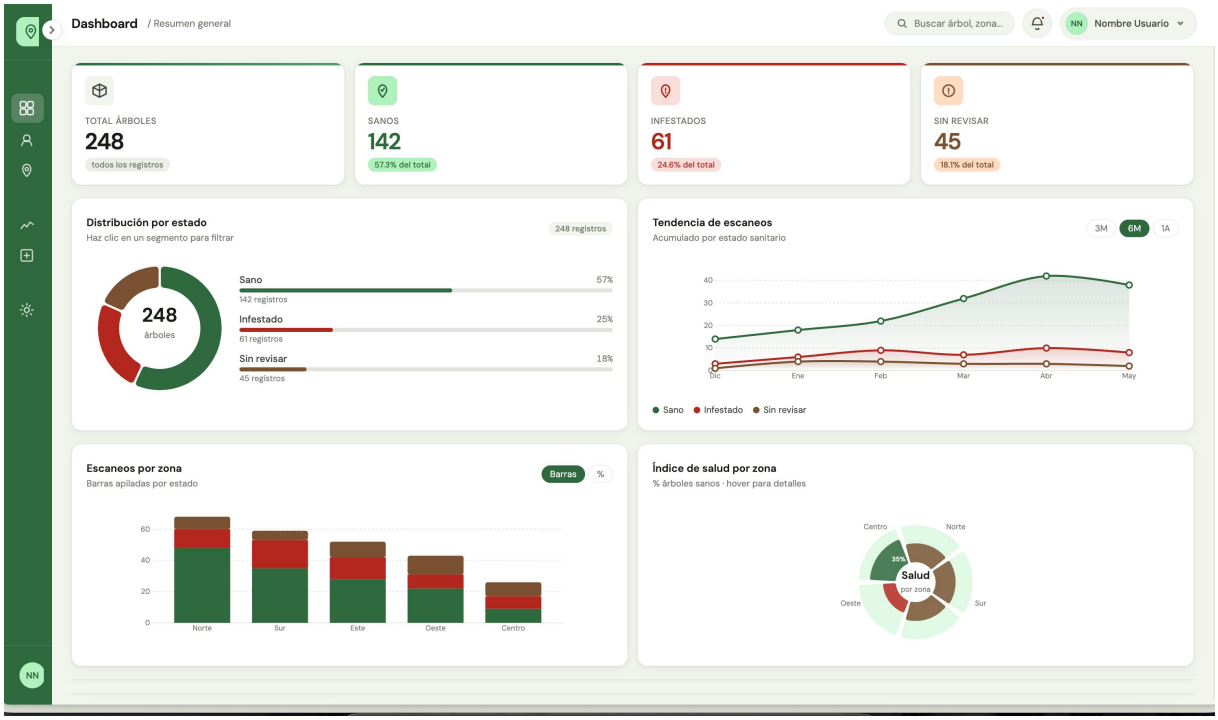


Figura 63: *Interfaz del panel de resumen (Dashboard)*

8.10.4 Análisis espacial e identificación de focos de infestación

El análisis geoespacial es fundamental para la toma de decisiones ecológicas. Se diseñó una interfaz interactiva cartográfica que renderiza marcadores para cada árbol analizado. Este módulo cuenta con controles de filtrado dinámico que permiten aislar árboles sanos de aquellos en riesgo, facilitando la visualización inmediata de focos de infestación activos sobre el terreno.

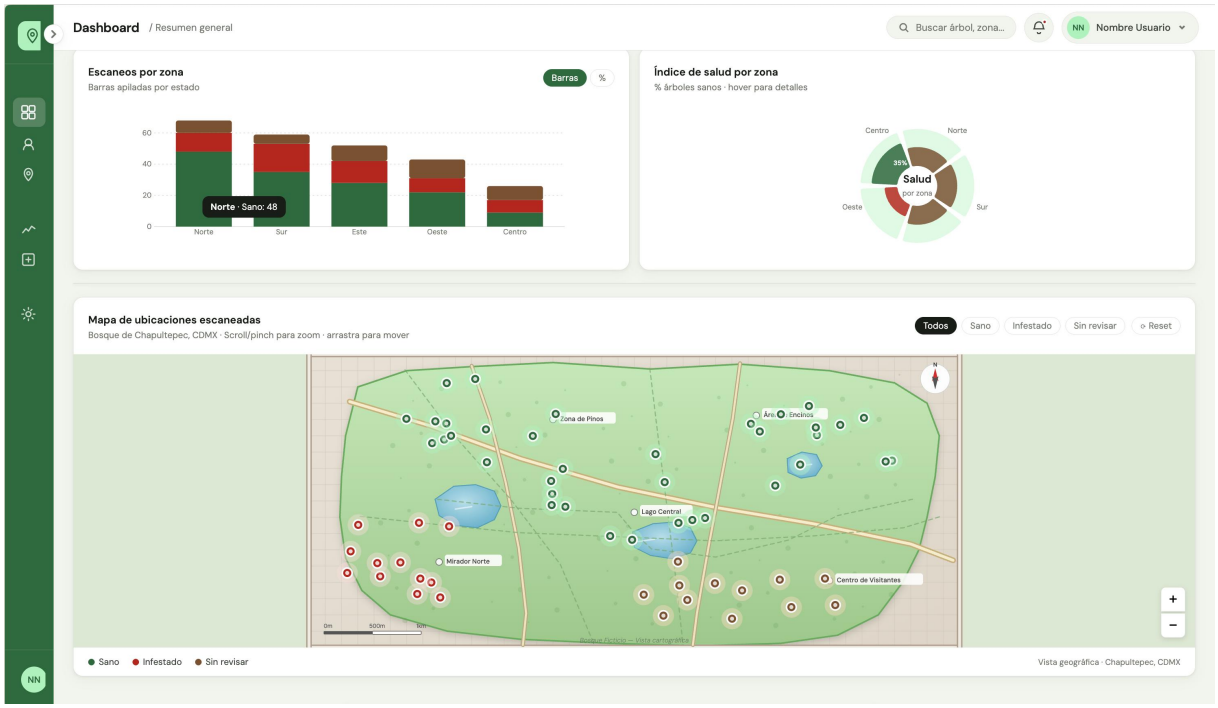


Figura 64: *Maqueta del mapa general de árboles y focos de infestación.*

8.10.5 Presentación de resultados analíticos (D3.js)

Para presentar el alto volumen de datos provenientes de la base de datos PostgreSQL de manera analítica e intuitiva, se integró la librería *D3.js*. Esta herramienta permite generar visualizaciones interactivas.

El sistema mapea los registros para renderizar gráficos de evolución temporal, distribución de especies y estados sanitarios, así como interfaces de análisis detallado por árbol (incluyendo evidencia fotográfica, resultados de clasificación neuronal y mapas de calor cromáticos), optimizando la interpretabilidad de los diagnósticos generados.

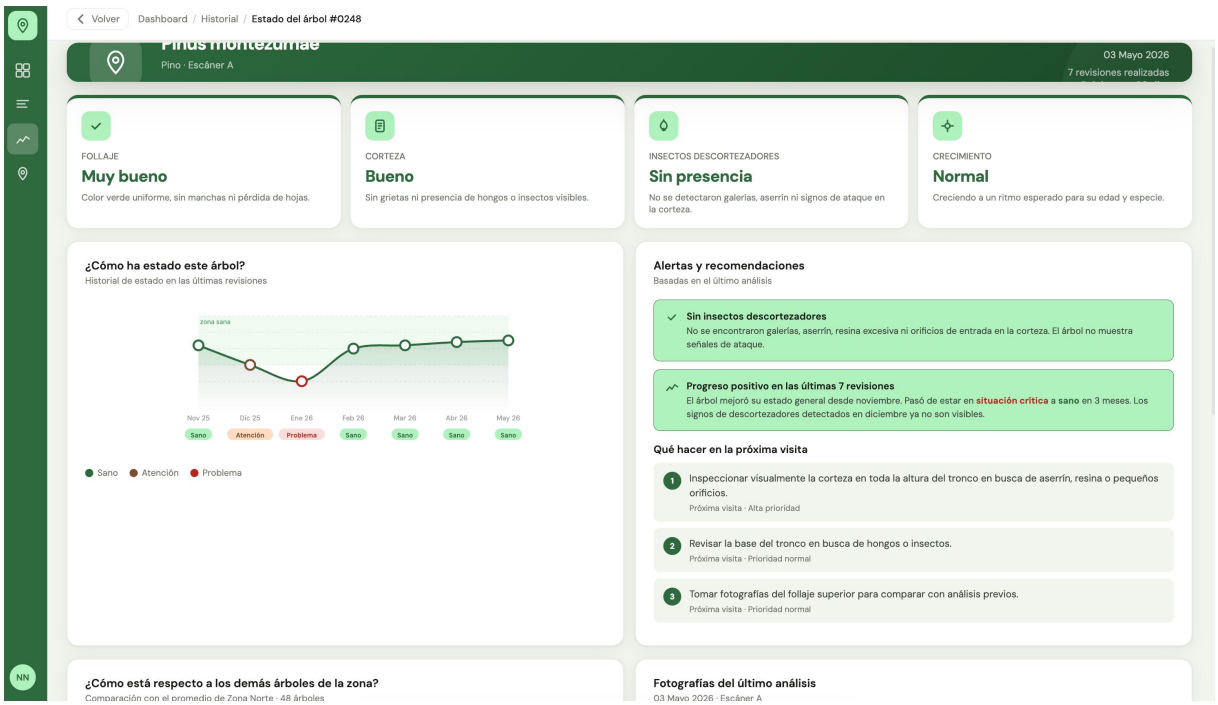


Figura 65: *Interfaces de análisis detallado y estadísticas.*

8.11 Diseño de base de datos

El sistema utiliza una estructura lógica común para representar las entidades principales del dominio, tanto en la base local SQLite como en la base central en la nube. La diferencia entre ambos entornos no radica principalmente en el modelo de datos, sino en el alcance de los registros almacenados. La base local SQLite conserva únicamente la información necesaria para que el dispositivo móvil opere sin conexión, mientras que la base en la nube concentra los registros sincronizados desde los distintos dispositivos asociados a las cuentas.

Debido al enfoque *offline-first*, los registros generados en la aplicación móvil se almacenan inicialmente en SQLite y posteriormente se sincronizan hacia la nube cuando existe conexión a internet.

Para conservar la trazabilidad, los identificadores generados localmente se mantienen durante la sincronización, permitiendo que un mismo registro tenga la misma identidad lógica en ambos entornos.

8.11.1 Modelo entidad-relación

El *Modelo Entidad-Relación (MER)* es una representación conceptual de la estructura de datos de un sistema, utilizada para identificar las entidades que lo conforman, sus atributos y las relaciones que existen entre ellas. Su propósito es modelar la información que el sistema necesita almacenar de forma organizada, independientemente de la tecnología de base de datos que se utilice para su implementación. En el MER, las entidades representan objetos del mundo real con existencia propia, los atributos describen sus características y las relaciones expresan cómo se vinculan entre sí.

El modelo entidad-relación del presente sistema está conformado por seis entidades principales que reflejan los elementos operativos del proceso de monitoreo forestal. La entidad **cuenta** identifica a los usuarios del sistema y su rol, mientras que la entidad **dispositivo** registra los equipos móviles autorizados para operar con la aplicación móvil. La vinculación entre ambas se gestiona mediante la entidad **cuenta_dispositivo**, que controla qué dispositivos están asociados a cada cuenta y su estado de asociación.

En cuanto a la información operativa, la entidad **árbol** almacena los registros individuales del arbolado monitoreado, y la entidad **zona_follaje** registra las áreas geográficas identificadas mediante análisis macro. Ambas se relacionan con la entidad **análisis**, que concentra los resultados de clasificación obtenidos a partir de las imágenes capturadas, diferenciando entre análisis de corteza a nivel de árbol individual y análisis de follaje a nivel de zona. Finalmente, la entidad **análisis_cromático** complementa cada análisis almacenando los porcentajes de coloración obtenidos mediante OpenCV, correspondientes a los rangos de evidencia visual definidos por la NOM-019-SEMARNAT-2017 [31].

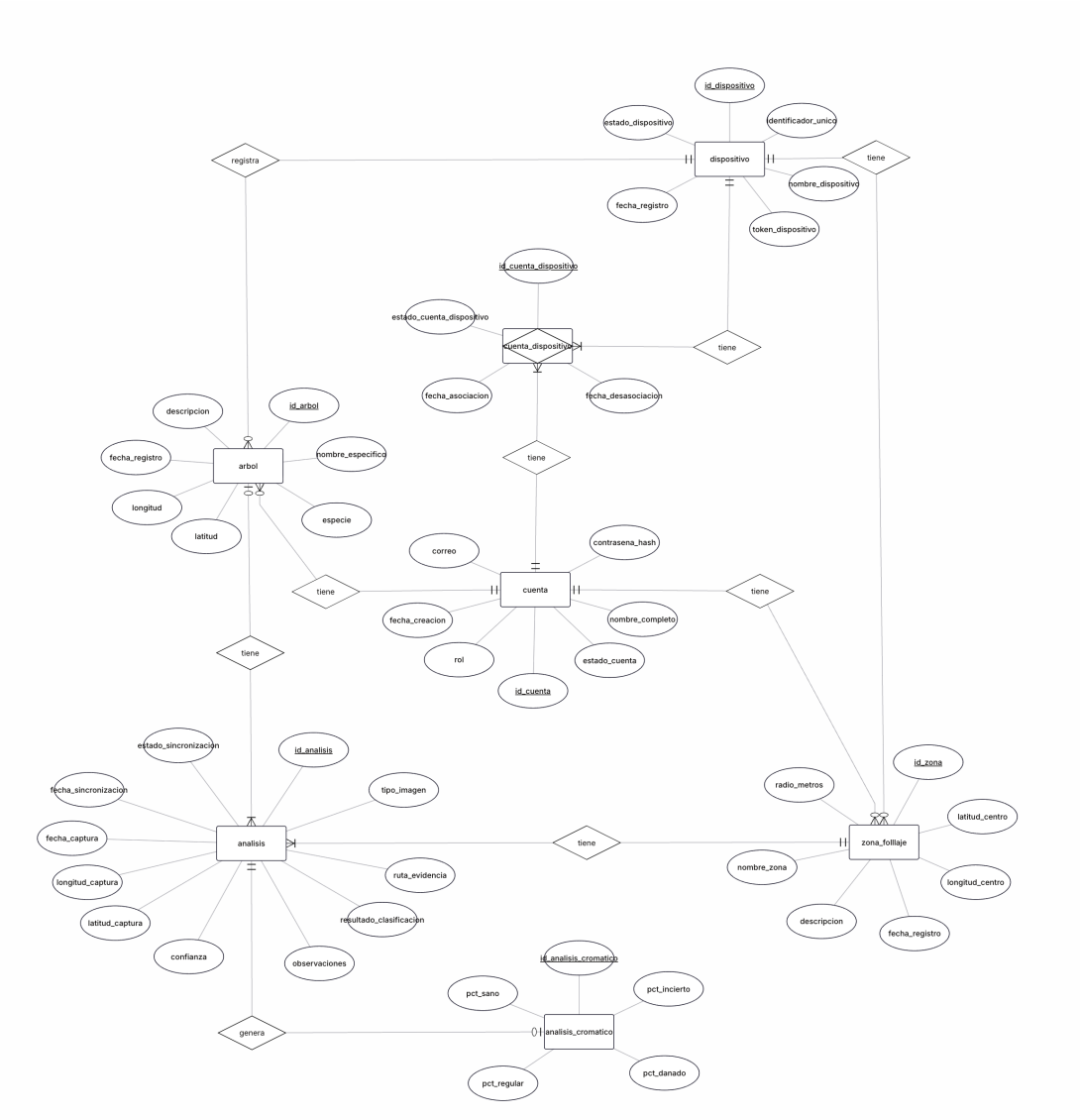


Figura 66: *MER: Sistema móvil y web*

8.11.2 Esquema relacional

El *esquema relacional* es la representación lógica de la base de datos derivada del modelo entidad-relación, expresada en términos de tablas, columnas, claves primarias y claves foráneas. A diferencia del MER, el esquema relacional está orientado a la implementación, definiendo con precisión cómo se organizan y vinculan los datos dentro del gestor de base de datos.

El esquema relacional del sistema se deriva del modelo entidad-relación y está conformado por siete tablas principales: **CUENTA**, **DISPOSITIVO**, **CUENTA_DISPOSITIVO**, **ARBOL**, **ZONA_FOLLAJE**, **ANALISIS** y **ANALISIS_CROMATICO**. Estas tablas permiten representar tanto la gestión de usuarios y dispositivos como los registros operativos generados durante el monitoreo forestal.

Las tablas **CUENTA**, **DISPOSITIVO** y **CUENTA_DISPOSITIVO** concentran la información de autenticación, autorización y asociación de dispositivos móviles. Por su parte, **ARBOL** y **ZONA_FOLLAJE** representan las entidades monitoreadas según el tipo de captura realizada: corteza para árboles individuales y follaje para zonas geográficas.

La tabla **ANALISIS** almacena los resultados de cada captura, incluyendo la referencia de la evi-

dencia, coordenadas de captura, resultado de clasificación, confianza y estado de sincronización. Esta tabla se asocia de forma condicional con **ARBOL** o **ZONA_FOLLAJE**, dependiendo de si el análisis corresponde a corteza o follaje. Finalmente, **ANALISIS_CROMATICO** almacena los porcentajes derivados del análisis complementario de color asociado a cada análisis.

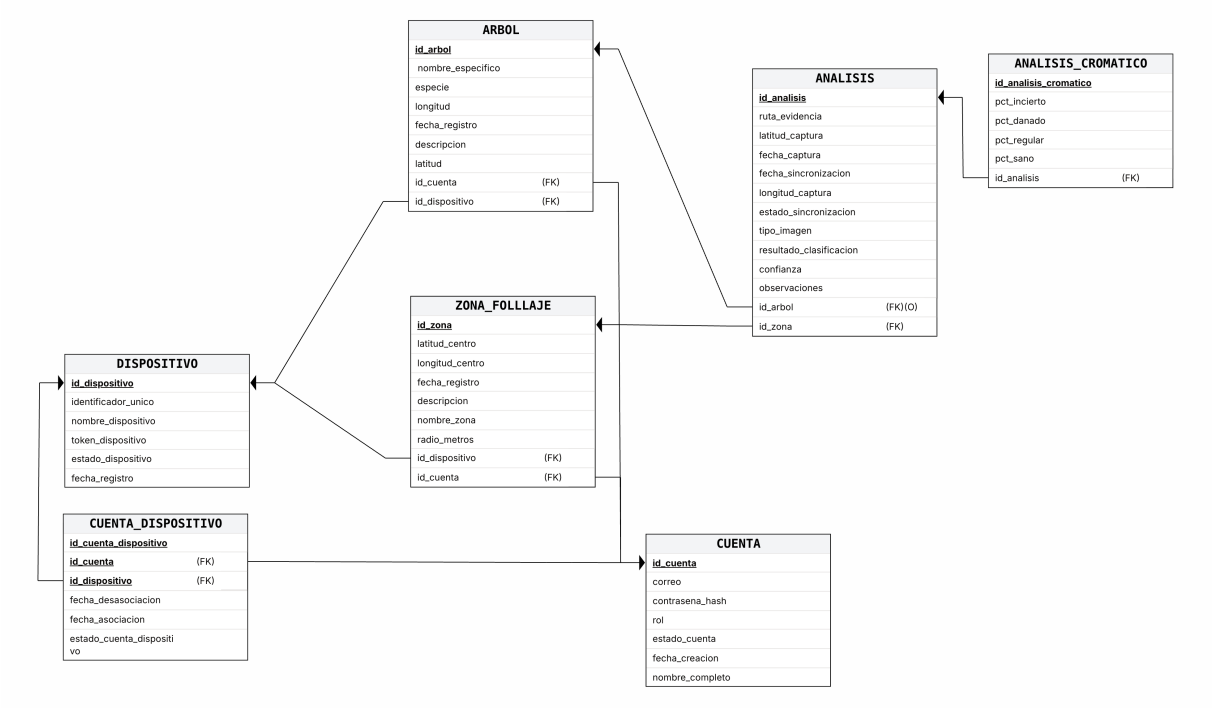


Figura 67: *ER: Sistema móvil y web*

8.11.3 Diccionario de datos

El diccionario de datos describe los atributos que integran cada tabla del esquema relacional, indicando su significado, tipo de dato sugerido, restricciones y función dentro del sistema. Esta sección complementa el modelo entidad-relación y el esquema relacional, ya que detalla la forma en que se almacenan e interpretan los datos utilizados por la aplicación móvil y la plataforma web.

Tabla 25: *Diccionario de datos de la tabla CUENTA*

Campo	Tipo	Descripción
id_cuenta	UUID	Identificador único de la cuenta. Corresponde a la clave primaria de la tabla.
correo	VARCHAR	Correo electrónico utilizado para iniciar sesión en el sistema. Debe ser único.
contrasena_hash	VARCHAR	Contraseña cifrada o hasheada de la cuenta. No se almacena en texto plano.
rol	VARCHAR	Rol asignado a la cuenta. Puede corresponder a Experto de campo o Administrador.
estado_cuenta	VARCHAR	Estado actual de la cuenta, por ejemplo: activa o desactivada.
fecha_creacion	TIMESTAMP	Fecha y hora en que la cuenta fue registrada en el sistema.

Tabla 25 – continuación

Campo	Tipo	Descripción
nombre_completo	VARCHAR	Nombre completo del usuario asociado a la cuenta.

Tabla 26: *Diccionario de datos de la tabla DISPOSITIVO*

Campo	Tipo	Descripción
id_dispositivo	UUID	Identificador único del dispositivo. Corresponde a la clave primaria de la tabla.
identificador_unico	VARCHAR	Identificador técnico del dispositivo móvil, utilizado para reconocer el equipo autorizado.
nombre_dispositivo	VARCHAR	Nombre descriptivo del dispositivo, asignado para facilitar su identificación.
token_dispositivo	VARCHAR	Token asociado al dispositivo, utilizado para identificación técnica o notificaciones.
estado_dispositivo	VARCHAR	Estado del dispositivo dentro del sistema, por ejemplo: activo, inactivo o bloqueado.
fecha_registro	TIMESTAMP	Fecha y hora en que el dispositivo fue registrado en el sistema.

Tabla 27: *Diccionario de datos de la tabla CUENTA_DISPOSITIVO*

Campo	Tipo	Descripción
id_cuenta_dispositivo	UUID	Identificador único de la asociación entre una cuenta y un dispositivo. Corresponde a la clave primaria.
id_cuenta	UUID	Clave foránea que referencia a la cuenta asociada.
id_dispositivo	UUID	Clave foránea que referencia al dispositivo asociado.
fecha_asociacion	TIMESTAMP	Fecha y hora en que el dispositivo fue asociado a la cuenta.
fecha_desasociacion	TIMESTAMP	Fecha y hora en que el dispositivo fue desasociado de la cuenta, si aplica.
estado_cuenta_dispositivo	VARCHAR	Estado de la relación entre cuenta y dispositivo, por ejemplo: activa o desasociada.

Tabla 28: *Diccionario de datos de la tabla ARBOL*

Campo	Tipo	Descripción
id_arbol	UUID	Identificador único del árbol registrado. Corresponde a la clave primaria de la tabla.
nombre_especifico	VARCHAR	Nombre o identificador descriptivo asignado al árbol por el Experto de campo.
especie	VARCHAR	Especie o tipo de árbol registrado, en caso de ser conocida.

Tabla 28 – continuación

Campo	Tipo	Descripción
longitud	FLOAT	Coordenada geográfica de longitud correspondiente a la ubicación general del árbol.
latitud	FLOAT	Coordenada geográfica de latitud correspondiente a la ubicación general del árbol.
fecha_registro	TIMESTAMP	Fecha y hora en que el árbol fue registrado en el sistema.
descripcion	TEXT	Información adicional u observaciones generales sobre el árbol registrado.
id_cuenta	UUID	Clave foránea que referencia la cuenta a la que pertenece el árbol.
id_dispositivo	UUID	Clave foránea que referencia el dispositivo desde el cual se registró el árbol.

Tabla 29: *Diccionario de datos de la tabla ZONA_FOLLAJE*

Campo	Tipo	Descripción
id_zona	UUID	Identificador único de la zona de follaje. Corresponde a la clave primaria de la tabla.
latitud_centro	FLOAT	Latitud del punto central de la zona de follaje.
longitud_centro	FLOAT	Longitud del punto central de la zona de follaje.
fecha_registro	TIMESTAMP	Fecha y hora en que la zona fue registrada o generada.
descripcion	TEXT	Descripción general de la zona de follaje.
nombre_zona	VARCHAR	Nombre descriptivo asignado a la zona para facilitar su identificación.
radio_metros	FLOAT	Radio aproximado de cobertura de la zona, expresado en metros.
id_dispositivo	UUID	Clave foránea que referencia el dispositivo desde el cual se registró la zona.
id_cuenta	UUID	Clave foránea que referencia la cuenta a la que pertenece la zona.

Tabla 30: *Diccionario de datos de la tabla ANALISIS*

Campo	Tipo	Descripción
id_analisis	UUID	Identificador único del análisis realizado. Corresponde a la clave primaria de la tabla.
ruta_evidencia	VARCHAR	Ruta relativa de la imagen asociada al análisis. En SQLite referencia el archivo temporal local; en la nube referencia la evidencia almacenada remotamente.
latitud_captura	FLOAT	Latitud exacta desde donde se realizó la captura del análisis.
longitud_captura	FLOAT	Longitud exacta desde donde se realizó la captura del análisis.

Tabla 30 – continuación

Campo	Tipo	Descripción
fecha_captura	TIMESTAMP	Fecha y hora en que se realizó la captura de la imagen.
fecha_sincronizacion	TIMESTAMP	Fecha y hora en que el análisis fue sincronizado con la nube. Puede estar vacío mientras el registro esté pendiente.
estado_sincronizacion	VARCHAR	Estado del proceso de sincronización del análisis, por ejemplo: pendiente, sincronizado o error.
tipo_imagen	VARCHAR	Tipo de imagen analizada. Puede corresponder a follaje o corteza.
resultado_clasificacion	VARCHAR	Resultado obtenido por el modelo de clasificación o análisis aplicado a la imagen.
confianza	FLOAT	Nivel de confianza asociado al resultado de clasificación.
observaciones	TEXT	Comentarios u observaciones agregadas por el Experto de campo.
id_arbol	UUID	Clave foránea opcional hacia la tabla ARBOL. Se utiliza cuando el análisis corresponde a corteza.
id_zona	UUID	Clave foránea opcional hacia la tabla ZONA_FOLLAJE. Se utiliza cuando el análisis corresponde a follaje.

Tabla 31: *Diccionario de datos de la tabla ANALISIS_CROMATICO*

Campo	Tipo	Descripción
id_analisis_cromatico	UUID	Identificador único del análisis cromático. Corresponde a la clave primaria de la tabla.
pct_incierto	FLOAT	Porcentaje de la imagen clasificado como incierto dentro del análisis cromático.
pct_danado	FLOAT	Porcentaje de la imagen clasificado como dañado.
pct_regular	FLOAT	Porcentaje de la imagen clasificado como regular.
pct_sano	FLOAT	Porcentaje de la imagen clasificado como sano.
id_analisis	UUID	Clave foránea que referencia el análisis al que pertenece el análisis cromático.

En la tabla **ANALISIS**, los campos **id_arbol** e **id_zona** se consideran condicionales. Cuando el análisis corresponde a corteza, el registro se asocia a un árbol mediante **id_arbol** y **id_zona** permanece vacío. Cuando el análisis corresponde a follaje, el registro se asocia a una zona mediante **id_zona** y **id_arbol** permanece vacío. Esta decisión permite utilizar una sola tabla de análisis para representar ambos tipos de captura sin duplicar estructuras.

Capítulo 9. Pruebas y validación del sistema

En este capítulo se presentan las pruebas y evaluaciones técnicas realizadas para verificar el cumplimiento de los requerimientos funcionales (**RF**) y no funcionales (**RNF**) del sistema *Eco-Verde*, establecidos en el Capítulo 7. Debido a que el proyecto adopta una estrategia *offline-first* para operar en zonas forestales sin conectividad, resulta fundamental validar el comportamiento de la aplicación móvil y determinar en qué tipos de dispositivos es viable su ejecución sin comprometer la estabilidad ni la precisión del análisis fitosanitario.

9.1 Pruebas de viabilidad de requisitos funcionales en dispositivos móviles

La aplicación móvil representa la herramienta central de trabajo en campo para el **Experto de campo**. A diferencia de la plataforma web, donde el procesamiento y las consultas complejas se delegan a los servicios en la nube (AWS), la aplicación móvil debe ejecutar tareas de alta exigencia de forma local y autónoma:

- **Captura y decodificación de fotografías (RF-03):** Procesamiento en memoria de imágenes en alta resolución capturadas con la cámara o seleccionadas desde la galería.
- **Geolocalización satelital precisa (RF-05):** Obtención de coordenadas geográficas mediante el sensor GNSS/GPS con al menos cinco decimales de precisión, requerimiento indispensable para delimitar el radio de alerta de 100 metros en apego a la norma NOM-019-SEMARNAT-2017 [31].
- **Almacenamiento local offline (RF-06, RF-18, RNF-15):** Persistencia transaccional de los registros y fotografías en una base de datos local SQLite mediante la librería **Room**, asegurando que la información no se pierda en campo.
- **Inferencia local con TensorFlow Lite (RF-08, RF-09):** Clasificación inmediata del estado sanitario del árbol mediante el modelo embebido *EfficientNet-B3* en formato `.tflite`, sin requerir conexión a internet.
- **Análisis cromático complementario (RF-11):** Segmentación y evaluación de color en espacio HSV mediante **OpenCV** para apoyar la detección de indicios visuales en la corteza.
- **Visualización de mapas fuera de línea (RNF-03, RNF-11):** Renderizado interactivo de mapas vectoriales mediante la librería nativa **MapLibre GL** y el archivo de teselas locales (`cdmx.pmtiles`).

Por esta razón, evaluar los límites de hardware y el rendimiento en distintos dispositivos móviles constituye una prueba directa de la viabilidad de estos requerimientos funcionales en condiciones de operación real.

9.1.1 Requisitos de hardware mínimos y recomendados

A partir de las mediciones de memoria, procesamiento y almacenamiento obtenidas en el dispositivo de prueba de referencia (*Google Pixel 7 Pro*), se definieron los requisitos técnicos necesarios para el funcionamiento de la aplicación móvil. La Tabla 32 presenta la comparativa entre los requisitos mínimos para una ejecución estable y los requisitos recomendados para un desempeño óptimo.

Tabla 32: *Requisitos de hardware mínimos y recomendados para la aplicación móvil.*

Componente	**Requisito mínimo (Entrada)**	**Requisito recomendado (Media/Alta)**	**Justificación técnica en el sistema**
Memoria RAM	3 GB (LPDDR4x)	6 GB a 12 GB (LPDDR5)	La aplicación consume en reposo ≈184 MB de memoria física real (*PSS*), alcanzando picos de 280 a 320 MB durante la captura con cámara, decodificación del bitmap y ejecución concurrente de TFLite y MapLibre. En dispositivos con 2 GB o menos (como Android Go), existe riesgo elevado de cierre forzoso por el *Low Memory Killer* (LMK) al procesar imágenes de 12 MP.
Espacio libre (ROM)	500 MB libres	2 GB a 4 GB libres	La aplicación instalada ocupa entre 90 y 140 MB. El archivo de teselas locales (`cdmx.pmtiles`) requiere 29 MB, el modelo embebido 21 MB y la base SQLite inicial de 1 a 5 MB. El espacio restante actúa como reserva para almacenar entre 150 y 200 fotografías en campo (1.5 MB en promedio cada una) antes de sincronizar con la nube (**RF-07**).
Procesador (SoC)	Octa-Core ARMv8-A de 64 bits (e.g., Helio G85, Snapdragon 680)	SoC con aceleración por NPU/DSP (e.g., Google Tensor G2/G3, Snapdragon 778G / 8 Gen 2)	En procesadores de gama de entrada con núcleos Cortex-A55 @ 2.0 GHz, la inferencia local de EfficientNet-B3 (300×300 px) toma entre 600 y 1,100 ms. En procesadores de gama alta toma de 90 a 180 ms. Un tiempo de aproximadamente un segundo resulta perfectamente aceptable para el trabajo en campo.
Cámara	8 MP a 12 MP con Autoenfoque (AF)	12 MP a 50 MP con OIS y sensor de mayor tamaño	El diagnóstico fitosanitario de corteza requiere fotografiar el fuste a corta distancia (macro a 15–30 cm) para distinguir orificios de entrada (1.5 a 4 mm) y grumos de resina. Sensores sin autoenfoque producen imágenes borrosas que impiden una clasificación correcta.

Tabla 32 – continuación

Componente	Requisito mínimo (Entrada)	Requisito recomendado (Media/Alta)	Justificación técnica en el sistema
Sensor GPS / GNSS	GPS compatible con A-GPS y GLONASS	GPS de doble frecuencia (L1 + L5) y Galileo	La norma NOM-019-SEMARNAT-2017 exige delimitar un radio de alerta de 100 metros alrededor de los árboles afectados. Se requiere una precisión satelital de al menos 5 cifras decimales ($\pm$3 a 5 m de margen de error).
Sistema Operativo	Android 8.0 (Oreo, API 26)	Android 12 a 15 (API 31–35)	Definido en la configuración del proyecto mediante `minSdk = 26` y `targetSdk = 35`, garantizando compatibilidad con la gran mayoría de los dispositivos Android activos.

9.1.2 Clasificación de dispositivos por gama de rendimiento

Con base en los requerimientos anteriores, los dispositivos móviles compatibles se clasificaron en tres gamas principales para estimar su comportamiento en campo:

1. **Gama de Entrada / Baja (Compatibilidad condicionada):**
   - *Modelos de referencia:* Samsung Galaxy A04s / A13, Xiaomi Redmi 9A / 10C, Motorola Moto G22.
   - *Comportamiento esperado:* La aplicación funciona de manera estable; la inferencia local tarda cerca de 1 segundo por imagen y la pantalla se refresca a 60 Hz o 90 Hz. En estos dispositivos se recomienda no mantener múltiples aplicaciones abiertas en segundo plano para evitar que el sistema libere memoria cerrando la aplicación durante la captura.
2. **Gama Media (Rendimiento óptimo en campo):**
   - *Modelos de referencia:* Samsung Galaxy A34 / A54, Xiaomi Redmi Note 12 / 13 Pro, Motorola Moto G84.
   - *Comportamiento esperado:* La inferencia local se completa en aproximadamente 250 a 350 ms, las transiciones y el desplazamiento del mapa son fluidos a 60–120 Hz y el almacenamiento permite varias semanas de trabajo continuo en campo sin requerir limpieza de datos.
3. **Gama Alta (Dispositivo de referencia de desarrollo):**
   - *Modelos de referencia:* Google Pixel 7 Pro (equipo empleado en las pruebas), Samsung Galaxy S23 / S24, Motorola Edge 40 Pro.
   - *Comportamiento validado:* Inferencia prácticamente instantánea ($<$150 ms) aprovechando la NPU integrada, visualización fluida a 120 Hz nativos sin sobrecalentamiento y fijación GPS rápida y precisa mediante sistemas de doble banda (L1+L5).

9.1.3 Metodología de validación y benchmark con dispositivo de prueba

Dado que para el desarrollo del proyecto se contó físicamente con un único dispositivo de gama alta (*Google Pixel 7 Pro*), se estableció una metodología de pruebas compuesta por cuatro herramientas y técnicas formales, permitiendo evaluar el rendimiento real y simular el comportamiento de la aplicación en dispositivos de menor gama sin necesidad de adquirir equipo adicional:

Método A: Simulación de gama baja en el dispositivo físico mediante ADB. A través de comandos de *Android Debug Bridge* (`adb`), es posible modificar de manera temporal la resolución de pantalla, la densidad de píxeles y el perfil de energía del procesador del Google Pixel 7 Pro. Esto permite emular las condiciones de un teléfono económico de gama de entrada:

```
# 1. Simular pantalla de gama baja (720p HD+ a 280 DPI)
adb shell wm size 720x1600
adb shell wm density 280

# 2. Reducir frecuencia de CPU (modo ahorro extremo)
adb shell cmd power set-adaptive-power-saver-enabled false
adb shell cmd power set-mode 1

# 3. Pruebas de captura, recorte e inferencia

# 4. Restaurar valores originales del Pixel 7 Pro
adb shell wm size reset
adb shell wm density reset
adb shell cmd power set-mode 0
```

Método B: Extracción de métricas de rendimiento reales mediante ADB Profiling. Durante la ejecución continua de las funciones de la aplicación (**RF-03**, **RF-08**, **RF-11**), se recolectaron los datos cuantitativos del sistema mediante los siguientes comandos:

- **Consumo de memoria RAM (`dumpsys meminfo`):** Permite obtener el consumo de memoria real del proceso `com.descortezadores.ecoverde`:

  ```
  adb shell dumpsys meminfo com.descortezadores.ecoverde
  ```

  Métricas reportadas:
  - *Total PSS (Proportional Set Size):* Memoria física real utilizada por la aplicación, incluyendo la porción proporcional de bibliotecas compartidas (lectura obtenida: 183.8 MB).
  - *Java Heap:* Memoria consumida por los objetos de la aplicación en Kotlin y Compose (lectura obtenida: 15.8 MB).
  - *Native Heap:* Memoria utilizada por librerías nativas en C++, incluyendo el motor de TensorFlow Lite y MapLibre GL (lectura obtenida: 2.3 MB).
- **Rendimiento gráfico y fluidez de la interfaz (`dumpsys gfxinfo`):** Permite medir los tiempos de renderizado y la presencia de cuadros retrasados (*jank*):

  ```
  adb shell dumpsys gfxinfo com.descortezadores.ecoverde
  ```

  Métricas reportadas:
  - *Percentil 50 (P50):* Tiempo típico de renderizado por cuadro (lectura obtenida: 6.0 ms, equivalente a más de 120 cuadros por segundo).

  - *Percentil 90 (P90):* Tiempo de renderizado en transiciones de pantalla (lectura obtenida: 15.0 ms).
  - *Janky frames:* Porcentaje de cuadros retrasados (lectura obtenida en modo depuración: 6.19%, por debajo del umbral de tolerancia del 10%).
- **Uso de CPU en reposo frente a inferencia (`top`):** Permite verificar la carga del procesador durante la ejecución del modelo:

```
adb shell top -n 1 -p $(adb shell pidof com.descortezadores.ecoverde)
```

Método C: Emuladores con restricciones de hardware en Android Studio (AVD). Dentro de Android Studio se crearon dispositivos virtuales (*Android Virtual Device*) con especificaciones intencionalmente limitadas: memoria RAM configurada en 2,048 MB (2 GB) o 3,072 MB (3 GB), *VM Heap* en 192 MB y procesador asignado a 2 núcleos virtuales. En este entorno se corroboró que el asistente de captura, la inferencia local y el mapa cargaran correctamente sin generar errores de falta de memoria (*OutOfMemoryError*).

Método D: Pruebas automatizadas en la nube con Firebase Test Lab. Con el propósito de validar la compatibilidad en terminales físicas de diferentes marcas y versiones del sistema operativo, se utilizó la plataforma gratuita *Firebase Test Lab* de Google. Mediante pruebas automatizadas de exploración (*Robo Tests*) sobre el paquete instalable (`app-debug.apk`), se verificó el funcionamiento en dispositivos físicos reales de gama baja (Samsung Galaxy A12 y A13) y gama media (Xiaomi Redmi Note 10 y Galaxy A52) con versiones de Android 10 a 14, comprobando la estabilidad de la aplicación y la ausencia de fallos críticos.

9.1.4 Resultados del benchmark y criterios de aceptación

A partir de las pruebas descritas, se estructuró la Tabla 33, donde se comparan los valores obtenidos en el dispositivo de prueba contra los umbrales de aceptación definidos para los requerimientos funcionales y de calidad del sistema.

Tabla 33: *Resultados del benchmark y evaluación frente a umbrales de aceptación.*

Métrica evaluada	**Entorno / Dispositivo**	**Valor obtenido**	**Umbral de aceptación**	**Estado**
Consumo de RAM en reposo (PSS)	Google Pixel 7 Pro	183.8 MB	$<$ 300 MB	**Aprobado**
Consumo de RAM en inferencia máxima	Google Pixel 7 Pro (Cámara + TFLite)	$\approx$295 MB	$<$ 450 MB	**Aprobado**
Tiempo de inferencia local (TFLite)	Google Tensor G2 (ARMv9 Cortex-X2)	$\approx$140 ms	$<$ 1,500 ms	**Aprobado**
Tiempo de inferencia estimado en gama baja	Emulación Cortex-A55 (con limitación ADB)	$\approx$850 ms	$<$ 2,000 ms	**Aprobado**

Tabla 33 – continuación

Métrica evaluada	Entorno / Dispositivo	Valor obtenido	Umbral de aceptación	Estado
Renderizado gráfico (Percentil 50)	Pantalla LTPO a 120 Hz	6.0 ms	$<$ 8.3 ms (120 Hz) / $<$ 16.6 ms (60 Hz)	**Aprobado**
Cuadros con retraso (*Janky Frames*)	Flujo completo de navegación (Debug)	6.19%	$<$ 10.0%	**Aprobado**
Espacio de instalación y datos locales	Almacenamiento interno privado	39 MB (Datos) + 110 MB (App)	$<$ 300 MB	**Aprobado**
Tolerancia a modo desconectado (*Offline*)	Operación sin conexión celular ni Wi-Fi	100% de funciones operativas	Operación 100% desconectado (**RF-09**)	**Aprobado**

9.1.5 Casos de incompatibilidad y limitaciones técnicas

Para delimitar el alcance del proyecto con claridad metodológica, es tan importante documentar en qué condiciones funciona el sistema como establecer en qué tipos de dispositivos no es viable su uso. Se identificaron seis tipos de dispositivos en los que la aplicación móvil no funcionará o presentará fallos críticos:

1. **Dispositivos con menos de 2.5 GB de RAM (versiones Android Go):**
   - *Dispositivos de ejemplo:* Samsung Galaxy A03 Core (2 GB), Motorola Moto E13 (2 GB), Xiaomi Redmi A1+ (2 GB), Nokia C02 (1 GB).
   - *Causa técnica:* Al capturar una fotografía de 12 a 50 MP, el sistema operativo debe decodificar el mapa de bits (*Bitmap*) en memoria RAM sin compresión (ARGB_8888), consumiendo entre 48 y 90 MB de forma instantánea. Al mismo tiempo, el intérprete de TensorFlow Lite asigna $\approx$21 MB nativos para los pesos de EfficientNet-B3 y MapLibre GL retiene de 40 a 60 MB en texturas gráficas. En dispositivos con 1 o 2 GB de RAM, el límite máximo de memoria por aplicación suele ser de solo 96 a 128 MB.
   - *Tipo de fallo:* Al tomar la fotografía o recortarla, la aplicación se cierra de forma inesperada debido a una excepción `OutOfMemoryError` o porque el kernel de Linux activa el *Low Memory Killer* (LMK).
2. **Tablets o dispositivos "Solo Wi-Fi" (sin chip GPS integrado):**
   - *Dispositivos de ejemplo:* Tablets económicas sin ranura SIM o de uso escolar (e.g., Amazon Fire 7/8/10, tablets genéricas de punto de venta).
   - *Causa técnica y biológica:* Estos dispositivos no cuentan con un receptor físico de satélites GNSS (GPS/GLONASS), de modo que solo pueden obtener su ubicación mediante la triangulación de redes Wi-Fi cercanas. En áreas forestales como el Bosque de Chapultepec o zonas de alta montaña no existen redes Wi-Fi disponibles.
   - *Tipo de fallo y afectación funcional:* El módulo de geolocalización no puede obtener coordenadas (*Location Unavailable*), incumpliendo el requerimiento **RF-05** y la norma NOM-019-SEMARNAT-2017 [31], la cual exige coordenadas exactas para delimitar el radio de alerta de 100 metros alrededor del ejemplar evaluado.

3. **Dispositivos con cámara de enfoque fijo (*Fixed-Focus* / sin autoenfoque):**
   - *Dispositivos de ejemplo:* Teléfonos celulares ultra-básicos o tablets infantiles de muy bajo costo.
   - *Causa técnica y biológica:* Las cámaras de enfoque fijo vienen calibradas para enfocar objetos a partir de un metro hacia el infinito. Sin embargo, el diagnóstico de corteza requiere enfocar a corta distancia (macro a 15–30 cm) para distinguir grumos de resina y orificios de entrada del insecto (de 1.5 a 4 mm).
   - *Tipo de fallo:* Las imágenes resultan desenfocadas y borrosas, lo que provoca que tanto el modelo de aprendizaje profundo como la segmentación cromática HSV arrojen falsos negativos o diagnósticos no determinados de forma constante.

4. **Dispositivos con Android 7.1.1 (Nougat, API 25) o versiones anteriores:**
   - *Dispositivos de ejemplo:* Equipos lanzados en 2016 o antes (e.g., Samsung Galaxy S6, Motorola Moto G4, Nexus 5).
   - *Causa técnica:* El archivo de configuración `app/build.gradle.kts` define como versión mínima `minSdk = 26` (Android 8.0 Oreo), requerida por las bibliotecas modernas de Jetpack Compose y las APIs de mapas y geolocalización.
   - *Tipo de fallo:* El sistema operativo bloquea la instalación de la aplicación mostrando el mensaje de error:

     ```
     INSTALL_FAILED_OLDER_SDK: The package requires an sdk
     version of at least 26 but target is 25
     ```

5. **Dispositivos con almacenamiento libre crítico ($<$ 300 MB libres):**
   - *Dispositivos de ejemplo:* Teléfonos de 16 o 32 GB cuyo almacenamiento se encuentra saturado por otras aplicaciones o archivos multimedia.
   - *Causa técnica:* La base de datos local SQLite implementada con *Room* opera mediante el modo *Write-Ahead Logging* (WAL), el cual requiere espacio libre para escribir archivos temporales de registro durante las transacciones. Además, las fotografías capturadas se guardan de forma provisional en el almacenamiento interno de la aplicación.
   - *Tipo de fallo:* Cuando el espacio disponible desciende por debajo de $\approx$150 MB, el sistema operativo bloquea la escritura de archivos y se genera la excepción:

     ```
     android.database.sqlite.SQLiteFullException:
     database or disk is full (code 13)
     ```

     Como consecuencia, la captura no puede guardarse en la base de datos local y el registro de inspección se pierde (**RF-06**).

6. **Dispositivos sin soporte para OpenGL ES 2.0 / 3.0:**
   - *Dispositivos de ejemplo:* Emuladores antiguos de computadora configurados en renderizado por software (*Software Rendering*) o dispositivos con procesadores gráficos obsoletos.
   - *Causa técnica:* El motor de mapas MapLibre GL Native (`libmaplibre.so`) dibuja las teselas vectoriales y los marcadores en pantalla a través de la GPU.
   - *Tipo de fallo:* La aplicación inicia con normalidad, pero al entrar a la vista del mapa, la pantalla se cierra inesperadamente con un error nativo:

     ```
     Fatal signal 11 (SIGSEGV) in libmaplibre.so:
     Failed to initialize EGL/OpenGL context
     ```

Como síntesis de las restricciones identificadas, la Tabla 34 resume los tipos de incompatibilidad, los síntomas observados en la aplicación, la causa técnica asociada y el requerimiento que se ve directamente comprometido.

Tabla 34: *Resumen de restricciones técnicas e incompatibilidades de hardware.*

Tipo de incompatibilidad	**Síntoma en la aplicación**	**Causa técnica y excepción**	**Requisito afectado**
RAM < 2.5 GB (Android Go)	Cierre inesperado al tomar foto o recortar la imagen	`OutOfMemoryError` o activación del LMK por decodificación de imagen + TFLite	**RF-03**, **RF-08**
Tablet "Solo Wi-Fi"	Imposibilidad de registrar coordenadas en el bosque	Ausencia de chip receptor satelital GNSS/GPS; excepción *Location Unavailable*	**RF-05**, NOM-019
Cámara sin autoenfoque (*Fixed-Focus*)	Diagnósticos erróneos o dictámenes de `NO_DETERMINADO`	Imposibilidad de enfocar grumos y orificios de 2 mm a corta distancia (20 cm)	**RF-03**, **RF-09**, **RF-11**
Android < 8.0 (API < 26)	El sistema no permite instalar la aplicación en el equipo	Error del instalador del sistema: `INSTALL_FAILED_OLDER_SDK`	**RNF-11**
Espacio < 300 MB libres	Error al guardar la inspección y pérdida del registro	Excepción `SQLiteFullException` por falta de espacio para fotos y base de datos	**RF-06**, **RF-18**, **RNF-15**
Sin soporte OpenGL ES 2.0 / 3.0	Cierre inesperado de la aplicación al abrir la pestaña del mapa	Fallo al inicializar el contexto gráfico en `libmaplibre.so` (`SIGSEGV`)	**RNF-03**, **RNF-11**

9.2 Pruebas y evaluación del modelo de aprendizaje profundo

En esta sección se presentan los resultados de las pruebas experimentales y la evaluación del modelo de visión por computadora propuesto para la detección del escarabajo descortezador en imágenes de corteza. Como se fundamentó en el Capítulo 5, la arquitectura seleccionada como modelo base para esta tarea corresponde a **EfficientNet-B3**, debido a su balance óptimo entre precisión diagnóstica, tamaño del archivo y eficiencia de inferencia en dispositivos móviles.

Cabe destacar que los resultados presentados a continuación corresponden a la fase inicial de validación del clasificador de corteza sobre el conjunto de datos limpio del proyecto. Esta sección se mantendrá abierta para incorporar evaluaciones complementarias en etapas posteriores, tales como pruebas de inferencia en hardware móvil heterogéneo y modelos especializados para el análisis de follaje.

9.2.1 Configuración experimental y estrategia de entrenamiento

El entrenamiento del clasificador binario (sano frente a infestado) se llevó a cabo utilizando un flujo de aprendizaje por transferencia estructurado en dos fases consecutivas:

1. **Fase 1 (Alineación de la capa clasificadora):** Se congelaron los pesos preentrenados de la base convolucional de EfficientNet-B3 en ImageNet, permitiendo que únicamente los pesos de la capa densa de clasificación superior se ajustaran durante 10 épocas iniciales con una tasa de aprendizaje de $\eta = 0.001$.
2. **Fase 2 (Ajuste fino profundo):** Se descongelaron las capas convolucionales superiores a partir de la capa 250, reduciendo la tasa de aprendizaje a $\eta = 10^{-5}$ y posteriormente a 5×10^{-6}. Se empleó una política de parada temprana (*Early Stopping*) basada en la pérdida de validación, alcanzando 18 épocas de entrenamiento en esta etapa.

La Tabla 35 resume los parámetros y condiciones operativas bajo las cuales se realizó el experimento.

Tabla 35: *Parámetros de configuración experimental del modelo.*

Parámetro	**Valor / Descripción**
Arquitectura base	EfficientNet-B3 preentrenada en ImageNet
Dimensión de entrada	300×300 píxeles (RGB)
Tamaño de lote (*batch size*)	16 imágenes
Semilla aleatoria (*seed*)	42 (para garantizar reproducibilidad)
Capas descongeladas (Fase 2)	A partir de la capa 250
Regularización	*Dropout* con probabilidad de 0.3
Suavizado de etiquetas	0.6
Pesos de clase aplicados	Clase 0 (Sano): 0.916 Clase 1 (Infestado): 1.117
Métrica de optimización	F_β con $\beta = 0.7$ (priorizando la precisión)

9.2.2 Distribución del conjunto de datos

El conjunto de datos procesado estuvo conformado por un total de 6,085 imágenes, correspondientes a 4,589 árboles o grupos detectados. Con el propósito de evitar la fuga de información entre particiones (*data leakage*), la división se realizó asegurando que las imágenes provenientes de un mismo árbol pertenecieran exclusivamente a una de las tres particiones:

- **Entrenamiento (*Train*):** 5,009 imágenes (2,912 sanas y 2,097 con indicios de infestación).
- **Validación (*Validation*):** 557 imágenes (281 sanas y 276 infestadas).
- **Prueba independiente (*Test*):** 519 imágenes (259 sanas y 260 infestadas), proporcionando un conjunto de evaluación final balanceado.

9.2.3 Curvas de entrenamiento y convergencia

La evolución de las funciones de pérdida (*loss*), exactitud (*accuracy*) y métricas de desempeño a lo largo de las 28 épocas de entrenamiento acumuladas se presenta en la Figura 68. Como se observa en la gráfica, el paso a la Fase 2 (a partir de la época 11) produjo una estabilización progresiva de la pérdida de validación en torno a 0.24, evitando problemas de sobreajuste severo gracias a la regularización por *dropout* y al esquema de pesos ponderados.

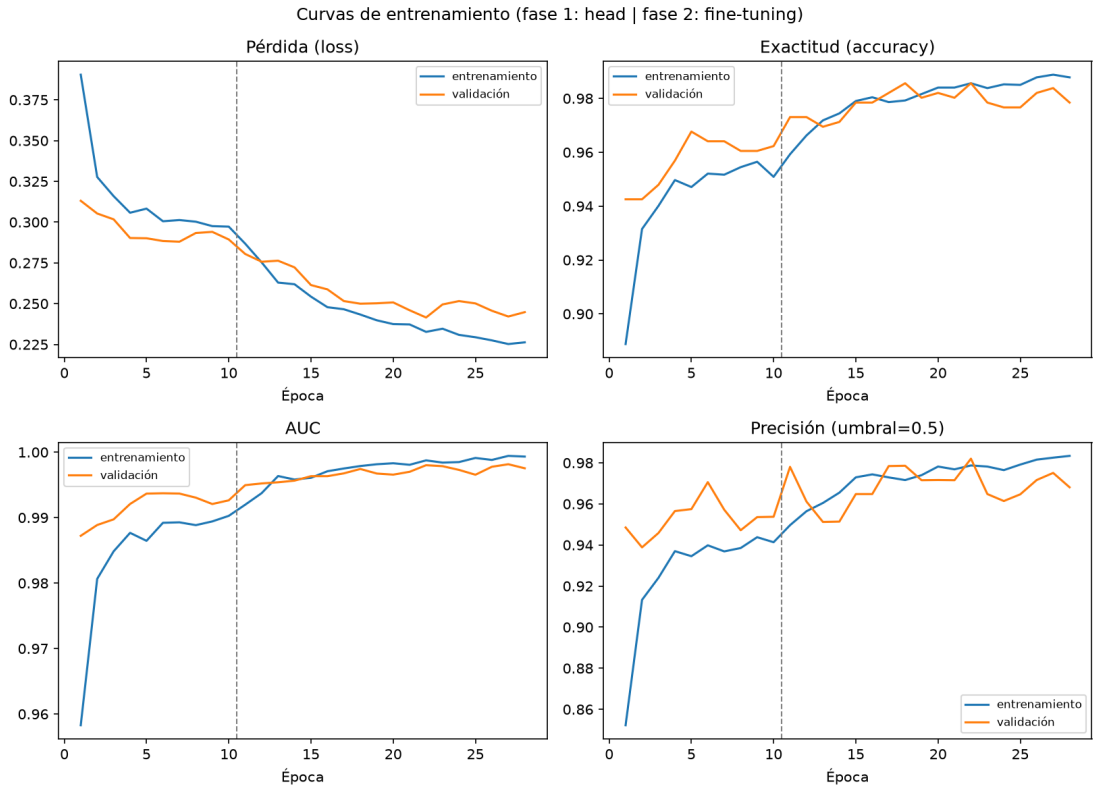


Figura 68: *Curvas de entrenamiento y validación para el clasificador de corteza (fases 1 y 2).*

Asimismo, la Figura 69 ilustra la evolución de la exactitud balanceada y del puntaje $F_{0.7}$ en el conjunto de validación durante ambas fases, corroborando que el modelo mejoró de forma continua su capacidad de discriminación sin favorecer artificialmente a la clase mayoritaria.

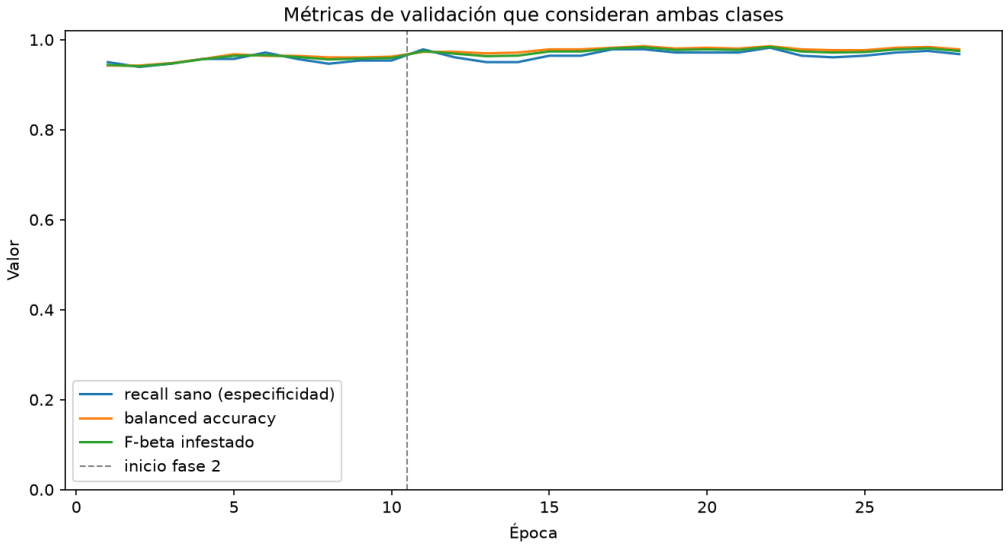


Figura 69: *Evolución de la exactitud balanceada y la puntuación F_β en validación.*

9.2.4 Optimización del umbral de decisión

En aplicaciones de detección fitosanitaria, utilizar el umbral probabilístico por defecto de 0.5 suele ser subóptimo debido a las implicaciones asimétricas entre los errores de clasificación: clasificar erróneamente un árbol sano como infestado genera costos de inspección innecesarios, mientras que no detectar un árbol infectado permite la dispersión de la plaga.

Por esta razón, se realizó una búsqueda exhaustiva del umbral óptimo de decisión evaluando la métrica $F_{0.7}$ de forma estricta sobre el **conjunto de validación** (sin utilizar el conjunto de prueba para preservar la objetividad de los resultados finales). A partir de este procedimiento, se determinó como umbral óptimo:

$$\tau_{\text{óptimo}} = 0.5791 \tag{8}$$

Con este umbral de corte, el modelo alcanzó en el conjunto de validación una precisión de 0.986, una exhaustividad (*recall*) de 0.986 y una puntuación $F_{0.7}$ de 0.986.

9.2.5 Evaluación final en el conjunto de prueba independiente

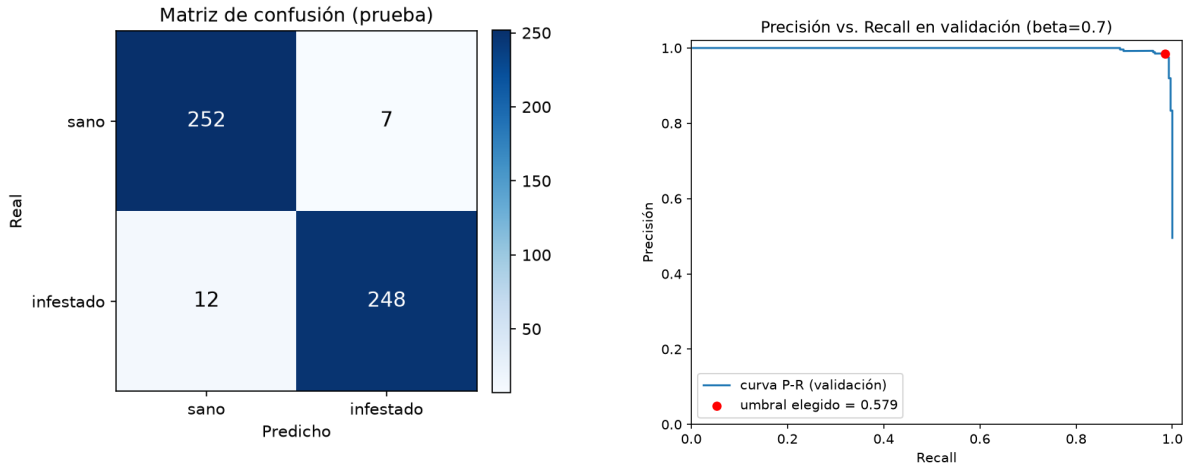
Una vez fijado el umbral óptimo $\tau = 0.5791$, se procedió a evaluar el modelo sobre el conjunto de prueba de 519 imágenes nunca antes vistas por la red. La Tabla 36 presenta las métricas cuantitativas desglosadas por clase.

Tabla 36: *Métricas de evaluación final en el conjunto de prueba con umbral $\tau = 0.5791$.*

Clase	**Precisión**	**Exhaustividad (*Recall*)**	**Puntaje F_1**	**Soporte**
Sano	95.45%	97.30%	96.37%	259
Infestado	97.25%	95.38%	96.31%	260
Exactitud global (*Accuracy*)	**96.34%**			519
Promedio macro (*Macro Avg*)	96.35%	96.34%	96.34%	519
Promedio ponderado (*Weighted Avg*)	96.36%	96.34%	96.34%	519

El modelo obtuvo una exactitud global del **96.34%**, superando holgadamente los criterios mínimos requeridos para herramientas de apoyo en campo. La clase infestado alcanzó una precisión del **97.25%**, lo que minimiza la generación de falsas alarmas para los brigadistas, junto con un *recall* del **95.38%**, garantizando la detección efectiva de la mayoría de los árboles afectados.

La Figura 70 muestra la matriz de confusión obtenida y la curva Precision-Recall correspondiente sobre el conjunto de prueba. De las 260 muestras reales de árboles infestados, 248 fueron clasificadas correctamente como positivas y únicamente 12 fueron clasificadas erróneamente como sanas. Asimismo, de los 259 árboles sanos, 252 fueron dictaminados correctamente, registrándose solo 7 falsos positivos.



(a) *Matriz de confusión (Test).* (b) *Curva Precision-Recall.*

Figura 70: *Evaluación diagnóstica en el conjunto de prueba independiente.*

Por otra parte, la Figura 71 presenta la distribución de probabilidades predichas para ambas clases, donde se evidencia una separación bimodal clara respecto a la frontera de decisión marcada por el umbral $\tau = 0.5791$.

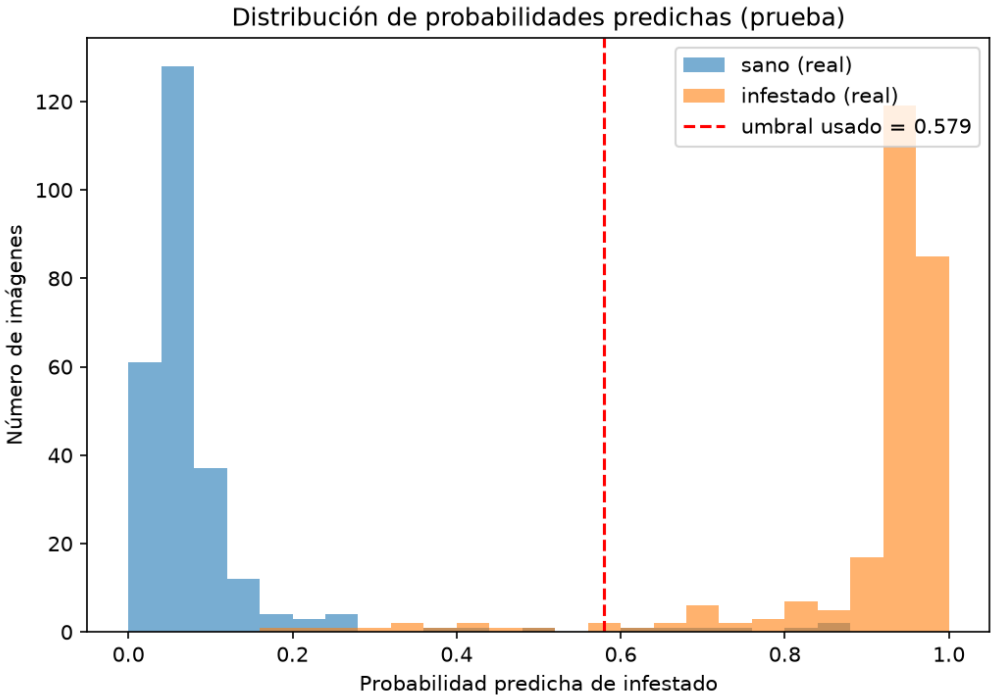


Figura 71: *Distribución de probabilidades predichas en el conjunto de prueba.*

9.2.6 Exportación a TensorFlow Lite y verificación de paridad numérica

Con el fin de integrar el modelo entrenado dentro de la aplicación móvil de acuerdo con los requerimientos **RF-08** y **RF-09**, el modelo final de Keras fue convertido al formato embebido **TensorFlow Lite** (`bark_classifier.tflite`), con un tamaño final en disco de **20.58 MB**, lo que cumple plenamente con la restricción de almacenamiento para dispositivos móviles.

Para verificar que el proceso de exportación y optimización no degradara la capacidad diagnóstica del clasificador, se ejecutó una prueba de paridad comparando las probabilidades predichas por el modelo original en Keras 3 frente al intérprete nativo de TensorFlow Lite sobre muestras del conjunto de prueba. La Tabla 37 ilustra una muestra representativa de estas comparaciones.

Tabla 37: *Prueba de paridad diagnóstica entre el modelo Keras y el modelo TensorFlow Lite.*

Muestra evaluada	Keras (%)	Clase Keras	TFLite (%)	Clase TFLite	Diferencia (%)
`inf.webp`	95.80%	Infestado	95.23%	Infestado	0.57%
`alder_2.jpg`	7.68%	Sano	7.55%	Sano	0.13%
`5549894.jpg`	7.23%	Sano	8.58%	Sano	1.36%
`alder_1.jpg`	6.35%	Sano	11.45%	Sano	5.10%
`5543613.jpg`	8.13%	Sano	7.41%	Sano	0.72%
`5454221.jpg`	7.41%	Sano	7.16%	Sano	0.25%
`5500295.jpg`	6.02%	Sano	6.21%	Sano	0.19%
`5526018.jpg`	3.69%	Sano	3.19%	Sano	0.50%

Los resultados confirman que la asignación de clase es 100% consistente entre ambos entornos, registrando diferencias porcentuales menores a nivel de probabilidad decimal atribuibles a la aritmética en punto flotante del runtime móvil, garantizando así una operación equivalente tanto en la nube como en el dispositivo de campo.

10 Conclusiones

El presente trabajo permitió establecer las bases conceptuales, técnicas y metodológicas para el desarrollo de un prototipo de sistema móvil orientado a la detección de indicios de infestación por insectos descortezadores en árboles, mediante el uso de visión por computadora y aprendizaje profundo. A partir del análisis del problema, se identificó que la detección de este tipo de plagas es una necesidad relevante para la conservación del arbolado urbano, debido a que los métodos tradicionales de inspección visual pueden resultar limitados ante la extensión de las áreas verdes, la falta de personal especializado y la rapidez con la que puede avanzar el deterioro de los árboles afectados.

La selección del Bosque de Chapultepec como caso de estudio permitió delimitar un escenario pertinente para el proyecto, al tratarse de un área verde urbana de alta importancia ecológica, social y cultural, en la que se ha documentado la presencia de afectaciones asociadas a insectos descortezadores. Asimismo, la caracterización de los hospederos, de los agentes causantes del daño y de las señales visuales presentes en el follaje y la corteza permitió definir los elementos observables que pueden ser aprovechados por un sistema computacional para apoyar la identificación de posibles casos de infestación.

Desde el punto de vista computacional, se concluye que la visión por computadora representa una alternativa viable para complementar las labores de monitoreo forestal, ya que permite analizar imágenes recolectadas en campo y extraer patrones visuales relacionados con el estado sanitario del arbolado. En este sentido, el enfoque de aprendizaje profundo resulta adecuado para el proyecto, debido a su capacidad para aprender características visuales complejas a partir de datos etiquetados, especialmente en tareas de clasificación de imágenes.

El análisis comparativo de arquitecturas permitió identificar modelos de referencia adecuados para el desarrollo del sistema, considerando criterios como precisión, eficiencia computacional y posibilidad de integración en dispositivos móviles. De igual forma, la definición del conjunto de datos, las etapas de preprocesamiento y la organización de imágenes para entrenamiento, validación y prueba constituyen elementos fundamentales para el desempeño del modelo de clasificación propuesto.

En cuanto al diseño del sistema, se estableció una arquitectura compuesta por una aplicación móvil para la captura y análisis de imágenes en campo, una plataforma web para la consulta y gestión de registros, y una infraestructura de soporte para el almacenamiento, procesamiento y seguimiento de la información generada. La definición de requerimientos funcionales y no funcionales, reglas de negocio, actores, casos de uso, flujos de operación y diseño de base de datos permitió estructurar una solución integral orientada tanto al análisis técnico como a la futura implementación del prototipo.

Se concluye que la propuesta es técnicamente viable como herramienta de apoyo para la detección y el seguimiento de árboles con posibles indicios de infestación. No obstante, su desempeño dependerá directamente de la calidad, cantidad y diversidad del conjunto de imágenes recolectadas, así como de la correcta validación de los resultados con especialistas en el área forestal. Factores como las condiciones de iluminación natural, el ángulo de captura, la variabilidad entre especies arbóreas y la similitud visual con otros tipos de daño representan limitaciones que deberán atenderse durante la fase de implementación.

Los resultados obtenidos en esta primera etapa sientan las bases necesarias para que en el Trabajo Terminal II se lleve a cabo la implementación, integración y validación del prototipo, abarcando el entrenamiento definitivo del modelo, el desarrollo de la aplicación móvil y la plataforma web, el despliegue de la infraestructura en la nube y la ejecución del plan de pruebas.

11 Trabajo a futuro

Para la siguiente fase del proyecto, se tiene planificada la ejecución de los Sprints 3 y 4 de la metodología Scrum establecida [7]. Estas etapas contemplan el desarrollo técnico, la integración de componentes y la validación final del prototipo en un entorno real. Las actividades se desglosan en las siguientes áreas de acción:

Optimización del modelo y mitigación de riesgos: Una prioridad inmediata es el incremento del volumen del conjunto de datos de otras temporadas del año. Se proyecta realizar campañas intensivas de captura en campo para alcanzar un umbral de entre 10,000 y 20,000 imágenes por clase, lo que permitirá reducir la brecha de generalización observada en las curvas de entrenamiento de las pruebas preliminares [28]. Estas capturas se realizarán bajo diversas condiciones de iluminación natural y ángulos para mitigar el riesgo de sesgo de dominio. Asimismo, se implementará un proceso de *fine-tuning* sobre la arquitectura EfficientNet-B3 [27], priorizando la mejora de la métrica de sensibilidad (*recall*), la cual es fundamental para minimizar los falsos negativos en la detección temprana de estas plagas [3].

Implementación de la arquitectura Cloud y Offline-First: Se llevará a cabo el despliegue de la infraestructura *serverless* en Amazon Web Services (AWS) [37, 38]. El núcleo del sistema será una API REST desarrollada en Micronaut y Java 21, la cual gestionará la lógica de negocio mediante el enfoque de *Clean Architecture* y administrará la persistencia políglota en Amazon RDS (PostgreSQL) [39] y Amazon S3. En el cliente móvil, se consolidará la estrategia *Offline-First*, asegurando que el modelo de clasificación [20] y el procesamiento de color con OpenCV operen de forma concurrente en el dispositivo. La sincronización de datos se programará mediante procesos en segundo plano que detecten de forma automática la disponibilidad de red para subir los registros locales alojados en SQLite hacia la nube.

Escalabilidad y proyección a ambiente productivo: El sistema ha sido diseñado desde su arquitectura base para escalar hacia un entorno de producción. A futuro, se contempla la transición desde los límites de la capa gratuita de AWS [32] hacia instancias de mayor capacidad, como *db.t3.medium* para base de datos [35, 40], y la implementación de AWS WAF para la protección contra amenazas externas en la red. Debido a la arquitectura modular empleada, el sistema podrá adaptarse a los cambios climaticos dada la naturaleza del dataset, que se compone de imagenes tomadas en todas las temporadas del año [2, 9] mediante el reentrenamiento del modelo en Amazon SageMaker, permitiendo que esta solución sea replicable en otros bosques y pulmones urbanos del país [1].

Validación integral y documentación: Finalmente, se ejecutará el plan general de pruebas funcionales, de integración y de sistema diseñado en la primera etapa. Se planea involucrar a personal forestal operativo en sesiones de prueba, con el objetivo de identificar áreas de mejora en la usabilidad tanto de la aplicación Android como de la plataforma web.

12 Referencias

[1] C. Forestal. *Bosques urbanos: aliados clave para ciudades más inclusivas, resilientes y sostenibles*. Último acceso: 13 septiembre 2025. 2025. URL: https://www.gob.mx/conafor/prensa/bosques-urbanos-aliados-clave-para-ciudades-mas-inclusivas-resilientes-y-sostenibles?idiom=es-MX.

[2] A. Vega. *Mongabay. Bosques más vulnerables a plagas: un efecto del cambio climático que ya se observa en México*. Último acceso: 13 septiembre 2025. 2022. URL: https://es.mongabay.com/2022/01/bosques-mas-vulnerables-a-plagas-un-efecto-del-cambio-climatico-mexico/.

[3] C. N. Forestal. *Insectos descortezadores*. Último acceso: 13 septiembre 2025. 2025. URL: https://www.gob.mx/conafor/documentos/insectos-descortezadores.

[4] R. N. *Radiografía de la Plaga que Amenaza los Árboles de México: el Escarabajo Descortezador*. Último acceso: 13 septiembre 2025. 2025. URL: https://www.nmas.com.mx/nacional/que-es-escarabajo-descortezador-y-como-afecta-plaga-alerta-mexico/.

[5] K. Mora. *El Sol de México. Chapultepec está enfermo: talarán 4 mil 400 árboles*. Último acceso: 13 septiembre 2025. 2022. URL: https://oem.com.mx/elsoldemexico/metropoli/chapultepec-esta-enfermo-talaran-4-mil-400-arboles-16411123.

[6] F. Reséndiz Martínez et al. «Salud forestal del arbolado de la segunda sección del Bosque de Chapultepec de la Ciudad de México». En: *SOCIEDADES RURALES, PRODUCCIÓN Y MEDIO AMBIENTE* (2019).

[7] K. S. Schwaber y J. Sutherland. *La Guía de Scrum*. Último acceso: 13 septiembre 2025. 2020. URL: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf.

[8] Scrum.org. *¿Qué es Scrum?* https://www.scrum.org/resources/blog/que-es-scrum. Accedido: 20-mar-2026.

[9] Ek del-Val y Cuauhtémoc Sáenz-Romero. «Insectos descortezadores (Coleoptera: Curculionidae) y cambio climático: problemática actual y perspectivas en los bosques templados». En: *TIP. Revista Especializada en Ciencias Químico-Biológicas* 20.2 (2017), págs. 53-60. DOI: 10.1016/j.recqb.2017.04.006.

[10] Erika Gómez-Pineda y M. Isabel Ramírez. «Descortezadores y cambio climático: una relación preocupante». En: *Revista Digital Universitaria* 24.2 (mar. de 2023). DOI: 10.22201/cuaieed.16076079e.2023.24.2.15.

[11] Microsoft Azure. *¿Qué es Computer Vision?* Último acceso: 20 marzo 2026. 2026. URL: https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-computer-vision.

[12] IBM. *¿Qué es la visión artificial?* Último acceso: 20 marzo 2026. 2026. URL: https://www.ibm.com/mx-es/think/topics/computer-vision.

[13] Jacob Murel y Eda Kavlakoglu. *¿Qué es la detección de objetos?* https://www.ibm.com/mx-es/think/topics/object-detection. Último acceso: 20 marzo 2026. 2026.

[14] Dave Bergmann. *¿Qué es la segmentación de imágenes?* https://www.ibm.com/think/topics/image-segmentation. Último acceso: 20 marzo 2026. 2026.

[15] Dave Bergmann. *¿Qué es el aprendizaje automático?* https://www.ibm.com/mx-es/think/topics/machine-learning. Último acceso: 24 marzo 2026. 2026.

[16] Ivan Belcic y Cole Stryker. *¿Qué es el aprendizaje supervisado?* https://www.ibm.com/mx-es/think/topics/supervised-learning. Último acceso: 24 marzo 2026. 2026.

[17] Ivan Belcic. *¿Qué es la clasificación en machine learning?* https://www.ibm.com/mx-es/think/topics/classification-machine-learning. Último acceso: 24 marzo 2026. 2026.

[18] AprendeIA. *Ventajas y Desventajas de los Algoritmos de Clasificación*. https://aprendeia.com/2019/12/20/ventajas-y-desventajas-de-los-algoritmos-de-clasificacion-machine-learning/. Accedido: 20-mar-2026. Dic. de 2019.

[19] Zhizhe Liu, Luo Sun y Qian Zhang. «High Similarity Image Recognition and Classification Algorithm Based on Convolutional Neural Network». En: *Computational Intelligence and Neuroscience* 2022 (2022), pág. 2836486. DOI: 10.1155/2022/2836486.

[20] Niall O'Mahony et al. «Deep Learning vs. Traditional Computer Vision». En: *Advances in Computer Vision*. Springer, 2019, págs. 128-144.

[21] Mingfei Wu, Chen Li y Zehuan Yao. «Deep Active Learning for Computer Vision Tasks: Methodologies, Applications, and Challenges». En: *Applied Sciences* 12.16 (2022), pág. 8103. DOI: 10.3390/app12168103.

[22] Amazon Web Services. *Aprendizaje profundo y machine learning: diferencia entre tecnologías de datos*. https://aws.amazon.com/es/compare/the-difference-between-machine-learning-and-deep-learning/. Último acceso: 24 marzo 2026. 2026.

[23] Bravent. *Conociendo las Redes Neuronales con Diego Calvo*. https://www.bravent.net/noticias/conociendo-las-redes-neuronales-con-diego-calvo/. Accedido: 20-mar-2026. Jun. de 2019.

[24] IBM. *¿Qué son las redes neuronales convolucionales?* https://www.ibm.com/mx-es/think/topics/convolutional-neural-networks. Último acceso: 25 marzo 2026. 2025.

[25] alka1974. *Vision Transformer (ViT) Architecture*. https://www.geeksforgeeks.org/deep-learning/vision-transformer-vit-architecture/. Último acceso: 25 marzo 2026. 2025.

[26] Andrew G. Howard et al. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. https://arxiv.org/abs/1704.04861. Último acceso: 25 marzo 2026. 2017.

[27] Mingxing Tan y Quoc V. Le. «EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks». En: *ArXiv* abs/1905.11946 (2019).

[28] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: http://www.deeplearningbook.org.

[29] Alex Krizhevsky, Ilya Sutskever y Geoffrey E Hinton. «Imagenet classification with deep convolutional neural networks». En: *Advances in neural information processing systems*. 2012, págs. 1097-1105.

[30] Sergey Ioffe y Christian Szegedy. «Batch normalization: Accelerating deep network training by reducing internal covariate shift». En: *International conference on machine learning*. PMLR. 2015, págs. 448-456.

[31] Secretaría de Medio Ambiente y Recursos Naturales. *Norma Oficial Mexicana NOM-019-SEMARNAT-2017, que establece los lineamientos técnicos para la prevención, combate y control de insectos descortezadores*. Diario Oficial de la Federación, 22 marzo 2018. Accedido: 20 marzo 2026. Mar. de 2018. URL: https://cepanaf.edomex.gob.mx/sites/cepanaf.edomex.gob.mx/files/files/Monte%20Alto/Normatividad/NOM-019-SEMARNAT-2017.pdf.

[32] Amazon Web Services. *AWS Free Tier*. Último acceso: 2 mayo 2026. 2026. URL: https://aws.amazon.com/free/.

[33] Amazon Web Services. *AWS Lambda Pricing*. Último acceso: 2 mayo 2026. 2026. URL: https://aws.amazon.com/lambda/pricing/.

[34] Amazon Web Services. *Amazon CloudFront Pricing*. Último acceso: 2 mayo 2026. 2026. URL: https://aws.amazon.com/cloudfront/pricing/.

[35] Amazon Web Services. *Estimación de costos del prototipo — AWS Pricing Calculator*. Último acceso: 2 mayo 2026. 2026. URL: https://calculator.aws/#/estimate?id=f1906b6e6936514d8325160110c97fc1d367d5de.

[36] Tech Insider. *AWS vs Azure vs Google Cloud 2026*. Último acceso: 2 mayo 2026. 2026. URL: https://tech-insider.org/aws-vs-azure-vs-google-cloud-2026/.

[37] Northflank. *AWS vs Azure vs Google Cloud: comprehensive comparison*. Último acceso: 2 mayo 2026. 2025. URL: https://northflank.com/blog/aws-vs-azure-vs-google-cloud.

[38] CloudPlexo. *Serverless Compute Comparison: AWS Lambda vs Azure Functions vs Google Cloud Functions*. Último acceso: 2 mayo 2026. 2024. URL: https://cloudplexo.com/blog/serverless-compute-comparison-aws-lambda-vs-azure-functions-vs-google-cloud-functions/.

[39] Amazon Web Services. *Amazon RDS for PostgreSQL Pricing*. Último acceso: 2 mayo 2026. 2026. URL: https://aws.amazon.com/rds/postgresql/pricing/.

[40] Amazon Web Services. *Estimación de costos en ambiente productivo — AWS Pricing Calculator*. Último acceso: 2 mayo 2026. 2026. URL: https://calculator.aws/#/estimate?id=65c11c7fb1e7ea52aab349e587a999504e736112.

[41] Yann LeCun et al. «Backpropagation applied to handwritten zip code recognition». En: *Neural computation* 1.4 (1989), págs. 541-551.

13 Glosario

Capítulo 1. Planteamiento del proyecto

Plaga forestal: Organismo o conjunto de organismos que afectan negativamente la salud, crecimiento o supervivencia del arbolado.

Insectos descortezadores: Escarabajos que viven y se desarrollan debajo de la corteza de los árboles, alimentándose de tejidos conductores y provocando debilitamiento o muerte del árbol.

Detección temprana: Identificación de señales iniciales de una afectación antes de que el daño sea severo o irreversible.

Arbolado: Conjunto de árboles presentes en una zona determinada, como un bosque, parque o área verde urbana.

Monitoreo: Proceso de observación, registro y seguimiento continuo del estado de un fenómeno, sistema o recurso natural.

Visión por computadora: Área de la inteligencia artificial que permite a los sistemas analizar e interpretar imágenes o videos para extraer información útil.

Inteligencia artificial: Campo de la computación orientado al desarrollo de sistemas capaces de realizar tareas que normalmente requieren inteligencia humana.

Procesamiento de imágenes: Conjunto de técnicas utilizadas para modificar, mejorar o analizar imágenes digitales.

Aplicación móvil: Software diseñado para ejecutarse en dispositivos móviles, como teléfonos inteligentes o tabletas.

Dataset: Conjunto de datos organizado que se utiliza para entrenar, validar o probar un modelo computacional.

Scrum: Metodología ágil de gestión de proyectos basada en ciclos iterativos llamados sprints.

Sprint: Periodo corto de trabajo dentro de Scrum en el que se desarrollan tareas específicas del proyecto.

Product Backlog: Lista priorizada de requerimientos, funcionalidades o tareas pendientes de un proyecto ágil.

Sprint Backlog: Conjunto de tareas seleccionadas para desarrollarse durante un sprint específico.

Capítulo 2. Contexto y caracterización del caso de estudio

Bosque urbano: Área verde con presencia significativa de árboles ubicada dentro o cerca de una zona urbana.

Servicios ecosistémicos: Beneficios que los ecosistemas proporcionan a la sociedad, como regulación de temperatura, mejora de calidad del aire y espacios de recreación.

Infestación: Presencia y desarrollo de organismos dañinos, como insectos descortezadores, dentro o sobre un árbol.

Hospedero: Árbol o planta que sirve como fuente de alimento, refugio o sitio de reproducción para un organismo.

Agente biótico: Organismo vivo que influye sobre otro organismo o sobre un ecosistema.

Coleóptero: Insecto perteneciente al orden Coleoptera, caracterizado por tener alas anteriores endurecidas llamadas élitros.

Curculionidae: Familia de insectos coleópteros a la que pertenecen diversos escarabajos, incluidos algunos descortezadores.

Dendroctonus: Género de insectos descortezadores de alta relevancia forestal debido a su capacidad para causar la muerte de árboles.

Ips: Género de insectos descortezadores asociado al ataque de árboles debilitados o estresados.

Hylesinus sp.: Género de descortezador asociado a afectaciones en fresnos, especialmente en ramas y tallos.

Phloeosinus sp.: Género de descortezador asociado a afectaciones en cedros y otras especies de la familia Cupressaceae.

Fraxinus uhdei: Nombre científico del fresno, especie arbórea presente en el Bosque de Chapultepec.

Cupressus lusitanica: Nombre científico del cedro blanco, especie susceptible a infestaciones por descortezadores.

Corteza: Capa externa del tronco y ramas del árbol que protege los tejidos internos.

Tejido subcortical: Región ubicada debajo de la corteza del árbol, donde pueden desarrollarse galerías producidas por descortezadores.

Floema: Tejido vegetal encargado de transportar nutrientes producidos por la fotosíntesis hacia distintas partes del árbol.

Xilema: Tejido vegetal encargado de transportar agua y minerales desde las raíces hacia el resto del árbol.

Resina: Sustancia viscosa producida por algunos árboles como mecanismo de defensa ante heridas o ataques de insectos.

Grumos de resina: Acumulaciones visibles de resina sobre la corteza que pueden indicar perforaciones o ataques de insectos.

Galerías subcorticales: Túneles excavados por insectos descortezadores debajo de la corteza durante su alimentación o reproducción.

Orificios de entrada: Perforaciones realizadas por los insectos para ingresar al interior de la corteza.

Orificios de salida: Perforaciones por donde emergen los insectos adultos después de completar su desarrollo.

Estrés hídrico: Condición en la que un árbol no dispone de suficiente agua para mantener sus funciones fisiológicas normales.

Colonización: Proceso mediante el cual los insectos descortezadores ingresan y se establecen en un árbol hospedero.

Follaje: Conjunto de hojas o acículas de un árbol, cuya coloración puede indicar su estado sanitario.

Dosel: Capa superior formada por las copas de los árboles en un bosque.

Infestación activa: Estado en el que los insectos se encuentran vivos, alimentándose o reproduciéndose dentro del árbol.

Fase de infestación: Etapa del proceso de daño del árbol, distinguible por señales visuales como cambios de color en follaje o corteza.

Capítulo 3. Delimitación computacional del problema

Información de entrada: Datos que recibe un sistema para realizar un procesamiento; en este proyecto corresponde principalmente a imágenes de árboles.

Imagen digital: Representación visual compuesta por píxeles que puede ser procesada por una computadora.

Píxel: Unidad mínima de una imagen digital, asociada a un valor de color o intensidad.

Clasificación de imágenes: Tarea de visión por computadora que asigna una imagen a una categoría o clase determinada.

Detección de objetos: Tarea de visión por computadora que localiza e identifica objetos específicos dentro de una imagen.

Segmentación de imágenes: Técnica que divide una imagen en regiones o zonas significativas para su análisis.

Características visuales: Rasgos presentes en una imagen, como color, textura, forma o patrones, que pueden ser utilizados para su análisis.

Extracción de características: Proceso mediante el cual se identifican y obtienen rasgos relevantes de una imagen.

Modelo computacional: Representación matemática o algorítmica capaz de procesar datos y producir resultados.

Flujo de trabajo: Secuencia organizada de pasos necesarios para desarrollar, entrenar, evaluar o implementar un sistema.

Capítulo 4. Fundamentación del enfoque de aprendizaje profundo

Aprendizaje automático: Rama de la inteligencia artificial que permite a los sistemas aprender patrones a partir de datos.

Aprendizaje supervisado: Tipo de aprendizaje automático en el que el modelo se entrena con datos previamente etiquetados.

Clasificación: Tarea de aprendizaje automático que consiste en asignar una entrada a una de varias clases definidas.

Clase: Categoría o etiqueta que representa el resultado esperado de una predicción.

Aprendizaje profundo: Subcampo del aprendizaje automático basado en redes neuronales con múltiples capas.

Red neuronal: Modelo computacional inspirado en el funcionamiento del cerebro, formado por unidades llamadas neuronas artificiales.

Red neuronal convolucional: Tipo de red neuronal especializada en el análisis de imágenes mediante operaciones de convolución.

Datos anotados: Datos que incluyen etiquetas o información adicional que indica la respuesta esperada para el entrenamiento de un modelo.

Entrenamiento: Proceso mediante el cual un modelo ajusta sus parámetros para aprender patrones a partir de datos.

Validación: Evaluación del modelo durante el desarrollo para medir su desempeño y ajustar decisiones de entrenamiento.

Prueba: Evaluación final del modelo con datos que no fueron utilizados durante el entrenamiento.

Generalización: Capacidad de un modelo para producir resultados correctos ante datos nuevos o no vistos previamente.

Sobreajuste: Fenómeno en el que un modelo aprende demasiado los datos de entrenamiento y pierde capacidad de generalizar.

Precisión: Métrica que indica la proporción de predicciones correctas realizadas por un modelo.

Predicción: Resultado generado por un modelo al analizar una entrada.

Inferencia: Proceso de utilizar un modelo entrenado para generar predicciones sobre nuevos datos.

Capítulo 5. Análisis comparativo de arquitecturas y selección del modelo

Arquitectura de modelo: Estructura interna de un modelo de aprendizaje profundo, definida por sus capas, conexiones y operaciones.

ResNet-50: Arquitectura de red neuronal profunda basada en conexiones residuales, utilizada en clasificación de imágenes.

MobileNetV3: Arquitectura de red neuronal optimizada para dispositivos móviles y sistemas con recursos limitados.

EfficientNet: Familia de modelos de aprendizaje profundo que busca equilibrar precisión y eficiencia computacional.

EfficientNet-B3: Variante de EfficientNet con mayor capacidad que modelos más pequeños de la misma familia, utilizada para clasificación de imágenes.

MobileViT: Arquitectura que combina redes convolucionales con mecanismos tipo Transformer para tareas visuales.

Transfer learning: Técnica que reutiliza un modelo previamente entrenado para adaptarlo a una nueva tarea.

Fine-tuning: Ajuste fino de un modelo preentrenado mediante entrenamiento adicional con datos específicos del problema.

Backbone: Red base de una arquitectura que extrae características generales de las imágenes.

Modelo preentrenado: Modelo que ya fue entrenado previamente con un conjunto de datos amplio y puede reutilizarse en otra tarea.

ImageNet: Conjunto de datos ampliamente utilizado para entrenar y evaluar modelos de clasificación de imágenes.

Convolución: Operación matemática que permite detectar patrones locales en una imagen, como bordes, texturas o formas.

Batch Normalization: Técnica que normaliza valores intermedios dentro de una red neuronal para estabilizar y acelerar el entrenamiento.

ReLU: Función de activación que transforma valores negativos en cero y conserva valores positivos.

Pooling: Operación que reduce el tamaño de los mapas de características conservando información relevante.

Flatten: Operación que convierte una matriz o mapa de características en un vector.

Capa densa: Capa neuronal en la que cada neurona está conectada con todas las neuronas de la capa anterior.

Softmax: Función que convierte los valores de salida de un modelo en probabilidades asociadas a cada clase.

Parámetros del modelo: Valores internos que el modelo ajusta durante el entrenamiento para aprender patrones.

Escalado compuesto: Estrategia utilizada en EfficientNet para aumentar de forma balanceada la profundidad, ancho y resolución del modelo.

Capítulo 6. Conjunto de datos y preprocesamiento

Recopilación de datos: Proceso de obtener imágenes o información necesaria para construir el conjunto de datos del proyecto.

Conjunto de datos: Colección organizada de ejemplos utilizados para entrenar, validar y probar un modelo.

Etiquetado de imágenes: Proceso de asignar una clase o categoría a cada imagen del conjunto de datos.

Preprocesamiento: Conjunto de operaciones aplicadas a los datos antes de utilizarlos en el entrenamiento de un modelo.

Normalización: Ajuste de los valores de una imagen para que se encuentren dentro de un rango común y faciliten el entrenamiento.

Redimensionamiento: Modificación del tamaño de una imagen para adaptarla a los requerimientos de entrada del modelo.

Aumentación de datos: Técnica que genera variaciones de imágenes existentes para incrementar la diversidad del conjunto de datos.

Conjunto de entrenamiento: Parte del dataset utilizada para que el modelo aprenda patrones.

Conjunto de validación: Parte del dataset utilizada para evaluar el comportamiento del modelo durante el desarrollo.

Conjunto de prueba: Parte del dataset utilizada para medir el desempeño final del modelo.

Análisis cromático: Evaluación de los colores presentes en una imagen para identificar patrones asociados al estado del árbol.

Coloración del follaje: Cambio visual en las hojas o copa del árbol que puede indicar una fase de deterioro o infestación.

Capítulo 7. Análisis del sistema

Requerimiento funcional: Función o comportamiento específico que el sistema debe realizar.

Requerimiento no funcional: Condición de calidad que debe cumplir el sistema, como rendimiento, seguridad, disponibilidad o usabilidad.

Regla de negocio: Condición o restricción que define cómo debe comportarse el sistema según el contexto del proyecto.

Actor del sistema: Usuario, entidad o componente externo que interactúa con el sistema.

Métrica de evaluación: Criterio cuantitativo utilizado para medir el desempeño del modelo o del sistema.

Criterio de aceptación: Condición que debe cumplirse para considerar válida una funcionalidad o entregable.

Riesgo técnico: Posible problema relacionado con la implementación, funcionamiento o integración del sistema.

Mitigación: Acción planificada para reducir la probabilidad o impacto de un riesgo.

Prueba funcional: Evaluación que verifica si una función del sistema opera correctamente.

Prueba de integración: Evaluación que verifica la comunicación y funcionamiento conjunto de varios módulos del sistema.

Infraestructura en la nube: Conjunto de recursos computacionales ofrecidos por un proveedor de nube para almacenamiento, procesamiento o despliegue.

AWS: Plataforma de servicios en la nube de Amazon utilizada para alojar, procesar o administrar componentes del sistema.

Azure: Plataforma de servicios en la nube de Microsoft considerada como alternativa de infraestructura.

GCP: Plataforma de servicios en la nube de Google considerada como alternativa de infraestructura.

Amazon SageMaker: Servicio de AWS orientado al desarrollo, entrenamiento y despliegue de modelos de aprendizaje automático.

AWS Lambda: Servicio serverless de AWS que permite ejecutar código sin administrar servidores directamente.

Amazon RDS: Servicio de AWS para administrar bases de datos relacionales.

PostgreSQL: Sistema de gestión de bases de datos relacional de código abierto.

Serverless: Modelo de computación en la nube en el que el proveedor administra la infraestructura y el usuario solo define la lógica de ejecución.

Contenedor: Unidad de software que empaqueta una aplicación y sus dependencias para ejecutarse de forma portable.

Capa gratuita: Conjunto de servicios o recursos ofrecidos sin costo limitado por un proveedor de nube.

AWS Pricing Calculator: Herramienta de AWS utilizada para estimar costos de infraestructura y servicios en la nube.

Capítulo 8. Diseño del sistema

Arquitectura del sistema: Organización general de los componentes que conforman el sistema y sus relaciones.

Backend: Parte del sistema encargada de la lógica de negocio, procesamiento, comunicación con bases de datos y servicios.

Frontend: Parte visual del sistema con la que interactúa el usuario.

API: Interfaz que permite la comunicación entre diferentes componentes de software.

Clean Architecture: Patrón de diseño que organiza el software en capas independientes para mejorar mantenibilidad y separación de responsabilidades.

CI/CD: Prácticas de integración continua y despliegue continuo utilizadas para automatizar pruebas, construcción y liberación de software.

Integración continua: Práctica que consiste en incorporar frecuentemente cambios de código y verificarlos mediante procesos automáticos.

Despliegue continuo: Práctica que automatiza la publicación de nuevas versiones del sistema.

Observabilidad: Capacidad de conocer el estado interno de un sistema mediante métricas, registros y trazas.

Monitoreo centralizado: Supervisión unificada de eventos, errores y métricas de los componentes del sistema.

Offline-First: Estrategia de diseño que permite que una aplicación funcione aun cuando no exista conexión a internet.

Sincronización: Proceso mediante el cual los datos locales y remotos se actualizan para mantener consistencia.

Inferencia local: Ejecución del modelo de clasificación directamente en el dispositivo móvil.

SPA: Aplicación web de una sola página que actualiza su contenido dinámicamente sin recargar completamente el sitio.

Autenticación: Proceso mediante el cual se verifica la identidad de un usuario.

Control de acceso: Mecanismo que determina qué acciones o recursos puede utilizar un usuario dentro del sistema.

Dashboard: Interfaz que presenta información resumida, indicadores o accesos principales del sistema.

UI: Interfaz de usuario, es decir, los elementos visuales con los que interactúa una persona.

UX: Experiencia de usuario, relacionada con la facilidad, claridad y satisfacción al utilizar el sistema.

Caso de uso: Descripción de una interacción entre un actor y el sistema para lograr un objetivo específico.

Modelo entidad-relación: Representación conceptual de las entidades, atributos y relaciones de una base de datos.

Esquema relacional: Organización lógica de una base de datos mediante tablas, columnas y relaciones.

Diccionario de datos: Documento que describe las tablas, campos, tipos de datos y restricciones de una base de datos.

Entidad: Objeto o concepto del mundo real representado dentro de una base de datos.

Atributo: Propiedad o característica de una entidad.

Llave primaria: Atributo que identifica de forma única cada registro de una tabla.

Llave foránea: Atributo que establece una relación entre dos tablas.

Persistencia: Capacidad de almacenar datos de forma duradera para consultarlos posteriormente.

Persistencia políglota: Uso de diferentes tecnologías de almacenamiento según el tipo o necesidad de los datos.

Análisis cromático: Proceso complementario que evalúa la información de color de las imágenes para apoyar la clasificación del estado del árbol.

Registro: Entrada almacenada en el sistema que contiene información asociada a un árbol, análisis o usuario.

Benchmark: Proceso sistemático de evaluación y comparación cuantitativa del rendimiento de un sistema frente a umbrales predefinidos.

PSS (Proportional Set Size): Medida de memoria física real atribuible a un proceso, calculada sumando la memoria privada más la fracción proporcional de memoria compartida con otros procesos.

Java Heap: Región de memoria administrada automáticamente por la máquina virtual de Android (ART) para instancias de objetos de la aplicación.

Native Heap: Espacio de memoria asignado dinámicamente fuera de la máquina virtual gestionada, comúnmente utilizado por bibliotecas en C y C++ como TensorFlow Lite o MapLibre GL Native.

Low Memory Killer (LMK): Mecanismo del kernel de Linux en Android encargado de terminar selectivamente procesos cuando la memoria física disponible alcanza niveles críticos.

Janky Frames: Cuadros de la interfaz de usuario cuyo tiempo de renderizado supera el presupuesto permitido por la tasa de refresco de la pantalla (e.g., más de 16.6 ms a 60 Hz o más de 8.3 ms a 120 Hz), produciendo tirones visibles.

Android Debug Bridge (ADB): Herramienta de línea de comandos que permite la comunicación, depuración y control de un dispositivo Android conectado.

AVD (Android Virtual Device): Configuración que emula las características de hardware y software de un dispositivo Android físico dentro del entorno de desarrollo.

Firebase Test Lab: Infraestructura basada en la nube de Google que permite ejecutar pruebas automatizadas en matrices de dispositivos móviles físicos y virtuales.

Fixed-Focus: Óptica de cámara con distancia de enfoque fija e invariable, usualmente calibrada a la distancia hiperfocal sin posibilidad de enfoque macro cercano.

Write-Ahead Logging (WAL): Mecanismo de persistencia transaccional en bases de datos relacionales donde las modificaciones se registran secuencialmente en un archivo de bitácora antes de consolidarse en el archivo principal.

OpenGL ES: Especificación estándar de interfaz de programación de aplicaciones para el renderizado de gráficos vectoriales 2D y 3D en dispositivos embebidos y móviles.

Fine-Tuning: Técnica de ajuste fino en aprendizaje por transferencia donde se descongelan y reentrenan capas convolucionales superiores a una tasa de aprendizaje reducida.

Early Stopping: Criterio de parada temprana que interrumpe el entrenamiento de una red neuronal cuando el desempeño en el conjunto de validación deja de mejorar, evitando el sobreajuste.

Data Leakage: Fuga de información que ocurre cuando datos del conjunto de prueba o validación influyen inadvertidamente en el entrenamiento del modelo.

F-beta Score: Métrica de evaluación que pondera la importancia relativa entre la precisión y la exhaustividad mediante un factor de ponderación β.

Curva Precision-Recall: Gráfica que ilustra el balance entre la precisión y la exhaustividad de un clasificador binario a través de diversos umbrales de decisión.

Paridad numérica: Grado de consistencia matemática en las predicciones generadas por un modelo al ser ejecutado en diferentes entornos o motores de inferencia (e.g., Keras frente a TensorFlow Lite).